



Universidad Veracruzana

Facultad de Estadística e Informática

Región Xalapa

Licenciatura en Ingeniería de Software

Análisis del uso de Jenkins en actividades de integración y despliegue continuo: Una Revisión Multivocal de la Literatura

Presentan:

Ángel de Jesús de la Cruz García
Tristán Eduardo Suárez Santiago

Director:

Saúl Domínguez Isidro

Codirector:

Juan Carlos Pérez Arriaga

Mayo de 2025

“Lis de Veracruz: Arte, Ciencia, Luz”



Universidad Veracruzana

Facultad de Estadística e Informática
Región Xalapa

Licenciatura en Ingeniería de Software

*Análisis del uso de Jenkins en actividades de integración y
despliegue continuo: Una Revisión Multivocal de la Literatura*

Presentan:

Ángel de Jesús de la Cruz García
Tristán Eduardo Suárez Santiago

Director:

Saúl Domínguez Isidro

Codirector:

Juan Carlos Pérez Arriaga

Índice

Índice	2
Resumen.....	4
1 Introducción	5
2 Antecedentes	7
2.1 Integración y despliegue continuo	9
2.2 Jenkins.....	10
3 Conducción del método	11
3.1 Planeación de la revisión multivocal de la literatura.....	13
3.1.1 Preguntas de investigación.....	13
3.2 Conducción de la revisión multivocal de la literatura	14
3.2.1 Identificación de estudios relevantes.....	14
3.2.1.1 Identificar venues y motores	14
3.2.1.2 Establecer el estándar Quasi-Gold.....	15
3.2.1.3 Identificar Términos de Búsqueda.....	18
3.2.1.4 Generar Cadenas de Búsqueda.....	19
3.2.1.5 Selección de fuentes.....	29
3.2.2 Selección de estudios primarios	30
3.2.2.1 Criterios de Inclusión y Exclusión.....	30
3.2.2.2 Procedimiento de selección de estudios	31
3.2.3 Evaluación de la calidad	33
3.2.4 Extracción de Datos	34
3.2.5 Síntesis de datos.....	35
3.2.6 Sesgos	39
4 Resultados	39
4.1 Estudios relevantes.....	39
4.2 Contextos y escenarios en los que se utiliza Jenkins para crear pipelines de integración y despliegue continuo	55
4.3 Herramientas complementarias a Jenkins para crear pipelines de integración y despliegue continuo	63
4.4 Prácticas de configuración y administración de Jenkins recomendadas por los expertos	85
4.5 Actividades de integración, prueba y despliegue integradas y automatizadas utilizando Jenkins	97

4.6	Aspectos de seguridad y principales estrategias para mitigar vulnerabilidades y asegurar los pipelines de CI/CD.....	103
5	Discusión	118
	Contextos y escenarios en los que se suele utilizar Jenkins para la creación de pipelines de Integración y Despliegue Continuo.....	119
	Herramientas que se suelen integrar con Jenkins para la creación de pipelines de Integración y Despliegue Continuo.....	125
	Prácticas de configuración y administración de Jenkins recomendadas por los expertos .	137
	Actividades de integración, prueba y despliegue integradas y automatizadas utilizando Jenkins.....	140
	Aspectos de seguridad y principales estrategias para mitigar vulnerabilidades y asegurar los pipelines de CI/CD.....	148
	Limitaciones	152
	Amenazas a la validez.....	153
	Conclusiones	155
	Referencias.....	157
6	Referencias de los estudios primarios (literatura blanca)	159
	Apéndice A) Protocolo de investigación del trabajo recepcional	166
	Apéndice B) Protocolos de la Revisión multivocal de la literatura	166
	Apéndice C) Rúbrica de Evaluación de la Calidad de la Literatura Gris.....	166
	Apéndice D) Formatos de extracción de datos.	167
	Apéndice E) Artefactos generados durante el desarrollo de la RSL.....	169

Resumen

Contexto: la integración y despliegue continuo (CI/CD) son prácticas de DevOps que permiten a las aplicaciones integrar y desplegar cambios, de manera constante, sin comprometer la calidad, mediante procesos automatizados. Para aplicar estas prácticas correctamente, es necesario el uso de herramientas de automatización. Jenkins es un servidor de automatización de código abierto y es la herramienta más utilizada en la actualidad para la automatización de actividades de integración, pruebas y despliegue de software.

Objetivo: el presente estudio pretende concebir un panorama actual del uso de la herramienta Jenkins en actividades de integración, prueba y despliegue de software, con el objetivo de identificar los contextos en los que se utiliza, los tipos de sistemas de software que se integran y despliegan utilizando Jenkins, las principales herramientas complementarias a Jenkins reportadas en la literatura, así como las prácticas recomendadas de configuración, administración y seguridad. Todo esto a través de cinco preguntas de investigación.

Método: se elaboró una revisión multivocal de la literatura (RML) en la que se revisaron diversos recursos de investigación, documentación, reportajes, videos, entre otros, publicados entre enero de 2016 y diciembre del 2024, identificando aquellos recursos que reportaran el uso de la herramienta Jenkins para la creación de pipelines de integración y despliegue continuo (CI/CD).

Resultados: de los 2536 estudios que se revisaron, y tras aplicar los criterios de inclusión y exclusión establecidos, se logró identificar evidencia del uso de Jenkins en la creación de pipelines integración y despliegue continuo en 163 estudios.

Conclusiones: Jenkins es una herramienta ampliamente utilizada en la industria, así como en los sectores académicos y de investigación. Además, brinda a los desarrolladores una gran flexibilidad, permitiendo integrar una gran diversidad de herramientas para automatizar una gran parte de las actividades y tareas de integración, prueba y despliegue de software. Tras el análisis de los 163 estudios, se logró contestar todas las preguntas de investigación, sin

embargo, la pregunta de investigación cinco reveló un hueco de investigación dada la poca cantidad de estudios; a pesar de haber estudios que daban respuesta a esta pregunta, estas no fueron amplias y los estudios no fueron abundantes.

Palabras clave: Jenkins, Continuous Integration, Continuous Deployment, Continuous Delivery, CI/CD Pipeline, Multivocal Literature Review, Narrative Synthesis, Quasi-Gold.

I Introducción

El despliegue de software es una de las actividades finales dentro del ciclo de vida de software, en la que se busca liberar actualizaciones o aplicaciones completas que han sido previamente aprobadas para su despliegue en producción. Con el acelerado ritmo de la industria de software actual, las empresas recurren a prácticas ágiles como DevOps para liberar software de calidad de manera constante y en ciclos cortos. Una de las prácticas de DevOps más utilizadas para lograr este objetivo es la integración y despliegue continuo (CI/CD), la cual busca lograr cierto grado de automatización en las actividades de integración, prueba y despliegue de software, empleando herramientas de automatización como Jenkins, un servidor de código abierto para la creación de *pipelines* de CI/CD.

A pesar de que Jenkins es la herramienta de CI/CD más utilizada en la industria dada su versatilidad y flexibilidad, actualmente no hay estudios que analicen su uso en la automatización de actividades de integración, pruebas y despliegue de software, así como los contextos y herramientas complementarias con los que comúnmente se integra.

Previamente se han realizado estudios secundarios sobre prácticas ágiles y DevOps, donde se abordan los beneficios y retos que conllevan su implementación, así como su integración con el ciclo de vida de software, sin embargo, estos estudios no se enfocan en el uso de una herramienta específica como Jenkins.

En esta investigación, se analiza el uso de Jenkins en la creación de *pipelines* de CI/CD para la automatización de actividades de integración, prueba y despliegue de software. Además, se exploran los contextos, escenarios, herramientas complementarias, prácticas recomendadas y consideraciones de seguridad que respaldan la implementación de estos pipelines.

Con el objetivo de definir la información que se esperan recopilar, se establecieron cinco preguntas de investigación que guían el desarrollo de este estudio:

RQ01. *¿En qué contextos y escenarios se suele utilizar Jenkins para crear pipelines de integración y despliegue continuo (CI/CD)?*

La primera pregunta tiene el objetivo de identificar en dónde y de qué forma se está utilizando Jenkins, para determinar el alcance de su aplicación y los casos más favorables para su implementación.

RQ02. *¿Cuáles son las herramientas complementarias que suelen utilizarse junto con Jenkins para crear los pipelines de integración y despliegue continuo (CI/CD)?*

La segunda pregunta tiene el objetivo de identificar las herramientas más utilizadas con Jenkins para la creación de pipelines de CI/CD, con el fin de identificar las herramientas más utilizadas, cuál es su rol dentro de los *pipelines* y cuáles potencian mejor las capacidades de Jenkins al integrarse con él.

RQ03. *¿Qué prácticas recomiendan los expertos y la comunidad para la configuración y administración de Jenkins?*

La tercera pregunta busca identificar las prácticas para la configuración y administración de Jenkins que se mencionan en la literatura para determinar la mejor forma de utilizar Jenkins, aprovechando las capacidades y características que ofrece esta herramienta al máximo.

RQ04. *¿Qué actividades de integración, prueba y despliegue son comúnmente integradas y automatizadas utilizando Jenkins en distintos proyectos de software?*

La pregunta RQ04 tiene el objetivo de identificar y clasificar las actividades que se suelen automatizar utilizando Jenkins, con el fin de identificar actividades claves que sean adecuadas para su automatización, e implementar *pipelines* efectivos.

RQ05. *¿Qué aspectos de seguridad deben considerarse al utilizar Jenkins, y cuáles son las principales estrategias para mitigar vulnerabilidades y asegurar los pipelines de Integración y Despliegue Continuo (CI/CD)?*

Finalmente, la última pregunta pretende identificar aquellos aspectos clave de seguridad que se deben tomar en cuenta al utilizar Jenkins, con el fin de crear *pipelines* de CI/CD que cuenten con las medidas necesarias para prevenir brechas de seguridad.

2 Antecedentes

Con el acelerado entorno de desarrollo de software actual, se torna crítico integrar métodos efectivos y confiables que permitan administrar, construir, probar y entregar código de manera continua. El uso de la integración, la entrega y el despliegue continuos (CI, CDE y CD, respectivamente) son las principales prácticas que permiten tales objetivos (Dileepkumar & Mathew, 2023).

Estos conceptos se engloban dentro la idea de DevOps, que consiste en un conjunto de prácticas y principios que permiten una mejor comunicación y colaboración entre los interesados, con el fin de especificar, desarrollar y operar productos de software (ISO/IEEE, 2021). DevOps intenta cerrar la brecha entre el desarrollo y las operaciones, además de cubrir aspectos que ayudan a una entrega de software rápida, optimizada y de alta calidad (Virmani, 2015). Implica un cambio organizacional entre desarrollo, QA y operaciones para pasar de equipos separados a equipos multifuncionales (Ebert et al., 2016).

En el contexto de DevOps, el uso de herramientas de automatización es una necesidad latente. Para realizar entregas de calidad en ciclos de tiempo corto, se requiere un alto grado de automatización (Ebert et al., 2016). Así, herramientas enfocadas en la automatización de procesos y actividades de integración, entrega y despliegue continuos tales como Hudson, Cruise Controll, Apache's Continuum, Team City, GitLab o Jenkins, se vuelven cada vez más relevantes.

Entre la basta variedad de herramientas de CI/CD que existen en la actualidad, Jenkins es por mucho la más popular en el mercado (Leszko, 2022). Jenkins es un servidor de automatización de código abierto que facilita la creación y mantenimiento de pipelines de CI/CD para la automatización de procesos de construcción, pruebas y despliegue de software, que provee una solución confiable y adaptable para los equipos de desarrollo contemporáneos, así como un ecosistema robusto, extensible y escalable a través de la implementación de *plugins* que expanden sus capacidades y permiten la interacción con otras plataformas y herramientas de desarrollo (Dileepkumar & Mathew, 2023).

Tal es la importancia de Jenkins en el mercado actual que, según datos del reporte de JetBrains (2023), donde se recopiló información de 26,348 desarrolladores a nivel mundial sobre herramientas de DevOps, reveló que Jenkins ocupa al menos el 50% de la cuota de mercado en el ámbito empresarial, consolidándose como la tecnología más utilizada para integración continua. Además, en un reporte similar de Enlyft (s/f), donde se analizó el uso de al menos 82 productos que caen en la categoría de entrega continua en más de 265,000 compañías, informó que la principal herramienta de entrega continúa utilizada en esas compañías es Jenkins, que alcanzó un 33% de la cuota del mercado, con más de 90,050 compañías que reportaron su uso, ubicadas principalmente en Estados Unidos, India y Reino Unido.

Actualmente existen varios trabajos secundarios sobre las prácticas ágiles y DevOps, como el desarrollado por Amaro et al. (2025) en el que analizan el impacto de las capacidades de DevOps y como estas pueden mejorar la entrega, calidad y confiabilidad del software. Otros estudios se enfocan en diferentes aspectos de las prácticas continuas, tales como el desarrollado por Theunissen et al. (2022), en el que analiza el panorama general de la actividad de documentación en el desarrollo de software continuo.

Aun con lo anterior, no se pudo identificar algún estudio que abordara el uso de una herramienta específica como Jenkins, y hasta la fecha de realización de esta investigación, todavía no hay un trabajo que analice el uso de Jenkins en proyectos de software. Es por esta razón que en esta RML nos centramos en explorar lo reportado en los estudios en los que utilizan Jenkins para la creación de pipelines de CI/CD para la automatización de procesos de integración, prueba y despliegue de software, analizando el uso de Jenkins en estas actividades.

Antes de describir el proceso que se siguió para realizar la RML, es necesario explicar algunos conceptos claves para entender el uso de Jenkins y el rol que desempeña en la adopción de las prácticas de CI/CD.

2.1 Integración y despliegue continuo

La integración y despliegue continuo (CI/CD) son una serie de actividades de DevOps que simplifican los procesos de integración de una aplicación y permiten integrar y desplegar cambios de forma constante (Kusumadewi & Adrian, 2023). Estos dos conceptos, junto con la entrega continua (CDE), forman parte de las llamadas prácticas continuas que tienen como objetivo ayudar a las organizaciones a acelerar el desarrollo y entrega de software sin comprometer la calidad (Shahin et al., 2017).

La integración continua (CI) es una práctica de desarrollo que fomenta el trabajo en equipo y la asistencia para identificar problemas de integración según surgen. Consiste en contar con un repositorio compartido que recibe actualizaciones continuas por parte de los desarrolladores (Dileepkumar & Mathew, 2023). Aunque esta idea se extiende según Bildiri & Akdemir (2021), que mencionan dicha práctica como un sistema de *pipeline* que permite la integración de nuevas versiones con versiones previas, pasando por determinadas etapas y pruebas. En términos generales, la integración continua tiene dos objetivos principales: reunir el código de los desarrolladores en un solo repositorio compartido y construir un software funcional a partir de este código (Jokinen, 2020), lo que resume las dos ideas previamente expuestas.

La entrega continua (CDE) se centra en asegurar que una aplicación siempre esté en condiciones de ser liberada en un entorno de producción después de haber pasado por pruebas automáticas y verificaciones de calidad. Esta práctica tiene un enfoque basado en demanda, en el cual el negocio decide qué y cuando desplegar, y en el que el último despliegue (normalmente a producción), es un paso totalmente manual (Shahin et al., 2017).

Por otra parte, el despliegue continuo (CD) contempla un paso más respecto a la entrega continua (CDE), pues de forma automática y continua se despliega el software en el ambiente de producción o del cliente. Tan pronto como un desarrollador realiza un *commit*, este pasa por un *pipeline* de despliegue para ser liberado a producción (Shahin et al., 2017).

El éxito de la adopción de estas prácticas continuas en la industria contemporánea se debe en gran parte a la implementación de *pipelines* de despliegue (Shahin et al., 2017). El concepto de *pipeline* se establece en el estándar 2675 de la ISO/IEEE (2021), como una técnica de diseño de software o hardware en la cual la salida de un proceso sirve como la entrada a un segundo proceso, la salida del segundo proceso sirve como la entrada del tercero y así sucesivamente.

En el contexto de CI/CD, un *pipeline* de despliegue se refiere a una secuencia de procesos desencadenados por cambios en el código fuente de un sistema de control de versiones, y que pueden ser ejecutados en múltiples sistemas o herramientas de automatización de despliegue, produciendo cambios de estado en el software y artefactos si es que los hay (Révész & Pataki, 2021).

2.2 Jenkins

Para poder aplicar de manera adecuada las prácticas de integración, entrega y despliegue continuos, es necesario utilizar herramientas de automatización, pues estas herramientas facilitan la implementación de los procesos de CI/CD.

Las herramientas de CI/CD se inicializan cuando detectan cambios en el sistema de control de versiones. A partir de ahí, se comienzan a ejecutar las tareas definidas en el *pipeline*, se realizan las compilaciones, se corren pruebas, se ejecutan análisis estáticos de código, se generan artefactos que se versionan y publican, y se despliega el producto en cada ambiente necesario, ya sea integración, pruebas o producción (Révész & Pataki, 2021).

Jenkins es un servidor de automatización de código abierto, escrito en Java y lanzado inicialmente en el 2011 como un proyecto derivado del proyecto Hudson. Es la herramienta más utilizada para implementar procesos de integración y entrega continua y cuenta con una comunidad muy activa y colaborativa, que durante años han desarrollado una cantidad inmensa de *plugins*, que permiten extender y adaptar las funcionalidades de Jenkins (Leszko, 2022).

Una de las características destacables de Jenkins en comparación con otros servidores de automatización, es su facilidad de integración con otras herramientas, gracias a los *plugins* (Gowda et al., 2024). Según la página oficial de Jenkins (s/f-a), un *plugin* es una extensión de la funcionalidad de Jenkins que se proporciona por separado de su núcleo. El núcleo de Jenkins hace referencia a la aplicación primaria de Jenkins, que provee la interfaz web básica, la configuración y la base sobre la que se pueden crear los *plugins*.

En el contexto de Jenkins, el concepto de *pipeline* cambia. Un *pipeline* de Jenkins es un conjunto de *plugins* que apoya la implementación e integración de *pipelines* de despliegue dentro de Jenkins. Los *pipelines* de Jenkins proveen un conjunto extensible de herramientas para modelar *pipelines* de despliegue como código, a través de archivos de texto llamados *Jenkinsfiles* (Jenkins, s/f-b).

3 Conducción del método

Para realizar la RML se siguió la metodología propuesta por Garousi et al. (2019), la cual extiende las directrices propuestas por B. A. Kitchenham et al. (2015) e incorpora recomendaciones específicas para incluir literatura gris.

Inicialmente se tomaron como referencia las tres fases primordiales que plantea B. A. Kitchenham et al. (2015), y para cada fase se realizó una serie de actividades que condujeron la investigación en la literatura blanca. Así, cubriendo únicamente la literatura blanca, se realizó una revisión sistemática de la literatura (RSL), que posteriormente se actualizó para incluir la literatura gris y extenderla a una RML.

Por lo anterior, varios artefactos y recursos desarrollados durante la RSL se adaptaron y reutilizaron en el desarrollo de la RML. A continuación, se enlistan las fases y actividades que se desarrollaron.

Planeación: durante esta primera fase se determina la necesidad de la revisión, factibilidad, objetivo, preguntas de investigación, criterios de selección, así como otros aspectos que guían la conducción de la revisión. Todas estas decisiones se condensan en un protocolo de investigación.

Las tareas que se realizaron en esta fase son:

- Especificar las preguntas de Investigación.
- Desarrollar el protocolo de las RSL y posteriormente de la RML.
- Validar ambos protocolos.

Conducción: la segunda fase de la metodología consiste en la ejecución del protocolo desarrollado en la fase previa y comprende, desde la identificación de los estudios relevantes para la investigación, hasta la síntesis de los datos extraídos. Las tareas que se realizaron en esta fase son:

- Identificar estudios relevantes.
- Seleccionar estudios primarios.

- Evaluación de la calidad (únicamente para la literatura gris).
- Extraer los datos requeridos.
- Sintetizar los datos.

Documentación: la última fase consiste en reportar los procesos y técnicas utilizadas durante la conducción de la revisión, así como los hallazgos encontrados al finalizar esta misma. La tarea que se realizó durante esta fase fue documentar los resultados.

Al término del proceso de la RML, se generaron algunos artefactos que evidencian y respaldan el proceso seguido y los resultados obtenidos, dichos artefactos son:

- Protocolo de la revisión sistemática de la literatura.
- Protocolo de la revisión multivocal de la literatura.
- Formatos de extracción de datos de la literatura blanca.
- Formatos de extracción de datos de la literatura gris.
- Formatos y hojas de cálculo con los resultados, datos y métricas obtenidos de la literatura blanca.
- Formatos y hojas de cálculo con los resultados, datos y métricas obtenidas de la literatura gris.
- Hojas de cálculo con la síntesis narrativa de la literatura blanca.
- Hojas de cálculo con la síntesis narrativa de la literatura gris.
- Reporte de la revisión sistemática de la literatura.
- Reporte de la revisión multivocal de la literatura.

Antes de describir los aspectos relevantes de la planeación, es necesario mencionar que esta RML forma parte de un trabajo de investigación que tiene el objetivo de desarrollar una guía práctica de Jenkins para el despliegue automatizado de software. Por esto anterior, previo a la planeación de la RML se realizó un protocolo de investigación en el que se definen el problema, objetivo, alcances y limitaciones del trabajo. El protocolo de investigación se puede consultar en el Apéndice A) Protocolo de investigación.

3.1 Planeación de la revisión multivocal de la literatura

Como primer paso, se desarrolló el protocolo específico de la RML, en el que se definieron las preguntas de investigación, la estrategia de búsqueda, el proceso de selección de estudios primarios, la rúbrica de evaluación de la calidad, los formatos de extracción de datos, la estrategia de síntesis de datos y las limitaciones de la investigación. El protocolo de la RML se puede consultar en el Apéndice B) Protocolo de la RML.

A continuación, se muestran las preguntas de investigación que guiaron el desarrollo de la RML. Es importante mencionar que estas preguntas se plantearon inicialmente para la revisión de la literatura blanca, y posteriormente se determinó que se usarían las mismas preguntas para la literatura gris.

3.1.1 Preguntas de investigación

Para determinar la información que se recopilaría, se plantearon un total de cinco preguntas de investigación para delimitar la información de valor (en esta investigación) relacionada con el uso de Jenkins en actividades de integración, prueba y despliegue en diversos proyectos de software. Las preguntas desarrolladas y su correspondiente motivación se muestran en la Tabla 1.

Tabla 1. Preguntas de investigación		
Identificador	Pregunta	Motivación
RQ01	¿En qué contextos y escenarios se suele utilizar Jenkins para crear pipelines de Integración y despliegue continuo (CI/CD)?	El objetivo es identificar cómo se está utilizado Jenkins para determinar el alcance de su aplicación y los casos más favorables para su implementación.
RQ02	¿Cuáles son las herramientas complementarias que suelen utilizarse junto con Jenkins para crear los pipelines de Integración y despliegue Continuo (CI/CD)?	El objetivo es identificar las herramientas más utilizadas con Jenkins para la creación de pipelines de CI/CD, con el fin de identificar aquellas altamente usadas en la industria, y cuáles potencian mejor las capacidades de Jenkins.
RQ03	¿Qué prácticas recomiendan los expertos y la comunidad para la configuración y administración de Jenkins?	El objetivo es identificar las prácticas para la configuración y administración de Jenkins que se mencionan en la literatura para determinar la mejor forma de utilizar Jenkins y aprovechar al máximo las capacidades que provee esta herramienta.
RQ04	¿Qué actividades de integración, prueba y despliegue son comúnmente integradas y automatizadas utilizando Jenkins en distintos proyectos de software?	El objetivo es identificar y clasificar las actividades que se suelen automatizar utilizando Jenkins, con el fin de identificar actividades claves adecuadas para la automatización e implementar pipelines efectivos.
RQ05	¿Qué aspectos de seguridad deben considerarse al utilizar Jenkins, y cuáles son las principales estrategias para mitigar vulnerabilidades y asegurar los pipelines de Integración y Despliegue Continuo (CI/CD)?	El objetivo es identificar aquellos aspectos clave de seguridad que se deben tomar en cuenta al utilizar Jenkins, con el fin de crear pipelines de CI/CD que cuenten con las medidas necesarias para prevenir brechas de seguridad.

3.2 Conducción de la revisión multivocal de la literatura

La conducción de la revisión comenzó con la búsqueda de estudios siguiendo las estrategias de búsqueda definidas en el protocolo de la RML. Posteriormente se realizó la selección de estudios con base en los criterios de inclusión y exclusión. Finalmente se realizó la extracción de datos y la síntesis de la información, siguiendo las técnicas y estrategias descritas en el protocolo de la RML. Cabe mencionar que para la literatura gris se consideró la etapa de evaluación de calidad, en la que se utilizó una adaptación de la rúbrica propuesta por Garousi et al. (2019).

3.2.1 Identificación de estudios relevantes

Identificar toda la literatura relevante posible para las preguntas de investigación es un paso crucial en una RML. Hablando particularmente de la búsqueda en la literatura blanca, se decidió seguir el proceso propuesto por Zhang et al. (2011). Este proceso consta de dos etapas: una búsqueda manual y una automatizada; la búsqueda manual sirve para establecer un estándar Quasi-Gold (QGS), que permite evaluar la búsqueda automatizada, mientras que esta última se realiza con el fin de complementar la búsqueda manual.

A su vez cada etapa se conforma de ciertas actividades que se pueden repetir hasta finalizar la búsqueda de estudios. A continuación, se describen las actividades que se realizaron para la identificación de estudio relevantes.

3.2.1.1 Identificar venues y motores

Venue es un término general para referir al lugar (*journals* o conferencias) donde los estudios relevantes son publicados (Zhang et al., 2011). Para cubrir la literatura blanca se realizó una búsqueda manual en *venues* populares según (Zhang et al., 2011), como IEEE Software, ESEM, ISESE, TSE e ICSE. Sin embargo, los estudios relevantes encontrados fueron escasos, por lo que se buscaron *venues* alternativos que dieran indicios de contener estudios relacionados, como ICSSAS, ICORIS, CSITSS, entre otros. Esto anterior aplicado a los motores de búsqueda IEEE Xplore y ACM Digital Library.

A pesar de que se revisaron los *venues* de cuatro bases de datos con el fin de identificar aquellos que dieran indicios de estar relacionados con las prácticas de DevOps, despliegue y automatización, no se pudieron encontrar una cantidad considerable de estudios relacionados al uso de Jenkins para la integración y despliegue continuo, por lo que se tomó la decisión se complementar la búsqueda manual con los resultados directos de las bases de datos.

3.2.1.2 Establecer el estándar *Quasi-Gold*

Para la búsqueda manual, se localizaron estudios que dieran indicios de utilizar la herramienta Jenkins para la creación de *pipelines* de integración y despliegue continuo. Esta búsqueda se realizó considerando aquellos estudios publicados entre enero de 2016 y diciembre de 2024. Se buscó en *journals*, conferencias, *workshops* y revistas, y para cada estudio se aplicaron los criterios de inclusión y exclusión definidos en el protocolo de la RML. Se leyó el título y resumen para identificar indicios de su utilidad en la respuesta de las preguntas de investigación y finalmente se leyeron completamente estos estudios para verificar que se diera respuesta a al menos una pregunta de investigación en ellos.

Una vez finalizado este proceso, se lograron obtener 10 estudios relevantes por cada uno los motores de búsqueda IEEE Xplore, SpringerLink y ScienceDirect, y 14 para el motor de búsqueda ACM Digital Library.

Con base en estos estudios, se establecieron cuatro estándares *Quasi-old*, uno para cada motor de búsqueda. Esto con el fin de disminuir los sesgos y desviaciones en las cadenas de búsqueda utilizadas para la búsqueda automatizada de cada base de datos. En las tablas 2, 3, 4 y 5, se observan los estudios relevantes que conforman los estándares *Quasi-Gold* para cada motor de búsqueda.

Tabla 2. Quasi-Gold IEEE Xplore	
ID	Estudio relevante
EP01	Vassallo, C. (2019). Enabling continuous improvement of a continuous integration process. Proceedings - 2019 34th IEEE/ACM International Conference on Automated Software Engineering, ASE 2019, 1246–1249. https://doi.org/10.1109/ASE.2019.00151
EP02	Balalaie, A., Heydarnoori, A., & Jamshidi, P. (2016). Microservices Architecture Enables DevOps: Migration to a Cloud-Native Architecture. IEEE Software, 33(3), 42–52. https://doi.org/10.1109/MS.2016.64
EP03	Dileepkumar, S. R., & Mathew, J. (2023). Transforming Software Development: Achieving Rapid Delivery, Quality, and Efficiency with Jenkins-Based CI/CD Pipelines. 2023 Annual International Conference on Emerging Research Areas: International Conference on Intelligent Systems, AICERA/ICIS 2023. https://doi.org/10.1109/AICERA/ICIS59538.2023.10420251
EP04	Naveen, B., Grandhi, J. K., Lasya, K., Reddy, E. M., Srinivasu, N., & Bulla, S. (2023). Efficient Automation of Web Application Development and Deployment Using Jenkins: A Comprehensive CI/CD Pipeline for Enhanced Productivity and Quality. International Conference on Self Sustainable Artificial Intelligence Systems, ICSSAS 2023 - Proceedings. https://doi.org/10.1109/ICSSAS57918.2023.10331631

EP05	Singh, N., Singh, A., & Rawat, V. (2022). Deploying Jenkins, Ansible and Kubernetes to Automate Continuous Integration and Continuous Deployment Pipeline. 2022 IEEE International Conference on Service Operations and Logistics, and Informatics (SOLI), 1–5. https://doi.org/10.1109/SOLI57430.2022.10294378
EP06	Mysari, S., & Bejgam, V. (2020). Continuous Integration and Continuous Deployment Pipeline Automation Using Jenkins Ansible. 2020 International Conference on Emerging Trends in Information Technology and Engineering (Ic-ETITE), 1–4. https://doi.org/10.1109/ic-ETITE47903.2020.239
EP07	Cepuc, A., Botez, R., Craciun, O., Ivanciu, I. A., & Dobrota, V. (2020). Implementation of a continuous integration and deployment pipeline for containerized applications in amazon web services using jenkins, ansible and kubernetes. Proceedings - RoEduNet IEEE International Conference, 2020-December. https://doi.org/10.1109/RoEduNet51892.2020.9324857
EP08	Vernekar, S. R., Kumar, L., & Kalnoor, G. (2024). Hotel Reservation App Integrated with Docker and Jenkins. 2024 International Conference on Emerging Technologies in Computer Science for Interdisciplinary Applications (ICETCS), 1–6. https://doi.org/10.1109/ICETCS61022.2024.10544088
EP09	Permana, P. A. G., Triandini, E., & Suwirmayanti, N. L. G. P. (2021). Implementation Jenkins Automation Deployment with Scheduler and Notification. 2021 3rd International Conference on Cybernetics and Intelligent System (ICORIS), 1–5. https://doi.org/10.1109/ICORIS52787.2021.9649555
EP10	Rakshitha, A. H., Jayanthi, P. N., & Manyam, S. (2023). Software Configuration Using Jenkins and Yocto Project. 2023 7th International Conference on Computation System and Information Technology for Sustainable Solutions (CSITSS), 1–4. https://doi.org/10.1109/CSITSS60515.2023.10334213

Tabla 3. Quasi-Gold ACM Digital Library

ID	Estudio relevante
EP43	Steffens, A., Lichter, H., & Döring, J. S. (2018). Designing a next-generation continuous software delivery system: Concepts and architecture. Proceedings - International Conference on Software Engineering, 1–7. https://doi.org/10.1145/3194760.3194768
EP44	Düllmann, T. F., Paule, C., & Hoorn, A. van. (2018). Exploiting DevOps Practices for Dependable and Secure Continuous Delivery Pipelines. 2018 IEEE/ACM 4th International Workshop on Rapid Continuous Software Engineering (RCoSE), 27–30.
EP45	Straubinger, P., & Fraser, G. (2024). Gamifying a Software Testing Course with Continuous Integration. 2024 IEEE/ACM 46th International Conference on Software Engineering: Software Engineering Education and Training (ICSE-SEET), 34–45. https://doi.org/10.1145/3639474.3640054
EP46	Philipp Straubinger and Gordon Fraser. 2024. An IDE Plugin for Gamified Continuous Integration. In 2024 First IDEWorkshop (IDE '24), April 20, 2024, Lisbon, Portugal. ACM, New York, NY, USA, 4 pages. https://doi.org/10.1145/3643796.3648462
EP47	Straubinger, P., & Fraser, G. (2022). Gamekins: Gamifying Software Testing in Jenkins. 2022 IEEE/ACM 44th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion), 85–89. https://doi.org/10.1145/3510454.3516862
EP48	Zampetti, F., Nardone, V., & di Penta, M. (2022). Problems and Solutions in Applying Continuous Integration and Delivery to 20 Open-Source Cyber-Physical Systems. 2022 IEEE/ACM 19th International Conference on Mining Software Repositories (MSR), 646–657. https://doi.org/10.1145/3524842.3527948
EP49	Light, J., Pfeiffer, P., & Bennett, B. (2021). An evaluation of continuous integration and delivery frameworks for classroom use. Proceedings of the 2021 ACM Southeast Conference, 204–208. https://doi.org/10.1145/3409334.3452085
EP50	Heckman, S., & King, J. (2018). Developing Software Engineering Skills using Real Tools for Automated Grading. Proceedings of the 49th ACM Technical Symposium on Computer Science Education, 794–799. https://doi.org/10.1145/3159450.3159595

EP51	Pecka, N., ben Othmane, L., & Valani, A. (2022). Privilege Escalation Attack Scenarios on the DevOps Pipeline Within a Kubernetes Environment. <i>Proceedings of the International Conference on Software and System Processes and International Conference on Global Software Engineering</i> , 45–49. https://doi.org/10.1145/3529320.3529325
EP52	Antunes, R. V., Navarro, G. M., & Hanazumi, S. (2018). Test Framework for Jenkins Shared Libraries. <i>Proceedings of the III Brazilian Symposium on Systematic and Automated Software Testing</i> , 13–19. https://doi.org/10.1145/3266003.3266008
EP53	Schloyer, P., Knauer, P., Bauer, B., & Merli, D. (2024). Automating Side-Channel Testing for Embedded Systems: A Continuous Integration Approach. <i>Proceedings of the 19th International Conference on Availability, Reliability and Security</i> . https://doi.org/10.1145/3664476.3670436
EP54	Sarmiento-Calisaya, E., Mamani-Aliaga, A., & do Prado Leite, J. C. S. (2024). Introducing Computer Science Undergraduate Students to DevOps Technologies from Software Engineering Fundamentals. <i>2024 IEEE/ACM 46th International Conference on Software Engineering: Software Engineering Education and Training (ICSE-SEET)</i> , 348–358. https://doi.org/10.1145/3639474.3640071
EP55	Eddy, B. P., Wilde, N., Cooper, N. A., Mishra, B., Gamboa, V. S., Patel, K. N., & Shah, K. M. (2017). CDEP: Continuous Delivery Educational Pipeline. <i>Proceedings of the 2017 ACM Southeast Conference</i> , 55–62. https://doi.org/10.1145/3077286.3077301
EP56	Amalfitano, D., de Simone, V., Fasolino, A. R., & Scala, S. (2017). Improving traceability management through tool integration: an experience in the automotive domain. <i>Proceedings of the 2017 International Conference on Software and System Process</i> , 5–14. https://doi.org/10.1145/3084100.3084101

Tabla 4. Quasi-Gold SpringerLink

ID	Estudio relevante
EP67	López-Huguet, S., García-Castro, F., Alberich-Bayarri, A., & Blanquer, I. (2019). A Cloud Architecture for the Execution of Medical Imaging Biomarkers. In J. M. F. Rodrigues, P. J. S. Cardoso, J. Monteiro, R. Lam, V. v Krzhizhanovskaya, M. H. Lees, J. J. Dongarra, & P. M. A. Sloot (Eds.), <i>Computational Science – ICCS 2019</i> (pp. 130–144). Springer International Publishing.
EP68	Berger, C. (2016). An Open Continuous Deployment Infrastructure for a Self-driving Vehicle Ecosystem. In K. Crowston, I. Hammouda, B. Lundell, G. Robles, J. Gamalielsson, & J. Lindman (Eds.), <i>Open Source Systems: Integrating Communities</i> (pp. 177–183). Springer International Publishing.
EP69	Biswas, N., Mondal, A. S., Kusumastuti, A., Saha, S., & Mondal, K. C. (2022). Automated credit assessment framework using ETL process and machine learning. <i>Innovations in Systems and Software Engineering</i> . https://doi.org/10.1007/s11334-022-00522-x
EP70	Wei, H., Rodriguez, J. S., & Garcia, O. N.-T. (2021). Deployment Management and Topology Discovery of Microservice Applications in the Multicloud Environment. <i>Journal of Grid Computing</i> , 19(1), 1. https://doi.org/10.1007/s10723-021-09539-1
EP71	Teixeira, J. A., & Karsten, H. (2019). Managing to release early, often and on time in the OpenStack software ecosystem. <i>Journal of Internet Services and Applications</i> , 10(1), 7. https://doi.org/10.1186/s13174-019-0105-z
EP72	Munir, H., Linåker, J., Wnuk, K., Runeson, P., & Regnell, B. (2018). Open innovation using open source tools: a case study at Sony Mobile. <i>Empirical Software Engineering</i> , 23(1), 186–223. https://doi.org/10.1007/s10664-017-9511-7
EP73	Altunel, H., & Say, B. (2021). Software Product System Model: A Customer-Value Oriented, Adaptable, DevOps-Based Product Model. <i>SN Computer Science</i> , 3(1), 38. https://doi.org/10.1007/s42979-021-00899-9
EP74	Kao, C. H. (2017). Testing and evaluation framework for virtualization technologies. <i>Computing</i> , 99(7), 657–677. https://doi.org/10.1007/s00607-016-0517-6
EP75	Song, H., Dautov, R., Ferry, N., Solberg, A., & Fleurey, F. (2022). Model-based fleet deployment in the IoT–edge–cloud continuum. <i>Software and Systems Modeling</i> , 21(5), 1931–1956. https://doi.org/10.1007/s10270-022-01006-z
P76	Sallin, M., Kropp, M., Anslow, C., Quilty, J. W., & Meier, A. (2021). Measuring Software Delivery Performance Using the Four Key Metrics of DevOps. In P. Gregory, C. Lassenius, X. Wang, & P. Kruchten (Eds.), <i>Agile Processes in Software Engineering and Extreme Programming</i> (pp. 103–119). Springer International Publishing.

Tabla 5-. Quasi-Gold ScienceDirect

ID	Estudio relevante
EP80	Bernardo, S., Orviz, P., David, M., Gomes, J., Arce, D., Naranjo, D., Blanquer, I., Campos, I., Moltó, G., & Pina, J. (2024). Software Quality Assurance as a Service: Encompassing the quality assessment of software and services. <i>Future Generation Computer Systems</i> , 156, 254–268. https://doi.org/10.1016/j.FUTURE.2024.03.024

EP81	Berger, C. (2016). An Open Continuous Deployment Infrastructure for a Self-driving Vehicle Ecosystem. In K. Crowston, I. Hammouda, B. Lundell, G. Robles, J. Gamalielsson, & J. Lindman (Eds.), <i>Open Source Systems: Integrating Communities</i> (pp. 177–183). Springer International Publishing.
EP82	Yang, D., Wang, D., Yang, D., Dong, Q., Wang, Y., Zhou, H., & Daocheng, H. (2020). DevOps in Practice for Education Management Information System at ECNU. <i>Procedia Computer Science</i> , 176, 1382–1391. https://doi.org/10.1016/j.PROCS.2020.09.148
EP83	Hermosilla, A., Gallego-Madrid, J., Martinez-Julia, P., Ortiz, J., Kafle, V. P., & Skarmeta, A. (2024). Advancing 5G Network Applications Lifecycle Security: An ML-Driven Approach. <i>CMES - Computer Modeling in Engineering and Sciences</i> , 141(2), 1447–1471. https://doi.org/10.32604/CMES.2024.053379
EP84	Chadwick, D. W., Fan, W., Costantino, G., de Lemos, R., di Cerbo, F., Herwono, I., Manea, M., Mori, P., Sajjad, A., & Wang, X. S. (2020). A cloud-edge based data security architecture for sharing and analysing cyber threat information. <i>Future Generation Computer Systems</i> , 102, 710–722. https://doi.org/10.1016/j.FUTURE.2019.06.026
EP85	Sadek, J., Craig, D., & Trenell, M. (2022). Design and Implementation of Medical Searching System Based on Microservices and Serverless Architectures. <i>Procedia Computer Science</i> , 196, 615–622. https://doi.org/10.1016/j.PROCS.2021.12.056
EP86	Krzemien, W., Gajos, A., Kacprzak, K., Rakoczy, K., & Korcyl, G. (2020). J-PET Framework: Software platform for PET tomography data reconstruction and analysis. <i>SoftwareX</i> , 11, 100487. https://doi.org/10.1016/j.SOFTX.2020.100487
EP87	Kayser, H., Herzke, T., Maanen, P., Zimmermann, M., Grimm, G., & Hohmann, V. (2022). Open community platform for hearing aid algorithm research: open Master Hearing Aid (openMHA). <i>SoftwareX</i> , 17, 100953. https://doi.org/10.1016/j.SOFTX.2021.100953
EP88	Gallego-Madrid, J., Sanchez-Iborra, R., & Skarmeta, A. (2021). From Network Functions to NetApps: The 5GASP Methodology. <i>Computers, Materials and Continua</i> , 71(2), 4115–4134. https://doi.org/10.32604/CMC.2022.021754
EP89	Hähner, U. R., Alvarez, G., Maier, T. A., Solcà, R., Staar, P., Summers, M. S., & Schulthess, T. C. (2020). DCA++: A software framework to solve correlated electron problems with modern quantum cluster methods. <i>Computer Physics Communications</i> , 246, 106709. https://doi.org/10.1016/j.CPC.2019.01.006

Como se muestra en las tablas previas, para las bases de datos de IEEE Xplore, SpringerLink y ScienceDirect se encontraron un total de 10 estudios relevantes mediante la búsqueda manual, mientras que para ACM Digital Library se identificaron un total de 14 estudios. Con base en estos estudios, se identificaron los términos de búsqueda que se utilizaron para generar las cadenas de búsqueda.

3.2.1.3 Identificar Términos de Búsqueda

Para identificar las palabras claves que sirvieron como términos de búsqueda, se trató de detectar aquellos conceptos más repetidos entre los estudios, así como aquellos de alto interés para la investigación. Esta detección de términos partió de analizar las palabras clave de los estudios, pero también se consideró el conocimiento de los investigadores con mayor trayectoria en el área, involucrados en la investigación, para definirlos, de acuerdo con (B. A. Kitchenham et al., 2015). Estos términos, así como sus sinónimos, se presentan en la Tabla 6, que se muestra a continuación.

Tabla 6. Términos de Búsqueda	
Palabra Clave	Sinónimos
Jenkins	• Jenkins-based
CI/CD	• CICD • Continuous integration and continuous deployment • Continuous integration and deployment • Continuous integration and delivery
Continuous integration	• CI • Continuous-integration
Continuous delivery	• CDE • CD • Continuous-delivery
Continuous deployment	• CD • Continuous-deployment
Pipeline	• Non-pipeline • Pipeline-as-code
DevOps	• Development-operations • Development operations • Development and operations

Como se puede observar en la tabla, se determinaron un total de siete términos de búsqueda, que sirvieron como base para desarrollar las cadenas de búsqueda, junto con los sinónimos asociados.

3.2.1.4 Generar Cadenas de Búsqueda

Las cadenas de búsqueda fueron generadas a partir de las palabras clave y sinónimos identificados en los estudios relevantes (tabla 6) y, con el fin de evaluar su calidad, se siguieron las dos métricas propuestas por (Zhang et al., 2011):

$$\text{Sensibilidad} = \frac{\text{Número de estudios relevantes recuperados (ERR)}}{\text{Número total de estudios relevantes (TER)}} 100\%$$

$$\text{Precision} = \frac{\text{Número de estudios relevantes recuperados (ERR)}}{\text{Número de estudios recuperados (TE)}} 100\%$$

A partir de estas métricas y usando como referencia la clasificación propuesta por Zhang et al. (2011), se evaluaron las cadenas de búsqueda, tratando de obtener valores de sensibilidad y precisión de al menos 80 y 10 por ciento, respectivamente. Esto anterior de

forma ideal, pero relativo para la precisión, pues valores mayores a 1 se consideraron como aceptables para esta métrica. Es importante mencionar que el proceso para desarrollar los *Quasi-Gold* solo se realizó para la literatura blanca, para la literatura gris no se hizo una búsqueda manual; en su lugar, se generó una cadena de búsqueda con base en los términos clave identificados y se guio una búsqueda automatizada únicamente.

La tabla 7 muestra las cadenas de búsqueda evaluadas en el motor IEEE Xplore, con los respectivos valores obtenidos para las dos métricas previamente mencionadas. Hay que recordar que el total de estudios relevantes identificados para IEEE Xplore, a través de la búsqueda manual, es 10 (TER = 10). Además, para el conteo del número total de estudios recuperados (TE), se aplicaron filtros sobre la fecha de publicación (2016 al 2024), el idioma de publicación (Inglés) y el tipo de *venue* del que proviene (*journal*, conferencia, *workshop* o revista).

Tabla 7. Cadenas de Búsqueda IEEE Xplore					
ID	Cadena	TE	ERR	Sensibilidad	Precisión
CB01	("Jenkins" OR "Jenkins-based") AND ("Pipeline" OR "Pipeline-as-code") AND ("CI/CD" OR "Continuous integration and continuous deployment" OR "Continuous integration and deployment" OR "Continuous integration and delivery")	21	5	50%	24%
CB02	("Jenkins" OR "Jenkins-based") AND ("Pipeline" OR "Pipeline-as-code") AND ("CI/CD" OR "Continuous integration and continuous deployment" OR "Continuous integration and deployment" OR "Continuous integration and delivery" OR "Continuous integration" OR "CI" OR "Continuous-integration")	30	5	50%	17%
CB03	("Jenkins" OR "Jenkins-based") AND ("Pipeline" OR "Pipeline-as-code" OR "CI/CD" OR "Continuous integration and continuous deployment" OR "Continuous integration and deployment" OR "Continuous integration and delivery")	56	7	70%	13%
CB04	("Jenkins" OR "Jenkins-based") AND ("CI/CD" OR "Continuous integration and	31	7	70%	23%

	continuous deployment" OR "Continuous integration and deployment" OR "Continuous integration and delivery")				
CB05	("Jenkins" OR "Jenkins-based") AND ("CI/CD" OR "Continuous integration and continuous deployment" OR "Continuous integration and deployment" OR "Continuous integration and delivery" OR "Continuous integration" OR "CI" OR "Continuous-integration")	75	8	80%	11%
CB06	("Jenkins" OR "Jenkins-based") AND ("CI/CD" OR "CICD" OR "Continuous integration and continuous deployment" OR "Continuous integration and deployment" OR "Continuous integration and delivery" OR "Continuous integration" OR "CI" OR "Pipeline" OR "Pipeline-as-code" OR "Continuous delivery" OR "CD" OR "CDE")	94	8	80%	9%
CB07	("Jenkins" OR "Jenkins-based") AND ("Continuous integration" OR "CI") AND (CI/CD" OR "CICD" OR "Continuous integration and continuous deployment" OR "Continuous integration and deployment" OR "Continuous integration and delivery" OR "Pipeline" OR "Pipeline-as-code")	40	7	70%	18%
CB08	("Jenkins" OR "Jenkins-based") AND ("CI/CD" OR "Continuous integration and continuous deployment" OR "Continuous integration" OR "CI" OR "DevOps")	81	8	80%	10%
CB09	"Jenkins" AND ("CI/CD" OR "Continuous integration" OR "CI" OR "Continuous delivery" OR "Continuous deployment" OR "CD")	68	8	80%	12%

Como se muestra en la tabla 7, se generaron y evaluaron nueve cadenas distintas para la base de datos de IEEE Xplore. En general, todas arrojaron una cantidad manejable de estudios y se distinguen las unas de las otras por la cantidad de estudios relevantes que obtienen. Con base en estos datos, se pudo concluir lo siguiente para seleccionar la mejor cadena de búsqueda:

- Las cadenas de búsqueda CB01, CB02, CB03, CB04 y CB07 no recuperan la cantidad suficiente de estudios relevantes del *Quasi-Gold*, por lo que se descartaron al no alcanzar el mínimo del 80% de sensibilidad.
- La cadena de búsqueda CB06 alcanza el 80% de sensibilidad deseado, pero tiene demasiados términos de búsqueda, por lo que recuperó demasiados estudios y esto afectó negativamente la precisión deseada, siendo esta menor al 10%.
- Los valores de sensibilidad y precisión de las cadenas CDB5, CB08 y CB09 son muy similares entre estas, 80% de sensibilidad y entre 10-12% de precisión. Es la cadena de

búsqueda CB09 la que recuperó el menor número de estudios, teniendo una mayor precisión (12%). Además, la cadena CB09 es una versión refinada de la cadena CB05, que incluye menos operadores (AND y OR).

A partir de estas inferencias es que se optó por la cadena de búsqueda CB09 para la base de datos de IEEE Xplore:

CB09. "Jenkins" AND ("CI/CD" OR "Continuous integration" OR "CI" OR "Continuous delivery" OR "Continuous deployment" OR "CD")

Una vez determinada la cadena de búsqueda para IEEE Xplore, se prosiguió con la generación de las cadenas de búsqueda en la base de datos de ACM Digital Library. En primera instancia, se probaron algunas de las cadenas generadas para la IEEE, para ver su rendimiento en ACM, sin embargo, no se obtuvieron los resultados deseados y se optó por generar las cadenas de búsqueda desde cero, con base en los términos de búsqueda.

La tabla 8 muestra la evaluación de las cadenas de búsqueda para el motor ACM Digital Library, con sus respectivos valores de sensibilidad y precisión. En el caso de ACM Digital Library, el total de estudios relevantes es 14 (TER = 14). Hay que tomar en cuenta que para el conteo del número total de estudios recuperados (TE), se aplicaron filtros sobre la fecha de publicación (2016 al 2024), el idioma de publicación (Inglés) y el tipo de *venue* del que proviene (*journal*, conferencia, *workshop* o revista).

Tabla 8. Cadenas de Búsqueda ACM Digital Library					
ID	Cadena	TE	ERR	Sensibilidad	Precisión
CB01	("Jenkins" OR "Jenkins-based") AND ("Pipeline" OR "Pipeline-as-code") AND ("CI/CD" OR "Continuous integration and continuous deployment" OR "Continuous integration and deployment" OR "Continuous integration and delivery")	131	7	50%	5%
CB02	("Jenkins" OR "Jenkins-based") AND ("Pipeline" OR "Pipeline-as-code") AND ("CI/CD" OR "Continuous integration and continuous deployment" OR "Continuous integration and deployment" OR	281	11	79%	4%

	"Continuous integration and delivery" OR "Continuous integration" OR "CI" OR "Continuous-integration")				
CB04	("Jenkins" OR "Jenkins-based") AND ("CI/CD" OR "Continuous integration and continuous deployment" OR "Continuous integration and deployment" OR "Continuous integration and delivery")	169	7	50%	4%
CB10	("Jenkins" OR "Jenkins-based") AND ("Pipeline" OR "Pipeline-as-code") AND ("Continuous integration" OR "CI" OR "Continuous-integration") AND ("CI/CD" OR CICD OR "Continuous integration and continuous deployment" OR "Continuous integration and deployment" OR "Continuous integration and delivery")	133	7	50%	5%
CB11	("Jenkins" OR "Jenkins-based") AND Abstract:("CI/CD" OR "Continuous integration and continuous deployment" OR "Continuous integration and deployment" OR "Continuous integration and delivery" OR "Continuous Integration" OR "CI" OR "Continuous-integration" OR "CD" OR "Pipeline")	122	14	100%	12%
CB12	(Title:"Jenkins" OR Abstract:"Jenkins" OR "Jenkins-based") AND ("Pipeline" OR "Pipeline-as-code" OR "CI/CD" OR "Continuous integration and continuous deployment" OR "Continuous integration and deployment" OR "Continuous integration and delivery" OR "Continuous integration" OR "CI" OR "Continuous- integration" OR "CD" OR "Continuous deployment" OR "Continuous-deployment" OR "CDE" OR "Continuous delivery" OR "Continuous-delivery" OR "DevOps" OR "Development-operations" OR "Development and operations")	21	9	64%	43%
CB13	("Jenkins" OR "Jenkins-based") AND (abstract:("CI/CD" OR "CD" OR "Continuous deployment" OR "Continuous-deployment" OR "CDE" OR "Continuous delivery" OR "Continuous- delivery"))	49	8	57%	16%

Como se puede observar en la tabla 8, se probaron tres de las cadenas de IEEE Xplore en el motor de búsqueda de ACM, sin embargo, los resultados no fueron óptimos considerando las medidas de sensibilidad y precisión. Por esto, se decidió desarrollar más cadenas de búsqueda, finalizando con un total de trece cadenas de búsqueda de las cuales cuatro son específicas de ACM Digital Library. Con base en los resultados obtenidos, se pudieron obtener las siguientes conclusiones:

- Las cadenas de búsqueda CB01, CB02, y CB04, probadas en IEEE Xplore y que se probaron en el motor de búsqueda de ACM, dieron resultados de sensibilidad y precisión

mucho menores a los deseados (80% y 10%, respectivamente), por lo que quedaron totalmente descartadas.

- Así mismo, la cadena de búsqueda CB10 también carece de la sensibilidad y precisión necesaria con apenas un 50% de sensibilidad y un 5% de precisión, por lo que quedó descartada.

- A pesar de que las cadenas de búsqueda CB12 Y CB13 tienen valores de precisión muy favorables, estas no cuentan con la sensibilidad adecuada. Esto se debe a que utilizan demasiados operadores, recuperando solo una pequeña porción de estudios, lo que podría dejar fuera varios estudios relevantes.

- Finalmente, los valores obtenidos por la cadena de búsqueda CB11 son los ideales, con un 100% de sensibilidad, lo que significa que recupera todos los estudios del *Quasi-Gold*, y un 12% de precisión, lo cual indica que recupera una cantidad manejable de estudios.

Es a partir de estas conclusiones que se tomó la decisión de seleccionar a la cadena de búsqueda CB11 para el motor de búsqueda ACM Digital Library:

CB11. ("Jenkins" OR "Jenkins-based") AND Abstract:("CI/CD" OR "Continuous integration and continuous deployment" OR "Continuous integration and deployment" OR "Continuous integration and delivery" OR "Continuous Integration" OR "CI" OR "Continuous-integration" OR "CD" OR "Pipeline")

Posteriormente, se realizó la evaluación de las cadenas de búsqueda para la base de datos de SpringerLink. Para esta base de datos, se decidió probar primero las cadenas de búsqueda seleccionadas para IEEE Xplore y ACM Digital library, sin embargo, la cantidad de estudios que se recuperaban no era manejable, por lo que se decidió crear más cadenas de búsqueda.

La tabla 9 muestra la evaluación de las cadenas propuestas para este motor de búsqueda, con sus respectivos valores de sensibilidad y precisión. Para esta base de datos, el total de estudios relevantes es de 10 (TER = 10). Hay que tomar en cuenta que para el conteo

del número total de estudios recuperados (TE), se aplicaron filtros sobre la fecha de publicación (2016 al 2024), el idioma de publicación (Inglés) y el tipo de *venue* del que proviene (journal, conferencia, workshop o revista).

Tabla 9. Cadenas de Búsqueda SpringerLink					
ID	Cadena	TE	ERR	Sensibilidad	Precisión
CB09	"Jenkins" AND ("CI/CD" OR "Continuous integration" OR "CI" OR "Continuous delivery" OR "Continuous deployment" OR "CD")	6001	9	90%	0.15%
CB11	("Jenkins" OR "Jenkins-based") AND Abstract:("CI/CD" OR "Continuous integration and continuous deployment" OR "Continuous integration and deployment" OR "Continuous integration and delivery" OR "Continuous Integration" OR "CI" OR "Continuous-integration" OR "CD" OR "Pipeline")	6753	8	80%	0.12%
CB14	Jenkins AND Continuous Integration OR CI OR Continuous Deployment OR CD	2819	6	60%	21%
CB15	(Title: Jenkins) AND (Continuous OR Integration OR CI OR Pipelines OR Continuous OR Deployment OR CD)	81	1	10%	1.23%
CB16	Jenkins AND ("Continuous Integration" OR "Continuous Deployment" OR "Continuous Delivery")	582	9	90%	1.55%
CB17	Jenkins AND Tool AND Automated AND Pipeline AND ("Continuous Integration" OR "Continuous Deployment" OR "Continuous Delivery")	847	5	50%	0.59%

Se generaron un total de cuatro cadenas de búsqueda adicionales para SpringerLink. Es necesario mencionar que se utilizó el motor de búsqueda antiguo de SpringerLink, y que en la mayoría de las búsquedas siempre se recuperaban una cantidad de estudios muy grande (superior a mil), por lo que mejorar la precisión sin sacrificar la sensibilidad resultó una tarea muy compleja. Por esto, se priorizó el valor de la sensibilidad sobre el de precisión.

Con base en los resultados obtenidos a partir de las búsquedas, se obtuvieron las siguientes conclusiones:

- Las cadenas de búsqueda CB09 y CB11, seleccionadas para la IEEE Xplore y ACM, respectivamente, dieron resultados de sensibilidad óptimos, iguales o superiores al 80%. Sin embargo, su precisión era incluso menor al 0.2%, lo que significa que recupera una cantidad de estudios muy elevada (arriba de 6,000), por lo que se descartaron.
- Las cadenas CB14, CB15 y CB17 mejoran un poco la precisión en comparación con las dos cadenas previamente descritas. A pesar de esto, las tres cadenas de búsqueda no

alcanzaron el mínimo de sensibilidad deseado, obteniendo un 60%, 10% y 50% respectivamente.

- Finalmente, la cadena de búsqueda CB16 demostró sensibilidad muy favorable y recuperó una cantidad de estudios considerada manejable por los investigadores involucrados. Esta cadena presentó las mejores métricas de sensibilidad y precisión entre todas las evaluadas para esta base de datos, alcanzando un 90% y 1.55%.

Por esto, se tomó la decisión de seleccionar la cadena de búsqueda CB16 para realizar la búsqueda automatizada en SpringerLink:

CB16. Jenkins AND ("Continuous Integration" OR "Continuous Deployment" OR "Continuous Delivery")

Finalmente, se realizó la evaluación de las cadenas de búsqueda para el motor de búsqueda ScienceDirect. Debido a que en las evaluaciones previas el usar las cadenas de búsqueda utilizadas en los otros motores no brindó los resultados deseados, para esta última base de datos se decidió desarrollar las cadenas de búsqueda desde cero y se omitió el utilizar las otras cadenas de búsqueda ya seleccionadas.

La tabla 10 muestra la evaluación de las cadenas desarrolladas, con sus respectivos valores de sensibilidad y precisión. Para este motor, el total de estudios relevantes es de 10 (TER = 10). Hay que tomar en cuenta que para el conteo del número total de estudios recuperados (TE), se aplicaron filtros sobre la fecha de publicación (2016 al 2024), el idioma de publicación (Inglés) y el tipo de *venue* del que proviene (*journal*, conferencia, *workshop* o revista). Sin embargo, es importante mencionar que, a diferencia de los tres motores de búsqueda previos, en ScienceDirect no es posible aplicar todos los filtros directamente en el buscador, por lo que esto se realizó el cálculo manualmente.

Tabla 10. Cadenas de Búsqueda ScienceDirect					
ID	Cadena	TE	ERR	Sensibilidad	Precisión
CB18	"Jenkins" AND ("CI/CD" OR "DevOps" OR "Continuous integration" OR "Continuous delivery" OR "Continuous deployment")	178	9	90%	5%
CB19	"Jenkins" AND ("CI/CD" OR "Continuous integration" OR "Continuous delivery" OR "Continuous deployment")	169	4	60%	4%

Tal como se muestra en la tabla 10, se generaron solamente dos cadenas de búsqueda para la base de datos de SceinceDirect. Estas recuperaron una cantidad de estudios similares debido a que las cadenas son muy similares entre sí, la única diferencia es la exclusión del término de búsqueda “DevOps” en la cadena CD19. Sin embargo, esto afecto de manera negativa la sensibilidad.

En este caso, no se decidió seguir probando otras cadenas de búsqueda, pues CB18 arrojó muy buenos valores de sensibilidad y precisión. Quitar términos de esta cadena resultaba en reducciones de sensibilidad, como ocurrió con CB19. Aumentar términos o el uso de sinónimos, afectaría negativamente la precisión, que de por sí ya es menor que el valor deseable (10%). Por esta razón, los investigadores seleccionaron la cadena de búsqueda CB18 para la base de datos de ScienceDirect:

CB18. "Jenkins" AND ("CI/CD" OR "DevOps" OR "Continuous integration" OR "Continuous delivery" OR "Continuous deployment")

Como se mencionó previamente, en lo que respecta a la literatura gris, no realizó un *Quasi-Gold*, por lo que no es posible aplicar las métricas de sensibilidad y precisión para evaluar las cadenas de búsqueda.

En el caso de la literatura gris, para seleccionar la cadena de búsqueda, se comparó el desempeño de las cadenas previamente seleccionadas para los otros motores, pero ahora en Google. Sin embargo, debido a que las cadenas anteriores no utilizan todos los términos de búsqueda y están diseñadas para funcionar de manera óptima en sus respectivos motores, se decidió generar nuevas búsqueda para Google y explorar la literatura gris. La tabla 11 muestra las cadenas de búsqueda que se probaron, así como los resultados obtenidos, donde no se aplicó ningún tipo de filtro para su revisión.

Tabla 11. Cadenas de Búsqueda Google

ID	Cadena	TE	Exploración inicial (Primero 50 resultados)
CB09	"Jenkins AND ("CI/CD" OR "Continuous integration" OR "CI" OR "Continuous delivery" OR "Continuous deployment" OR "CD")	285	Páginas oficiales: 8 Blogs: 12 Página web tipo tutorial: 17 Estudios: 2 Videos: 5 Publicidad: 3 Sitios de preguntas y respuestas: 2 Observaciones: a partir de la página 6-9 se empiezan a repetir los temas de las publicaciones, la mayoría son blogs o páginas web donde explican que es Jenkins, que es CI/CD, beneficios, y la creación de un pipeline simple, y configuraciones básicas con GitHub o Docker.
CB11	("Jenkins" OR "Jenkins-based") AND ("CI/CD" OR "Continuous integration and continuous deployment" OR "Continuous integration and deployment" OR "Continuous integration and delivery" OR "Continuous Integration" OR "CI" OR "Continuous-integration" OR "CD" OR "Pipeline")	283	Páginas oficiales: 8 Blogs: 19 Página web tipo tutorial: 14 Estudios: 2 Sitios de preguntas y respuestas: 2 Videos: 2 Cursos/plataformas de aprendizaje: 2 Publicidad: 1 Observaciones: pasa algo similar que con la cadena previa, los temas entre los estudios parecen repetirse, explicando qué es Jenkins o la configuración básica de un pipeline CI/CD con dicha herramienta.
CB16	Jenkins AND ("Continuous Integration" OR "Continuous Deployment" OR "Continuous Delivery")	279	Páginas oficiales: 5 Blogs: 14 Página web tipo tutorial: 13 Libros: 2 Estudios: 2 Videos: 2 Publicidad: 5 Sitios de preguntas y respuestas: 3 Observaciones: esta cadena parece que trae más hits relacionados a venta de cursos y publicidad de Jenkins. Aunque parece que los temas son más variados, hay bastante repetido sobre qué es Jenkins y configuración básica, también se notan otra publicaciones sobre otros aspectos menos comunes.
CB20	(Jenkins pipeline) AND ("CI/CD" OR "Continuous integration" OR "CI" OR "Continuous delivery" OR "Continuous deployment" OR "CD" OR DevOps)	253	Páginas oficiales: 5 Blogs: 22 Páginas web tipo tutorial: 13 Cursos/plataformas de aprendizaje: 3 Publicidad: 3 Videos: 4

CB21	Jenkins AND ("CI/CD" OR "Continuous integration" OR "CI" OR "Continuous delivery" OR "Continuous deployment" OR "CD" OR DevOps OR Pipeline)	278	Páginas oficiales: 9 Blogs: 19 Páginas web tipo tutorial: 13 Cursos/plataformas de aprendizaje: 3 Videos: 4 Sitios de preguntas y respuestas: 1 Estudios: 1
------	---	-----	---

Las primeras cadenas probadas, se desarrollaron específicamente para cada uno de los motores de búsqueda de la literatura blanca, por lo que existe un cierto grado de sesgo en ese aspecto, ya que están optimizadas para la base de datos respectiva. Además, estas cadenas no ocupan todos los términos de búsqueda clave; por estas razones, se decidió generar una nueva cadena de búsqueda donde se utilizaron todos los términos clave, la cadena CB21.

A pesar de no haber realizado una evaluación formal como con las cadenas de la literatura blanca, hubo un proceso de exploración y comparación previo a la selección final de la cadena de la literatura gris. Se analizaron los 50 primeros resultados, así como el total de estos en las primeras páginas del motor de búsqueda, lo que permitió tomar una decisión más informada sobre la selección de la cadena.

Los resultados de las cadenas probadas fueron bastante similares, pues la cantidad de estudios que recuperan oscilan entre los 250 y 290 recursos. Así mismo, la mayoría de los resultados son blogs o páginas web que presentan tutoriales de la herramienta, también se encuentran algunos videos, sitios de preguntas y respuestas, páginas del sitio oficial de Jenkins y publicidad de libros o cursos.

Considerando los puntos previamente expuestos, se optó por la cadena búsqueda CB21 para la literatura gris:

CB21. Jenkins AND ("CI/CD" OR "Continuous integration" OR "CI" OR "Continuous delivery" OR "Continuous deployment" OR "CD" OR DevOps OR Pipeline)

3.2.1.5 Selección de fuentes

Como se pudo notar en la sección previa, se seleccionaron un total de cuatro fuentes de información para llevar a cabo la búsqueda automatizada en la literatura blanca, y una fuente de información para la literatura gris. Sin embargo, durante la conducción de la búsqueda automatizada en la literatura gris (Google), se encontró un repositorio oficial de Jenkins con más de 200 historias de éxito del uso de Jenkins, las cuales contenían descripciones breves pero muy útiles de organizaciones, empresas y desarrolladores que utilizaron Jenkins para resolver algún problema de automatización. Por esa razón, se tomó la decisión de revisar

cada una de estas historias de éxito (*Succes-stories*). A continuación se enumeran las fuentes de información seleccionadas:

- IEEE Xplore,
- ACM Digital Library,
- SpringerLink,
- ScienceDirect.
- Google y
- Repositorio de historia de éxito de Jenkins.

Para la selección de los primeros cuatro motores de búsqueda se tomó como referencia el ranking propuesto por Zhang et al. (2011), dándole prioridad a aquellos motores especializados en ingeniería de software y de libre acceso con el uso de las credenciales de la Universidad Veracruzana (IEEE Xplore y ACM Digital Libray). La selección del motor de búsqueda de Google se decidió partiendo de lo propuesto por Garousi et al. (2019) y lo recomendado por los investigadores involucrados con mayor experiencia.

Las cinco cadenas de búsqueda seleccionadas (CB09, CB11, CB16, CB18, CB21) fueron utilizadas para realizar la búsqueda automatizada en los motores de IEEE Xplore, ACM Digital Library, SpringerLink, ScienceDirect y Google, respectivamente.

3.2.2 Selección de estudios primarios

La fase de selección de estudios se centra en que los investigadores apliquen los criterios de inclusión y exclusión para descartar aquellos estudios que no son de relevancia para la investigación.

3.2.2.1 Criterios de Inclusión y Exclusión

Para la selección de estudios, se determinaron seis criterios de inclusión, que se muestran en la tabla 12, y se plantearon cuatro criterios de exclusión, que se muestran en la tabla 13.

Es importante mencionar que los criterios de inclusión y exclusión se utilizaron tanto para la literatura blanca como la gris, por lo que se tuvieron que realizar ciertas adaptaciones a algunos de estos, particularmente en el CI V. En otros casos, se omitió el uso de algunos criterio debido a que no era aplicables, como el CI III, CE III, CE IV.

Tabla 12. Criterios de Inclusión		
ID	Criterio	Aplicable a literatura Blanca o Gris
CI I	El estudio fue publicado entre enero de 2016 y diciembre de 2024.	Ambas
CI II	El estudio fue publicado en el idioma inglés.	Ambas
CI III	El estudio fue publicado en un <i>journal</i> , conferencia, <i>workshop</i> o revista.	Literatura Blanca
CI IV	El título del estudio alude a la implementación de pipelines de integración o despliegue continuo, o bien al uso de la herramienta de automatización Jenkins.	Ambas
CI V	A: El resumen (<i>abstract</i>) del estudio da indicios de responder al menos una de las preguntas de investigación.	Literatura Blanca
	B: Tras una revisión rápida del recurso (lectura superficial en textos, visualización parcial en videos o exploración general en otros formatos), este da indicios de responder al menos a una de las preguntas de investigación.	Literatura Gris
CI VI	El texto del estudio proporciona una respuesta a al menos una de las preguntas de investigación.	Ambas

Tabla 13. Criterios de exclusión		
ID	Criterio	Aplicable a literatura Blanca o Gris
CE I	El estudio se encuentra duplicado.	Ambas
CE II	No es posible acceder al estudio completo.	Ambas
CE III	Para las bases de datos multidisciplinarias: el estudio no proviene de una fuente especializada en ingeniería de software, ciencias computacionales o áreas a fines. Esto incluye publicaciones de otras diciplinas como ciencias generales, ciencias sociales o cualquier otra área no relacionada directamente con la ingeniería de software.	Literatura Blanca
CE IV	El estudio no es un estudio primario.	Literatura Blanca

3.2.2.2 Procedimiento de selección de estudios

Para llevar a cabo la selección de estudios primarios de forma sistemática y ordenada, se establecieron cinco etapas secuenciales en las que se aplicaron los criterios de inclusión y exclusión previamente mencionados.

Etapas 1. Primero, se utilizaron las opciones de filtrado que ya proporcionan los motores de búsqueda (IEEE, ACM, SpringerLink y ScienceDirect) para seleccionar solo aquellos estudios publicados entre 2016 y 2024 y que se encuentren escritos en el idioma inglés. Así mismo, se utilizaron los filtros de los motores de búsqueda para descartar los

estudios que no provinieran de un *journal*, conferencia, *workshop* o revista. En el caso de Google y las *success stories*, no se utilizaron estos filtros, por lo que se realizó el filtrado de manera manual, aplicando los mismos criterios, a excepción del CI III.

Etapas 2. Posteriormente, se realizó una exploración rápida para descartar aquellos estudios que se encontraban repetidos en distintos motores de búsqueda, así como los estudios a los que no se tenía acceso completo.

Etapas 3. Esta solo se aplicó a la literatura blanca, ya que se enfocó en revisar conferencias, *journals*, *workshops* y revistas de la que provienen los estudios, dando prioridad a aquellos *venues* que estén estrechamente relacionados con la ingeniería de software y que aseguren un grado de calidad, como lo es la ICSE o la IEEE Software.

Etapas 4. Consistió en realizar una revisión rápida de los estudios restantes para identificar aquellos que presentan indicios de responder al menos una pregunta de investigación. Esta revisión se hizo leyendo el título y *abstract* de los estudios de la literatura blanca o realizando un *skimming* de los estudios de la literatura gris.

Etapas 5. Finalmente, se leyó completamente el contenido de los estudios para realizar un último filtro, descartando aquellos estudios que no respondían al menos una pregunta de investigación.

En la figura 1, se muestran las etapas del proceso de selección y los criterios aplicados para cada una.



Figura 1 Etapas del proceso de selección de estudios primarios

En la tabla 14, se muestran con mayor detalle los criterios de inclusión y exclusión que se aplicaron en cada una de las etapas previamente descritas, así como el propósito de cada una.

Tabla 14. Propósito de las etapas de selección		
Etapas	Criterios por considerar	Propósito
Etapas 1	CI I, CI II, CI III	Descartar los estudios publicados previo al lanzamiento de la actualización 2.0 de la herramienta Jenkins. Así como mantener cierto nivel de calidad de los estudios al asegurarse de que provienen de un <i>journal</i> , <i>workshop</i> , conferencia o revista.
Etapas 2	CE I, CE II	Descartar los estudios repetidos y aquellos que no son accesibles completamente.
Etapas 3	CE III	Garantizar cierto nivel de calidad de los estudios con base en la conferencia, revista, <i>journal</i> o <i>workshop</i> del que provienen, asegurando que estén relacionados a la Ingeniería de software.
Etapas 4	CE IV, CI IV, CI V.	Descartar los estudios secundarios, así como los estudios que no presentan indicios de ser relevantes para la revisión con base en una revisión rápida del estudio.
Etapas 5	CI VI	Filtrar de manera granular los estudios que responden o aportan información relevante a al menos una de las preguntas de investigación con base en la lectura completa del mismo.

3.2.3 Evaluación de la calidad

Una vez que se identificaron los estudios primarios tanto de la literatura blanca como de la literatura gris se realizó una evaluación de la calidad de los estudios de la literatura gris. Se tomó la decisión de omitir una evaluación de la calidad de la literatura blanca debido a que estos fueron recuperados de congresos, *journals*, *workshops* y revistas en los que se sometieron a una evaluación previa para su publicación y que en la mayoría de los casos estas fueron revisiones por pares, lo que garantiza un grado de calidad de los estudios.

Por el otro lado, los recursos de la literatura gris no tienen ningún tipo de filtro, revisión, control de calidad o aspecto que legitímese su veracidad, por lo que es necesario evaluar ciertos aspectos y criterios metodológicos, de contenido, autoría, entre otros, de los estudios, para minimizar los sesgos y tratar de garantizar que los resultados y conclusiones se generaron a partir de recursos que cumplen con un grado mínimo de calidad.

Para realizar dicha evaluación de la calidad se siguieron las pautas sugeridas por Garousi et al. (2019). En conjunto con los directores de trabajo, se adaptaron algunos aspectos de la rúbrica de evaluación propuesta por Garousi et al. (2019), se eliminaron y adaptaron aspectos relacionados a la credibilidad de los autores de los recursos, se tomó la decisión de considerar la reputación de las organizaciones que publicaron el estudio en lugar de los autores mismo, ya que es más complicado determinar si un autor específico es experto o no en el área de ingeniería de software. También se modificaron algunos criterios de

evaluación relaciones a la metodología y la objetividad para que se adaptaran de mejor manera a los tipos de recursos que se recuperaron durante la selección de estudios. De esta forma de determinaron un total de trece criterios de calidad con un valor maximo de un punto cada uno, y un posible valor parcial de 0.5 en algunos casos.

Estos criterios buscan asistir a los investigadores a evaluar la reputación de la organización que realiza las publicaciones, determinar si el estudio tiene un objetivo claro, metodología, límites y fecha de publicación establecida, identificar si el estudio refiere a una implementación particular, valorar los posibles conflictos de interés así como las fuentes formales de literatura que se citen, además de revisar si es que el estudio aporta algo nuevo a la investigación o fortalece alguna idea o posición actual en la investigación, y finalmente evaluar el impacto del recurso en la comunidad mediante métricas de vistas, *backlinks* o citas, y asignar un último punto dependiendo del nivel de literatura gris que se encuentra el recurso, tal y como lo marca Garousi et al. (2019). Es importante mencionar que para la evaluación de los criterios que califican el impacto con base en el número de vistas, *backlinks* o citas (Dependiendo el tipo de recurso) se hizo una normalización con base en el recurso que tenga la mayor cantidad de vistas, *backlinks* o citas respectivamente.

Finalmente, al término de la evaluación de la calidad y de haber asignado una calificación a cada uno de los estudios, estos se revisaron para determinar el puntaje mínimo para incluirse en el trabajo, en conjunto con los directores se determinó que una calificación normalizada igual o mayor a seis (Al menos ocho de los trece puntos posibles) es suficiente para incluirse en la revisión multivocal de la literatura. En el Apéndice C) Rúbrica de Evaluación de la Calidad se puede ver con detalle la rúbrica de evaluación de la calidad que se utilizó en esta RML.

3.2.4 Extracción de Datos

Posterior a haber finalizado la evaluación de la calidad de los estudios de la literatura gris se continuó con la extracción de datos. Para esto, se establecieron dos formatos en Excel (una

pág. 34

para cada nivel de literatura) en el que ambos responsables de la RML registraron los datos relevantes de los estudios. En el Apéndice D), se puede consultar los formatos de extracción de datos con los campos de interés que se establecieron.

De manera general, los datos que se extrajeron para la literatura blanca son: título, autores, año de publicación, fuente, DOI, base de datos, tipo de publicación, *venue*, palabras clave y *abstract*. En lo que respecta a la literatura gris, se extrajeron los siguientes datos: título, organización que publica, año de publicación, URL, base de datos, nivel de literatura gris y tipo de recurso. Así mismo, ambos formatos de extracción contemplan campos para capturar las respuestas a cada una de las preguntas de investigación.

3.2.5 Síntesis de datos

Un elemento clave de toda revisión es la síntesis de la información. Este es el proceso que permite reunir los hallazgos encontrados en los estudios seleccionados, con el fin de desarrollar conclusiones basadas en evidencia (Popay et al., 2006). De la variedad de métodos que existen para realizar síntesis cualitativas, la más usada por los investigadores de ingeniería de software es la síntesis narrativa (Cruzes & Dybå, 2011).

Por lo anterior, en esta RML se decidió desarrollar una síntesis narrativa para reportar los hallazgos, siguiendo las pautas, método y técnicas propuesta por Popay et al. (2006) en la que se identifican cuatro etapas principales.

1. Desarrollo de un modelo teórico.
2. Desarrollo de una síntesis preliminar.
3. Exploración de las relaciones dentro y entre los estudios.
4. Evaluación de la robustez de la síntesis.

Tomando estas etapas como referencia, se tomó la decisión de únicamente desarrollar la etapas dos y tres, las cuales se describen a continuación.

Desarrollo de una síntesis preliminar: el propósito de esta actividad es desarrollar una descripción inicial para resumir y organizar los resultados obtenidos en la revisión. Funciona como una base en la que posteriormente se debe profundizar con el fin de identificar patrones y tendencias en los datos, así como factores que pudieron influir en los resultados de los estudios.

Durante esta etapa se utilizaron dos de las técnicas y herramientas propuestas por Popay et al. (2006). En primera instancia, se realizó una tabulación de los datos para

identificar la información relevante específica de los fragmentos extraído de los estudios, como se muestra en la figura 2. Posteriormente se analizaron los datos para clasificar los estudios y agruparlos en *clústeres*, como se muestra en la figura 3. Este proceso se llevó a cabo para cada una de las preguntas de investigación.

Explorar las relaciones dentro y entre los estudios: Esta actividad se realiza a partir de los patrones y tendencia identificados en la síntesis preliminar; consiste en explorar la relación entre los estudios, analizando las diferencias y similitudes para entender por qué los estudios reportan resultados diferentes, así como identificar qué factores influyen en los resultados. Esta parte del proceso de síntesis narrativa contribuye en gran medida a la calidad de la revisión.

Para la segunda etapa se utilizaron dos técnicas para explorar y representar las relaciones entre los estudios. Primero, se realizaron conteos para cada uno de los clústeres previamente desarrollados, como se muestra en la figura 4. Posteriormente, se utilizaron gráficas para representar los hallazgos, lo que ayudó a interpretar y entender las tendencias reportadas en los estudios, como se muestra en la figura 5.

CATEGORÍA DE USUARIOS PARA PREGUNTAS			
ID	Contextos	Escenarios	Fragmento de texto del estudio.
EP02	* PegahTech * Provides back-end services to mobile developers * Written in Java using Spring framework * Oracle Database * Microservices	Relational database management system: * Backtory * RDBMS functioning as a service*	Backtory, which was developed at PegahTech (www.pegahtech.ir), provides back-end services to mobile developers who don't know any server-side programming languages. It originally was an RDBMS (relational database management system) functioning as a service. [...] Backtory is written in Java using the Spring framework. The underlying RDBMS is Oracle Database 11g. Backtory uses Maven to fetch dependencies and build the project. Before migration, all the services were in a Git repository, and Backtory used Maven's modules to build services. Deployment of services to development machines was done using Maven's Jetty plug-in. However, deployment to the production machine was a manual task. [...] We transformed Backtory's core architecture to the target architecture through refactorings. These changes included introducing microservices specific components and rearchitecting the system. [...] Backtory's new technology stack included Spring Boot for the embedded application server and fast service initialization, the OS's environment variables for configuration, and Spring Cloud Context and the Spring Cloud Config server to separate the configuration from the source code, as continuous delivery (CD) practices recommend.[...]
EP04	* To evaluate the effectiveness of Jenkins in automating various stages of development * Angular framework	Web application	To evaluate the effectiveness of Jenkins in automating various stages of development, a practical pipeline was executed using the Angular framework for web applications. [...] The results of this research have shown that using Jenkins to automate the development of web applications is a highly effective approach.
EP05	* A comparison of Jenkins with popular CI/CD tools * AWS deployment. * Java-based.	Web application	This research paper presents a Continuous Integration/Continuous Deployment pipeline which is fully automated for AWS deployment of a Java-based web application.[...] A comparison of Jenkins with popular CI/CD tools is given in table 1. This comparison proves Jenkins to be a popular choice to handle the CI/CD process, as it comes with multiple options for execution environments, it is open source and 1000's of plugins for integration. To further estimate the effectiveness of the CI tools we did a performance testing of Gitlab specific runner, shared runner, Teamcity, Bamboo, CircleCI and Jenkins slave.[...]
EP06	This paper focuses on building and integration using Jenkins with pipeline methodology	Web applications	[...]This paper focuses on building and integration using Jenkins with pipeline methodology and deploying using the ansible tool and gives you a basic skeleton of it.[...] [...] Whereas using Jenkins and ansible you can automate your integration and deployment process with a single button click after making changes in your project and it would be useful in companies and universities where all the systems are connected using LAN.[...] [...] CI/CD using Jenkins ansible is an efficient and optimized way of building a web applications and hosting it as they make the development faster with a better software quality.
EP07	* Presents an entire automated pipeline * Java and Apache Maven archetype * For hosting 5G network core functions	Web application	[...]This paper presents an entire automated pipeline, starting with detecting changes in the Java-based web application source code, creating new resources in the Kubernetes cluster to host this new version and finally deploying the containerized application in AWS.[...] The web application was developed using Java and an Apache Maven archetype which helps describe the software project, its dependencies and other external components in a Project Object Model (POM) .xml file. Once created, the source code of the application was loaded in GitHub. [...] While especially tailored for a Java-based web application, the techniques and processes described herein are a good starting point for hosting 5G network core functions in Docker containers and managed by Kubernetes. This, in turn,
☰ Estudios relevantes IEEE ACM RQ01 RQ01-Conteo RQ02 RQ03 RQ04 RQ05 +			

Figura 2. Tabulación de datos

Extracción de datos para preguntas		Clustering	
Fragmento de texto del estudio.		Contexto	Escenario
Backtory, which was developed at PegahTech (www.pegahtech.ir), provides back-end services to mobile developers who don't know any server-side programming languages. It originally was an RDBMS (relational database management system) functioning as a service. [...] Backtory is written in Java using the Spring framework. The underlying RDBMS is Oracle Database 11g. Backtory uses Maven to fetch dependencies and build the project. Before migration, all the services were in a Git repository, and Backtory used Maven's modules to build services. Deployment of services to development machines was done using Maven's Jetty plug-in. However, deployment to the production machine was a manual task. [...] We transformed Backtory's core architecture to the target architecture through refactorings. These changes included introducing microservices specific components and rearchitecting the system. [...] Backtory's new technology stack included Spring Boot for the embedded application server and fast service initialization, the OS's environment variables for configuration, and Spring Cloud Context and the Spring Cloud Config server to separate the configuration from the source code, as continuous delivery (CD) practices recommend.[...]		Industria	Sistemas basados en la nube
To evaluate the effectiveness of Jenkins in automating various stages of development, a practical pipeline was executed using the Angular framework for web applications. [...] The results of this research have shown that using Jenkins to automate the development of web applications is a highly effective approach.		Laboratorio	Sistemas Web
This research paper presents a Continuous Integration/Continuous Deployment pipeline which is fully automated for AWS deployment of a Java-based web application.[...] A comparison of Jenkins with popular CI/CD tools is given in table 1. This comparison proves Jenkins to be a popular choice to handle the CI/CD process, as it comes with multiple options for execution environments, it is open source and 1000's of plugins for integration. To further estimate the effectiveness of the CI tools we did a performance testing of Gitlab specific runner, shared runner, Teamcity, Bamboo, CircleCI and Jenkins slave.[...]		Laboratorio	Sistemas Web
[...]This paper focuses on building and integration using Jenkins with pipeline methodology and deploying using the ansible tool and gives you a basic skeleton of it.[...] [...] Whereas using Jenkins and ansible you can automate your integration and deployment process with a single button click after making changes in your project and it would be useful in companies and universities where all the systems are connected using LAN.[...] [...] CI/CD using Jenkins ansible is an efficient and optimized way of building a web applications and hosting it as they make the development faster with a better software quality.		Laboratorio	Sistemas Web
[...]This paper presents an entire automated pipeline, starting with detecting changes in the Java-based web application source code, creating new resources in the Kubernetes cluster to host this new version and finally deploying the containerized application in AWS.[...] The web application was developed using Java and an Apache Maven archetype which helps describe the software project, its dependencies and other external components in a Project Object Model (POM) .xml file. Once created, the source code of the application was loaded in GitHub. [...] While especially tailored for a Java-based web application, the techniques and processes described herein are a good starting point for hosting 5G network core functions in Docker containers and managed by Kubernetes. This, in turn,		Laboratorio	Sistemas Web
RQ02 RQ03 RQ04 RQ05 +			

Figura 3. Clústeres de estudios

Contextos en los que se utiliza Jenkins en los pipelines de CI/CD	
Contexto	Estudios que responden
Laboratorio	34
Industria	12
Academia	9
Escenarios en los que se utiliza Jenkins en los pipelines de CI/CD	
Escenario	Estudios que responden
Sistemas Web	21
Sistemas basados en la nube	6
Sistemas de apoyo	6
Software embebido	5
Sistemas AI/ML/NPL	5
Sistemas Ciber-Físicos	3
Sistemas de IoT	3
Aplicación Móvil	3
Sistemas para construir software	1
Sistemas de información	1
Sistemas de base de datos	1
Computación de alto rendimiento (HPC)	1

Estudios relevantes IEEE ACM RQ01 **RQ01-Conteo** RQ02 RQ03

Figura 4. Conteo de hallazgos en clústeres

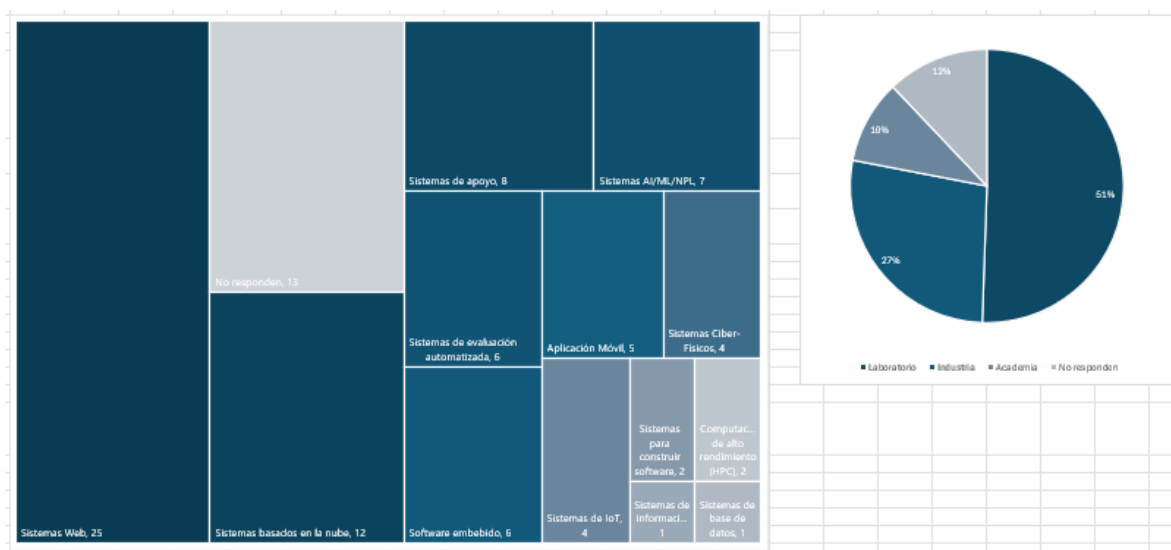


Figura 5. Graficas de hallazgos.

3.2.6 Sesgos

Como en todo trabajo de investigación, es importante considerar los posibles sesgos que puedan presentarse al seleccionar los estudios relevantes para la investigación.

Durante la aplicación de los criterios de inclusión y exclusión de la etapa cuatro, es posible que se hayan introducido algunos sesgos. Al analizar los títulos y resúmenes de los estudios, muchos fueron descartados por no mostrar indicios de responder al menos una de las preguntas de investigación. Sin embargo, este proceso omitió la revisión completa de dichos estudios, lo que da lugar a que exista más de un estudio rechazado durante esta etapa que sí respondiera alguna de las preguntas de investigación. Además, esta evaluación depende de manera significativa del conocimiento y capacidades subjetivas de cada investigador.

Este aspecto se mitigó mediante la comunicación continua entre los dos investigadores, para que formaran una opinión conjunta y se pudieran tomar decisiones más adecuadas respecto a la selección de los estudios. Además, ambos investigadores revisaron algunos de los estudios examinados por el otro para verificar que se hayan descartado o incluido correctamente.

4 Resultados

A continuación se presentan los resultados de la conducción del método de la revisión multivocal de la literatura (RML), así como las respuestas a las preguntas de investigación.

4.1 Estudios relevantes

Tras aplicar los criterios de inclusión y exclusión para todos los estudios recuperados con las cadenas de búsqueda, finalizamos con un total de 163 estudios, 102 en la literatura blanca y 61 en la gris.

La búsqueda inicial de estudios, sin aplicar ningún tipo de filtro más que las propias cadenas de búsqueda, permitió recuperar un total de 2,536 estudios, de los cuales: 102 se encontraron en IEEE Xplore Digital Library, 155 en ACM Digital Library, 1,522 en SpringerLink, 312 en ScienceDirect, alrededor de 252 en Google y 193 en la biblioteca de historias de usuarios de Jenkins. Destacando que en Google solo se consideraron las principales entradas, pues de forma general, Google arrojó más de 125 millones de resultados.

Esta búsqueda inicial se presenta a continuación, a través de las cinco figuras que muestran las cadenas de búsqueda y los resultados iniciales en cada biblioteca digital (o motor de búsqueda).

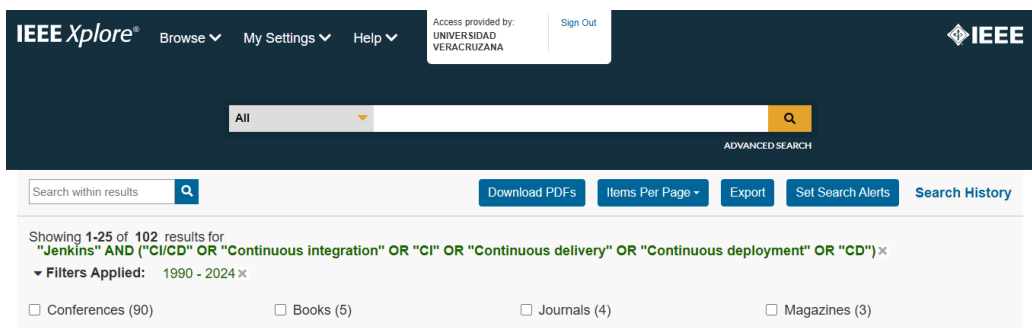


Figura 6. Búsqueda inicial en IEEE Xplore

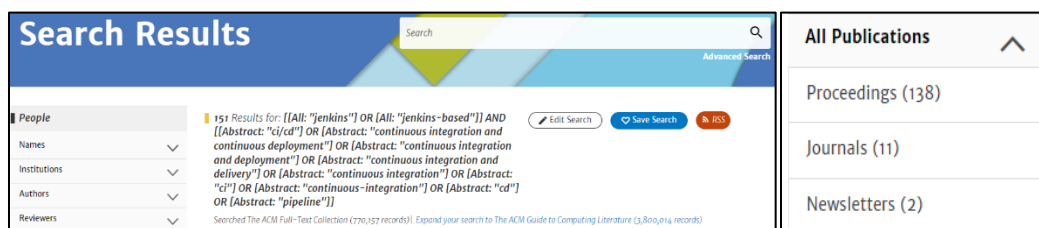


Figura 7. Búsqueda inicial en ACM



Figura 8. Búsqueda inicial en SpringerLink

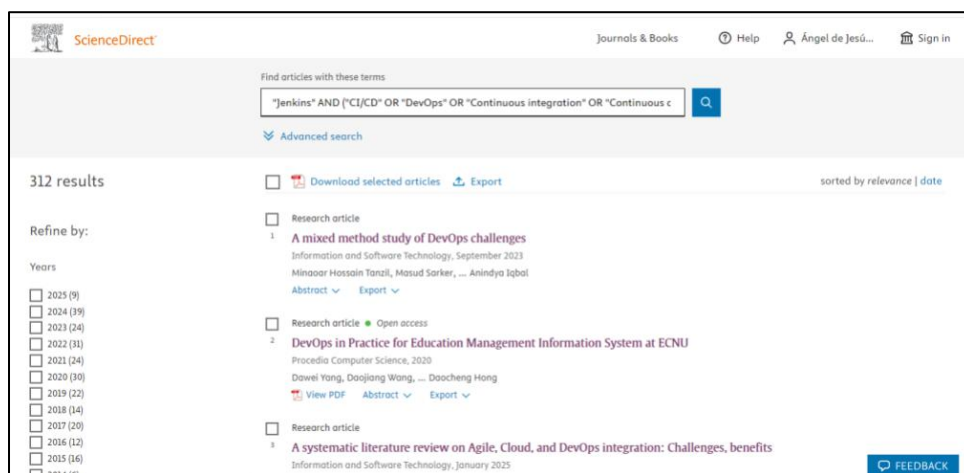


Figura 9. Búsqueda inicial en ScienceDirect

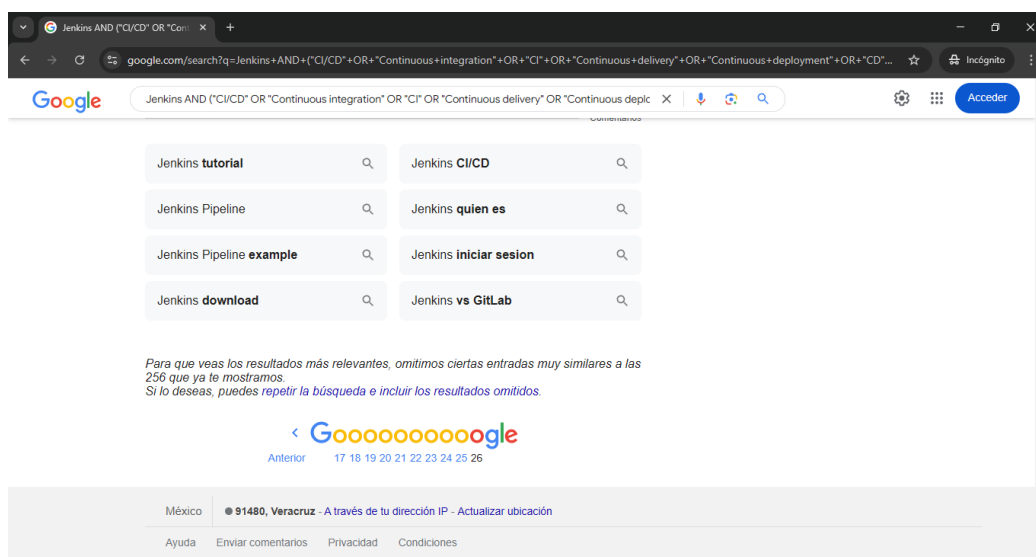


Figura 10. Búsqueda inicial en Google

El resumen de la selección de estudios por etapa se presenta a continuación de forma gráfica, con ayuda de un diagrama de flujo “prisma”. En el diagrama se puede observar que en la búsqueda automatizada, la etapa uno fue la que más estudios descartó. Además, resalta que la búsqueda automatizada no recuperó todos los estudios de los estándares *Quasi-gold*, por lo que aquellos faltantes, se incluyeron tras haber aplicado las etapas de selección.

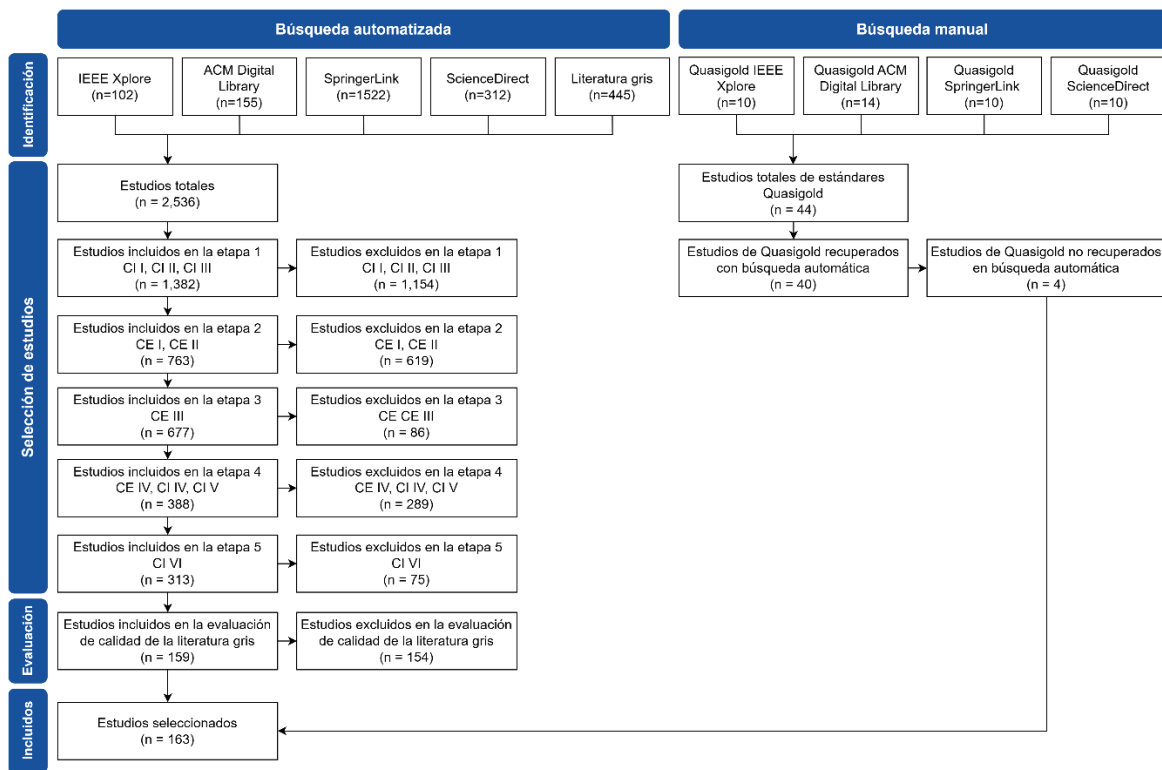


Figura 11. Diagrama de flujo prisma de la selección de estudios por etapa

El desglose de las etapas por cada biblioteca digital se presenta en la tabla 15. Como se puede apreciar, IEEE Xplore fue en la que más estudios relevantes se encontraron, seguida de ACM Digital Library, SpringerLink y ScienceDirect, en ese orden. Tal como ocurrió en la generalidad según lo mostrado en el diagrama de flujo prisma, la etapa uno fue en la que más estudios se descartaron.

Algo importante para destacar en la tabla, es la columna de la etapa cinco, donde se muestran sumas en las celdas. Estas sumas indican el total de estudios que quedaron en la etapa cinco a partir de la búsqueda automática, sumando aquellos pertenecientes al *Quasigold*, pero no recuperados en la búsqueda.

Tabla 15. Selección de estudio de la literatura blanca.

Biblioteca	Estudios recuperados		Etapas 1	Etapas 2	Etapas 3	Etapas 4	Etapas 5	Total
IEEE	102	Aceptados	77	74	69	54	47 + 2	49
		Rechazados	25	3	5	15	7	-
ACM	155	Aceptados	127	125	122	47	28	28
		Rechazados	28	2	3	75	19	-
SpringerLink	1522	Aceptados	582	97	57	21	12 + 1	13
		Rechazados	940	465	40	36	9	-
ScienceDirect	312	Aceptados	178	98	60	17	11 + 1	12
		Rechazados	134	80	38	43	6	-
								102

Es relevante mencionar que de los 49 estudios que se recuperaron en IEEE, diez son parte del *Quasi-gold*, mientras que en el caso de ACM Digital Libray, más de la mitad (14) pertenecen al *Quasi-gold* correspondiente, por lo que en realidad solo se obtuvieron diez estudios nuevos con la búsqueda automatizada en esta biblioteca digital. Casos similares los de SpringerLink y ScienceDirect, pues la mayoría de los estudios se obtuvieron mediante la búsqueda manual al desarrollar los estándares *Quasi-gold* correspondientes (cada uno con diez estudios), obteniendo únicamente tres y dos estudios más con la búsqueda automatizada, respectivamente.

La tabla 16 es muy similar a la anterior, pero muestra el resumen de la selección de estudios en la literatura gris. Aunque la búsqueda automatizada se realizó en el motor de búsqueda Google, obteniendo 252 estudios con tendencias a la variación en el total de estudios, por la forma en la que funciona el motor, se agregaron otros 193 estudios provenientes de la biblioteca de historias de usuarios de Jenkins (o *success storie*). El incluir estos recursos fue una decisión en conjunto dado el contenido de estos, centrado en el uso de Jenkins en diversos contextos y para diversos propósitos.

A diferencia de la tabla previa, en esta no se muestra la columna respectiva para la etapa 3. Esto se debe a que, como se explicó previamente, los criterios de inclusión/exclusión de esta etapa no fueron aplicables para la literatura gris, por lo que se omitió en la selección de estudios de este grupo. Además, en esta tabla también resalta la columna de evaluación, etapa aplicada solamente para la literatura gris, donde se descartaron 154 estudios de los que

pasaron la etapa cinco, lo que representa alrededor del 70% de estudios de la literatura gris que respondían al menos una pregunta de investigación.

Tabla 16. Selección de estudio de la literatura gris.								
Fuente	Estudios recuperados		Etapas 1	Etapas 2	Etapas 4	Etapas 5	Evaluación	Total
Google	252	Aceptados	225	182	112	78	61	61
		Rechazados	27	43	70	34		
Biblioteca de historias de usuarios de Jenkins	193	Aceptados	193	187	137	137	154	
		Rechazados	0	6	50	0		

El proceso de selección de estudios culminó con un total de 163 estudios relevantes, 102 de la literatura blanca y 61 de la gris. En la tabla que se muestra a continuación, se enlistan los 163 estudios relevantes, con su respectivo ID único definido de acuerdo con el tipo de literatura donde se identificaron, su título, año de publicación y tipo de publicación, así como la biblioteca de la que se recuperaron.

Tabla 17. Estudios Relevantes				
ID	Título del estudio	Año de publicación	Tipo de publicación	Biblioteca
EP01	Enabling Continuous Improvement of a Continuous Integration Process	2019	Conferencia	IEEE Xplore
EP02	Microservices Architecture Enables DevOps: Migration to a Cloud-Native Architecture	2016	Revista	IEEE Xplore
EP03	Transforming Software Development: Achieving Rapid Delivery, Quality, and Efficiency with Jenkins-Based CI/CD Pipelines	2023	Conferencia	IEEE Xplore
EP04	Efficient Automation of Web Application Development and Deployment Using Jenkins: A Comprehensive CI/CD Pipeline for Enhanced Productivity and Quality	2023	Conferencia	IEEE Xplore
EP05	Deploying Jenkins, Ansible and Kubernetes to Automate Continuous Integration and Continuous Deployment Pipeline	2022	Conferencia	IEEE Xplore
EP06	Continuous Integration and Continuous Deployment Pipeline Automation Using Jenkins Ansible	2020	Conferencia	IEEE Xplore
EP07	Implementation of a Continuous Integration and Deployment Pipeline for Containerized Applications in Amazon Web Services Using Jenkins, Ansible and Kubernetes	2020	Conferencia	IEEE Xplore
EP08	Hotel Reservation App Integrated with Docker and Jenkins	2024	Conferencia	IEEE Xplore

EP09	Implementation Jenkins Automation Deployment with Scheduler and Notification	2021	Conferencia	IEEE Xplore
EP10	Software Configuration Using Jenkins and Yocto Project	2023	Conferencia	IEEE Xplore
EP11	Creation of Continuous Integration Continuous Deployment Pipeline Using Cloud	2024	Conferencia	IEEE Xplore
EP12	Comparison of Different CI/CD Tools Integrated with Cloud Platform	2019	Conferencia	IEEE Xplore
EP13	Test Automation and Continuous Integration using Jenkins for Smart Card OS	2021	Conferencia	IEEE Xplore
EP14	Distributing Parallel Virtual Image Application using Continuous Integrity/Continuous Delivery Based on Cloud Infrastructure	2020	Conferencia	IEEE Xplore
EP15	Improving Business deliveries using Continuous Integration and Continuous Delivery using Jenkins and an Advanced Version control system for Microservices-based system	2022	Conferencia	IEEE Xplore
EP16	Automated Development and Testing of ECUs in Automotive Industry with Jenkins	2020	Conferencia	IEEE Xplore
EP17	Design and implementation of continuous integration scheme based on Jenkins and Ansible	2018	Conferencia	IEEE Xplore
EP18	A New Approach for End to End Automation Testing Platform with Cloud Computing for 5G Product	2021	Conferencia	IEEE Xplore
EP19	Continuous Integration and Continuous Delivery Pipeline Automation for Agile Software Project Management	2018	Conferencia	IEEE Xplore
EP20	Building, Deploying and Validating a Home Location Register (HLR) using Jenkins under the Docker and Container Environment	2020	Conferencia	IEEE Xplore
EP21	Design and Practice of DevOps Platform via Cloud Native Technology	2022	Conferencia	IEEE Xplore
EP22	A Survey of DevOps tools for Networking	2018	Conferencia	IEEE Xplore
EP23	Angular and Jenkins Based System for Remote Execution of Tasks on Raspberry Pi Cluster	2020	Conferencia	IEEE Xplore
EP24	Setting Up A CICD Pipeline in The Cloud for A Web Application	2024	Conferencia	IEEE Xplore
EP25	Integrating Jenkins for Efficient Deployment and Orchestration across Multi-Cloud Environments	2023	Conferencia	IEEE Xplore
EP26	Automatic Deployment Pipeline for Containerized Application of IoT Device	2022	Conferencia	IEEE Xplore
EP27	Jenkins Pipelines: A Novel Approach to Machine Learning Operations (MLOps)	2022	Conferencia	IEEE Xplore
EP28	Cost-Effective Integration and Deployment of Enterprise Application Using Azure Cloud Devops	2022	Conferencia	IEEE Xplore
EP29	IoT Apiary Fleet Management with Jenkins	2022	Conferencia	IEEE Xplore
EP30	Visual GUI testing in continuous integration environment	2016	Conferencia	IEEE Xplore
EP31	Deploying a Build Job Orchestration in Docker	2023	Conferencia	IEEE Xplore
EP32	Tool support for traceability management of software artefacts with DevOps practices	2017	Conferencia	IEEE Xplore
EP33	Enhancing Devops Infrastructure For Efficient Management Of Microservice Applications	2023	Conferencia	IEEE Xplore
EP34	Streamlining mobile app deployment with Jenkins and Fastlane in the case of Catrobat's pocket code	2018	Conferencia	IEEE Xplore
EP35	Continuous Integration in Automation Testing	2020	Conferencia	IEEE Xplore
EP36	Study on the Automated Unit Testing Solution on the Linux Platform	2019	Conferencia	IEEE Xplore
EP37	Development of Artificial Intelligence Supported Tool for Anomaly Detection in Cloud Computing Systems	2023	Conferencia	IEEE Xplore
EP38	Unit Test Generation During Software Development: EvoSuite Plugins for Maven, IntelliJ and Jenkins	2016	Conferencia	IEEE Xplore

EP39	PCCTE: A portable component conformance test environment based on container cloud for avionics software development	2016	Conferencia	IEEE Xplore
EP40	Cloud Computing in Automation Testing	2022	Conferencia	IEEE Xplore
EP41	Vulnerability Analysis Using The Interactive Application Security Testing (IAST) Approach For Government X Website Applications	2020	Conferencia	IEEE Xplore
EP42	TEASER: Simulation-Based CAN Bus Regression Testing for Self-Driving Cars Software	2023	Conferencia	IEEE Xplore
EP43	Designing a Next-Generation Continuous Software Delivery System: Concepts and Architecture	2018	Workshop	ACM Digital Library
EP44	Exploiting DevOps Practices for Dependable and Secure Continuous Delivery Pipelines	2018	Workshop	ACM Digital Library
EP45	Gamifying a Software Testing Course with Continuous Integration	2024	Conferencia	ACM Digital Library
EP46	An IDE Plugin for Gamified Continuous Integration	2024	Workshop	ACM Digital Library
EP47	Gamekins: Gamifying Software Testing in Jenkins	2017	Conferencia	ACM Digital Library
EP48	Problems and Solutions in Applying Continuous Integration and Delivery to 20 Open-Source Cyber-Physical Systems	2022	Conferencia	ACM Digital Library
EP49	An evaluation of continuous integration and delivery frameworks for classroom use	2021	Conferencia	ACM Digital Library
EP50	Developing Software Engineering Skills using Real Tools for Automated Grading	2018	Conferencia	ACM Digital Library
EP51	Privilege Escalation Attack Scenarios on the DevOps Pipeline Within a Kubernetes Environment	2022	Conferencia	ACM Digital Library
EP52	Test Framework for Jenkins Shared Libraries	2018	Conferencia	ACM Digital Library
EP53	Automating Side-Channel Testing for Embedded Systems: A Continuous Integration Approach	2024	Conferencia	ACM Digital Library
EP54	Introducing Computer Science Undergraduate Students to DevOps Technologies from Software Engineering Fundamentals	2024	Conferencia	ACM Digital Library
EP55	CDEP: Continuous Delivery Educational Pipeline	2017	Conferencia	ACM Digital Library
EP56	Improving traceability management through tool integration: an experience in the automotive domain	2017	Conferencia	ACM Digital Library
EP57	Cost-efficient quality assurance of natural language processing tools through continuous monitoring with continuous integration	2016	Workshop	ACM Digital Library
EP58	Software engineer education support system ALECSS utilizing DevOps tools	2016	Conferencia	ACM Digital Library
EP59	Hands-Free Research Workflow	2017	Conferencia	ACM Digital Library
EP60	A Continuous Software Quality Monitoring Approach for Small and Medium Enterprises	2017	Conferencia	ACM Digital Library
EP61	Strong agile metrics: mining log data to determine predictive power of software metrics for continuous delivery teams	2017	Conferencia	ACM Digital Library
EP62	Continuous Integration and Delivery for HPC: Using Singularity and Jenkins	2018	Conferencia	ACM Digital Library
EP63	Introducing a Deployment Pipeline for Continuous Delivery in a Software Architecture Course	2018	Conferencia	ACM Digital Library

EP64	Developing Anti-tamper Functionalities through Continuous Integration	2020	Conferencia	ACM Digital Library
EP65	The Necessity of Interdisciplinary Software Development for Building Viable Research Platforms: Case Study in Automated Drug Delivery in Diabetes	2020	Conferencia	ACM Digital Library
EP66	Detecting architectural issues during the continuous integration pipeline	2021	Conferencia	ACM Digital Library
EP67	A Cloud Architecture for the Execution of Medical Imaging Biomarkers	2019	Conferencia	SpringerLink
EP68	An Open Continuous Deployment Infrastructure for a Self-driving Vehicle Ecosystem	2016	Conferencia	SpringerLink
EP69	Automated credit assessment framework using ETL process and machine learning	2022	Journal	SpringerLink
EP70	Deployment Management and Topology Discovery of Microservice Applications in the Multicloud Environment	2021	Journal	SpringerLink
EP71	Managing to release early, often and on time in the OpenStack software ecosystem	2019	Journal	SpringerLink
EP72	Open innovation using open source tools: a case study at Sony Mobile	2018	Journal	SpringerLink
EP73	Software Product System Model: A Customer-Value Oriented, Adaptable, DevOps-Based Product Model	2021	Journal	SpringerLink
EP74	Testing and evaluation framework for virtualization technologies	2017	Journal	SpringerLink
EP75	Model-based fleet deployment in the IoT-edge-cloud continuum	2022	Journal	SpringerLink
EP76	Measuring Software Delivery Performance Using the Four Key Metrics of DevOps	2021	Journal	SpringerLink
EP77	Achieving traceability in large scale continuous integration and delivery deployment, usage and validation of the eiffel framework	2017	Journal	SpringerLink
EP78	Traceable Multi-view Model Integration: A Transformation Pipeline for Agile Production Systems Engineering	2023	Journal	SpringerLink
EP79	um-d-verification: Automation of Software Validation for the EGI Federated e-Infrastructure	2018	Journal	SpringerLink
EP80	Software Quality Assurance as a Service: Encompassing the quality assessment of software and services	2024	Journal	Science Direct
EP81	Modern Build Automation for an Insurance Company Tool Selection	2023	Conferencia	Science Direct
EP82	DevOps in Practice for Education Management Information System at ECNU	2020	Conferencia	Science Direct
EP83	Advancing 5G Network Applications Lifecycle Security: An ML-Driven Approach	2024	Journal	Science Direct
EP84	A cloud-edge based data security architecture for sharing and analysing cyber threat information	2020	Journal	Science Direct
EP85	Design and Implementation of Medical Searching System Based on Microservices and Serverless Architectures	2022	Conferencia	Science Direct
EP86	J-PET Framework: Software platform for PET tomography data reconstruction and analysis	2020	Journal	Science Direct
EP87	Open community platform for hearing aid algorithm research: open Master Hearing Aid (openMHA)	2022	Journal	Science Direct
EP88	From Network Functions to NetApps: The 5GASP Methodology	2021	Journal	Science Direct
EP89	DCA++: A software framework to solve correlated electron problems with modern quantum cluster methods	2020	Journal	Science Direct
EP90	A Method to Estimate Software Strategic Indicators in Software Development: An Industrial Application	2021	Journal	Science Direct
EP91	Programming framework and infrastructure for self-adaptation and optimized evolution method for microservice systems in cloud-edge environments	2021	Journal	Science Direct
EP92	Streamlining Application Deployment: A CI/CD Pipeline for Kubernetes	2024	Conferencia	IEEE Xplore

EP93	DeepRIoT: Continuous Integration and Deployment of Robotic-IoT Applications	2024	Conferencia	ACM
EP94	On Rapid Application Deployment with Kubernetes	2024	Conferencia	ACM
EP95	Automation Framework Foundation for Continuous Integration with Docker	2024	Conferencia	ACM
EP96	Advancing Software Security and Reliability in Cloud Platforms through AI-based Anomaly Detection	2024	Workshop	ACM
EP97	Cloud-Native CDN Monitoring Using CI/CD	2024	Conferencia	IEEE Xplore
EP98	Framework for Integrating Threat Modeling into a DevOps Pipeline for Enhanced Software Development	2024	Conferencia	IEEE Xplore
EP99	Streamlining Software Development: A Comprehensive Study on CI/CD Automation	2024	Conferencia	IEEE Xplore
EPI00	Cloud-Native Continuous Integration/Continuous Deployment (CI/CD) Pipeline	2024	Conferencia	IEEE Xplore
EPI01	A comparative analysis of Jenkins as a data pipeline tool in relation to dedicated data pipeline frameworks	2024	Conferencia	IEEE Xplore
EPI02	Maintaining Mars Rover Operations Software on a Budget	2024	Conferencia	IEEE Xplore
ERLG01	Building a Continuous Delivery Pipeline Using Jenkins	2021	Blog	Google
ERLG02	Configuring a CD pipeline for your Jenkins CI	2023	Artículo Electrónico	Google
ERLG03	Continuous Integration and Deployment using Jenkins	s.f.	White-Paper	Google
ERLG04	Real-Time CI CD Pipeline Project CI CD Pipeline Jenkins CI CD Pipeline	2023	Video	Google
ERLG05	Creating and Managing CI/CD Pipelines with Jenkins	2024	Artículo Electrónico	Google
ERLG06	JENKINS END TO END CICD Implementation with Detailed Notes BEST CICD PROJECT	2023	Video	Google
ERLG07	ASP.NET Core CI/CD on Azure Web App with Jenkins - Day One	2024	Blog	Google
ERLG08	CI/CD Pipeline Using Jenkins, Git and Maven	2023	Artículo Electrónico	Google
ERLG09	State of the art Continuous Integration and Deployment Pipeline with Jenkins, GitHub, and Docker	2019	Blog	Google
ERLG10	Jenkins Best Practices - Practical Continuous Deployment in the Real World	2018	Blog	Google
ERLG11	Build a pipeline with Jenkins, Dependency Based Build, and UrbanCode Deploy	2021	Artículo Electrónico	Google
ERLG12	CI/CD JENKINS PIPELINE	2020	White-Paper	Google
ERLG13	DAY-2 Real-Time CI CD Pipeline From Scratch with Jenkins DevOps Shack	2023	Video	Google
ERLG14	CI/CD with Jenkins: A Comprehensive Guide	2024	Blog	Google
ERLG15	Jenkins CI CD Pipeline Script Tutorial CI CD Pipeline Using Jenkins	2023	Video	Google

ERLG16	Day-3 LIVE PROJECT CI CD Pipeline with Jenkins DevOps Shack	2023	Video	Google
ERLG17	CI/CD with Jenkins on Databricks	2024	Blog	Google
ERLG18	Set up a CI/CD pipeline for cloud deployments with Jenkins	2022	Blog	Google
ERLG19	DAY-1 Real-Time CI CD Pipeline From Scratch with Jenkins DevOps Shack	2023	Video	Google
ERLG20	DAY-5 Real-Time CI CD Pipeline From Scratch with Jenkins DevOps Shack	2023	Video	Google
ERLG21	A Developer's Guide on How to Create a CI/CD Pipeline with Jenkins	2023	Blog	Google
ERLG22	Jenkins is the way to build CI/CD that fits advanced and unique use cases	2021	Succes-Story	Google
ERLG23	Getting Started with Pipelines	s.f.	White-Paper	Google
ERLG24	Jenkins is the way to build a scalable CI/CD orchestration platform	2021	Succes-Story	Google
ERLG25	Simple DevOps Project-2 CI/CD pipeline using GIT, Jenkins & Ansible	2018	Video	Google
ERLG26	Complete Jenkins Pipeline Tutorial Jenkinsfile explained	2020	Video	Google
ERLG27	30 Days Of DevOps Zero To Hero Jenkins CI CD Pipeline Day-4	2023	Video	Google
ERLG28	Pulumi CI/CD & Jenkins Pipelines	s.f.	White-Paper	Google
ERLG29	GitHub - RitheeshBaradwaj/JenkinsPipeline: A simple application is developed to understand the DevOps CI/CD Pipeline	2020	Informe	Google
ERLG30	Learn Jenkins! Complete Jenkins Course - Zero to Hero	2022	Video	Google
ERLG31	How to optimize Jenkins pipeline performance	2023	Blog	Google
ERLG32	Git	2024	White-Paper	Google
ERLG33	Implement a CI/CD Pipeline with Jenkins and Vultr Kubernetes Engine	2024	Artículo Electrónico	Google
ERLG34	CI/CD Pipeline - Hands-on	2020	White-Paper	Google
ERLG35	Publishing to Connect using Jenkins CI/CD	2024	White-Paper	Google
ERLG36	A developer's guide to CI/CD and GitOps with Jenkins Pipelines	2023	Blog	Google
ERLG37	How to deploy a Mule application with a Maven and Jenkins pipeline	2018	Blog	Google
ERLG38	Building Your First CI/CD Pipeline with Jenkins	2023	Blog	Google
ERLG39	Jenkins Pipeline: Getting Started Tutorial For Beginners [With Examples]	2024	Blog	Google
ERLG40	Jenkins is the way to facilitate day-to-day work	2020	Succes-Story	Google
ERLG41	CICD - Creating a DevOps Pipeline with Jenkins	2022	Succes-Story	Google
ERLG42	Jenkins is the way to make CI testing management easy and enjoyable	2020	Succes-Story	Google
ERLG43	How to create CI CD Pipeline Jenkins Step by Step Guide	2024	Succes-Story	Google
ERLG44	Jenkins Continuous Integration Tutorial	2023	Succes-Story	Google
ERLG45	Jenkins is the way to keep IBM always-on	2020	Succes-Story	Google
ERLG46	Jenkins Handbook	s.f.	White-Paper	Google

ERLG47	Jenkins is the way to run technical simulations for developer/engineer interviews	2021	Succes-Story	Google
ERLG48	Jenkins is the way to automate all the things	2021	Succes-Story	Google
ERLG49	enkins is the way to automate almost anything in an enterprise	2021	Succes-Story	Google
ERLG50	Jenkins is the way to keep the world spinning	2021	Succes-Story	Google
ERLG51	Jenkins is the way to faster product release	2021	Succes-Story	Google
ERLG52	Jenkins is the way to migrate telecom applications to the cloud	2021	Succes-Story	Google
ERLG53	Jenkins is the way to faster and safer shipping to clients	2021	Succes-Story	Google
ERLG54	Jenkins is the way to build and deploy ML models fast and free	2021	Succes-Story	Google
ERLG55	Jenkins is the way to achieve greater flexibility in projects & faster deployments	2021	Succes-Story	Google
ERLG56	Jenkins is the way to create temporary review application via full automation	2021	Succes-Story	Google
ERLG57	Jenkins is the way to build multi-platform NUT	2022	Succes-Story	Google
ERLG58	Facilitating Customers' Digital Transformation Based on Jenkins	2024	Succes-Story	Google
ERLG59	Jenkins for MLOps: A Complete CI/CD Tutorial	2024	Blog	Google
ERLG60	Jenkins Continuous Integration With Katalon: A Complete Guide	2024	Artículo Electrónico	Google
ERLG61	LabVIEW Continuous Integration With Jenkins/GitHub	2024	Artículo Electrónico	Google

Partiendo de los estudios previamente mencionados y con apoyo de la siguiente figura, se pudo observar que donde se recuperaron más estudios fue en la revisión de la literatura gris, seguida de la biblioteca IEEE Xplore en la literatura blanca. Esto último es de sorprender, ya que esta biblioteca se especializa en el área de Ingeniería de Software, a diferencia de las bases de datos multidisciplinarias como ACM Digital Library, SpringerLink y ScienceDirect, en las cuales se encontraron menos estudios relevantes.

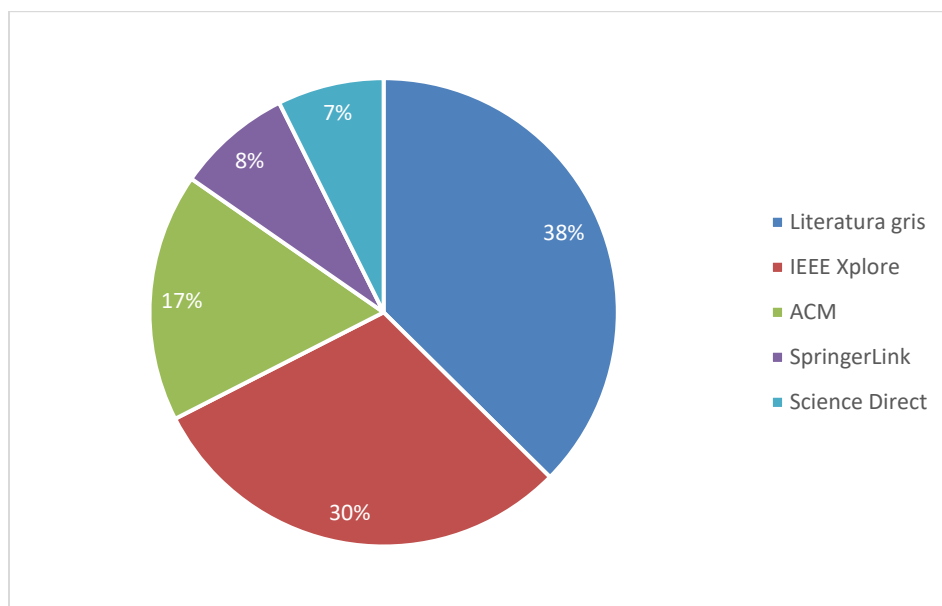


Figura 12. Distribución de estudios por base de datos

Pasando a la distribución de estudios por año, la siguiente figura ofrece un panorama útil sobre la situación. La publicación de estudios donde se reporta el uso de Jenkins se ha mantenido relevante y con un aparente incremento con el pasar de los años, con excepciones en los años 2019 y 2022, donde hubo diferencias importantes en el número de estudios respecto a los demás años.

Considerando que la herramienta fue lanzada al mercado en el 2011, es de destacar que la cantidad de estudios que hablen de ella vaya en incremento en años recientes, alcanzando el máximo en 2024 con 33 estudios. Esto habla del interés de las empresas e investigadores en el uso de herramientas de automatización, particularmente en Jenkins.

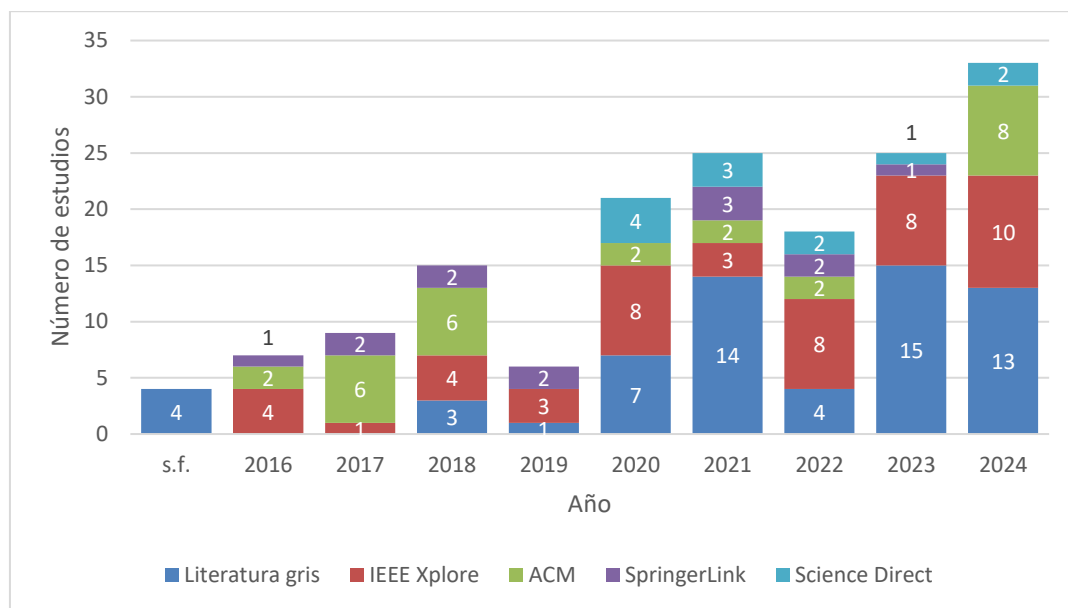


Figura 13. Distribución de estudios por años

Siguiendo con lo presentado en la figura que muestra la distribución de estudios por año, la investigación en el tema dentro de ACM fue más abundante en los primeros años (2017 y 20218). Conforme avanzó el tiempo, parece que se fue perdiendo el interés en el tema dentro de dicha biblioteca, o al menos no se publicaron muchos estudios relacionados. Sin embargo, en años recientes parece que eso está cambiando, ya que en 2024 se alcanzó el máximo de estudios publicados relacionados con el tema, en esa biblioteca (y también en general). Resaltamos esto porque podría indicar que Jenkins está recobrando importancia en el área de investigación.

Esta idea también se respalda por las tendencias en IEEE Xplore, donde antes de 2019 existía un interés por el tema, pero a partir del año 2020, la cantidad de estudios sobre Jenkins se disparó exponencialmente, casi triplicando la cantidad de estudios publicados en comparación con los primeros años. Caso similar el de ScienceDirect, pues es a partir de 2020 que se empiezan a encontrar estudios sobre Jenkins.

Pasando a la distribución de estudios por tipo de publicación, como se mencionó en la sección sobre criterios de inclusión y exclusión, en la literatura blanca solo se buscaron estudios que provinieran de *journals*, conferencias, *workshops* y revistas. En la siguiente figura, se muestra la distribución de estudios de acuerdo con estos tipos de publicaciones.

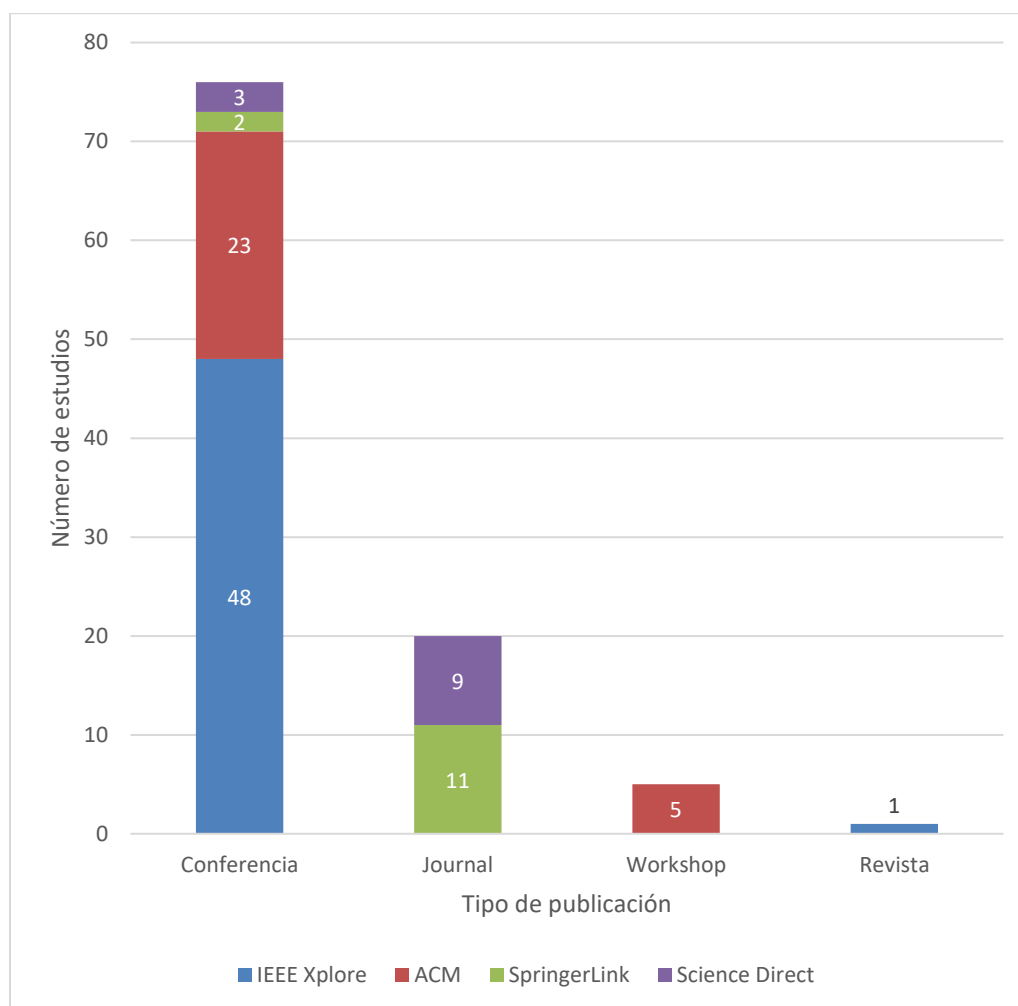


Figura 14. Distribución de estudios de la literatura blanca por tipo de publicación

Tal como se aprecia en la figura anterior, la mayoría de los estudios de la literatura blanca provienen de conferencias. Solamente un estudio proviene de una revista y es importante mencionar que dicho estudio fue identificado a partir del *Quasigold* en IEEE Xplore, pues nunca fue recuperado mediante la búsqueda automatizada. Las bases de datos de SpringerLink y ScienceDirect proporcionaron todos los estudios de *journal* y solamente en ACM Digital Library se encontraron estudios provenientes de *workshops*. Una visión más clara sobre la distribución de los estudios por cada biblioteca la ofrece la siguiente figura.

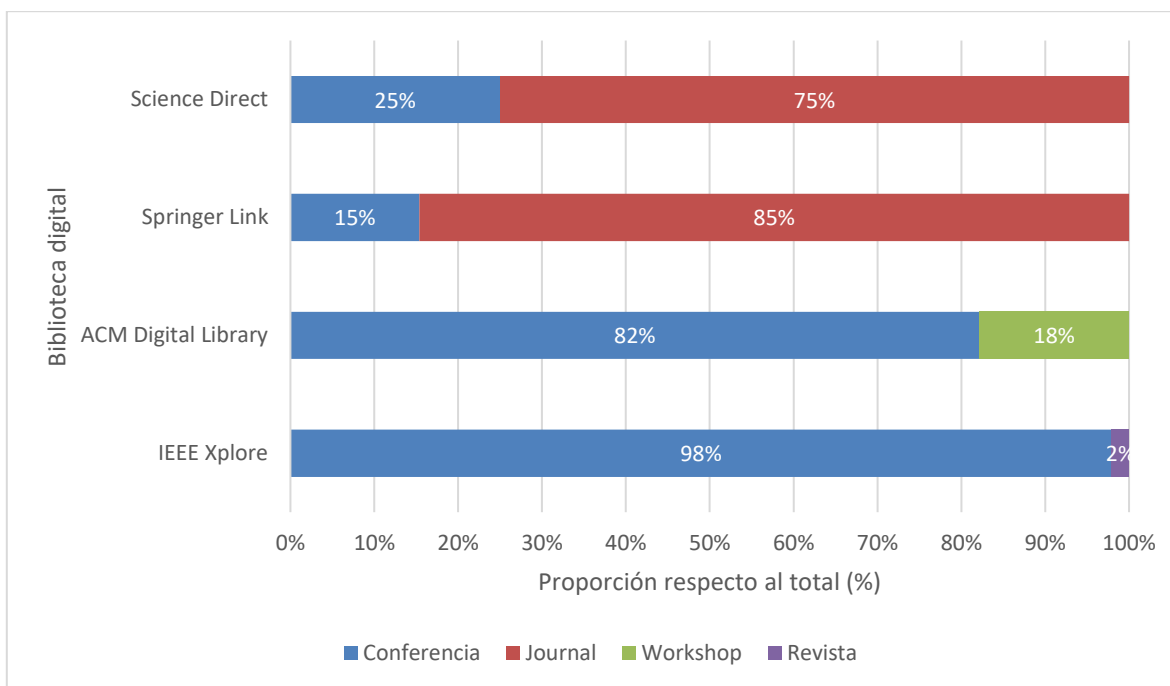


Figura 15. Proporción de tipo de publicación de estudios de la literatura blanca por biblioteca

Los resultados previos pueden revelar un bajo interés por parte de los *journals*, *workshops* y revistas por el tema de esta investigación. Sin embargo, también pone en manifiesto el considerable panorama y el creciente interés que existe entre las conferencias en relación con la investigación sobre el uso de Jenkins.

En cuanto a la distribución de estudios de la literatura gris según su tipo de publicación, la mayoría de estos (33%) provienen de las *success stories*, es decir, de la biblioteca de historias de usuarios de Jenkins. Le siguen los blogs, videos, *white papers* y artículos electrónicos con porcentajes similares entre el 10% y 20%. Los informes, en cambio, fueron los que abarcaron menos proporción del total de estudios, con solo un 2%. Esto se puede ver gráficamente a través de la siguiente figura.

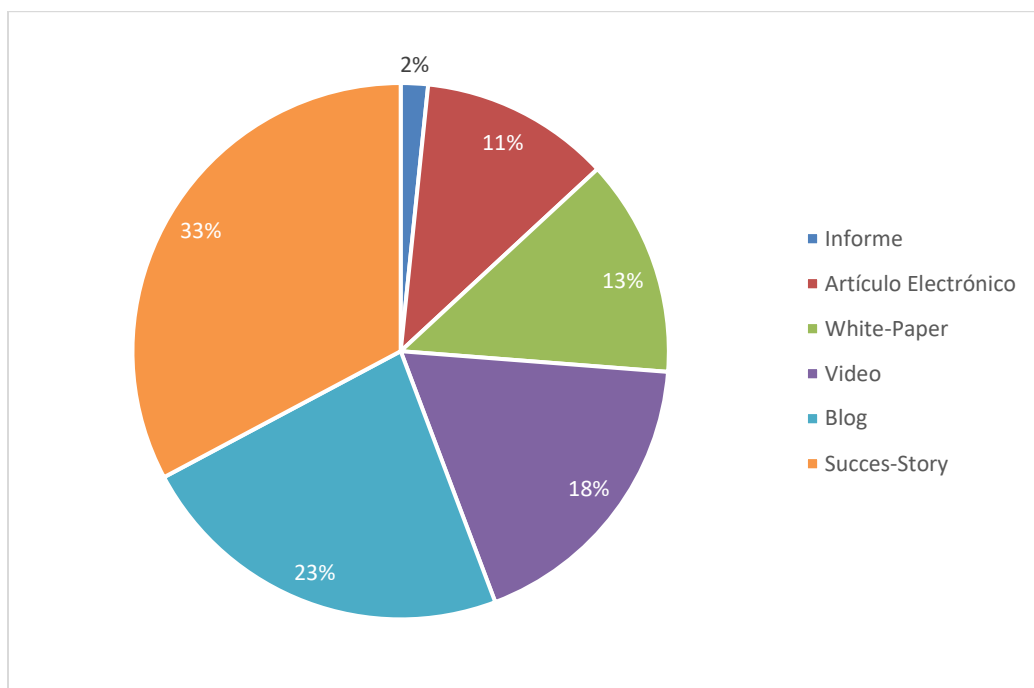


Figura 16. Distribución de estudios de la literatura gris por tipo de publicación

En general, la cantidad de estudios relevantes identificados refleja el alto interés en el tema de investigación. Pero no solo ahí, pues una porción considerable de estudios de la literatura gris proviene directamente de historias de usuarios. Los resultados confirman que el uso de Jenkins es un tema relevante en la industria de software actual.

4.2 Contextos y escenarios en los que se utiliza Jenkins para crear pipelines de integración y despliegue continuo

Esta sección tiene como propósito responder a la pregunta de investigación RQ01: **¿En qué contextos y escenarios se suele utilizar Jenkins para crear pipelines de Integración y Despliegue Continuo (CI/CD)?**

Se acordó interpretar “contexto” como toda aquella descripción del ambiente en el que se implementa el pipeline. En la mayoría de los estudios se identificó que dicho ambiente usualmente se describía a través del propósito que se le daba al pipeline de CI/CD o del estudio en sí. Tomando esto en cuenta, se lograron identificar tres contextos, presentados en la Tabla 18.

Tabla 18. Contextos de uso de Jenkins			
Contexto	Descripción	Estudios que lo mencionan	Total de estudios

Laboratorio	La implementación del pipeline de Jenkins se hizo con fines ilustrativos o propositivos. No se presenta una implementación en un ambiente real, sino en un ambiente controlado, con el fin de proponer un marco de trabajo o mostrar el uso de la herramienta y/o su integración con otras, con un fin de investigación.	EP04, EP05, EP06, EP07, EP08, EP11, EP12, EP14, EP15, EP16, EP17, EP18, EP19, EP22, EP23, EP24, EP27, EP28, EP30, EP31, EP32, EP33, EP35, EP36, EP37, EP39, EP40, EP48, EP51, EP53, EP57, EP59, EP60, EP64, EP66, EP67, EP68, EP69, EP70, EP74, EP77, EP78, EP83, EP84, EP88, EP89, EP91, EP92, EP93, EP94, EP95, EP96, EP97, EP98, EP99, EP100, EP101, ERLG04, ERLG06, ERLG07, ERLG09, ERLG11, ERLG16, ERLG18, ERLG19, ERLG20, ERLG27, ERLG29, ERLG33, ERLG34, ERLG35, ERLG36, ERLG37, ERLG39, ERLG43, ERLG59.	76
Industria	La implementación del pipeline de Jenkins se presenta como el resultado de una mejora para un sistema que se encuentra en operación. Dicha implementación tiene el fin de mostrar los resultados y/o beneficios de haber implementado el pipeline dentro de una organización o empresa.	EP02, EP03, EP09, EP10, EP20, EP21, EP26, EP34, EP41, EP42, EP56, EP61, EP62, EP71, EP72, EP73, EP75, EP76, EP79, EP80, EP81, EP82, EP85, EP86, EP87, EP90, EP102, ERLG10, ERLG12, ERLG22, ERLG40, ERLG42, ERLG44, ERLG45, ERLG47, ERLG48, ERLG49, ERLG50, ERLG51, ERLG52, ERLG53, ERLG55, ERLG56, ERLG57, ERLG58.	45
Academia	La implementación del pipeline de Jenkins se realizó con fines educativos, en un salón de clases, para un curso específico o similar.	EP29, EP45, EP49, EP50, EP54, EP55, EP58, EP63, EP65, ERLG54.	10
No responden	No se pudo identificar un contexto particular, ya sea por una mención explícita o una interpretación. En ocasiones, por la falta de implementación de un pipeline como tal.	EP01, EP13, EP25, EP38, EP43, EP44, EP46, EP47, EP52, ERLG01, ERLG02, ERLG03, ERLG05, ERLG08, ERLG13, ERLG14, ERLG15, ERLG17, ERLG21, ERLG23, ERLG24, ERLG25, ERLG26, ERLG28, ERLG30, ERLG31, ERLG32, ERLG38, ERLG41, ERLG46, ERLG60, ERLG61.	32

Como se puede observar en la Tabla 18, la mayoría de los estudios reportan el uso de Jenkins en la implementación de pipelines de CI/CD en un contexto de laboratorio, es decir, puramente con fines de investigación. Esto se puede ver de forma más clara a través de la Figura 17.

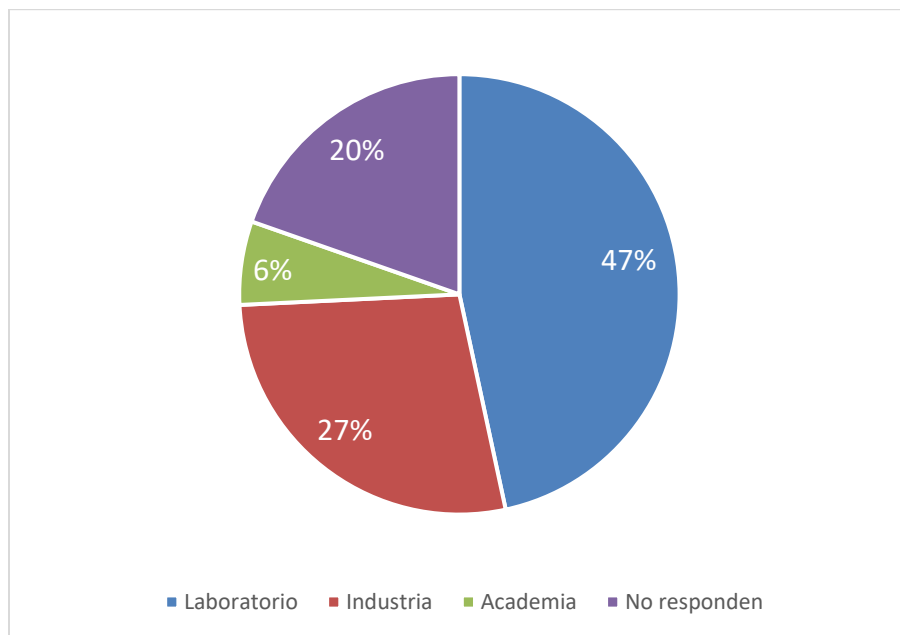


Figura 17. Contextos de uso de Jenkins para la creación de pipelines de CI/CD

La gráfica muestra que el 47% de los estudios reporta el uso de Jenkins para crear pipelines de CI/CD en contextos de laboratorio; lo anterior podría revelar el interés de la comunidad científica por el uso de la herramienta. Le sigue, con un 27%, los estudios que reportaron el uso de Jenkins en contextos de industria; esto puede revelar que la herramienta tiene relevancia en proyectos que se encuentran operando. Con un 20%, siguen los estudios que no revelan explícitamente un contexto del uso de Jenkins. Finalmente, con un 6%, se encuentran los estudios que revelan el uso de Jenkins en un contexto educativo. Esto último podría revelar la falta de material que aborde el uso de la herramienta en pipelines de integración y despliegue continuo, con un enfoque en la educación.

Ahora bien, se acordó interpretar por “escenario” el tipo de sistema al que apoyaba el pipeline de CI/CD. Por ejemplo, en un pipeline en el que se desplegaba un sistema web, se consideraba “Sistema web” como el “escenario”.

Los escenarios identificados y plasmados en la Tabla 19, se extrajeron directamente de la mayoría de los estudios, es decir, los estudios mencionaban explícitamente el tipo de sistema. Sin embargo, para definir los escenarios “Sistema basado en la nube”, “Sistema de apoyo” y “Sistemas de evaluación automatizada”, se realizaron agrupaciones de tipos de sistema mencionados en los estudios primarios, de acuerdo con su similitud. El escenario “Sistema basado en la nube” agrupó sistemas reportados como microservicios y sistemas “as a service”. Mientras que el escenario “Sistema de apoyo”, sirvió para agrupar todos aquellos

sistemas reportados en la literatura que tenían el objetivo de apoyar en la automatización de algún proceso específico, como la ejecución remota de tareas (EP23), apoyo en la trazabilidad de artefactos (EP32), marcos de trabajo (EP67), entre otros. Por otra parte, el escenario “Sistemas de evaluación automatizada”, comprende el uso de Jenkins para establecer modelos o sistemas con el único fin de analizar y evaluar de manera automática otros proyectos, artefactos o sistemas de software.

Tabla 19. Escenarios de uso de Jenkins

Escenario	Estudios que lo mencionan	Total de estudios
Sistemas Web	EP04, EP05, EP06, EP07, EP08, EP09, EP19, EP21, EP22, EP24, EP26, EP28, EP30, EP31, EP35, EP41, EP45, EP51, EP54, EP55, EP63, EP73, EP81, EP82, EP85, EP94, EP95, EP99, ERLG06, ERLG07, ERLG09, ERLG10, ERLG12, ERLG16, ERLG18, ERLG19, ERLG20, ERLG22, ERLG27, ERLG29, ERLG33, ERLG35, ERLG36, ERLG39, ERLG40, ERLG42, ERLG43, ERLG44, ERLG48, ERLG52, ERLG53, ERLG56, ERLG58.	53
Sistemas basados en la nube	EP02, EP12, EP15, EP33, EP49, EP63, EP70, EP71, EP79, EP85, EP88, EP91, EP92, EP96, ERLG45, ERLG47, ERLG49, ERLG50, ERLG51, ERLG52.	20
Sistemas AI/ML/NPL	EP14, EP27, EP37, EP57, EP59, EP69, EP83, EP93, EP96, EP100, EP101, ERLG54, ERLG59.	13
Sistemas de apoyo	EP23, EP32, EP60, EP67, EP77, EP84, EP87, EP90, EP102.	9
Software embebido	EP11, EP36, EP39, EP53, EP56, EP68, EP98, ERLG57	8
Aplicación Móvil	EP21, EP34, EP65, EP72, EP73, ERLG04, ERLG10, ERLG55.	8
Sistemas de evaluación automatizada	EP50, EP58, EP66, EP74, EP76, EP80.	6
Sistemas de IoT	EP18, EP29, EP64, EP75, EP93.	5
Sistemas Ciber-Físicos	EP16, EP42, EP48, EP78.	4
Sistemas para construir software	EP10, EP86, ERLG11.	3
Computación de alto rendimiento (HPC)	EP62, EP89.	2
Sistemas de base de datos	EP20.	1
Sistemas de información	EP17.	1
Sistemas de Entrega Multimedia	EP97.	1

Sistemas de Escritorio	ERLG34.	1
Middleware	ERLG37.	1
No responden	EP01, EP13, EP25, EP38, EP43, EP44, EP46, EP47, EP52, ERLG01, ERLG02, ERLG03, ERLG05, ERLG08, ERLG13, ERLG14, ERLG15, ERLG17, ERLG21, ERLG23, ERLG24, ERLG25, ERLG26, ERLG28, ERLG30, ERLG31, ERLG32, ERLG38, ERLG41, ERLG46, ERLG60, ERLG61, EP03, EP40, EP61.	35

Hay una tendencia marcada para los escenarios en los que se suele utilizar Jenkins en los pipelines, pues tal como lo muestra la Tabla 19, los sistemas web superan por mucho a otro tipo de sistema, con un total de 53 estudios que reportan el uso de Jenkins en un escenario de sistema web. También, es importante mencionar que hay 35 estudios en los que no se logró identificar el escenario, siendo estos en su mayoría recursos de la Literatura Gris.

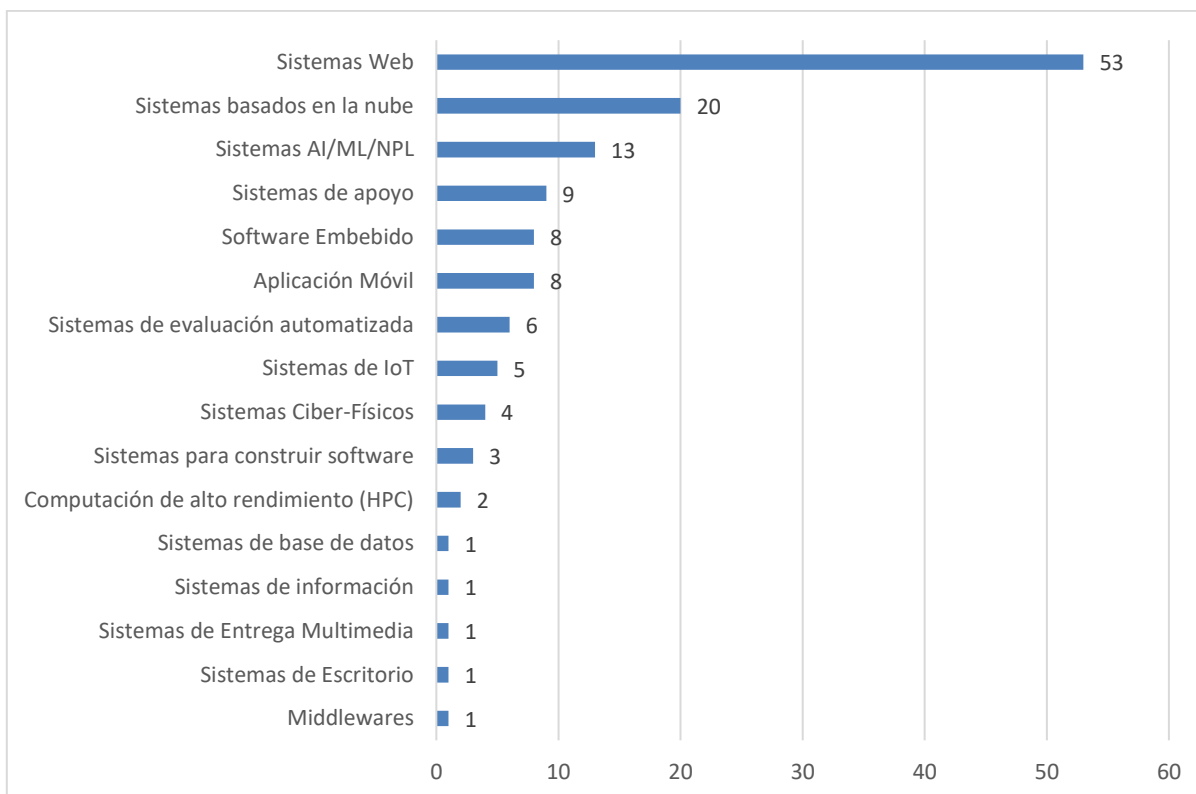


Figura 18. Escenarios de uso de Jenkins para la creación de pipelines de CI/CD

Como la figura anterior lo refleja, después de los sistemas web, los sistemas basados en la nube y los sistemas de AI/ML/NPL fueron los más reportados en los estudios, con 20 y 13 estudios que reportan dichos escenarios respectivamente. Los sistemas de apoyo se reportan un total de nueve veces. Muy de cerca le sigue los sistemas con software embebido y los sistemas de aplicaciones móviles, con ocho estudios reportando estos escenarios. seis

estudios reportaron el uso de sistemas de evaluación automatizada, mientras que cinco estudios reportan el uso de sistemas de IoT, y cuatro reportan sistemas Ciber-Físicos. Con un estudio menos, se encuentran los escenarios de sistemas para construir software, así mismo, dos estudios reportan el uso de sistemas de computación de alto rendimiento. Y Finalmente, los sistemas de información, sistemas de base de datos, sistemas de entrega multimedia, sistemas de escritorio y Middlewares solo fueron reportados en un estudio cada uno.

Es importante mencionar que un par de estudios reportaron más de un escenario. Tales fueron los casos de los estudios: EP21, donde se identificaron un sistema web y una aplicación móvil; el estudio EP63, donde se identificaron los escenarios de un sistema web y un sistema basado en la nube; el estudio EP73, donde se identificaron sistemas web y aplicación móvil; el estudio EP85, donde se identificaron sistemas web y sistemas basados en la nube; el estudio EP93, donde se identificaron sistemas de IoT y sistemas AI/ML/NPL; el estudio EP96, donde se identificaron Sistemas basados en la nube y sistemas de AI/ML/NPL; de los recursos de la literatura gris, el estudio ERLG10, donde se identificaron sistemas web y de aplicación móvil; y finalmente el estudio ERGL52, donde se identificaron sistemas web y sistemas basados en la nube. Es importante mencionar que estos escenarios fueron identificados textualmente en los estudios o se interpretaron a partir de las tecnologías, frameworks y herramientas reportadas en los estudios.

Por otra parte, con el objetivo de obtener un panorama general del uso de Jenkins en los diversos sectores productivos y de servicios, se identificaron y clasificaron los sectores o ámbitos de aplicación en el que se desarrollaron los sistemas o proyectos presentados en los estudios, estos sectores se denominaron como “Sector de aplicación” y se muestran en la siguiente Tabla.

Tabla 20. Sector de aplicación del uso de Jenkins		
Sector de Aplicación	Estudios que lo mencionan	Total de estudios

Automotriz	EP11, EP16, EP42, EP48, EP56, EP68, EP78, EP93, ERLG49.	9
Desarrollo de Software Comercial	EP02, EP10, EP34, EP71, EP90, ERLG10, ERLG56, ERLG57.	8
Telecomunicaciones	EP07, EP18, EP20, EP72, EP77, EP88, EP97, ERLG52,	8
Salud	EP30, EP65, EP67, EP75, EP85, EP86, EP87.	7
Educativo	EP50, EP54, EP55, EP58, EP63, EP82, ERLG54.	7
Investigación	EP62, EP79, EP80, EP81, EP89, ERLG12, ERLG40.	7
Financiero	EP21, EP61, EP69, EP73, ERLG11, ERLG19, ERLG58.	7
CiberSeguridad	EP64, EP83, EP96, EP98, ERLG51.	5
Aeronáutica	EP36, EP39, EP48, EP102.	4
Gestión Empresarial	ERLG22, ERLG45, ERLG47.	3
e-commerce	ERLG16, ERLG42, ERLG55.	3
Gubernamental	EP41, EP76.	2
Agropecuario	EP26, EP29.	2
Consultoría y Análisis de Datos	EP03, ERLG53.	2
Proyectos Personales	ERLG48.	1
Ambiental	ERLG44.	1
Turismo	EP08.	1
No responden	EP01, EP13, EP25, EP38, EP43, EP44, EP46, EP47, EP52, ERLG01, ERLG02, ERLG03, ERLG05, ERLG08, ERLG13, ERLG14, ERLG15, ERLG17, ERLG21, ERLG23, ERLG24, ERLG25, ERLG26, ERLG28, ERLG30, ERLG31, ERLG32, ERLG38, ERLG41, ERLG46, ERLG60, ERLG61, EP04, EP05, EP06, EP09, EP12, EP14, EP15, EP17, EP19, EP22, EP23, EP24, EP27, EP28, EP31, EP32, EP33, EP35, EP37, EP40, EP45, EP49, EP51, EP53, EP57, EP59, EP60, EP66, EP70, EP74, EP84, EP91, EP92, EP94, EP95, EP99, EP100, EP101, ERLG04, ERLG06, ERLG07, ERLG09, ERLG18, ERLG20, ERLG27, ERLG29, ERLG33, ERLG34, ERLG35, ERLG36, ERLG37, ERLG39, ERLG43, ERLG50, ERLG59.	55

Como se puede observar no existe una tendencia marcada hacia alguno de los sectores descritos en la tabla anterior, a pesar de que el sector automotriz es el que más estudios reporta el uso de Jenkins, otros ámbitos como las Telecomunicaciones, área de la Salud, sector Financiero, de Investigación, y comercial también presentan una frecuencia similar respecto al uso de la herramienta Jenkins, estas frecuencias permiten inferir que Jenkins es una herramienta que se utiliza en una amplia variedad de sectores, tanto comerciales, educativos o de investigación, y que la herramienta no está confinada al área de desarrollo de software comercial.

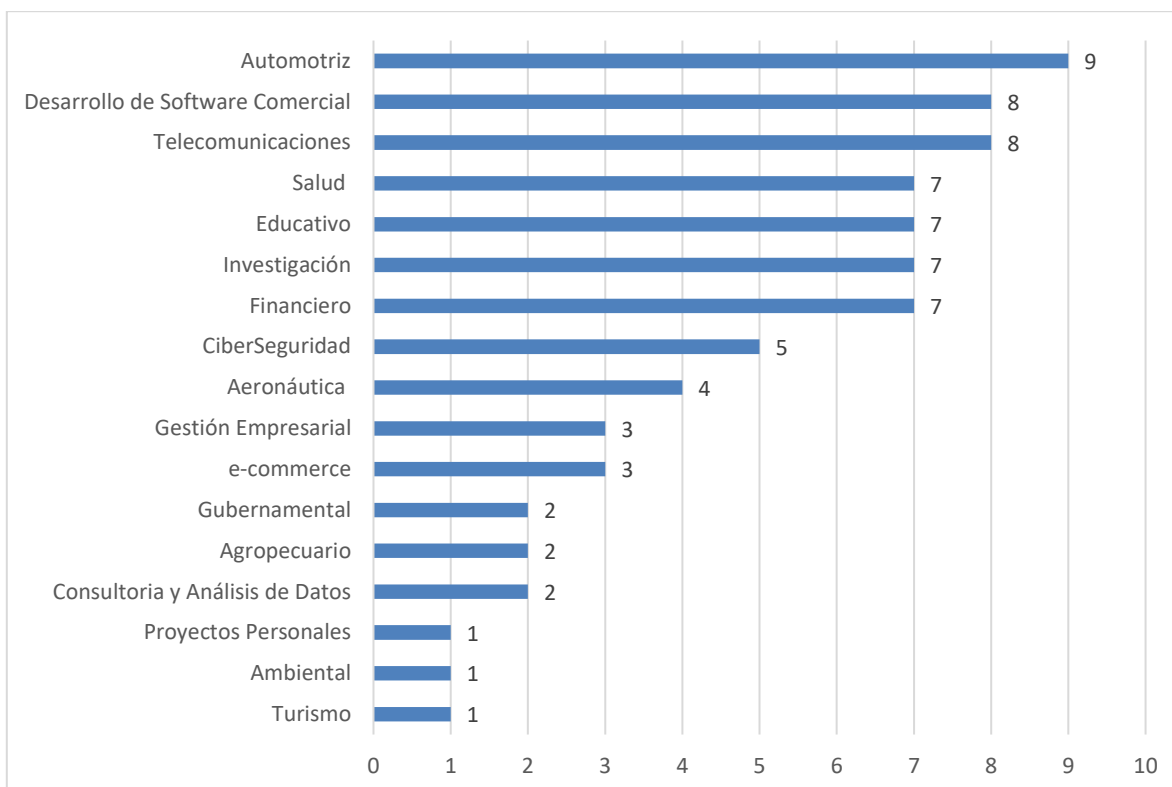


Figura 19 Sectores de aplicación de Jenkins

Como se puede observar en la Figura anterior, el sector automotriz es el que más se reporta el uso de Jenkins con un total de nueve estudios, seguido muy de cerca con un estudio menos, los sectores de Telecomunicaciones y Desarrollo de Software Comercial, con siete estudios cada uno les siguen el área de la salud, el ámbito educativo, de investigación, y el Financiero. Cinco estudios reportaron el uso de Jenkins en el campo de la ciberseguridad, mientras que cuatro estudios utilizaron Jenkins para dar soporte a software de aeronáutica, los ámbitos de gestión empresarial y de e-commerce obtuvieron una frecuencia de tres, mientras que el sector Gubernamental, agropecuario y de Consultoría y análisis de datos obtuvieron una frecuencia de dos. Finalmente, únicamente un estudio respectivamente reportó el uso de Jenkins para proyectos personales, ambientales y de Turismo.

4.3 Herramientas complementarias a Jenkins para crear pipelines de integración y despliegue continuo

En la presente sección se da respuesta a la pregunta de investigación **RQ02: ¿Cuáles son las herramientas complementarias que suelen utilizarse junto con Jenkins para crear los pipelines de Integración y despliegue Continuo (CI/CD)?**

Con el fin de dar respuesta a esta pregunta, se recopilaron y agruparon las herramientas reportadas en los estudios que se utilizaron en conjunto con Jenkins para crear pipelines de integración y despliegue continuo. Estas herramientas se clasificaron según el objetivo que cumplen dentro del pipeline.

También se realizó un ejercicio de trazabilidad para los *plugins* que se reportaron en los estudios, aquellos *plugins* que pudieron ser trazados a una herramienta concreta fueron incluidos en las estadísticas finales.

Además, es importante mencionar que casi todos los estudios mencionan más de una herramienta, solamente 19 estudios no reportan el uso de alguna herramienta complementaria, mientras que el resto mencionan una o más herramientas que caen en alguna de las categorías (clústeres) propuestas.

A continuación, en la Tabla 21 se describen los clústeres que se definieron y los estudios que reportan el uso de herramientas complementarias.

Tabla 21. Herramientas complementarias con Jenkins			
Contexto	Descripción	Estudios que lo mencionan	Total de estudios
Sistema de control de versiones (SCV)	Herramientas que permiten la gestión y control del código fuente que se utiliza en el pipeline, ya sea de manera colaborativa o local, y que funciona como disparador de los procesos del pipeline.	EP02, EP05, EP07, EP09, EP10, EP11, EP13, EP14, EP15, EP16, EP17, EP18, EP19, EP20, EP21, EP22, EP24, EP25, EP26, EP31, EP32, EP33, EP34, EP35, EP39, EP49, EP50, EP51, EP53, EP54, EP55, EP58, EP59, EP60, EP61, EP65, EP66, EP67, EP68, EP71, EP72, EP76, EP80, EP82, EP86, EP87, EP89, EP92, EP93, EP94, EP95, EP99, EP100, EP102, ERLG01, ERLG02, ERLG03, ERLG04, ERLG06, ERLG07, ERLG08, ERLG09, ERLG10, ERLG12, ERLG15, ERLG17, ERLG18, ERLG19, ERLG20, ERLG21, ERLG22, ERLG24, ERLG28, ERLG32, ERLG34, ERLG35, ERLG36, ERLG39, ERLG40, ERLG41, ERLG42, ERLG43, ERLG45, ERLG46, ERLG47, ERLG48, ERLG49, ERLG50, ERLG51, ERLG52, ERLG53, ERLG54, ERLG55, ERLG56, ERLG57, ERLG58, ERLG61.	97

Contenerización y orquestación	Herramientas que permiten realizar prácticas de Contenerización de software. Y herramientas que permiten gestionar múltiples contenedores.	EP02, EP04, EP05, EP07, EP08, EP09, EP12, EP14, EP18, EP21, EP22, EP24, EP25, EP26, EP27, EP31, EP32, EP33, EP37, EP39, EP48, EP49, EP51, EP54, EP55, EP62, EP63, EP65, EP67, EP68, EP76, EP80, EP81, EP82, EP85, EP91, EP93, EP94, EP95, EP97, EP99, EP100, EP101, ERLG03, ERLG04, ERLG06, ERLG09, ERLG12, ERLG13, ERLG15, ERLG16, ERLG18, ERLG19, ERLG20, ERLG22, ERLG24, ERLG27, ERLG33, ERLG36, ERLG40, ERLG45, ERLG47, ERLG49, ERLG52, ERLG54, ERLG56, ERLG58.	67
Gestión de compilación y paquetes	Herramientas que permiten la creación de software y su empaquetado en versiones en alguna de las etapas del pipeline.	EP07, EP08, EP24, EP31, EP58, EP81, EP85, EP87, EP89, EP95, ERLG01, ERLG02, ERLG03, ERLG04, ERLG06, ERLG08, ERLG13, ERLG15, ERLG16, ERLG21, ERLG22, ERLG27, ERLG34, ERLG37, ERLG39, ERLG40, ERLG42, ERLG43, ERLG45, ERLG48, ERLG49, ERLG51, ERLG52, ERLG53, ERLG54, ERLG55, ERLG56, ERLG57, ERLG58.	39
Infraestructura	Herramientas que dan soporte a los procesos y funcionamiento del pipeline.	EP07, EP12, EP15, EP21, EP24, EP25, EP26, EP28, EP33, EP39, EP55, EP61, EP67, EP70, EP71, EP73, EP85, EP97, EP99, EP100, EP101, ERLG06, ERLG09, ERLG12, ERLG17, ERLG18, ERLG24, ERLG25, ERLG28, ERLG37, ERLG39, ERLG41, ERLG43, ERLG49, ERLG55.	35
Pruebas	Herramientas que se usan en alguna de las etapas del pipeline para ejecutar los casos de prueba del software que se está procesando en el pipeline.	EP11, EP18, EP30, EP35, EP36, EP38, EP41, EP45, EP46, EP47, EP53, EP54, EP55, EP56, EP58, EP61, EP63, EP68, EP74, EP84, EP87, EP88, ERLG01, ERLG11, ERLG39, ERLG42, ERLG46, ERLG52, ERLG60.	29
Revisión de código	Herramientas que se usan en alguna de las etapas del pipeline para analizar el código fuente del software que se está procesando en el pipeline.	EP10, EP41, EP50, EP54, EP55, EP57, EP58, EP60, EP61, EP71, EP72, EP73, EP77, EP82, EP84, EP95, EP97, EP99, EP102, ERLG04, ERLG06, ERLG11, ERLG13, ERLG15, ERLG16, ERLG19, ERLG20, ERLG27.	28
Repositorio de artefactos	Herramientas que permiten el almacenamiento de artefactos de software creados durante alguna de las etapas del pipeline, tales como: binarios, bibliotecas, paquetes, reportes, etc.	EP02, EP04, EP06, EP11, EP13, EP19, EP20, EP21, EP49, EP61, EP67, EP73, EP76, EP77, EP93, EP94, EP100, ERLG11, ERLG16, ERLG29, ERLG40.	21
Monitoreo	Herramientas que permiten a los desarrolladores conocer el estatus de los procesos y etapas del pipeline.	EP01, EP07, EP11, EP15, EP19, EP21, EP33, EP71, EP73, EP74, EP75, EP76, EP82, EP85, EP86, EP91, EP97, EP100, ERLG10, ERLG31.	20
Gestión de la configuración	Herramientas que ayudan a hacer más eficiente la gestión de la configuración del pipeline, contribuyendo a la consistencia y el control.	EP05, EP06, EP07, EP17, EP19, EP22, EP70, EP73, EP75, EP79, ERLG25, ERLG40, ERLG49, ERLG51.	14

Herramientas de Seguridad	Herramientas que realizan pruebas o escaneos de seguridad con el fin de identificar vulnerabilidades, amenazas o ataques que puedan comprometer la integridad tanto del pipeline como del software desplegado.	EP54, EP61, EP79, EP84, EP97, EP98, ERLG04, ERLG13, ERLG15, ERLG16, ERLG19, ERLG20, ERLG27.	13
Notificaciones	Herramientas que envían notificaciones, correos, o mensajes a los equipos de trabajo involucrados en los procesos del pipeline.	EP07, EP09, EP20, EP27, EP54, ERLG10, ERLG46, ERLG48, ERLG52, ERLG53.	10
Virtualización	Herramientas que permiten la virtualización de recursos físicos durante alguna de las etapas del pipeline.	EP22, EP63, EP68, EP74, EP94.	5
Otros	Herramientas con objetivos de uso específico que no se pudieron categorizar en ninguna de los clústeres anteriores.	EP20, EP32, EP34, EP56, EP64, EP66, EP67, EP69, EP74, EP76, EP83, EP86, EP100, ERLG46.	14
No responde	No se pudo identificar ninguna herramienta, ya sea porque no se reporta su uso, o se menciona el uso de herramientas fuera del alcance de la implementación del Pipeline.	EP03, EP23, EP29, EP40, EP42, EP43, EP44, EP52, EP78, EP90, EP96, ERLG05, ERLG14, ERLG23, ERLG26, ERLG30, ERLG38, ERLG44, ERLG59.	19

Como se puede observar en la tabla anterior, se establecieron un total de doce categorías en las que recaen una o más herramientas de software que se utilizaron en conjunto con Jenkins, cabe mencionar que el clúster nombrado como “otros” agrupa una diversidad de herramientas que pueden no tener relación entre sí, y que se encuentran en esta categoría debido a que no se identificaron suficientes herramientas similares como para crear un clúster nuevo, entre las herramientas que se agruparon en dicha categoría, se encuentran dos ofuscadores de código (Stunnix y Starfoce), un marco de trabajo de ingeniería inversa (Modisco), un gestor del ciclo de vida de aplicaciones (Polarion ALM), una gestor de bases de datos de grafos (Neo4j), un marco de trabajo de liberación de aplicaciones móviles (Fastlane), un servicio de directorios en red (Active Directory), un balanceador de cargas (HAProxy), un gestor de esquemas de bases de datos (Liquidbase), un integrador de datos (Informatica), una librería de script interactivos (Expect), una plataforma de big data (Splunk), un servicio distribuido de datos (OpenSlice), una plataforma de Machine Learning (SageMaker), un proxy inverso (ngrok) y un generador de documentación (Doxygen).

En la Figura 20. se muestra gráficamente la distribución de las herramientas reportadas en la literatura.

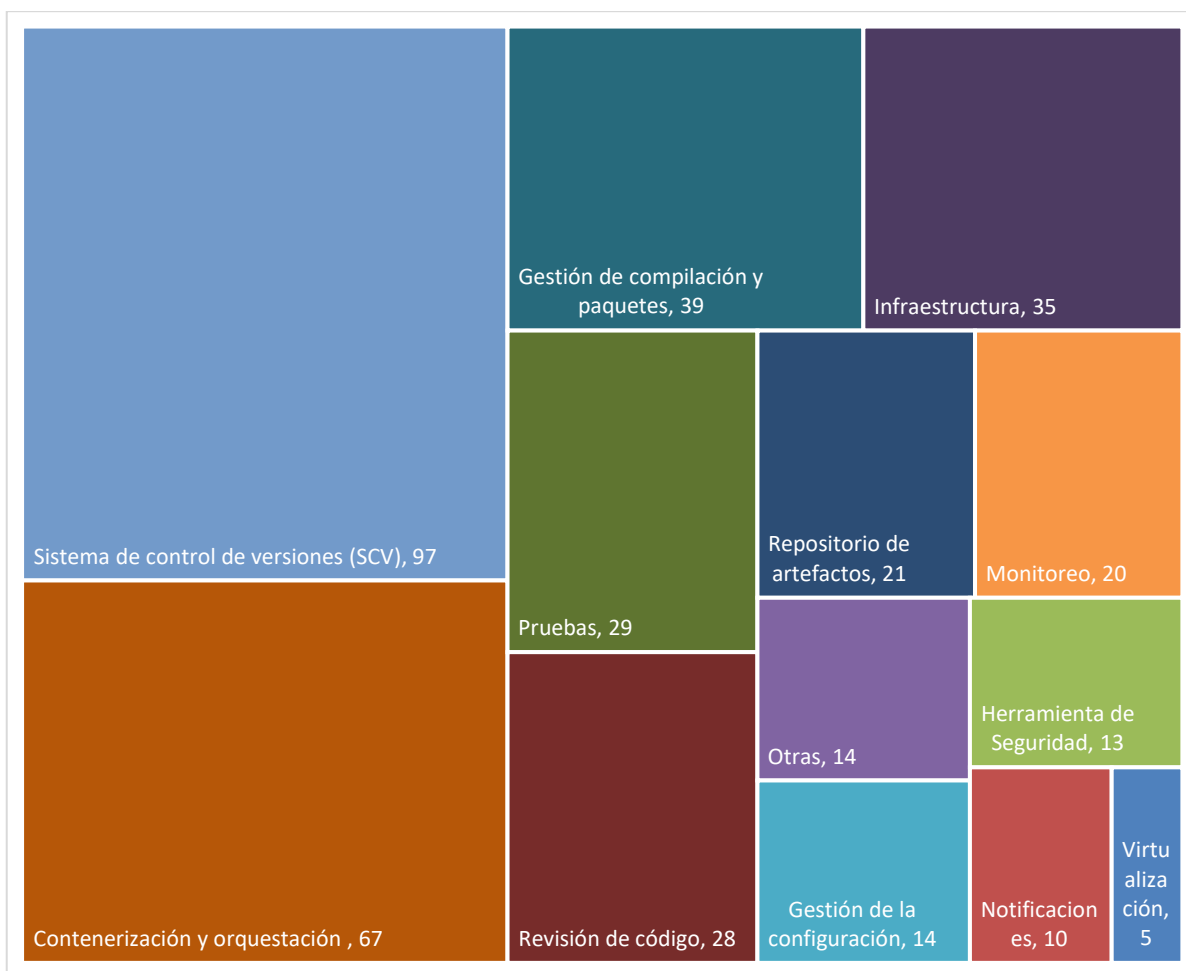


Figura 20. Herramientas complementarias a Jenkins para la creación de pipelines de CI/CD

Se puede observar que el uso de sistemas de control de versiones es el tipo de herramienta que más predomina al crear pipelines de CI/CD, el motivo de este hecho es que los pipelines de integración tiene como fin recuperar, integrar y verificar los cambios en el código fuente, para lo que se requiere necesariamente algún tipo de sistema de control de versiones, además de que normalmente este tipo de herramientas funcionan como el punto de partida y disparador de los procesos de los pipelines integración y despliegue continuo,

específicamente en Jenkins, este monitorea constantemente estos sistemas en busca de cambios o *commits* en el código fuente.

El segundo tipo de herramienta que más se reporta en la literatura con 67, son las herramientas de Contenerización y orquestación, tales como Docker, Singularity y Docker Compose. 39 estudios reportan el uso de herramientas para la compilación y empaquetado del código, mientras que otros 35 estudios hacen énfasis el uso de herramientas de infraestructura que dan soporte a alguno de los procesos del pipeline. 29 estudios reportan el uso de herramientas de pruebas, ya sean para ejecutar pruebas unitarias, de integración, aceptación, rendimiento u otro tipo. Por otra parte, 29 estudios reportan el uso de herramientas de revisión de código para detectar *smells* de código y bugs. Muy de cerca y con un estudio menos se encuentran las herramientas para almacenar artefactos de software producidos durante la ejecución de los procesos del pipeline. Mientras que veinte estudios utilizaron alguna herramienta para monitorear alguna de las etapas del pipeline. También, catorce estudios mencionan el uso de alguna herramienta para ayudar a la gestión de la configuración. Por otra parte, trece estudios hacen uso de herramientas de seguridad para escanear y detectar vulnerabilidades en el pipeline y en el código. 10 estudios reportan el uso de herramientas para enviar notificaciones a los miembros involucrados en el pipeline. Y únicamente cinco estudios mencionan el uso de herramientas de virtualización de recursos físicos.

Como se mencionó previamente, el tipo de herramienta complementaria con Jenkins más utilizada para la creación de pipelines de integración y despliegue continuo son los sistemas de control de versiones. En la Tabla 22 se muestran los estudios que reportaron el uso de un sistema de control de versiones.

Tabla 22. Sistemas de control de versiones		
SCV	Estudios que lo mencionan	Total de estudios
Git	EP02, EP05, EP07, EP10, EP11, EP14, EP15, EP16, EP18, EP19, EP20, EP21, EP22, EP24, EP25, EP26, EP31, EP32, EP33, EP34, EP35, EP39, EP49, EP50, EP51, EP53, EP54, EP55, EP58, EP59, EP60, EP61, EP65, EP66, EP67, EP68, EP71, EP72, EP76, EP80, EP82, EP86, EP87, EP89, EP92, EP93, EP94, EP95, EP99, EP100, EP102, ERLG01, ERLG02, ERLG03, ERLG04, ERLG06, ERLG07, ERLG08, ERLG09, ERLG10, ERLG12, ERLG15, ERLG17, ERLG18, ERLG19, ERLG20, ERLG21, ERLG22, ERLG24, ERLG28, ERLG32, ERLG34, ERLG35, ERLG36, ERLG39, ERLG40, ERLG41, ERLG42, ERLG43, ERLG45, ERLG46, ERLG47, ERLG48, ERLG49, ERLG50, ERLG51, ERLG52,	94

	ERLG53, ERLG54, ERLG55, ERLG56, ERLG57, ERLG58, ERLG61.	
Apache Subversion	EP09, EP13, EP17	3

Además del sistema control de versiones, 74 de los 94 estudios que mencionan el uso de Git también mencionan el uso de alguna plataforma de gestión de repositorios para interactuar con Git. En la Tabla 23 se muestran dichos estudios.

Tabla 23. Plataformas de gestión de repositorios		
SCV	Estudios que lo mencionan	Total de estudios
GitHub	EP05, EP07, EP15, EP32, EP33, EP34, EP49, EP50, EP51, EP54, EP55, EP59, EP65, EP66, EP67, EP68, EP80, EP86, EP89, EP94, EP95, EP99, EP100, EP102, ERLG04, ERLG09, ERLG12, ERLG18, ERLG21, ERLG22, ERLG24, ERLG28, ERLG35, ERLG36, ERLG39, ERLG41, ERLG43, ERLG45, ERLG46, ERLG47, ERLG48, ERLG50, ERLG51, ERLG52, ERLG54, ERLG56, ERLG57, ERLG58, ERLG61.	49
GitLab	EP02, EP11, EP14, EP18, EP21, EP26, EP39, EP53, EP60, EP61, EP82, EP93, ERLG40, ERLG45, ERLG46, ERLG49, ERLG51, ERLG58.	18
Bitbucket	EP76, ERLG42, ERLG46, ERLG53, ERLG55.	5
Gitea	ERLG46.	1
Azure Repos	ERLG35.	1

A pesar de que existen varias herramientas de este tipo, en la literatura revisada predomina el uso de Git, abarcando más del 90% de los estudios que reportan un SCV. En la Figura 21. se pudo observar cuantos estudios de los 44 identificados hacen uso de un sistema Git, y en la Figura 22 se muestran las plataformas de control de versiones y colaboración basadas en Git que se reportaron en los estudios.

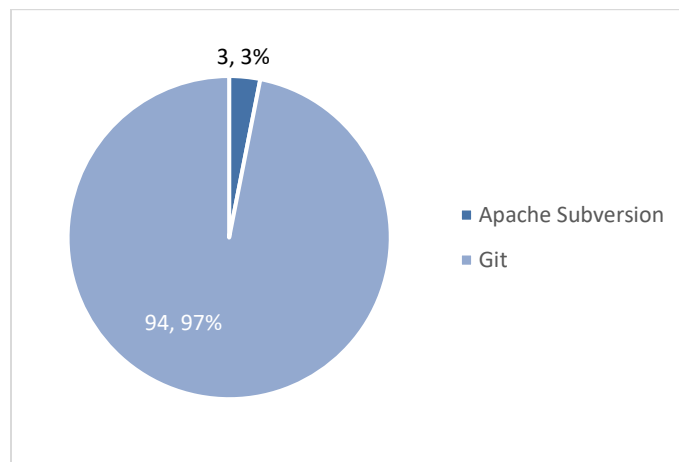


Figura 21. Sistemas de control de versiones

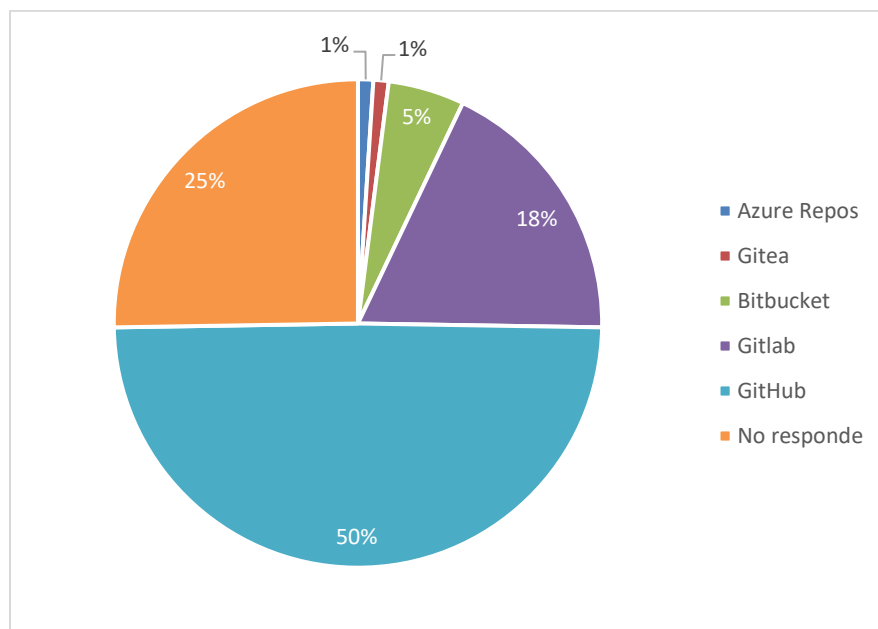


Figura 22. Plataformas de colaboración y gestión de versiones basadas en Git

Es bastante notorio que el sistema control de versiones más usado con Jenkins es Git, en comparación con Apache subversión cuyo uso se reportó únicamente tres veces y en ninguna de estas se menciona si se usa una herramienta de interfaz gráfica para interactuar con el SCV.

Por otro lado, de los 94 estudios que reportan el uso de Git, 49 hacen uso de GitHub mientras que 18 usan GitLab, cinco utilizan Bitbucket, y tan solo un estudio reportó el uso de Gitea y Azure Repos, también existen 25 estudios que, aunque mencionan el uso de Git, no hacen mención explícita de haber utilizado GitHub, GitLab o cualquier otra plataforma para interactuar con el SCV.

Otra de las herramientas más utilizadas en conjunto con Jenkins son las herramientas de contenerización, se identificaron 67 estudios que reportan el uso de herramientas que apoyan a las prácticas de contenerización. En la Tabla 24. Se pueden observar dichas herramientas.

Tabla 24. Herramientas de contenerización		
Herramienta	Estudios que lo mencionan	Total de estudios
Singularity	EP62	1
Docker	EP02, EP04, EP05, EP08, EP09, EP12, EP14, EP18, EP21, EP22, EP24, EP26, EP27, EP31, EP32, EP33, EP37, EP39, EP48, EP49, EP51, EP54, EP55, EP62, EP63, EP65, EP67, EP68, EP80, EP81, EP82, EP85, EP91, EP93, EP94, EP95, EP97, EP99, EP100, ERLG03, ERLG04, ERLG06, ERLG09, ERLG12, ERLG13, ERLG15, ERLG16, ERLG19, ERLG20, ERLG22, ERLG24, ERLG27, ERLG36, ERLG40, ERLG45, ERLG47, ERLG49, ERLG52, ERLG54, ERLG56, ERLG58.	61
Docker Registry	EP02, EP08, EP33, EP67, EP81	5
GitLab Registry	EP26	1
Docker Hub	EP51, ERLG06, ERLG12, ERLG15, ERLG16, ERLG20.	6
Kubernetes	EP05, EP07, EP08, EP21, EP24, EP25, EP33, EP37, EP39, EP51, EP81, EP82, EP91, EP94, EP97, EP100, EP101, ERLG06, ERLG18, ERLG22, ERLG24, ERLG33, ERLG40, ERLG45, ERLG47, ERLG49, ERLG54, ERLG56, ERLG58.	29
DockerCompose	EP08, EP09, EP80, ERLG13, ERLG19.	5
OpenShift	EP76	1

Como se puede observar, hay estudios que reportan el uso de más de una herramienta de contenerización, y en aquellos estudios que utilizan Docker Hub, Docker Registry, Docker Compose o Kubernetes normalmente también usan Docker. En la Figura 23 se muestra la distribución de las herramientas previamente mencionadas.

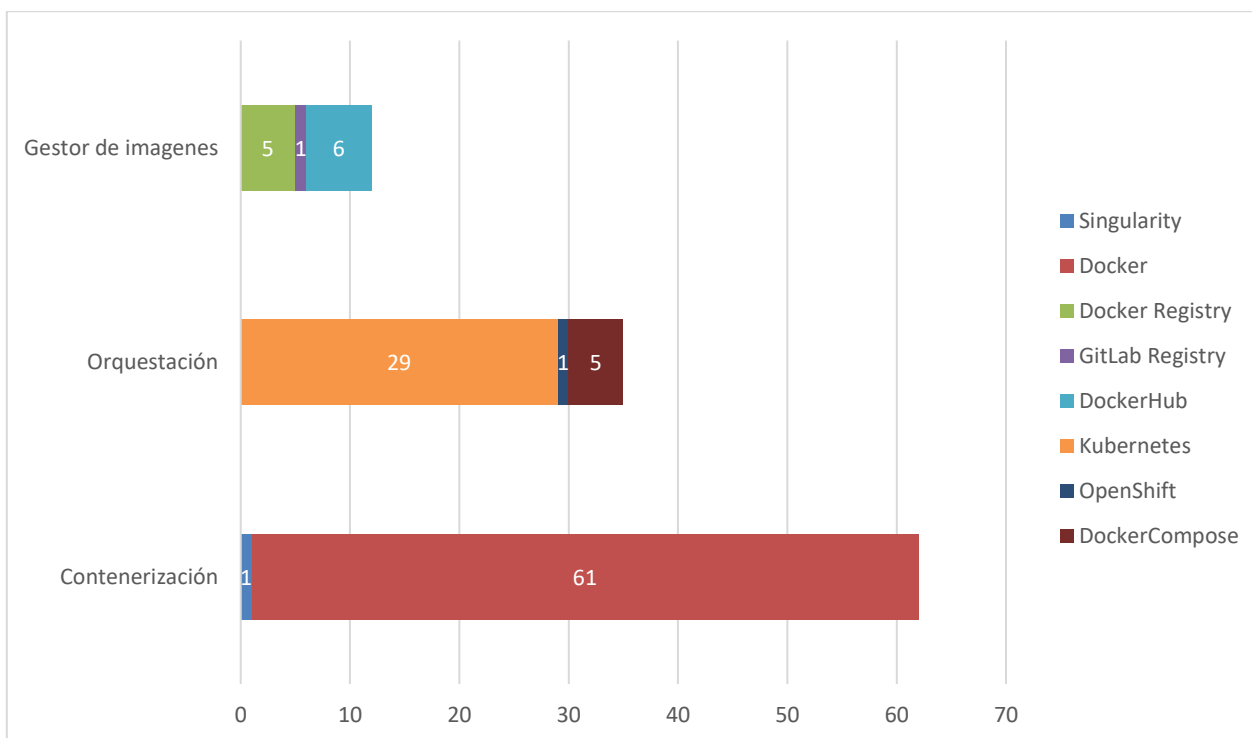


Figura 23. Herramientas de contenerización

Como se puede observar en la Figura anterior, de las dos herramientas para realizar la contenerización, Docker es la más usada con 61 menciones, mientras que Singularity solo se utilizó en un solo estudio (en el cual también se usó Docker). De los 34 estudios que además utilizan un orquestador de contenedores, 29 utilizaron Kubernetes, cinco mencionan haber utilizado Docker Compose (del cual uno también utilizó Kubernetes) y solamente un estudio menciona OpenShift. Finalmente, doce estudios reportan el uso de herramientas para gestionar las imágenes de los contenedores, de los cuales, el más usado es DockerHub con seis estudios, seguido de Docker Registry con cinco, y GitLab Registry lo cual solo se reporta una vez.

De los 163 estudios primarios son 39 los que hacen mención explícita del uso de una herramienta de compilación. Las herramientas: EasyBuild, Rake, MSBuild, Packer, Poetry, Rails, Autotools, y GCC/CLANG son solamente mencionados una sola vez, mientras que GNU Make es reportada un total de tres veces, Apache Ant y Gradle un total de cuatro veces y finalmente, Apache Maven es utilizado en 28 estudios distintos como se muestra en la Tabla 25.

Tabla 25. Herramientas de gestión de la compilación y empaquetado.

Herramienta	Estudios que lo mencionan	Total de estudios
Apache Maven	EP07, EP08, EP24, EP31, EP81, EP85, EP95, ERLG02, ERLG03, ERLG04, ERLG06, ERLG08, ERLG15, ERLG16, ERLG21, ERLG22, ERLG27, ERLG34, ERLG37, ERLG39, ERLG42, ERLG43, ERLG48, ERLG51, ERLG52, ERLG53, ERLG55, ERLG58.	28
Gradle	ERLG13, ERLG40, ERLG55, ERLG58.	4
Apache Ant	EP58, ERLG45, ERLG51, ERLG53.	4
GNU Make	EP87, ERLG57, ERLG58.	3
GCC/CLANG	ERLG57.	1
Autotools	ERLG57.	1
Rails	ERLG56.	1
Poetry	ERLG54.	1
Packer	ERLG49.	1
MSBuild	ERLG49.	1
Rake	ERLG45.	1
EasyBuild	EP89.	1

El uso de estas herramientas se ve influenciado en gran parte por lenguaje de programación que se usó para codificar el software que se está procesando en el pipeline, esto se pudo entender debido a que en los estudios mencionados se hace alusión a software escrito en Java. Además, de que el plugin de la herramienta Maven es parte de los plugins “iniciales” que se recomiendan al instalar Jenkins por primera vez, lo que explicaría porque su uso es tan masivo en comparación con los otros compiladores.

El cuarto tipo de herramientas que más se reportó en los estudios fueron aquellas que dan soporte a la infraestructura del pipeline y que ayudan a Jenkins con tareas de despliegue. En la Tabla 26 se enumeran los estudios que reportan el uso de estas herramientas y en la Figura 24 se muestra la frecuencia de uso de las herramientas de infraestructura identificadas.

Tabla 26. Herramientas de Infraestructura del pipeline

Herramienta	Estudios que lo mencionan	Total de estudios
AWS	EP07, EP12, EP24, EP25, EP55, EP67, EP70, EP85, EP97, EP99, EP100, EP101, ERLG06, ERLG12, ERLG25.	15
Plataforma Azure	EP15, EP25, EP28, EP100, ERLG28, ERLG39, ERLG43, ERLG49.	8
Terraform	EP25, ERLG09, ERLG18, ERLG24, ERLG49.	5
ArgoCD	EP33, EP97, ERLG06.	3
Google Cloud Plataform	EP25, EP100.	2
GitLab CI	EP12, EP26,	2
CloudHub	ERLG37.	1
Diawi	ERLG55.	1
Urban Code Deploy	ERLG41.	1
OracleCloud	ERLG18.	1
Apache Zookeeper	EP39.	1
Pulumi	ERLG28.	1
DataBricks	ERLG17.	1
Minio	EP21.	1
Nolio	EP61.	1
Ivalidate	EP61.	1
Hashicorp Nomad	EP67.	1
Infrastructure Manager	EP67.	1
CLUES	EP67.	1
XebiaLabs	EP73.	1
Zuul	EP71.	1
OpenStack	EP70.	1

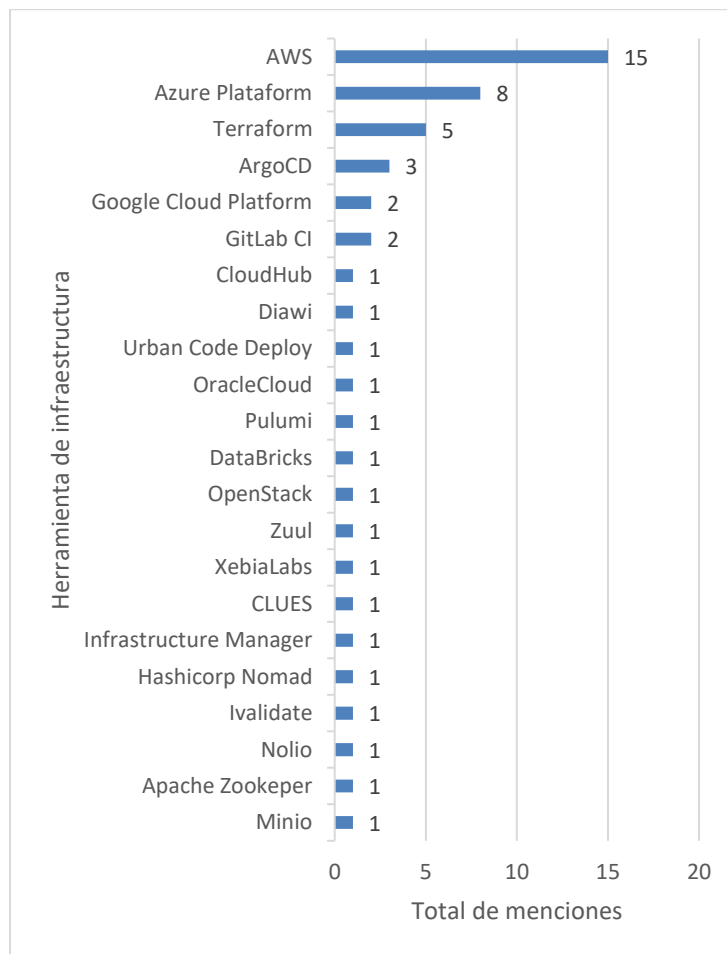


Figura 24. Herramientas de infraestructura del pipeline

Las principales herramientas que se identificaron son AWS y Azure los cuales se utilizaron debido a que ofrecen los servicios de infraestructura necesarios para hospedar las aplicaciones desarrolladas en los estudios. Otras herramientas como Terraform, ArgoCD, también se utilizaron de manera considerable en conjunto con Jenkins. Por otro lado, también se utilizaron una amplia variedad de herramientas como Nolio, Urban Code Deploy, Zuul, Pulumi para asistir a Jenkins en tareas de despliegue. Mientras que utilizaron herramientas como OpenStack, DataBrick, OracleCloud, o Minio para proporcionar a Jenkins la infraestructura en la nube requerida.

Una de las actividades que más se suele automatizar en lo pipelines de integración y despliegue continuo son las pruebas, como se puede observar en la Figura 25, son 29 los estudios que reportan el uso de al menos una herramienta de pruebas, y en casos específicos se reportan hasta ocho herramientas de pruebas en un solo estudio (EP54). En la Figura 25. se muestran la frecuencia de las herramientas de pruebas identificadas.

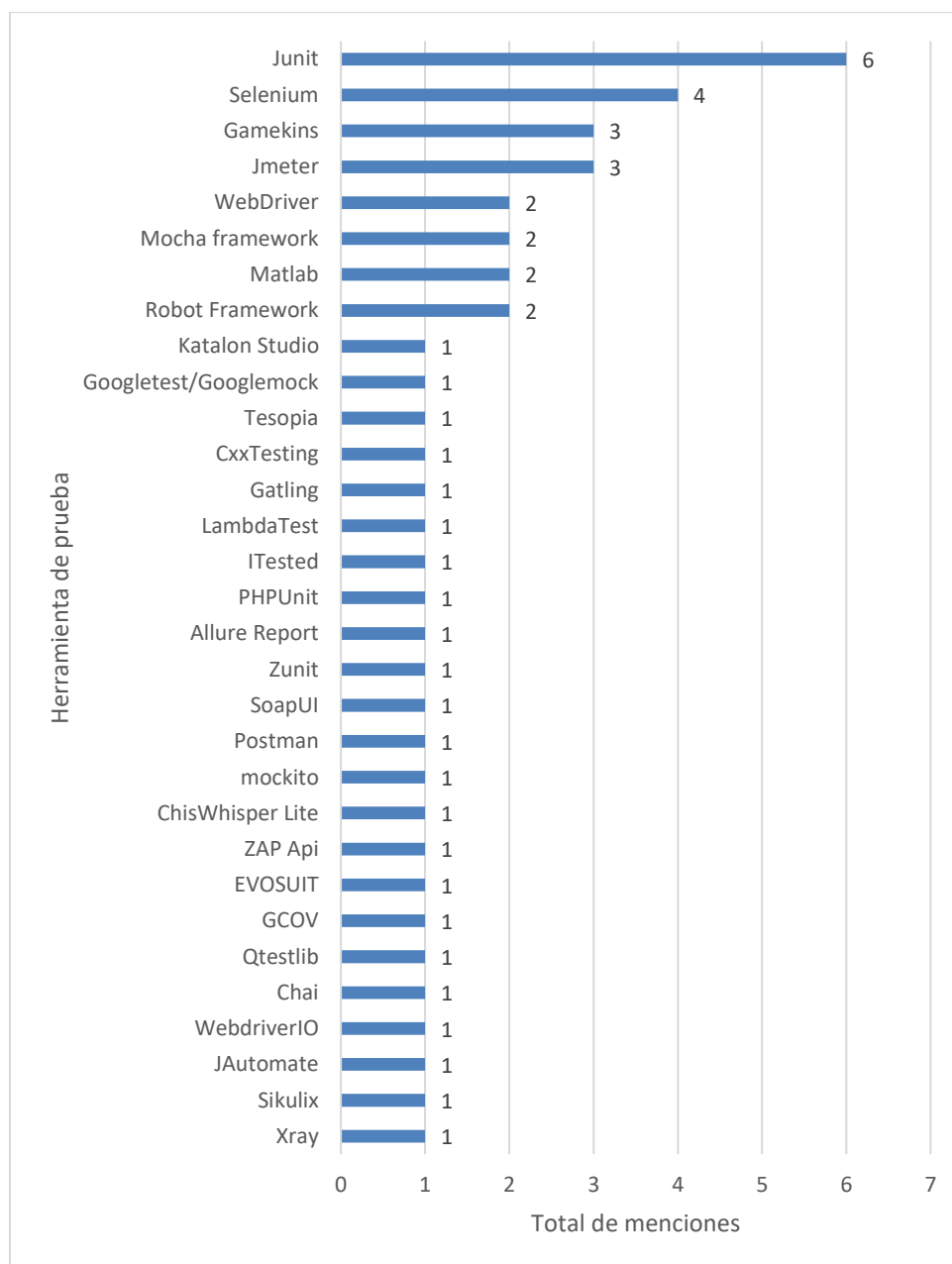


Figura 25. Herramientas de Prueba

Con base en la gráfica de la Figura 25 se puede ver que la mayoría de las herramientas de pruebas reportadas solo se mencionan en un único estudio, con excepción de ocho: Junit,

seis veces mencionada, utilizada para ejecutar pruebas unitarias; Selenium, cuatro veces reportada, la cual es utilizada en los estudios para pruebas funcionales y de aceptación; Gamekins, tres veces mencionada, es una herramienta para la generación de casos de pruebas; JMeter, reportada tres veces para pruebas de carga; y finalmente Robot Framework, MatLab, WebDriver, Mocha Framework son reportados en dos estudios cada una.

También se encontró una cantidad considerable herramientas de análisis de código tal como lo muestra la Tabla 27. Cabe aclarar que de los 28 estudios en los que se identificaron el uso de estas herramientas, hay algunos que reportan el uso de más de una herramienta como es el caso del estudio EP50.

Tabla 27. Herramientas de análisis de código.		
Herramienta	Estudios que lo mencionan	Total de estudios
SonarQube	EP41, EP54, EP57, EP60, EP61, EP73, EP82, EP97, EP99, EP102, ERLG04, ERLG06, ERLG13, ERLG15, ERLG16, ERLG19, ERLG20, ERLG27.	18
Gerrit	EP10, EP71, EP72, EP77.	4
CheckStyle	EP50, EP58, EP84, EP95.	4
FindBugs	EP50, EP58, EP84.	3
PMD	EP50.	1
SpotBugs	EP50.	1
PHPCodeSniffer	EP55.	1
CheckMarx	EP73.	1
Cobertura	EP84.	1
IDZ Code Review	ERLG11.	1

En total se identificaron diez herramientas de análisis de código distintas. En el caso específico del estudio EP50 se utilizan cuatro de estas herramientas en un curso de ingeniería de software con alumnos, es por esta razón que se mencionan tantas herramientas de análisis

de código en dicho estudio. En la Figura 26, se puede ver la tendencia de uso de dichas herramientas.

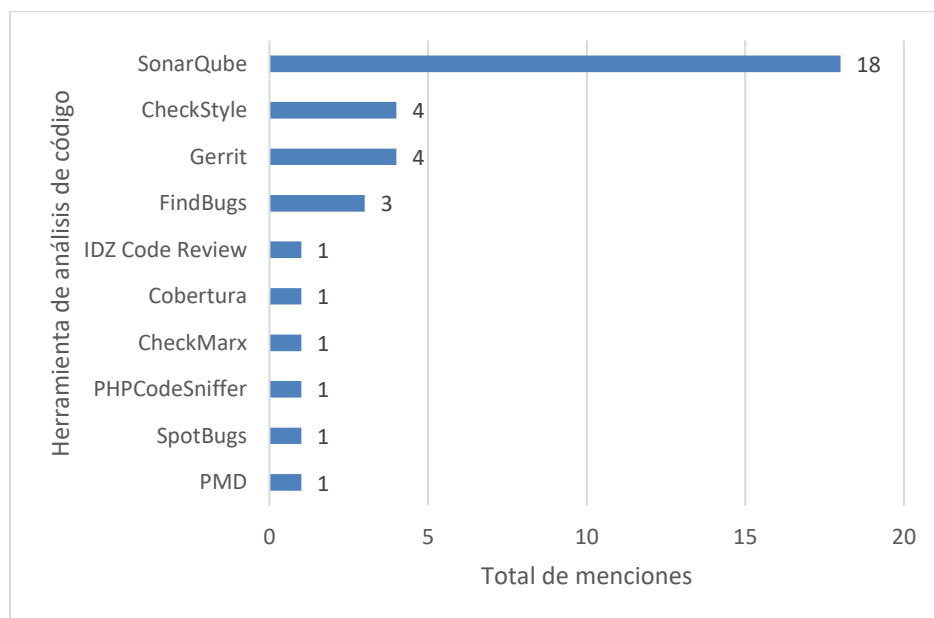


Figura 26. Herramientas de análisis de código

Es evidente que SonarQube es la herramienta más utilizada, esto se puede deber a su popularidad, así como a su facilidad de integración con distintas tecnologías como Jenkins. Y es probablemente que debido a esto las demás herramientas se mencionen en menor medida, ya que SonarQube descarta la necesidad de usar otra herramienta de análisis de código. Aun así, herramientas como Gerrit, CheckStyle, y FindBugs parecen tener cierta popularidad.

Para el almacenamiento de artefactos de software, se identificaron solamente seis herramientas en 21 estudios, los estudios que reportan el uso de estas herramientas se pueden consultar en la Tabla 28.

Tabla 28. Repositorios de artefactos		
Herramienta	Estudios que lo mencionan	Total de estudios
Jfrog Artifactory	EP02, EP04, EP06, EP11, EP49, EP61, EP76, EP77, ERLG11, ERLG29, ERLG40.	11
Nexus	EP13, EP19, EP20, EP21, EP73, EP77, ERLG16.	7
Harbor	EP21, EP93.	2
Aliyun	EP94.	1
Azure Blob Storage	E EP100.	1

Azure Files	EP67.	I
-------------	-------	---

De los 21 estudios que hacen uso de repositorios de artefactos, dos en particular reportan el uso de dos herramientas, el estudio EP21 hace uso de Harbor para almacenar las imágenes de contenedores, mientras que usa Nexus para almacenar otros artefactos de software. Así mismo, el estudio EP77 reporta el uso de dos repositorios de artefactos, sin embargo, no detalla en que usa cada uno específicamente.

En la Figura 27. Se muestran las cifras de uso de cada una de estas herramientas.

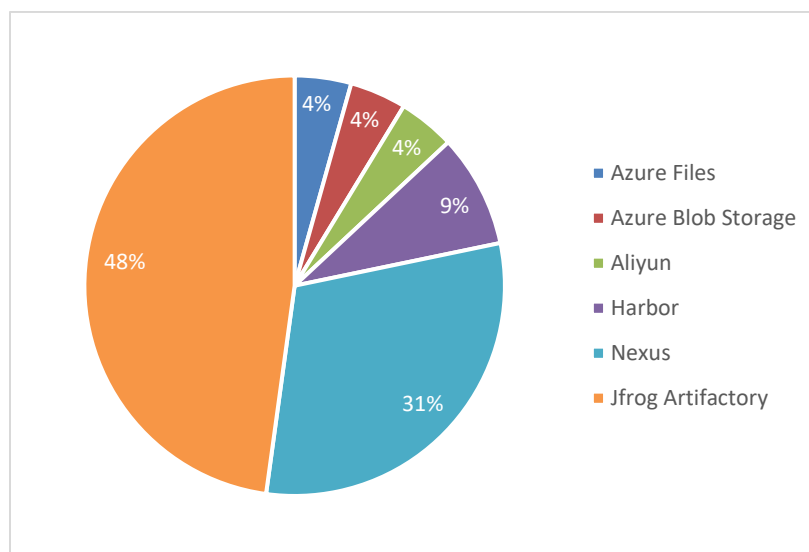


Figura 27. Repositorios de artefactos

Como se puede observar, Jfrog Artifactory es el repositorio de artefactos más utilizado, aunque no en todos los estudios se menciona explícitamente que tipo de artefactos de software se almacenan, algunos de los que se mencionan son: imágenes de Docker, nuevas actualizaciones y plantillas de proyectos. Mientras que los estudios que menciona a Nexus lo utilizan para almacenar Logs y archivos ejecutables. Por otro lado, solamente un estudio menciona el uso de Azure Blob Storage para guardar archivos pesados, y otro estudio utiliza Azure files.

En cuanto a herramientas de monitoreo, se encontraron resultados en veinte estudios que reportan la automatización de actividades de monitoreo en ciertas etapas del proceso del pipeline. En la Tabla 29 se muestran los estudios identificados, mientras que en la Figura 28 se muestra la distribución de uso de las herramientas reportadas.

Tabla 29. Herramientas de monitoreo.		
Herramienta	Estudios que lo mencionan	Total de estudios
Logstash	EP71, EP82, EP85, EP91.	4
Jira	EP11, EP21, EP73, EP76.	4
Grafana	EP15, EP33, EP97, ERLG31.	4
Prometheus	EP21, EP33, EP97, ERLG31.	4
Elasticsearch	EP82, EP85.	2
Kibana	EP82, EP85.	2
CloudWatch	EP07, EP100.	4
New Relic	ERLG10.	1
Sensu	ERLG10.	1
BART	EP01.	1
Nagios	EP19	1
Zabbix	EP21	1
Cacti	EP74	1
Azure IoT Edge	EP75	1
Redmine	EP86	1

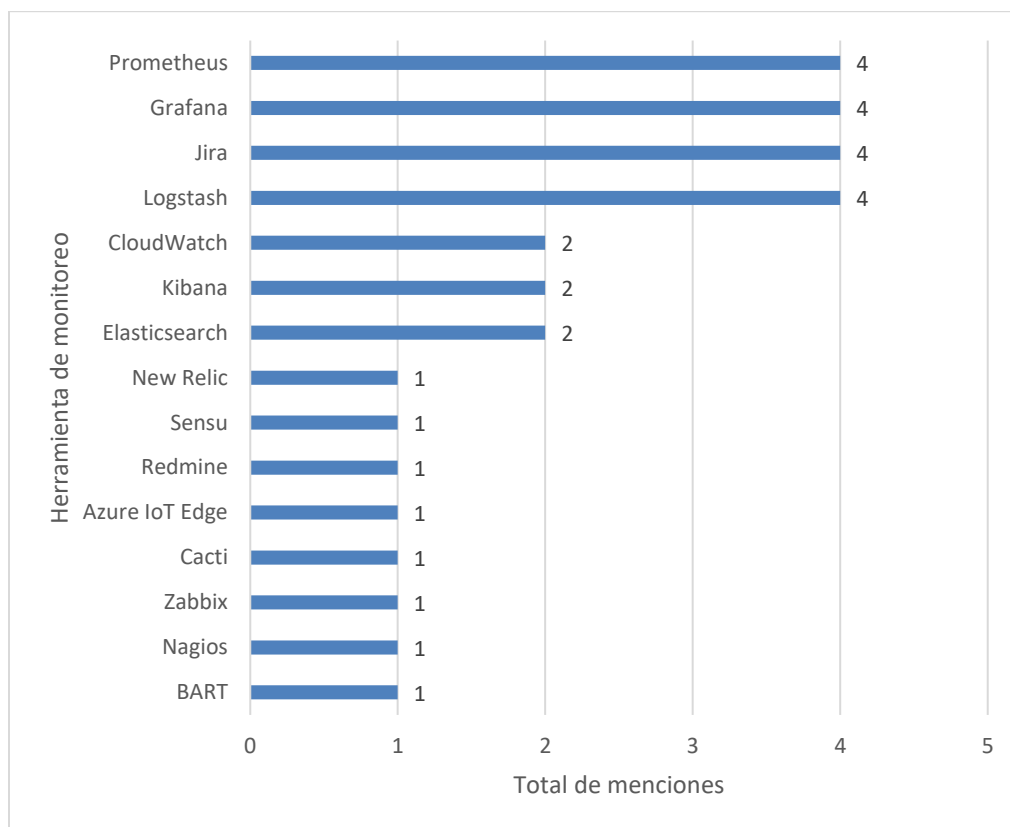


Figura 28. Distribución de uso de herramientas de monitoreo

Logstash, Jira, Grafana y Prometheus, son las principales herramientas de monitoreo que se reportan en los estudios para el monitoreo del pipeline con cuatro menciones cada una, seguido de otras tres herramientas: Elasticsearch, Kibana, CCloudWatch las cuales se mencionan tres veces cada una. Es importante mencionar que Logstash, ElasticSearch y Kibana se utilizaron en ambos estudios (EP82, EP85) en conjunto como parte de un sistema de gestión de logs *open-source*. Por otra parte, el uso de Jira se enfocó en monitorear todos los procesos del pipeline en el que se integraron. Así mismo, el estudio EP33, menciona el uso de Grafana y de Prometheus con el fin de monitorear métricas y el estatus de los contenedores de Kubernetes, que se despliegan durante la ejecución del pipeline. Las demás herramientas reportadas, se utilizan igualmente para monitorear una o más etapas de la

ejecución del pipeline y proveer de información a los desarrolladores de cualquier inconveniente que suceda durante la ejecución.

En cuanto a las herramientas para la gestión de la configuración, son catorce los estudios que mencionan el uso de alguna herramienta de este tipo. De los catorce estudios que reportan el uso de una de estas herramientas, trece utilizan Ansible y tan solo un estudio reporta el uso de Team Foundation Server, mientras que otro estudio menciona el uso de Puppet (Dicho estudio también usa Ansible). En la Tabla 30 se muestran las herramientas reportadas. Y en la Figura 29 se puede ver la distribución de uso de estas.

Tabla 30. Herramientas para la gestión de la configuración		
Herramienta	Estudios que lo mencionan	Total de estudios
Ansible	EP05, EP06, EP07, EP17, EP19, EP22, EP70, EP75, EP79, ERLG25, ERLG40, ERLG49, ERLG51.	13
Team Foundation Server	EP73	1
Puppet	EP79	1

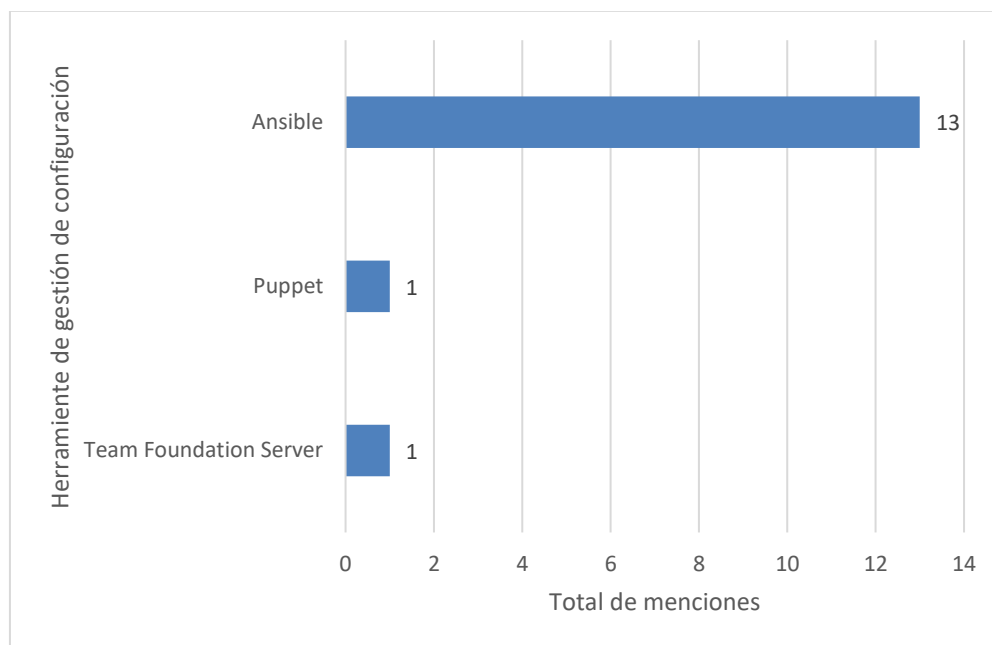


Figura 29. Herramientas para la Gestión de la configuración

Como se puede observar en la gráfica anterior, Ansible es claramente la principal herramienta de gestión de la configuración que se usa con Jenkins. Debido a que esta es una herramienta con una gran variedad de funciones, varios de los estudios reportan su uso para

dar soporte a diversas actividades de despliegue y automatización de tareas. Por otra parte, Team Foundation Server y Puppet solo se reportaron en una ocasión cada una.

En lo que respecta a la seguridad del pipeline, son solamente trece estudios los que mencionan alguna herramienta de seguridad para realizar pruebas, escaneos de vulnerabilidades o chequeos de dependencias. En la Tabla 31 se puede visualizar las herramientas reportadas en cada estudio, y en la Figura 30 se muestra la frecuencia de uso de estas.

Tabla 31. Herramientas de Seguridad		
Herramienta	Estudios que lo mencionan	Total de estudios
OWASP Dependency Check	EP84, EP97, ERLG04, ERLG13, ERLG15, ERLG16, ERLG19, ERLG20, ERLG27.	11
Trivy	EP97, ERLG04, ERLG15, ERLG19, ERLG20.	7
OWASP zed Attack proxy	EP54.	2
Qwasp	EP61.	1
Glue-validator	EP79,	1
OWASP Threat Dragon	EP98.	1

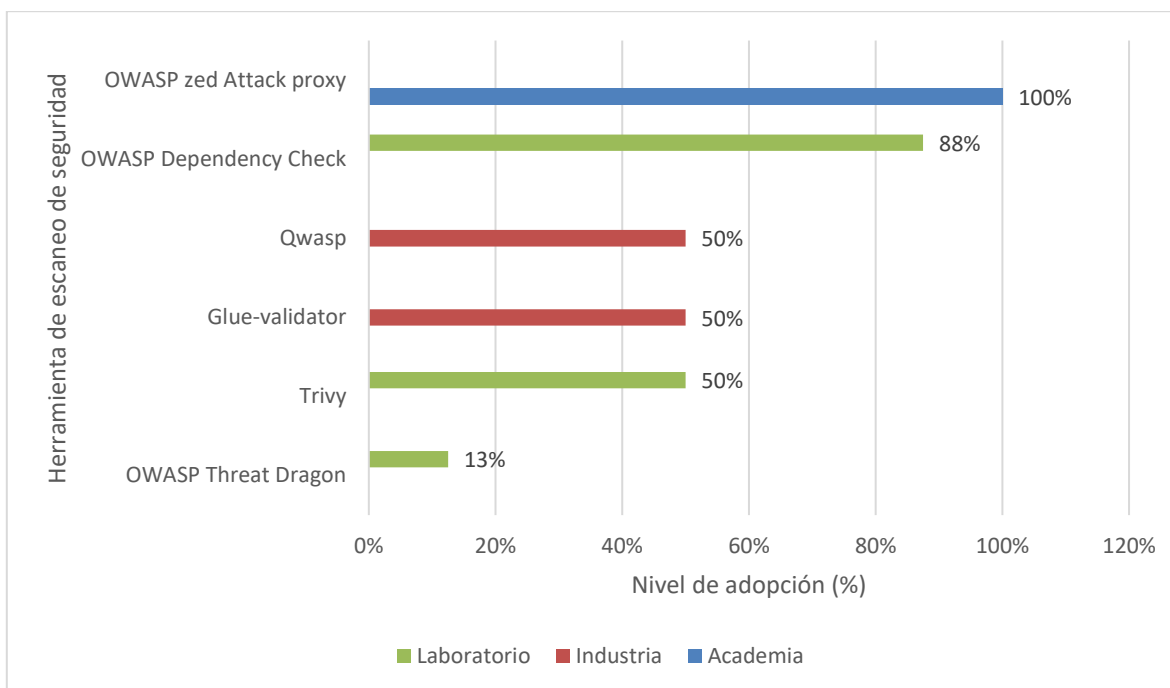


Figura 30. Herramientas de Seguridad

Como se puede observar en la tabla anterior, las herramientas de seguridad que más se integran con Jenkins son desarrolladas por la organización OWASP, esto se puede deber a que dicha organización tiene una amplia variedad de herramientas de seguridad y cuenta con un alto reconocimiento y reputación a nivel mundial, lo que garantiza la calidad de sus productos. La popularidad en la adopción de la herramienta OWASP Dependency Check se debe a que su respectivo plugin se encuentra entre los plugins “iniciales” que Jenkins recomienda descargar al instalarlo por primera vez. Por otra parte, la segunda herramienta que más se menciona es Trivy la cual depende del uso de Docker, ya que es una herramienta para analizar vulnerabilidades en las imágenes de Docker, debido al amplio uso de Docker es que también se utiliza en buena medida Trivy.

En cuanto a las herramientas para enviar notificaciones, son diez los estudios (EP07, EP09, EP20, EP27, EP54, ERLG10, ERLG46, ERLG48, ERLG52, ERLG53) que envían notificaciones sobre el estado del pipeline a los desarrolladores interesados. En el estudio EP09 se logra esto mediante la integración de *plugins* para enviar notificaciones por medio de WhatsApp y Telegram, además el estudio ERLG53 también usa Telegram, mientras que los estudios EP54, ERLG10, ERLG46, ERLG48, ERLG52 utilizaron Slack para enviar notificaciones sobre el ciclo completo del pipeline, por otra parte el estudio ERLG48 hace uso de la herramienta ChatOps para el envío de notificaciones. El resto de los estudios (EP07, EP20, EP27) mencionan el uso de un plugin de SMTP para recibir notificaciones mediante

correo electrónico sobre logs de errores, reportes de éxito, y el estado del pipeline. En la Figura 31 se pudo observar la distribución del uso de estas herramientas.

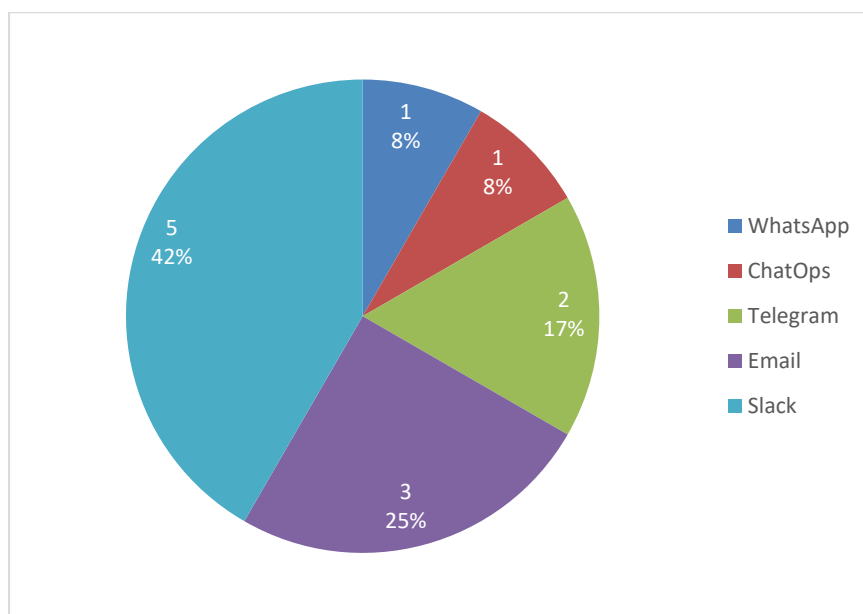


Figura 31. Herramientas de notificaciones

Finalmente, solamente son cinco los estudios: EP22, EP63, EP68, EP74 y EP94, que mencionan el uso de alguna tecnología para la virtualización de recursos físicos, en la Tabla 32 se muestran las tecnologías identificadas, y en la Figura 32 se muestra la distribución grafica del uso de estas herramientas.

Tabla 32. Herramientas de gestión de la compilación y empaquetado.		
Herramienta	Estudios que lo mencionan	Total de estudios
VirtualBox	EP22, EP63, EP68, EP94.	4
Vagrant	EP22, EP63,	2
libvirt	EP74,	1

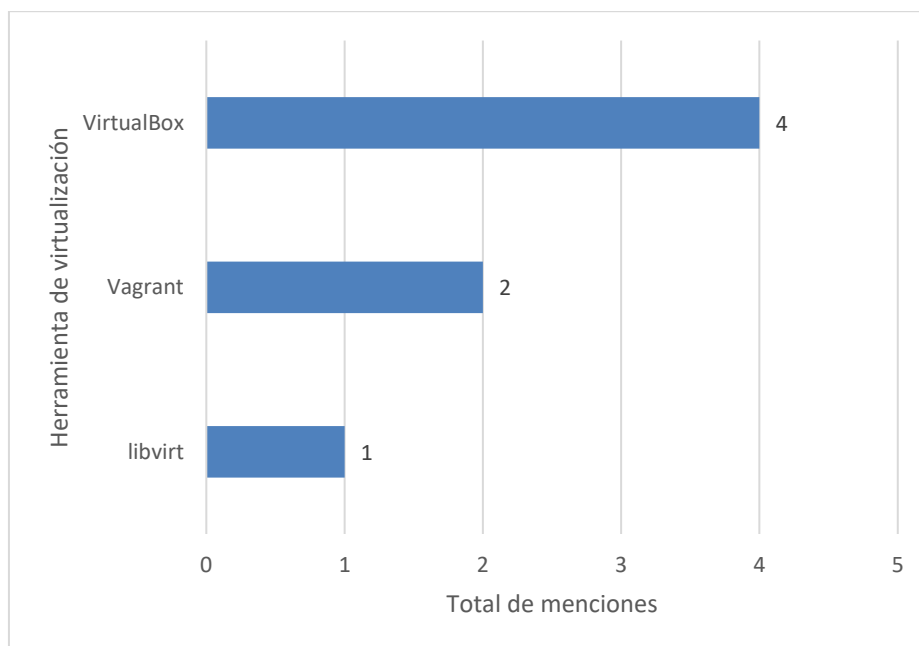


Figura 32. Herramientas de virtualización

Los estudios EP22 y EP63 describen el uso de Vagrant y de VirtualBox para crear máquinas virtuales, en el estudio EP63 se menciona explícitamente que estas se crean automáticamente, mientras que en el estudio EP22 solamente se menciona que se crean 5 máquinas virtuales. Mientras que el estudio EP68 utiliza VirtualBox para correr el servidor de Jenkins. Por otra parte, el estudio EP74 simplemente menciona que hace uso de la API libvirt, la cual se usa comúnmente para gestionar la virtualización de plataformas y otras tecnologías de virtualización.

4.4 Prácticas de configuración y administración de Jenkins recomendadas por los expertos

Esta sección tiene como objetivo responder a la pregunta de investigación RQ03: **¿Qué prácticas recomiendan los expertos y la comunidad para la configuración y administración de Jenkins?**

Se identificaron 106 prácticas de administración y configuración de Jenkins, que fueron agrupadas siguiendo la división temática propuesta por el *handbook* de Jenkins, distribuyéndolas en seis categorías: pipelines en Jenkins, gestión de Jenkins, administración

del sistema, uso de Jenkins, instalación de Jenkins y escalado de Jenkins. Además, también se aplicó una organización en grupos dentro de cada categoría.

La siguiente tabla muestra la distribución de las prácticas por categoría, destacando aquellas asociadas con pipelines en Jenkins, que representan casi la mitad del total de las prácticas identificadas, siendo 51 de 109 parte de esta categoría. Le siguen las clases gestión de Jenkins con 15 prácticas, administración del sistema y uso de Jenkins, con 13 cada una, instalación de Jenkins con nueve y finalmente escalado de Jenkins, con solo cinco prácticas.

Tabla 33. Conteo general de prácticas de administración y configuración de Jenkins	
Clasificación de la práctica	Total de prácticas reportadas
Escalado de Jenkins	5
Instalación de Jenkins	9
Uso de Jenkins	13
Administración del sistema	13
Gestión de Jenkins	15
Pipelines en Jenkins	51
Total de prácticas	106

En la siguiente figura se muestra la distribución de forma gráfica, con un 48% representado por prácticas relacionadas con pipelines en Jenkins y el otro 52% distribuido entre los otros cinco grupos de forma no tan proporcional.

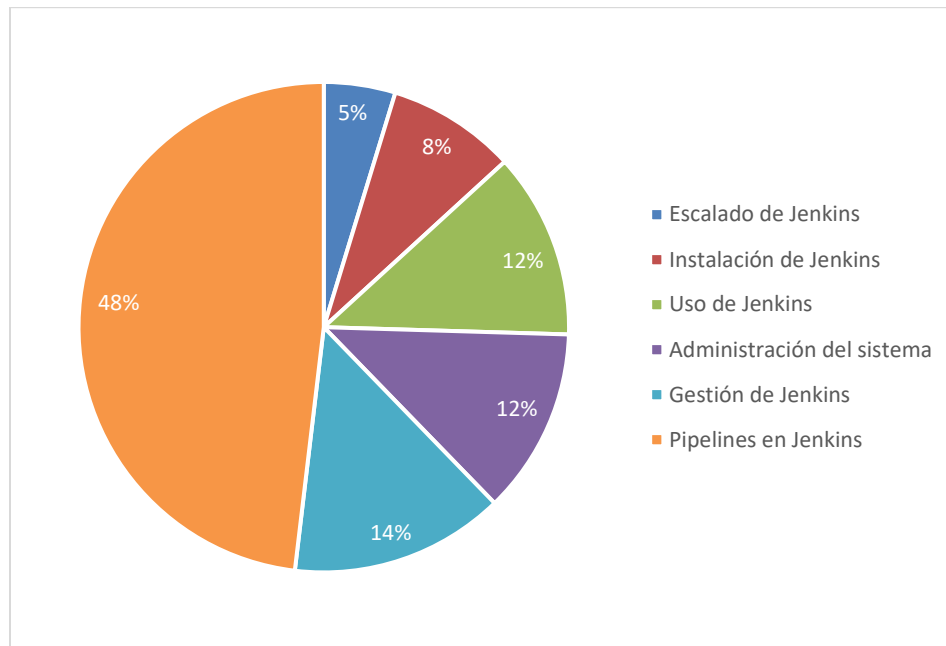


Figura 33. Distribución de prácticas de configuración y administración de Jenkins por categoría

Las 51 prácticas de configuración y administración asociadas con pipelines en Jenkins se clasificaron en siete grupos: herramientas de desarrollo de pipelines, uso de bibliotecas compartidas, ramas y *pull requests*, uso de *Jenkinsfile*, sintaxis del pipeline, ejecución de los pipelines y escalado de pipelines. A través de la siguiente tabla se presentan estas prácticas ordenadas de la menos a la más mencionada.

Tabla 34. Prácticas de configuración y administración relacionadas con pipelines en Jenkins			
Grupo	Práctica	Estudios	Total de menciones
Ejecución de los pipelines	Evitar el uso de pipelines de Jenkins para ejecutar servidores de forma indefinida	ERLG59	1
Ejecución de los pipelines	Evitar compartir espacios de trabajo entre múltiples ejecuciones de pipelines	ERLG46	1
Ejecución de los pipelines	Versionar los artefactos utilizando la variable de entorno BUILD_NUMBER de Jenkins	ERLG38	1
Ejecución de los pipelines	Probar implementaciones de pipelines con pruebas unitarias	EP52	1
Escalado de pipelines	Implementar estrategias de caché en Jenkins para acelerar los pipelines y reducir el uso de recursos	ERLG31	1
Escalado de pipelines	Ajustar las opciones de recolección de basura cuando Jenkins utilice más de 6 GB de memoria heap	ERLG46	1
Escalado de pipelines	Limitar la cantidad de datos escritos en los registros (logs) de los pipelines	ERLG46	1
Escalado de pipelines	Simplificar la matriz de construcción (uso de la directiva matrix)	EP48	1
Escalado de pipelines	Supervisar la estrategia de paralelismo del pipeline	EP03	1
Escalado de pipelines	Ejecutar etapas del pipeline de forma incremental según los cambios detectados	EP53	1

Escalado de pipelines	Optimizar continuamente los pasos individuales del pipeline	ERLG31	I
Escalado de pipelines	Reducir los steps en los pipelines, combinando aquellos similares en uno solo para minimizar la sobrecarga	ERLG46	I
Escalado de pipelines	Configurar la durabilidad de los pipelines en "Performance Optimized" cuando estos necesiten ser ejecutados múltiples veces	ERLG46	I
Escalado de pipelines	Configurar la durabilidad de los pipelines en "maximum durability" o "less durable" cuando se necesite un registro detallado de su ejecución	ERLG46	I
Escalado de pipelines	Revisar y actualizar regularmente la definición del pipeline	ERLG05	I
Escalado de pipelines	Asegurar que las unidades reutilizables de los pipelines estén disponibles a nivel organizacional	ERLG24	I
Escalado de pipelines	Ejecutar los comandos Docker en modo "detached" para evitar bloquear la ejecución del pipeline	ERLG13	I
Escalado de pipelines	Implementar manejo de errores dentro del pipeline para gestionar fallos de forma controlada	ERLG14	I
Escalado de pipelines	Integrar estrategias de rollback en el pipeline	ERLG05	I
Herramientas de desarrollo de pipelines	Validar los archivos de definición con sintaxis declarativa desde la línea de comandos antes de ejecutarlos	ERLG46	I
Ramas y pull requests	Especificar un tiempo límite para el checkout y así evitar bloqueos prolongados	ERLG32	I
Ramas y pull requests	Realizar el checkout en una subcarpeta dentro del workspace en lugar de usar la raíz del workspace	ERLG32	I
Ramas y pull requests	Optimizar el proceso de checkout de repositorios utilizando técnicas como el shallow clone y el narrow refspect	ERLG32	I
Sintaxis del pipeline	Seguir la convención de utilizar mayúsculas y guiones bajos para el nombrado de las variables de entorno	ERLG46	I
Sintaxis del pipeline	Evitar archivos grandes de declaración de variables globales	ERLG46	I
Sintaxis del pipeline	Usar la directiva withEnv en vez de sobrescribir directamente env.PATH cuando se utilizan múltiples agentes	ERLG23	I
Sintaxis del pipeline	Evitar sobrescribir los steps predeterminados de los pipelines	ERLG46	I
Uso de bibliotecas compartidas	Destinar repositorios especializados para el almacenado de shared libraries	ERLG46	I
Uso de bibliotecas compartidas	Probar las bibliotecas compartidas con pruebas funcionales	EP52	I
Uso de bibliotecas compartidas	Mantener las shares libraries simples	ERLG46	I
Uso de Jenkinsfile	Utilizar comillas dobles para envolver argumentos en el Jenkinsfile	ERLG28	I

Uso de Jenkinsfile	Agregar la línea <code>#!/usr/bin/env groovy</code> al inicio del Jenkinsfile para mejorar el resaltado de sintaxis	ERLG46	1
Ejecución de los pipelines	Configurar las variables de entorno y herramientas necesarias previo a la creación del pipeline	ERLG03, ERLG04	2
Ejecución de los pipelines	Validar la ejecución del pipeline de forma manual antes de automatizarla	EP14, EP52	2
Escalado de pipelines	Automatizar tantos procesos del pipeline como sea posible	EP03, ERLG38	2
Escalado de pipelines	Eliminar tareas innecesarias dentro del pipeline	EP48, ERLG38	2
Sintaxis del pipeline	Minimizar el uso de Groovy para funcionalidades clave del pipeline	ERLG21, ERLG46	2
Escalado de pipelines	Evitar el uso de scripts complejos dentro del pipeline	ERLG14, ERLG26, ERLG46	3
Escalado de pipelines	Integrar estrategias de despliegue controlado en el pipeline	EP44, EP63, ERLG05	3
Herramientas de desarrollo de pipelines	Utilizar la herramienta "Pipeline Syntax" para generar bloques de código del pipeline	ERLG15, ERLG20, ERLG27	3
Herramientas de desarrollo de pipelines	Utilizar el "Declarative Snippet Generator" para construir y mantener pipelines declarativos	ERLG22, ERLG23, ERLG39	3
Sintaxis del pipeline	Estructurar el pipeline en etapas lógicas	EP03, EP48, ERLG46	3
Sintaxis del pipeline	Utilizar plantillas de pipelines para mantener una estructura consistente	EP03, ERLG13, ERLG19	3
Ejecución de los pipelines	Limpiar el entorno entre ejecuciones del pipeline	EP48, ERLG30, ERLG31, ERLG32	4
Ejecución de los pipelines	Preservar artefactos y resultados generados en cada etapa del pipeline	EP53, ERLG01, ERLG23, ERLG46	4
Escalado de pipelines	Paralelizar procesos independientes	EP03, EP29, ERLG10, ERLG31	4
Ramas y pull requests	Configurar los disparadores del pipeline para que solo se activen en eventos importantes	ERLG31, ERLG32, ERLG33, ERLG39, ERLG46	5
Uso de Jenkinsfile	Utilizar sintaxis declarativa para definir los archivos de definición de pipelines (Jenkinsfiles)	EP29, ERLG10, ERLG14, ERLG21, ERLG30, ERLG46	6
Escalado de pipelines	Encapsular la lógica del pipeline en unidades reutilizables	EP03, EP29, ERLG05, ERLG14, ERLG21, ERLG22, ERLG31	7
Ejecución de los pipelines	Realizar limpieza periódica del historial de builds y artefactos	ERLG05, ERLG13, ERLG15, ERLG16, ERLG19, ERLG20, ERLG21, ERLG46	8
Uso de Jenkinsfile	Modelar el pipeline mediante archivos de definición (Jenkinsfiles)	EP04, EP80, EP81, ERLG10, ERLG14, ERLG15, ERLG23, ERLG24, ERLG36, ERLG38, ERLG39, ERLG46, ERLG51	13

La práctica relacionada con *pipelines* en Jenkins más reportada fue la de modelar el *pipeline* (o *pipelines*) mediante archivos de definición, también conocidos como *Jenkinsfiles*. Quienes reportaron esta práctica, destacaron el valor de gestionar dichos archivos con herramientas de control de versiones, ya que esto permite resguardar la integridad del *pipeline* y facilitar su mantenimiento.

Dentro de las prácticas relacionadas con pipelines en Jenkins, el grupo en el que más prácticas se reportaron fue el de escalado de pipelines, con un 41% del total de estas prácticas.

El otro 59% se distribuyó entre los otros siete grupos de esta clase de prácticas, tal como lo presenta la siguiente figura.

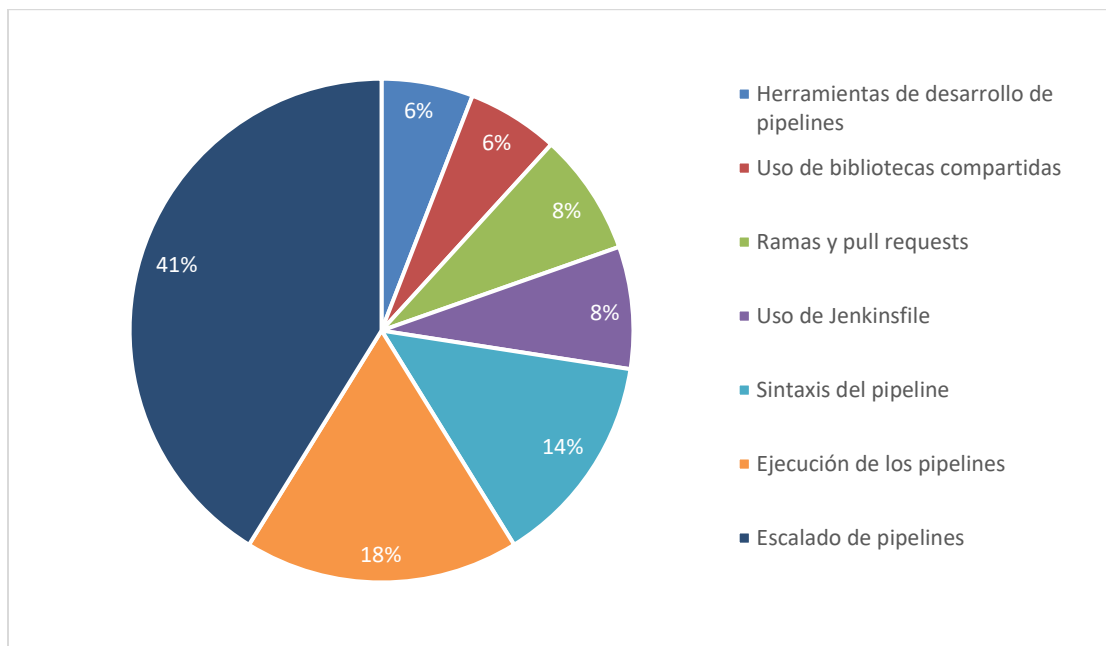


Figura 34. Distribución por grupo de prácticas relacionadas con pipelines en Jenkins

Las 15 prácticas de gestión de Jenkins se clasificaron en cinco grupos: CLI (interfaz de línea de comandos) de Jenkins, configuración como código, configuración del sistema, gestión de nodos y gestión de *plugins*. En la siguiente tabla se presentan las prácticas identificadas para esta clase.

Tabla 35. Prácticas de configuración y administración relacionadas con la gestión de Jenkins			
Grupo	Práctica	Estudios	Total de menciones
CLI de Jenkins	Usar jenkins-cli.jar en lugar del cliente SSH para la administración remota de Jenkins	ERLG46	1
Configuración como código	Almacenar los archivos de configuración como código (JCasC) en un SCM	ERLG46	1
Configuración del sistema	Alojar el servidor Jenkins en un entorno dedicado con recursos adecuados	ERLG12	1
Configuración del sistema	Gestionar adecuadamente el acceso a recursos en Jenkins para evitar colisiones entre trabajos concurrentes	ERLG46	1

Gestión de nodos	Preferir la configuración de agentes dinámicos en la nube sobre agentes permanentes	ERLG30	1
Gestión de nodos	Configurar un máximo de un ejecutor por CPU en cada nodo	ERLG46	1
Gestión de plugins	Utilizar la herramienta "Plugin Installation Manager" para la instalación de plugins sin conexión	ERLG46	1
Gestión de plugins	Utilizar herramientas de administración de plugins para optimizar la gestión de Jenkins	ERLG31	1
Gestión de plugins	Simplificar la ejecución remota de comandos Unix con el plugin SSH Pipeline Steps	EP29	1
Configuración del sistema	Reiniciar Jenkins tras realizar cambios de configuración	ERLG06, ERLG46	2
Gestión de nodos	Configurar una capacidad de instancias adecuada para evitar sobrecargar el sistema	ERLG30, ERLG31	2
Gestión de plugins	Seleccionar los plugins considerando las restricciones del pipeline y su entorno	ERLG12, ERLG15	2
Gestión de nodos	Evitar ejecutar pipelines en el nodo maestro y delegar su uso exclusivamente a tareas de administración	ERLG30, ERLG31, ERLG46	3
Gestión de plugins	Reducir el número de plugins utilizados tanto como sea posible	ERLG24, ERLG31, ERLG46	3
Gestión de plugins	Actualizar periódicamente Jenkins y sus plugins	ERLG05, ERLG14, ERLG31, ERLG38, ERLG46	5

La práctica de gestión de Jenkins más mencionada en los estudios fue una relacionada con la gestión de *plugins*, que recomienda actualizar periódicamente estos, así como Jenkins.

Continuando con las prácticas de gestión de Jenkins relacionadas con la gestión de *plugins*, este grupo fue en el que más prácticas se reportaron dentro de esta clase de prácticas. Su representación en el total de las prácticas de gestión de Jenkins fue del 40%, seguido de las prácticas de gestión de nodos, que ocuparon el 27% de las menciones como se puede observar en la siguiente figura.

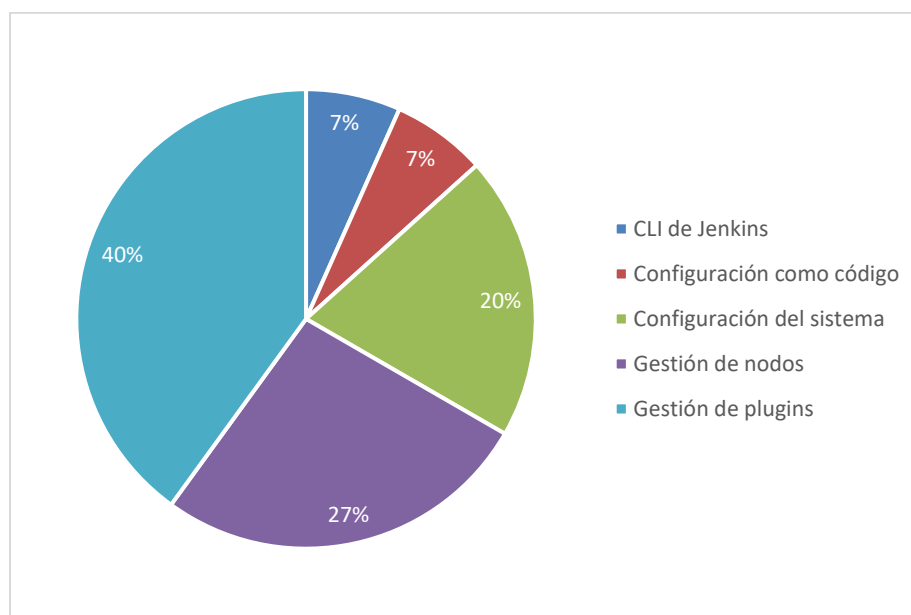


Figura 35. Distribución por grupo de prácticas de gestión de Jenkins

Las 13 prácticas de administración del sistema se dividieron en cuatro grupos: configuración con *Proxy* inverso, consulta de logs, monitoreo de Jenkins y copias de seguridad/Restauración de Jenkins. Esta clase de prácticas es muy similar a la de “gestión de Jenkins” en nombre, sin embargo, estas se centran en el correcto funcionamiento del sistema, mientras que las de gestión de Jenkins se centran en la gestión de los recursos y partes que conforman el sistema. Con esto en mente, a través de la siguiente tabla se presentan las prácticas de administración del sistema.

Tabla 36. Prácticas de configuración y administración relacionadas con la gestión de Jenkins			
Grupo	Práctica	Estudios	Total de menciones
Configuración con Proxy inverso	Aumentar el valor de proxy_read_timeout si se experimentan timeouts al ejecutar comandos a través de un proxy en Jenkins	ERLG46	1
Configuración con Proxy inverso	Asegurarse de que Jenkins se ejecute en la ruta de contexto en la que el proxy inverso está sirviendo Jenkins	ERLG46	1
Consulta de logs	Consultar los logs de Jenkins para identificar y solucionar errores	ERLG38	1
Consulta de logs	Evitar el uso del nivel de registro (logs) "DEBUG" en entornos de producción	ERLG46	1
Consulta de logs	Preparar el servidor de Jenkins previo a su reinicio o apagado para evitar la interrupción de trabajos en ejecución	ERLG30	1
Copias de seguridad/Restauración de Jenkins	Actualizar Jenkins corriendo el instalador, no solo cambiando el archivo .war	ERLG46	1
Copias de seguridad/Restauración de Jenkins	Evitar realizar copias de seguridad en la carpeta /tmp	ERLG46	1
Copias de seguridad/Restauración de Jenkins	Establecer una política de copias de seguridad para los archivos críticos de Jenkins	ERLG46	1
Monitoreo de Jenkins	Monitorear y escalar los recursos de Jenkins según la carga de trabajo	ERLG05	1
Monitoreo de Jenkins	Instalar el plugin Warnings Next Generation para visualizar y gestionar las advertencias y errores en Jenkins	ERLG46	1
Copias de seguridad/Restauración de Jenkins	Probar periódicamente la restauración desde copias de seguridad	ERLG05, ERLG46	2

Copias de seguridad/Restauración de Jenkins	Realizar copias de seguridad regulares de archivos críticos de Jenkins	ERLG05, ERLG21, ERLG38, ERLG46	4
Monitoreo de Jenkins	Monitorear el rendimiento de Jenkins con herramientas especializadas	ERLG05, ERLG14, ERLG31, ERLG38	4

Dentro de estas prácticas de administración del sistema, hubo dos con más menciones entre los estudios, ambas con cuatro menciones. Una de ellas fue monitorear el rendimiento de Jenkins con herramientas especializadas; herramientas como Nagios, Datalog, Prometheus y Grafana, o *plugins* como “Jenkins Monitoring Plugin” o “Green Balls” fueron mencionados para realizar esta práctica (ERLG05 y ERLG38). La otra práctica fue realizar copias de seguridad regulares de archivos críticos de Jenkins.

Las dos prácticas de administración previamente mencionadas se relacionan con el monitoreo y las copias de seguridad de Jenkins. De forma general, la distribución de prácticas de administración del sistema siguió esta tendencia, con el grupo de copias de seguridad de Jenkins representando el 38%, seguido del de monitoreo de Jenkins y consulta de logs, cada uno con un 23% del total de prácticas de administración del sistema. La siguiente figura presenta esta distribución entre grupos de forma gráfica.

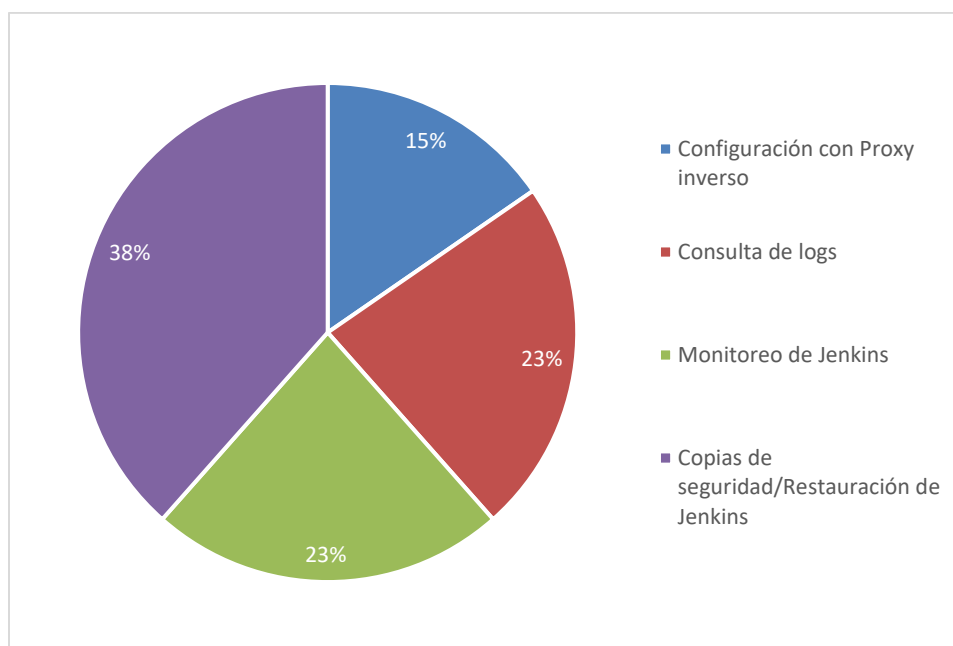


Figura 36. Distribución por grupo de prácticas de administración del sistema

Pasando a las 13 prácticas asociadas con el uso de Jenkins, estas se organizaron en cuatro grupos: uso de credenciales, uso de *fingerprints*, uso de agentes y proyectos de Jenkins. Con ayuda de la siguiente tabla se presentan estas 13 prácticas del uso de Jenkins.

Tabla 37. Prácticas de configuración y administración relacionadas con el uso de Jenkins

Grupo	Práctica	Estudios	Total de menciones
Proyectos de Jenkins	Utilizar carpetas de organización para gestionar los jobs en Jenkins	ERLG46	1
Proyectos de Jenkins	Evitar la creación de proyectos de tipo "Maven Project"	ERLG46	1
Proyectos de Jenkins	Automatizar el envío de notificaciones sobre el estado de los pipelines para las personas encargadas	ERLG46	1
Uso de agentes	Mantener la misma versión de JDK en el controlador y en los agentes para evitar problemas de compatibilidad	ERLG46	1
Uso de agentes	Establecer el directorio raíz del sistema de archivos remoto en una ruta estándar para los agentes	ERLG30	1
Uso de agentes	Usar etiquetas (labels) en Jenkins para evitar atar los builds a agentes específicos	ERLG46	1
Uso de credenciales	Establecer una estrategia consistente para el nombrado de los identificadores de credenciales	ERLG46	1
Uso de Fingerprints	Utilizar moderadamente file fingerprinting en Jenkins para gestionar las dependencias entre proyectos	ERLG46	1
Proyectos de Jenkins	Limitar el nombrado de los proyectos a caracteres alfanuméricos	ERLG30, ERLG46	2
Proyectos de Jenkins	Generar pipelines modulares por unidad de despliegue	EP02, ERLG12, ERLG36	3
Proyectos de Jenkins	Separar los procesos de integración y entrega en pipelines distintos	EP18, EP33, ERLG11	3
Proyectos de Jenkins	Organizar y ejecutar pipelines por rama usando proyectos multibranch	ERLG05, ERLG10, ERLG27, ERLG46	4
Uso de agentes	Utilizar Docker como agente	ERLG06, ERLG09, ERLG21, ERLG24, ERLG46	5

Dentro de las prácticas de uso de Jenkins, la más mencionada fue utilizar Docker como agente. Los estudios resaltaban su aplicación para *stages* específicos o para todo el pipeline, pero enfatizaban los beneficios de su uso, principalmente la facilidad de repetir el proceso gracias al encapsulamiento proporcionado por Docker.

Aunque la práctica previamente mencionada es del grupo de prácticas de uso de agentes, en la generalidad la mayoría de las prácticas de uso de Jenkins pertenecen al grupo de proyectos de Jenkins, con un 54% del total. Le siguen con un 31% las prácticas de uso de agentes y al final quedan las de uso de credenciales y uso de *fingerprints*, cada una con el 8% de las prácticas totales. Esta distribución se muestra gráficamente a través de la siguiente figura.

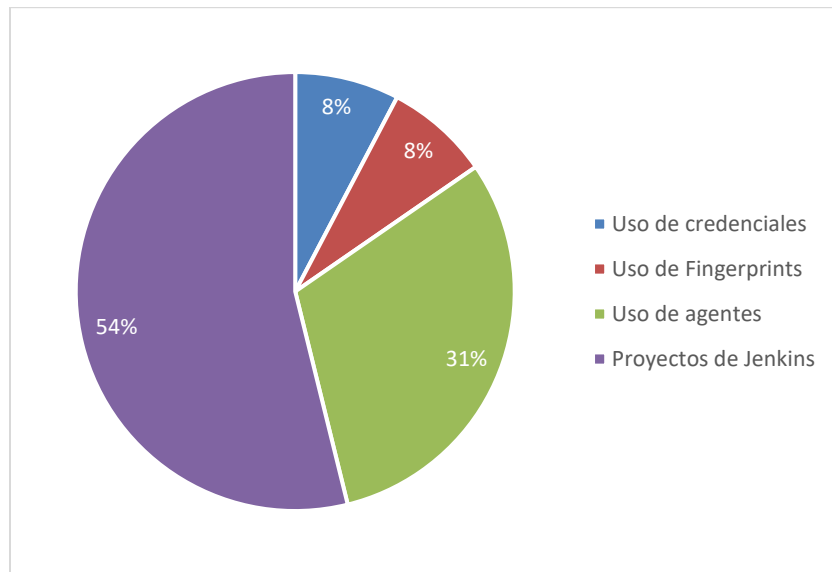


Figura 37. Distribución por grupo de prácticas asociadas con el uso de Jenkins

Las nueve prácticas de instalación de Jenkins no se agruparon internamente. En la división temática de la que partimos se proponían temas dentro de la instalación, pero asociados con el entorno en el que se instalaba Jenkins. Es por esto, que decidimos presentar las prácticas de forma general, sin la agrupación interna como en las otras clases de prácticas. A continuación se presentan estas nueve prácticas de instalación.

Tabla 38. Prácticas de configuración y administración relacionadas con el uso de Jenkins

Práctica	Estudios	Total de menciones
Probar actualizaciones relacionadas con Jenkins en un entorno de prueba antes de aplicarlas en producción	ERLG31	1
Utilizar el instalador oficial de Jenkins para Windows para hacer la instalación simple	ERLG46	1
Optar por el despliegue de Jenkins utilizando su archivo WAR	ERLG46	1
Utilizar la imagen oficial Jenkins de Docker Hub basada en la versión LTS si está configurando Jenkins dentro de Docker	ERLG46	1
Verificar que Docker está configurado para usar contenedores Linux para poder usar la imagen de Jenkins de Docker en Windows	ERLG46	1
Verificar los parámetros de línea de comandos al configurar Jenkins, ya que los incorrectos son ignorados	ERLG46	1
Utilizar la interfaz gráfica de Blue Ocean para configurar proyectos de Jenkins al iniciar en el uso de la herramienta	ERLG46	1
Instalar los plugins recomendados durante la configuración inicial de Jenkins	ERLG39, ERLG46	2
Ejecutar la infraestructura de Jenkins utilizando un enfoque basado en contenedores	ERLG24, ERLG30, ERLG38, ERLG51	4

La práctica más mencionada de instalación de Jenkins fue ejecutar la infraestructura de Jenkins utilizando un enfoque basado en contenedores. Dicho de otra forma, utilizar la imagen oficial de Docker de Jenkins para ejecutar el servidor de la herramienta. Se menciona

Docker porque los estudios así lo recomendaban, sin embargo, algunos de ellos también mencionaron el uso de Kubernetes para implementar esta práctica.

Finalmente, las cinco prácticas de escalado de Jenkins, fueron organizadas en tres grupos internos: arquitectura para la facilidad de gestión, recomendaciones de *hardware* y arquitectura del escalado. En la tabla siguiente se presentan estas cinco prácticas.

Tabla 39. Prácticas de configuración y administración relacionadas con el escalado de Jenkins			
Grupo	Práctica	Estudios	Total de menciones
Arquitectura del escalado	Implementar estrategias de alta disponibilidad para Jenkins	ERLG05	1
Arquitectura del escalado	Utilizar el período de espera de Jenkins (quiet period) para agrupar ejecuciones de jobs	ERLG31	1
Arquitectura para la facilidad de gestión	Descentralizar la administración de Jenkins y delegarla a cada equipo para mejorar su gestión	ERLG24	1
Recomendaciones de Hardware	Ejecutar Jenkins en un equipo con almacenamiento SSD en la medida de lo posible	ERLG46	1
Arquitectura del escalado	Aprovechar los sistemas de ejecución distribuidos	EP03, EP55, EP59, ERLG05, ERLG14, ERLG18	6

La práctica más mencionada fue identificada en seis estudios y aborda el aprovechar los sistemas de ejecución distribuidos. En los estudios donde se menciona, destacan el uso de la arquitectura maestro-agente para ejecutar los pipelines y así aprovechar esta idea de sistemas de ejecución distribuidos.

Dentro de la clasificación interna de prácticas de escalado de Jenkins, el grupo en el que se agruparon más prácticas, con tres de ellas y donde se clasificó la práctica previamente mencionada, es el de arquitectura de escalado. Después, con una sola práctica cada grupo, se encuentran los de recomendaciones de *hardware* y arquitectura para la facilidad de gestión. La siguiente figura muestra esta distribución.

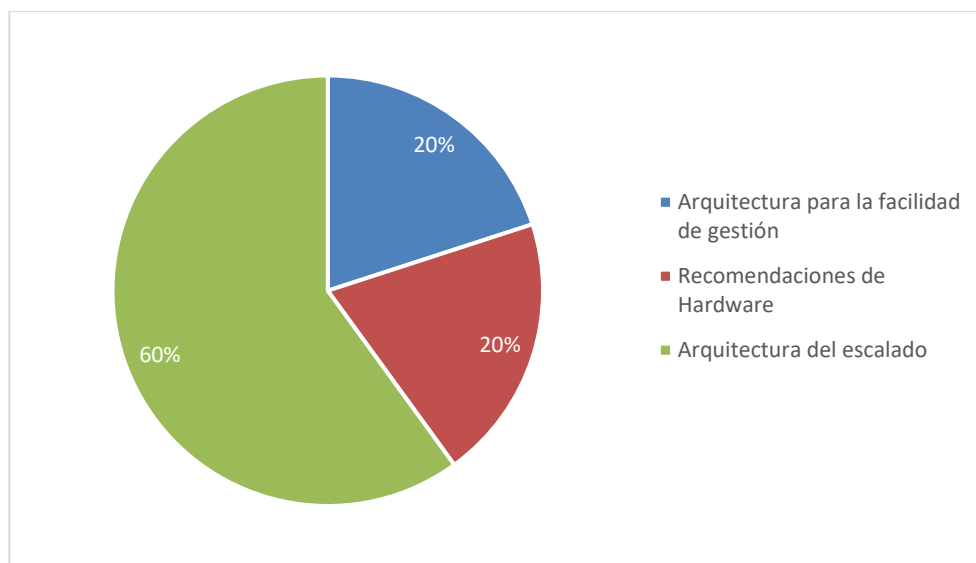


Figura 38. Distribución por grupo de prácticas de escalado de Jenkins

4.5 Actividades de integración, prueba y despliegue integradas y automatizadas utilizando Jenkins

Esta sección tiene el objetivo de responder la pregunta de investigación RQ04: **¿Qué actividades de integración, prueba y despliegue son comúnmente integradas y automatizadas utilizando Jenkins en distintos proyectos de software?**

Se identificó la automatización e integración de siete actividades de integración, cinco de prueba y siete de despliegue, así como otras ocho actividades fuera de estos tres grupos. Fue común que en los estudios se reportara la automatización de más de una actividad por grupo, además de actividades en diversos grupos. Como lo muestra la siguiente figura, las actividades de integración fueron las que más se automatizaron en la literatura, seguidas de las actividades de prueba y despliegue, respectivamente.

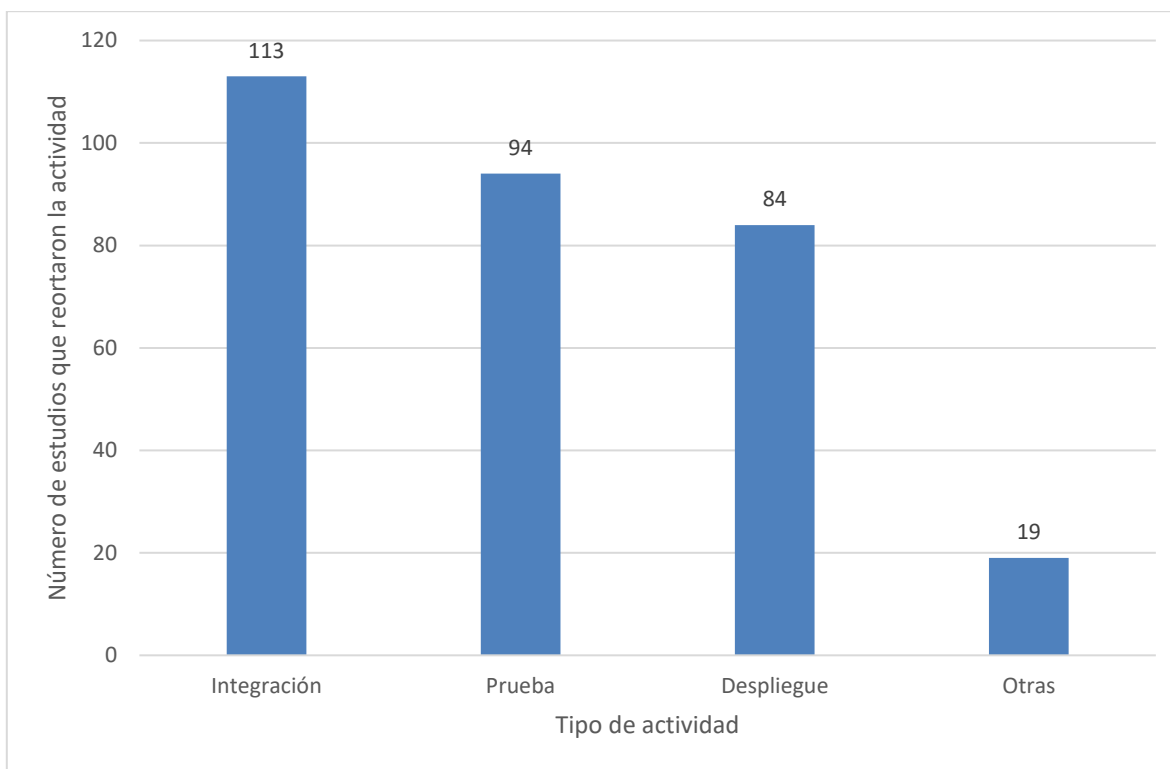


Figura 39. Menciones de actividades integradas y automatizadas por tipo

Las actividades de integración se identificaron siguiendo lo propuesto por Humble & Farley, 2010), que para el proceso de integración continua identifican y establecen siete actividades: consultar el sistema de control de versiones, descargar el código del repositorio, compilar la aplicación, realizar análisis de código fuente, ejecutar pruebas automatizadas (unitarias), notificar el estado de la compilación y almacenar los artefactos generados. La siguiente tabla muestra el conteo de estudios que reportaron la automatización e integración de estas actividades con Jenkins.

Tabla 40. Actividades de integración automatizadas e integradas con Jenkins		
Actividad de integración	Estudios donde se menciona	Total de menciones

Notificar el estado de la compilación	EP09, EP11, EP12, EP16, EP19, EP20, EP27, EP34, EP54, EP58, EP59, EP62, EP64, EP65, EP78, EP100, ERLG17, ERLG45, ERLG47, ERLG48, ERLG52, ERLG53, ERLG55, ERLG61	24
Ejecutar pruebas automatizadas (unitarias)	EP17, EP18, EP21, EP43, EP49, EP54, EP55, EP58, EP63, EP65, EP68, EP71, EP73, EP78, EP82, EP84, EP86, EP87, EP89, EP100, EP102, ERLG02, ERLG10, ERLG15, ERLG17, ERLG37, ERLG58, ERLG60	28
Realizar análisis de código fuente	EP04, EP08, EP17, EP18, EP21, EP22, EP24, EP34, EP36, EP41, EP43, EP50, EP54, EP55, EP58, EP60, EP71, EP73, EP84, EP92, EP95, EP97, ERLG04, ERLG06, ERLG10, ERLG15, ERLG16, ERLG19, ERLG20, ERLG45, ERLG54	31
Almacenar los artefactos generados	EP07, EP16, EP19, EP26, EP31, EP34, EP41, EP51, EP54, EP59, EP60, EP76, EP78, EP81, EP82, EP91, EP94, EP95, EP97, EP98, ERLG04, ERLG06, ERLG10, ERLG11, ERLG12, ERLG15, ERLG16, ERLG18, ERLG20, ERLG25, ERLG29, ERLG33, ERLG36, ERLG41, ERLG43, ERLG48, ERLG49	37
Consultar el sistema de control de versiones	EP02, EP07, EP09, EP12, EP14, EP16, EP17, EP18, EP19, EP21, EP22, EP26, EP28, EP31, EP34, EP36, EP41, EP45, EP51, EP54, EP55, EP58, EP62, EP63, EP64, EP65, EP66, EP77, EP78, EP81, EP82, EP86, EP89, EP91, EP92, EP93, EP94, EP95, EP96, EP97, EP98, EP99, EP100, ERLG04, ERLG06, ERLG07, ERLG12, ERLG25, ERLG40, ERLG45, ERLG56, ERLG61	52
Descargar el código del repositorio	EP02, EP07, EP09, EP12, EP14, EP16, EP17, EP18, EP19, EP21, EP22, EP26, EP28, EP31, EP34, EP36, EP41, EP45, EP51, EP54, EP55, EP58, EP62, EP63, EP64, EP65, EP66, EP81, EP82, EP89, EP91, EP92, EP93, EP94, EP95, EP96, EP97, EP98, EP99, EP100, ERLG04, ERLG06, ERLG07, ERLG11, ERLG12, ERLG13, ERLG15, ERLG16, ERLG17, ERLG18, ERLG19, ERLG20, ERLG25, ERLG27, ERLG33, ERLG36, ERLG37, ERLG39, ERLG40, ERLG41, ERLG42, ERLG43, ERLG45, ERLG56, ERLG58, ERLG60, ERLG61	67
Compilar la aplicación	EP02, EP03, EP04, EP07, EP08, EP09, EP10, EP11, EP12, EP16, EP17, EP18, EP20, EP21, EP22, EP23, EP24, EP26, EP27, EP28, EP31, EP34, EP36, EP37, EP41, EP43, EP45, EP49, EP50, EP51, EP53, EP54, EP58, EP61, EP62, EP63, EP64, EP65, EP67, EP68, EP71, EP73, EP81, EP82, EP87, EP89, EP90, EP91, EP92, EP93, EP94, EP95, EP97, EP100, ERLG04, ERLG06, ERLG07, ERLG08, ERLG09, ERLG10, ERLG11, ERLG12, ERLG13, ERLG15, ERLG16, ERLG17, ERLG18, ERLG19, ERLG20, ERLG25, ERLG27, ERLG29, ERLG33, ERLG34, ERLG36, ERLG39, ERLG40, ERLG41, ERLG42, ERLG43, ERLG44, ERLG45, ERLG47, ERLG49, ERLG50, ERLG51, ERLG52, ERLG53, ERLG54, ERLG55, ERLG56, ERLG57, ERLG58, ERLG60, ERLG61	95

A continuación se muestra una representación gráfica de las actividades de integración automatizadas con Jenkins, ordenadas según el número de menciones totales entre los estudios. Se puede observar que la actividad de integración más automatizada es la compilación de código, con 95 menciones entre los estudios. La menos automatizada fue la notificación del estado de la compilación, con solo 24 menciones entre los estudios.

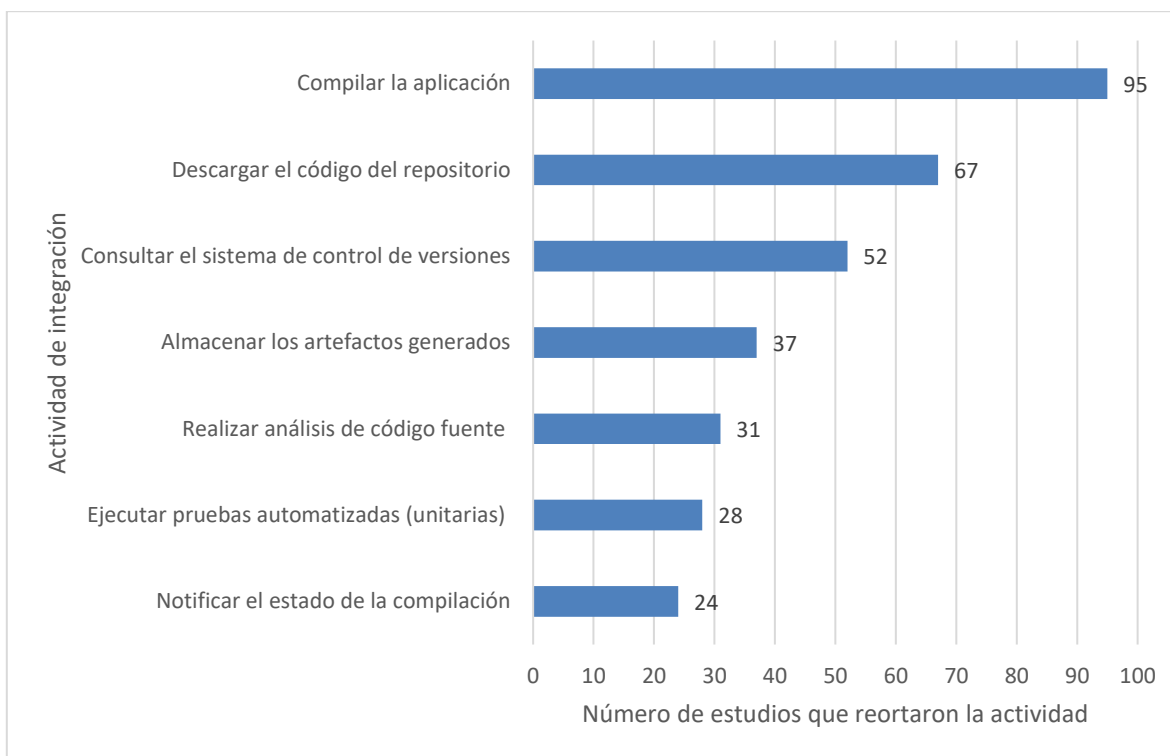


Figura 40. Menciones de actividades de integración automatizadas con Jenkins

Las actividades de prueba se identificaron siguiendo lo propuesto por la International Organization for Standardization. et al., (2013), en su estándar 29119 parte 2, donde se describen los procesos de prueba con sus respectivas actividades. De todas las actividades que presenta el estándar, se identificó la automatización de cinco de ellas: preparar datos de prueba, establecer el entorno de pruebas, registrar la ejecución de las pruebas, comparar la ejecución de las pruebas y ejecutar los procedimientos de prueba. A continuación se muestra el conteo de los estudios que reportaron la automatización de alguna de estas actividades.

Tabla 41. Actividades de prueba automatizadas e integradas con Jenkins

Actividad de prueba	Estudios donde se menciona	Total de menciones
Preparar datos de prueba	EP27, EP31, EP63	3

Establecer el entorno de pruebas	EP16, EP17, EP22, EP31, EP34, EP35, EP55, EP56, EP63, EP68, EP74, EP87, ERLG10, ERLG42, ERLG60	15
Registrar la ejecución de las pruebas	EP11, EP16, EP17, EP20, EP27, EP35, EP36, EP40, EP41, EP56, EP57, EP58, EP68, EP71, EP73, EP83, EP84, EP86, EP89, EP98, ERLG17, ERLG42, ERLG44	23
Comparar la ejecución de las pruebas	EP11, EP17, EP19, EP35, EP36, EP40, EP41, EP56, EP57, EP58, EP63, EP65, EP68, EP71, EP73, EP79, EP83, EP84, EP86, EP87, EP89, EP98, EP102, ERLG17, ERLG42, ERLG44	26
Ejecutar los procedimientos de prueba	EP03, EP04, EP07, EP08, EP11, EP12, EP16, EP17, EP18, EP19, EP20, EP21, EP24, EP28, EP31, EP34, EP35, EP36, EP37, EP39, EP40, EP41, EP45, EP49, EP50, EP54, EP55, EP56, EP57, EP58, EP62, EP63, EP65, EP68, EP71, EP73, EP74, EP77, EP78, EP79, EP81, EP82, EP83, EP84, EP86, EP87, EP88, EP89, EP90, EP96, EP97, EP98, EP99, EP100, EP102, ERLG02, ERLG04, ERLG07, ERLG10, ERLG12, ERLG13, ERLG15, ERLG16, ERLG17, ERLG19, ERLG20, ERLG34, ERLG36, ERLG37, ERLG39, ERLG40, ERLG42, ERLG44, ERLG45, ERLG49, ERLG52, ERLG54, ERLG58, ERLG59, ERLG60, ERLG61	81
Automatizada sin especificar	EP02, EP10, EP53, EP61, EP67, EP75, EP92, ERLG06, ERLG09, ERLG11, ERLG29,	11

La actividad de prueba más automatizada fue la ejecución de los procedimientos de prueba, siendo reportada en 81 de los estudios, mientras que la menos automatizada fue la preparación de datos de prueba, automatizada solamente en tres de los estudios. La siguiente figura muestra esta distribución de forma más clara, pero la ejecución de procedimientos de prueba fue la actividad más automatizada por mucho.

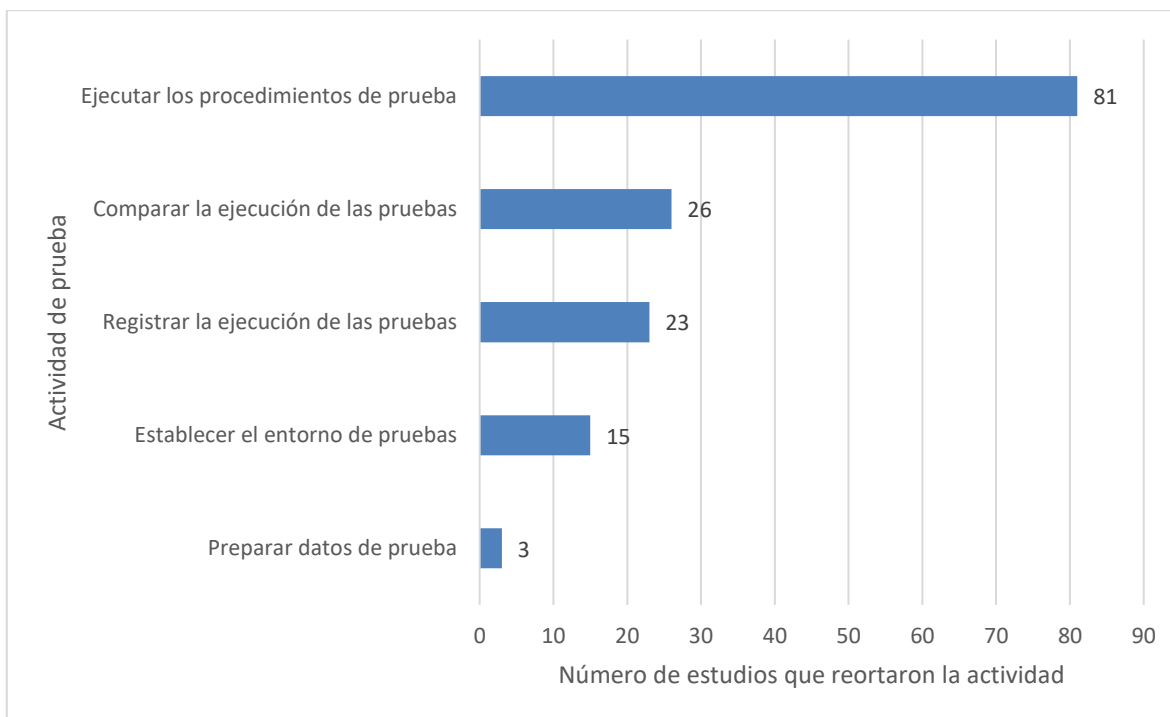


Figura 41. Menciones de actividades de prueba automatizadas con Jenkins

Las actividades de despliegue fueron identificadas partiendo del trabajo de Washizaki, (2024) en la cuarta versión del SWEBOK. Fueron siete actividades de despliegue

las que se automatizaron con Jenkins: ejecutar pruebas de humo, generar archivos de configuración, reiniciar los servidores, configurar los servidores y las bases de datos, iniciar la ejecución de la aplicación, empaquetar el código e instalar el software en diferentes servidores.

Tabla 42. Actividades de despliegue automatizadas e integradas con Jenkins		
Actividad de despliegue	Estudios donde se menciona	Total de menciones
Ejecutar pruebas de humo	EP18, EP20, ERLG44,	3
Generar archivos de configuración	EP22, EP34, EP75, EP92, ERLG25, ERLG33,	6
Reiniciar los servidores	EP26, EP80, ERLG11, ERLG12, ERLG33, ERLG36, ERLG53,	7
Configurar los servidores y las bases de datos	EP07, EP22, EP26, EP33, EP65, EP70, EP74, EP75, EP79, EP80, EP97, ERLG33, ERLG47,	13
Iniciar la ejecución de la aplicación	EP02, EP07, EP11, EP14, EP18, EP21, EP26, EP28, EP33, EP51, EP55, EP57, EP61, EP63, EP65, EP73, EP79, EP80, EP94, EP95, EP97, EP100, ERLG04, ERLG10, ERLG11, ERLG12, ERLG15, ERLG16, ERLG18, ERLG20, ERLG25, ERLG27, ERLG36, ERLG40, ERLG41, ERLG44, ERLG49, ERLG50, ERLG51,	39
Empaquetar el código	EP02, EP07, EP08, EP09, EP12, EP14, EP18, EP20, EP21, EP26, EP27, EP31, EP34, EP43, EP49, EP51, EP54, EP62, EP63, EP68, EP80, EP81, EP82, EP91, EP92, EP94, EP95, EP97, EP100, ERLG04, ERLG06, ERLG08, ERLG10, ERLG12, ERLG13, ERLG15, ERLG16, ERLG18, ERLG19, ERLG20, ERLG25, ERLG27, ERLG36, ERLG43, ERLG47, ERLG54, ERLG56,	47
Instalar el software en diferentes servidores	EP02, EP04, EP07, EP08, EP09, EP11, EP12, EP14, EP18, EP19, EP21, EP22, EP24, EP26, EP28, EP31, EP33, EP34, EP49, EP51, EP54, EP55, EP57, EP61, EP62, EP63, EP65, EP67, EP73, EP74, EP75, EP79, EP80, EP81, EP92, EP93, EP94, EP95, EP97, EP100, ERLG04, ERLG10, ERLG11, ERLG12, ERLG13, ERLG15, ERLG16, ERLG18, ERLG19, ERLG20, ERLG25, ERLG27, ERLG33, ERLG36, ERLG37, ERLG40, ERLG41, ERLG43, ERLG44, ERLG47, ERLG49, ERLG50, ERLG51, ERLG52, ERLG56, ERLG58, ERLG60,	67
Automatizada sin especificar	EP03, EP10, EP37, EP99, ERLG09, ERLG59,	6

De las actividades de despliegue previamente mencionadas, la más reportada entre los estudios fue la de instalación del software en diferentes servidores, con un total de 67 menciones, mientras que la menos reportada fue la ejecución de pruebas de humo, encontrada solamente en tres estudios. A continuación se muestra la distribución general de las actividades.

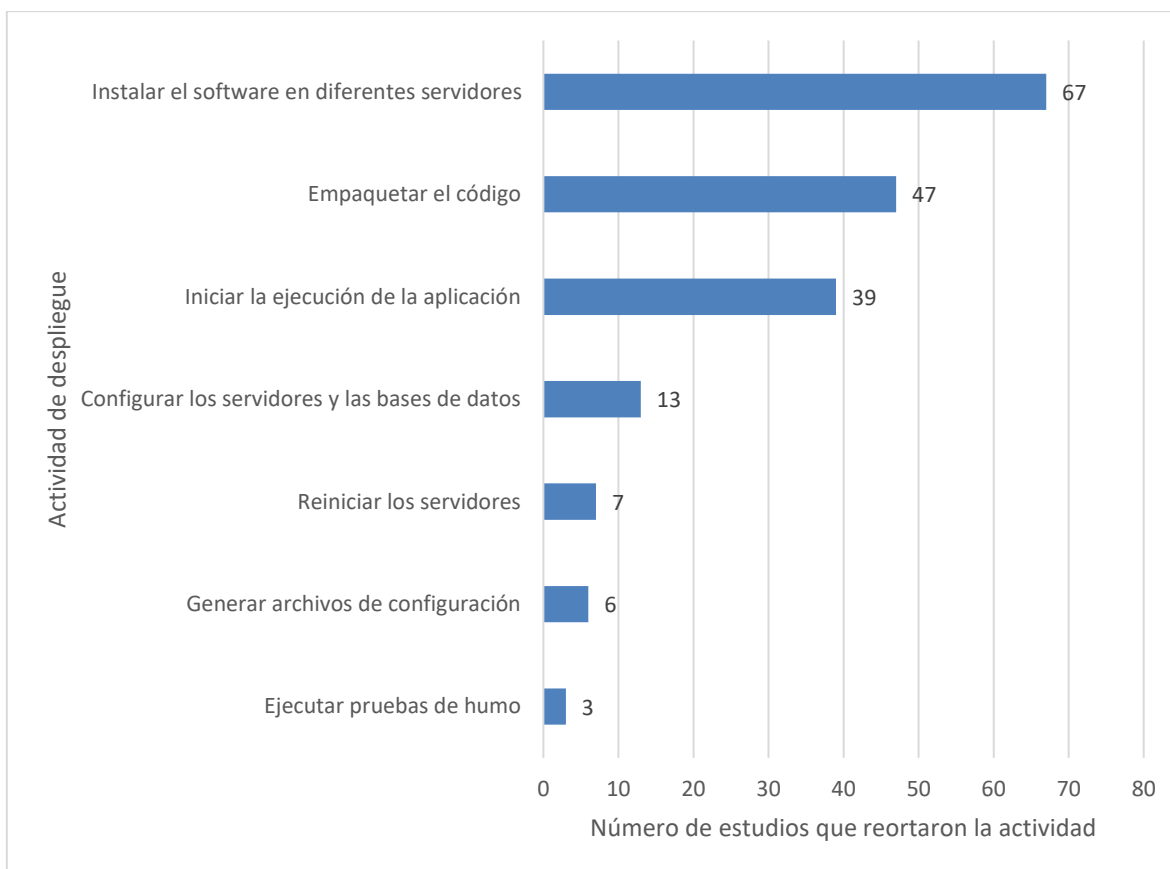


Figura 42. Menciones de actividades de despliegue automatizados con Jenkins

4.6 Aspectos de seguridad y principales estrategias para mitigar vulnerabilidades y asegurar los pipelines de CI/CD

Esta sección tiene el propósito de responder la pregunta de investigación RQ05: **¿Qué aspectos de seguridad deben considerarse al utilizar Jenkins, y cuáles son las principales estrategias para mitigar vulnerabilidades y asegurar los pipelines de Integración y Despliegue Continuo (CI/CD)?**

Vale la pena mencionar que, de todas las preguntas de investigación, esta fue la que contó con la menor cantidad de estudios identificados que la abordaran, únicamente 35 de los 163 estudios respondieron a esta pregunta de investigación, de los cuales solo doce son parte de la literatura blanca, mientras que el resto pertenecen a la literatura gris, siendo el Jenkins Handbook el recurso que más aspectos y estrategias de seguridad aportó.

Comenzando con los aspectos de seguridad que se deben considerar al utilizar Jenkins, se identificaron un total de 58 aspectos de seguridad, estos aspectos se agruparon en clusters más generales que se obtuvieron a partir de los capítulos enfocados en seguridad del

Handbook de Jenkins. Se determinaron un total de 8 clusters, los cuales son: Control de Accesos (Autorización y Autenticación), Seguridad del Controlador, Seguridad de los Builds, Protección CSRF, Gestion de Variables de Entorno, Gestion de Credenciales en Jenkins, Puertos y Servicios Expuestos, y Seguridad del Entorno de Jenkins. En la Tabla 43 se muestran los cluster antes mencionados y sus descripciones correspondientes.

Tabla 43. Aspectos de seguridad para considerar en Jenkins	
Dominio de Jenkins	Descripción del Dominio.
Puertos y Servicios Expuestos	Aspectos de seguridad relacionados a los puertos y servicios que Jenkins deja accesibles en la red, restringiendo servicios innecesarios o mal configurados.
Gestión de Variables de Entorno.	Aspectos de Jenkins a tomar en cuenta durante la definición, uso y protección de las variables de entorno.
Seguridad del Entorno de Jenkins.	Aspectos de seguridad a nivel de infraestructura y configuración general del ambiente en el que se ejecuta Jenkins.
Control de Accesos (Autorización y Autenticación).	Aspectos de seguridad que se deben considerar al implementar y configurar mecanismos de autorización y autenticación.
Gestión de Credenciales en Jenkins.	Aspectos de seguridad a contemplar para el almacenamiento y protección adecuado de las credenciales para conectarse con otras herramientas y nodos de Jenkins.
Seguridad del Controlador.	Aspectos de seguridad a considerar para el aislamiento y protección correcto del núcleo de Jenkins.
Seguridad de los Builds.	Aspectos de seguridad que cubren características importantes para el aseguramiento de código que se ejecuta en los pipelines y la protección de la integridad de los builds.
Protección CSRF.	Aspectos de seguridad centrados en la protección contra ataques de <i>cross site scripting</i> .

Como se mencionó previamente, se identificaron 58 aspectos de seguridad, de los cuales la mayoría se enfocan en aspectos relacionados al control de accesos en Jenkins, la seguridad del entorno en el que se ejecuta Jenkins, la seguridad en los builds que se ejecutan en los pipelines de CI/CD y la gestión de las credenciales para interactuar y conectarse con otras herramientas externas. En la Figura 43 se puede ver esta distribución de manera más clara.

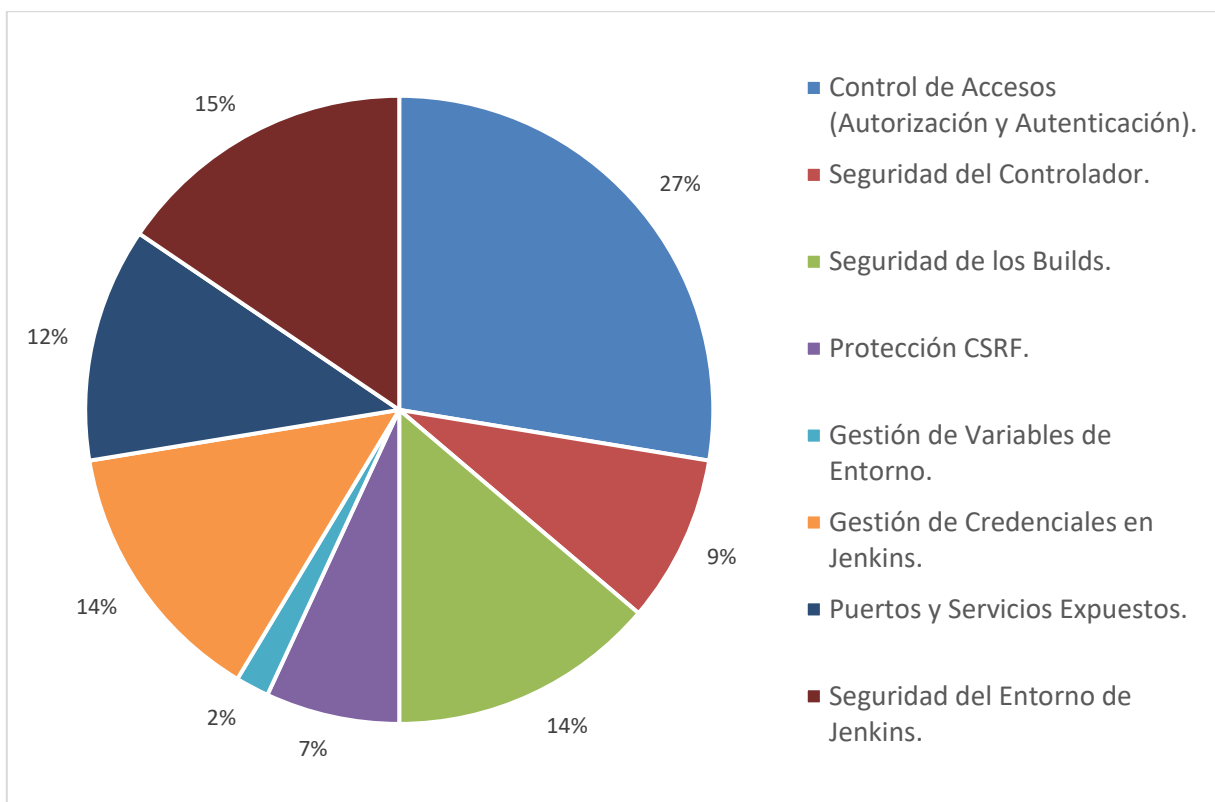


Figura 43 Aspectos de Seguridad de Jenkins por Dominio.

Aspectos de seguridad relacionados a las gestion de variables de entorno y de protección CSRF son apenas mencionados en la literatura, mientras que la aseguramiento del controlador principal de Jenkins y los puertos y servicios expuestos tiene un poco más relevancia.

En lo que respecta al Control de Accesos dentro de Jenkins, se encontraron 16 elementos a considerar, siendo los más reportados: Jenkins tiene una sección para gestionar a los usuarios para configurar mecanismos de autenticación y autorización, y La integración de herramientas y servidores externos con Jenkins (como SonarQube, Nexus, DockerHub y Docker) requiere el uso de credenciales de autenticación para garantizar una vinculación segura, con tres menciones cada uno. En la Tabla 44 se puede ver la frecuencia de los aspectos de seguridad de la agrupación de Control de accesos en Jenkins.

Tabla 44. Aspectos de seguridad para considerar en Jenkins enfocados en el control de accesos (Autorización y Autenticación)

Dominio de Jenkins	Aspecto de seguridad	Estudios que reportan	Total
Control de Accesos (Autorización y Autenticación).	Jenkins tiene una sección para gestionar a los usuarios para configurar mecanismos de autenticación y autorización.	ERLG30, ERLG38, ERLG46.	3

	La integración de herramientas y servidores externos con Jenkins (como SonarQube, Nexus, DockerHub y Docker) requiere el uso de credenciales de autenticación para garantizar una vinculación segura.	ERLG15, ERLG19, ERLG20.	3
	Existen diversos alcances en los permisos de Jenkins, aplicables de manera global, o a un objeto o proceso específico.	ERLG46.	1
	Jenkins no soporta la autenticación JWT.	ERLG46.	1
	El nivel máximo de permisos de usuario son Overall/Administer.	ERLG46.	1
	Los permisos de usuario más básicos son Overall/Read.	ERLG46.	1
	Si se usa el instalador de Jenkins, la configuración por defecto es que cualquier usuario loggeado tiene permiso de administrador.	ERLG46.	1
	Si se tiene permisos de colaborador en el repositorio es posible obtener acceso ilimitado a Jenkins.	ERLG46.	1
	Para permitir la comunicación con el Docker demon es necesario agregar el usuario de Jenkins al Grupo de Docker correspondiente.	ERLG46.	1
	Las conexiones SSH a los repositorios requieren una llave privada SSH.	ERLG32.	1
	Las conexiones HTTPS a los repositorios requieren credenciales con usuario y contraseña.	ERLG32.	1
	Para conectarse con un repositorio privado de Git es necesario proporcionar credenciales.	ERLG30.	1
	Para proporcionar acceso a Jenkins solo se requiere crear el usuario y compartir la URL.	ERLG27.	1
	Para conectarse con nodos o servidores es necesario especificar un usuario y una llave privada.	ERLG27.	1
	Los Grupos de Seguridad se configuran con las roles y políticas IAM.	EP101.	1
	Se accede a docker con un usuario root por defecto.	EP48.	1

Otros aspectos de seguridad relacionados al control de accesos reportados están directamente relacionados con el nivel de privilegios de los usuarios, configuraciones y permisos de usuarios preestablecidos por defecto, y mecanismos de autenticación soportados por Jenkins.

En lo que respecta a la seguridad del entorno en el que se ejecuta Jenkins, se identificaron nueve aspectos de seguridad siendo la actualización de plugins y extensiones el más repetido como se muestra en la Tabla 45.

Tabla 45. Aspectos de seguridad para considerar en Jenkins enfocados en la seguridad del entorno de ejecución.

Dominio de Jenkins	Aspecto de seguridad	Estudios que reportan	Total
Seguridad del Entorno de Jenkins.	Actualización y gestión de Plugins y Extensiones	ERLG05, ERLG14.	2
	Existen operaciones intrínsecamente seguras en Jenkins y que no requieren aprobación para ejecutarse.	ERLG23.	1

	Los entornos de Jenkins al crecer requieren que los modelos de confianza evolucionen también.	ERLG46.	I
	Los scripts aprobados pueden ejecutarse en cualquier funcionalidad o plugin de Jenkins fuera del Sandbox.	ERLG46.	I
	Versiones antiguas del CLI de Jenkins son consideradas inseguras.	ERLG46.	I
	Utilizar la API de Jenkins sin las precauciones adecuadas puede generar problemas de seguridad y rendimiento.	ERLG46.	I
	Jenkins requiere permisos suficientes al ejecutarse como Servicio.	ERLG46.	I
	Existen diversas Dependencias dentro del Pipeline.	EP44.	I
	Jenkins sigue el modelo de comunicación master-slave	EP23.	I

Entre los otros aspectos de Seguridad del entorno de Jenkins se encuentran aspectos relacionados la integración de dependencias y scripts externos o la propia API de Jenkins, al uso de versiones inseguras de Jenkins, y al modelo de comunicación que se utiliza para interactuar entre los nodos.

Uno de los principales beneficios de Jenkins sobre otras herramientas de CI/CD es su flexibilidad para integrar una amplia variedad de herramientas externas, para lograr esto es necesario realizar las configuraciones de seguridad necesarias para gestionar y utilizar de manera adecuada y segura las credenciales de autenticación de cada herramienta. En la Tabla 46 se presentan los aspectos de seguridad enfocados en la gestión de credenciales que se identificaron.

Tabla 46. Aspectos de seguridad para considerar en Jenkins enfocadas a la gestión de credenciales.			
Dominio de Jenkins	Aspecto de seguridad	Estudios que reportan	Total
Gestión de Credenciales en Jenkins.	Jenkins tiene un Sistema Gestor de Credenciales propio.	ERLG05, ERLG06, ERLG15, ERLG27, ERLG30, ERLG32, ERLG37.	3
	Los secretos nunca cambian después de ser creados.	ERLG46.	I
	La llave para descryptar secretos se encuentra en \$JENKINS_HOME/secrets/directory.	ERLG46.	I
	Las credenciales tienen un alcance limitado a los pipelines del folder en el que se definieron.	ERLG46.	I
	El acceso al Jenkinsfile, también puede permitir el accesos a las credenciales del SCM de la organización	ERLG46.	I
	El mal uso de la función WithCredential puede ocasionar la filtración de contraseñas.	ERLG46.	I
	Para poder registra credenciales en Jenkins es necesario el Plugin Jenkins Credentials	ERLG32.	I
	Los token de autenticación se registran en la sección de credenciales.	ERLG16.	I

Como se puede observar, el aspecto a de seguridad a considerar más reportado es que Jenkins ya cuenta con una sección específica para gestionar las credenciales, por lo que no es necesario la instalación de plugins externos o harcodear las credenciales directamente en los scripts.

Cada que se ejecuta un pipeline de integración o despliegue continuo en Jenkins este realiza una serie de builds definidos por el administrador en el Jenkinsfile, durante estos builds se realizan de manera automática diversas acciones y actividades las cuales según su naturaleza pueden manejar información confidencial, por lo que su correcto aseguramiento es algo que se debe de tomar en cuenta al desarrollar los pipelines de CI/CD. En la Tabla 47 se muestran los aspectos de Seguridad que se deben considerar al realizar los builds.

Tabla 47. Aspectos de seguridad para considerar en Jenkins enfocados en el aseguramiento de los builds.			
Dominio de Jenkins	Aspecto de seguridad	Estudios que reportan	Total
Seguridad de los Builds.	Jenkins permite el uso de comillas simples o dobles durante la interpolación de Groovy.	ERLG46.	I
	Jenkins permite la entrada de parámetros de usuario, al usarse con la interpolación de cadenas de Groovy puede generar vulnerabilidades de inyección SQL.	ERLG46.	I
	Utilizar la interpolación de Groovy para pasar credenciales con caracteres especiales puede resultar en pérdida de credenciales.	ERLG46.	I
	Las librerías marcadas como "Confiables" pueden ejecutar métodos en todo Jenkins.	ERLG46.	I
	Existen dos mecanismo de seguridad principales en el proceso de scripting: Groovy SandBox y Aprobación de Script.	ERLG46.	I
	Groovy SandBox permite ejecutar scripts y pipelines declarativos sin necesidad de permisos root.	ERLG46.	I
	Al utilizar multibranching Jenkins realiza builds automáticamente por cada pull Request.	ERLG46.	I
	Los builds se ejecutan como usuarios internos del sistema.	ERLG46.	I

Como se muestra en la figura anterior, estos aspectos de seguridad estan principalmente relacionados con la forma que se ejecutan los builds, en específico con la compilación del código Groovy de los Jenkinsfile la cual si se realiza de manera inadecuada puede provocar la filtración de información sensible.

En lo que respecta a los puertos y servicios que se exponen al utilizar Jenkins, se identificaron siete aspectos de seguridad a considerar, en la Tabla 48 se puede apreciar la frecuencia con que se reportó cada uno de estos.

Tabla 48. Aspectos de seguridad para considerar en Jenkins enfocadas en los puertos y servicios expuestos.

Dominio de Jenkins	Aspecto de seguridad	Estudios que reportan	Total
Puertos y Servicios Expuestos	Jenkins se expone en el puerto 8080.	EP05, EP07, EPI01, ERLG02, ERLG12, ERLG27, ERLG46.	8
	Para acceder a Jenkins por HTTPS o SSH se deben crear grupos de seguridad.	ERLG46.	1
	Los agentes se comunican mediante WebSocetk por defecto.	ERLG46.	1
	El puerto de comunicación TCP para conectarse con agentes esta deshabilitado por defecto.	ERLG46.	1
	Los recursos estáticos (URL) no requieren autenticación en Jenkins.	ERLG46.	1
	El puerto TCP 5000 está expuesto por default en la imagen de Docker de Jenkins.	ERLG30.	1
	Para acceder a Jenkins por HTTPS o SSH se deben crear grupos de seguridad.	EP97.	1

Como se puede apreciar en la gráfica anterior, el aspecto más reportado es el hecho de que Jenkins se expone por defecto en el puerto 8080, lo que implica que es necesario crear grupos de seguridad y reglas para acceder de manera segura a este puerto. Otros de los aspectos identificados estan relacionados con otros puertos expuestos como el TCP, asi como los distintos mecanismos de comunicación para para ingresar e interactuar con la instancia de Jenkins.

El controlador de Jenkins es el núcleo principal de esta herramienta, por lo que es necesario asegurar su protección, integridad y buen funcionamiento al realizar configuraciones en el entorno que Jenkins que puedan afectar al controlador principal, o ejecuciones que tengan acceso a este elemento tan crucial, en la Tabla 49 se muestran los aspectos seguridad a considerar respecto al Controlador de Jenkins.

Tabla 49. Aspectos de seguridad para considerar en Jenkins enfocadas en el aseguramiento del controlador.			
Dominio de Jenkins	Aspecto de seguridad	Estudios que reportan	Total
Seguridad del Controlador.	Si se usa un proxy reverso, este debe tener la misma ruta de contexto que el controlador principal.	ERLG46.	1
	La llave del controlador principal usada para encriptar información se encuentra en \$JENKINS_HOME/secrets/hudson.ut.il.secret_file.	ERLG46.	1
	Los procesos que se ejecutan en el nodo build-in tienen accesos al sistema de archivos del controlador principal.	ERLG46.	1
	El sistema de control de accesos del controlador principal evita que los agentes envíen comandos maliciosos.	ERLG46.	1
	El Jenkins Controller es el núcleo principal de Jenkins.	ERLG46.	1

Un aspecto que se menciona principalmente en la documentación oficial de Jenkins es la exposición a vulnerabilidades de Falsificación de Solicitud entre Sitios (CSRF), por lo

que Jenkins implementa por defecto mecanismos de protección contra esta vulnerabilidad, sin embargo el uso de estos mecanismos puede limitar las capacidades y funcionalidades de Jenkins (Como la generación de reportes HTML), por lo que es necesario tomar estos mecanismos en consideración al momento de determinar que actividades de planea automatizar.

Tabla 50. Aspectos de seguridad para considerar en Jenkins enfocados en la protección CSRF			
Dominio de Jenkins	Aspecto de seguridad	Estudios que reportan	Total
Protección CSRF.	Cada petición a Jenkins requiere de un token (crumb) para ser aceptada.	ERLG46.	1
	Jenkins utiliza protección CSRF por defecto.	ERLG46.	1
	Para archivos que provienen de fuentes sin confianza, Jenkins utiliza cabeceras CSP, permitiendo únicamente CSS e imágenes.	ERLG46.	1
	Cualquier cuenta puede editar su perfil, bajo ciertas circunstancias se puede ocasionar ataques XSS.	ERLG46.	1

Finalmente, solo un estudio reportó algo relacionado a la gestión de las variables de entorno en Jenkins, el estudio ERLG46 menciona que “Las variables de entorno se pueden filtrar según lo que defina el administrador”, por lo que el administrador debe tener cuidado al realizar dichas configuraciones para no dejar expuesta información sensible.

En lo que respecta a las estrategias de mitigación de vulnerabilidades, se lograron identificar un total de 76 estrategias enfocadas en diversos aspectos y areas de Jenkins. Con base en las clasificaciones propuestas en el reporte Secure Coding Practices Quick Reference Guide publicada por la OWASP Foundation (2010) se clasificaron las 76 estrategias según el tipo de práctica de seguridad, esta clasificación contempla un total de catorce tipos de prácticas de seguridad, las cuales son: Validación de entradas, Control de acceso, Seguridad en la comunicación, Codificación de salidas, Prácticas criptográficas, Configuración del sistema, Autenticación y gestión de contraseñas, Manejo de errores y registros, Seguridad de bases de datos, Gestión de sesiones, Protección de datos, Gestión de archivos y Prácticas Generales de Codificación. Además, se decidió agregar otra clasificación denominada “Pruebas de Seguridad” para agrupar estrategias enfocadas en detectar vulnerabilidades,

fallos y amenazas de seguridad de manera activa en el pipeline de CI/CD. En la Tabla 51 se muestran clasificaciones y su respectiva descripción.

Tabla 51. Estrategias de seguridad enfocados en la configuración del sistema.	
Clasificación de Prácticas de Codificación Segura.	Descripción de la práctica.
Configuración del sistema	Estrategias de seguridad enfocadas en la configuración adecuada del entorno general del sistema y de los componentes que lo rodean.
Autenticación y gestión de contraseñas	Estrategias orientadas a establecer mecanismos de seguridad con el fin de verificar la identidad de los usuarios de manera segura y eficaz, así como la creación almacenamiento y uso adecuado de las credenciales y contraseñas de autenticación.
Control de acceso	Estrategias dirigidas en restringir en limitar el accionar de los usuarios restringiendo las funcionalidad y datos que permite el sistema según el nivel y tipo de privilegios definidos. Asegurando que solo usuarios autorizados puedan ejecutar ciertas acciones.
Seguridad en la comunicación	Estrategias de seguridad que buscan asegurar los canales de comunicación y de transmisión de información mediante mecanismos de cifrado, certificados, entre otros.
Protección de datos	Estrategias de seguridad que tienen el objetivo de garantizar el manejo seguro de los datos e información sensible,
Pruebas de Seguridad	Estrategias de seguridad enfocadas en identificar vulnerabilidades y fallos tanto en la infraestructura del pipeline, en la propia herramienta de Jenkins y en el software que se está compilando y desplegando
Manejo de errores y registros	Estrategias de seguridad centradas en implementar mecanismo y lógicas que permitan al sistema prevenir fallas e interrupciones de servicios, o en su caso fallar de manera adecuada, permitiendo la recuperación y puesta en marcha nuevamente.
Validación de entradas	Estrategias de seguridad que tiene el objetivo de asegurar que los datos de entrada cumplan con los tipos, longitudes, rangos y formatos esperados por el sistema.
Gestión de archivos	Estrategias de seguridad orientadas al manejo adecuado y aseguramiento de archivos de sistema. así como en la prevención de carga de archivos externos maliciosos.
Gestión de sesiones	Estrategias de seguridad centradas en definir mecanismos para crear, mantener y destruir las sesiones de los usuarios de manera segura, tales como la protección de cookies, generación de identificadores o límites de tiempo.
Prácticas criptográficas	Estrategias enfocadas en implementar mecanismos de criptografía de manera adecuada para el aseguramiento de la información.
Codificación de salidas	Estrategias de seguridad en las que se aplican mecanismos de codificación previo a devolver los datos directamente al cliente, asegurando que los caracteres no se interpreten como comando maliciosos que pongan en riesgo la integridad del equipo del cliente.
Seguridad de bases de datos	Estrategias de seguridad centradas en la protección exclusivamente de la base de datos.

Como se puede observar en la tabla anterior, las estrategias de seguridad identificadas estan principalmente relacionadas con configuraciones generales de Jenkins, con los mecanismos de autenticación para ingresar a la instancia de Jenkins, asi como la gestion de las credenciales que se usan para interactuar con otras herramientas externas, con el control de accesos y privilegios de accionar de los usuarios en Jenkins, y con mecanismos para asegurar la comunicación entre los distintos componente s de Jenkins. Esta distribución se puede observar en la Figura 44.

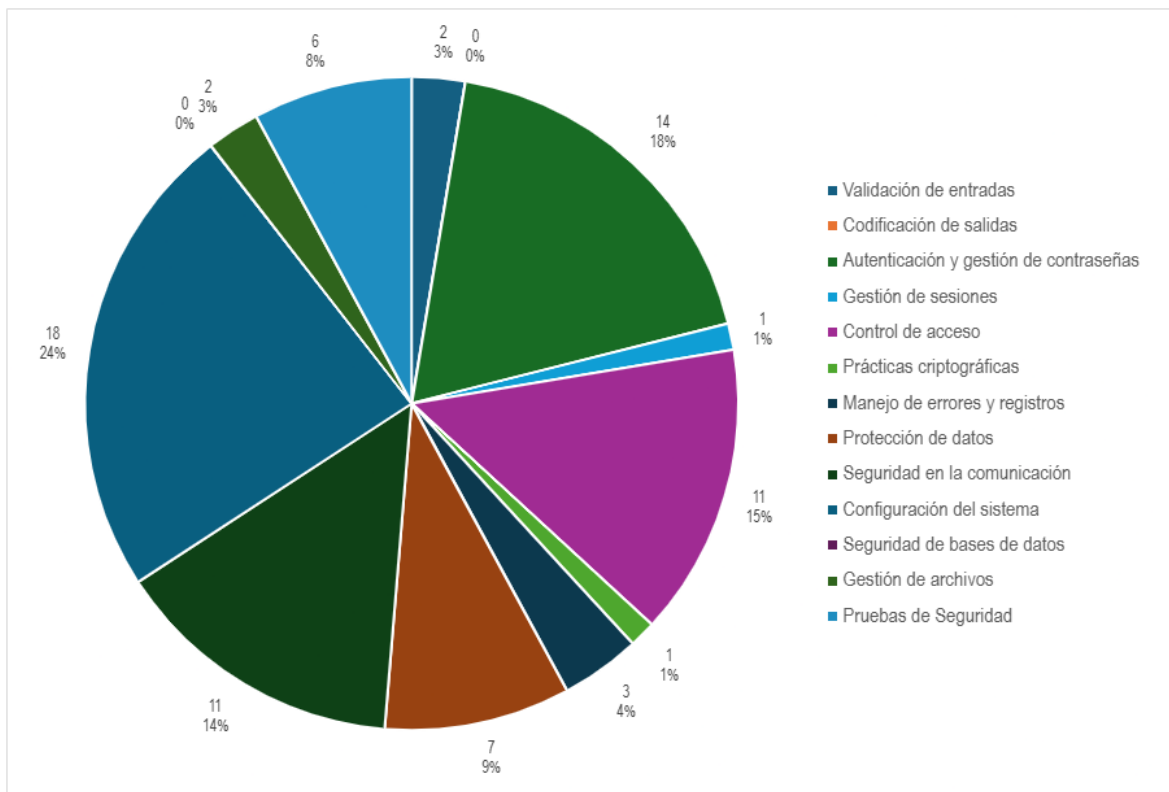


Figura 44. Distribución de estrategias de seguridad.

Otros tipos de estrategias reportados estan estrechamente relacionados con la validación de las entradas, prácticas criptográficas, manejo de errores y logs, protección de datos y gestión de archivos, es importante mencionar que no se identificó ninguna practica relacionada a la codificación de salidas, ni de seguridad de la base de datos.

En la Tabla 52 se puede consultar la frecuencia de las estrategias de seguridad para la configuración del sistema.

Tabla 52. Estrategias de seguridad enfocados en la configuración del sistema.			
Dominio de Jenkins	Aspecto de seguridad	Estudios que reportan	Total
Configuración del sistema	Actualizar regularmente Jenkins y sus plugins	ERLG05, ERLG14.	2
	Cambiar el puerto default si se usa un firewall.	ERLG46.	1
	Asegurar aislamiento de la red desplegando las instancias en un Ambiente privado	EP101.	1
	Limitar el uso de plugins lo más posible.	ERLG14.	1

	Los administradores no deben desviarse de los mecanismo de seguridad Groovy Sandbox y Script Approval de no ser estrictamente necesario.	ERLG46.	I
	Utilizar plugins de seguridad	ERLG50.	I
	Utilizar una cuenta genérica de GitHub para conectarse a Jenkins	ERLG39.	I
	Bloquear la UI de Jenkins para permitir una autenticación obligatoria y control de accesos.	ERLG46.	I
	Crear ejecutables autónomos para acceder y utilizar a librerías de terceros, en lugar de la función @Grab.	ERLG46.	I
	Revisar los permisos de los plugins para agregar variables de entorno de fuentes que no requieren permisos de configuración de Jobs	ERLG46.	I
	Utilizar una cuenta de servicio para correr el Job de Jenkins	EP51.	I
	Si se ejecuta Jenkins como un servicio de Windows independiente, se debe utilizar un usuario/dominio local	ERLG46.	I
	No ejecutar las tareas/procesos en el nodo maestro de Jenkins	ERLG46.	I
	Emplear contenerización para aislar los distintos ambientes.	ERLG14.	I
	Utilizar agentes con distintos OS para alcanzar un efecto de aislamiento.	ERLG46.	I
	Implementar builds distribuidos para evitar scripts maliciosos.	ERLG46.	I
	Utilizar servicios de nube que creen nuevos agentes con cada compilación.	ERLG46.	I
	Realizar chequeos regulares de las configuraciones de seguridad deshabilitadas.	ERLG46.	I

Como se puede observar, esta categoría agrupa estrategias relacionadas con la gestion y administración de plugins, configuraciones del entorno en el que se ejecuta Jenkins que tienen el objetivo de asegurar los pipelines de CI/CD evitando scripts malicioso, cuellos de botellas, ataques de negación de servicios, entre otros. Algunas de estas estrategias pueden trazarse tambien a otras de las categorías como el control de accesos o la seguridad en la comunicación, sin embargo se decidió dejarlas en este grupo debido a que su enfoque esta más relacionado con el entorno en el que se ejecuta Jenkins y los pipelines de CI/CD.

El segundo grupo de estrategias que más se reportó fueron las relacionadas a la autenticación en Jenkins y la gestión de las credenciales. En la Tabla 53 se muestra la frecuencia de las once estrategias identificadas.

Tabla 53. Estrategias de seguridad enfocados en la Autenticación y Gestión de contraseñas.

Dominio de Jenkins	Aspecto de seguridad	Estudios que reportan	Total
Autenticación y gestión de contraseñas	Restringir el acceso a la instancia de Jenkins	EP51, EP97, ERLG38, ERLG53.	4
	Utilizar el sistema manejador de credenciales de Jenkins para almacenar información sensible.	ERLG05, ERLG37.	2
	Asegurar el controlador de Jenkins al cambiar la contraseña.	ERLG46.	I
	Implementar Key pair Auth para acceder a los recursos de pipeline	EP97.	I
	Utilizar un archivo para cargar las credenciales de autenticación.	ERLG46.	I
	Definir cada credencial al mínimo nivel posible	ERLG46.	I
	Utilizar tokens en lugar de la contraseña real para invocar clientes automatizados.	ERLG46.	I
	Es preferible usar Tokens para la autenticación.	ERLG46.	I
	Utilizar factor de autenticación en dos pasos en el correo de notificaciones	EP97.	I
	Limitar el alcance de los usuarios para acceder a las credenciales.	ERLG46.	I
	Evitar que los Jobs no confiables usen credenciales confiables.	ERLG46.	I
	Utilizar el Snippet Generator para especificar las credenciales en lugar del step WithCredentials	ERLG46.	I

	No almacenar las credenciales en el SCV.	ERLG46.	I
	Modificar las credenciales frecuentemente.	ERLG46.	I

Como se puede observar en la tabla anterior, la estrategia de seguridad más mencionada es “Restringir el Acceso a la instancia de Jenkins” mediante el uso de algun metodo de autenticación, los estudios que mencionan esta práctica no especifican que mecanismo es el mejor, sin embargo otros estudios mencionan el uso de archivos para cargar las credenciales de autenticación, utilizar tokens de autenticación o utilizar autenticación “Key pair”. Asi mismo se mencionan estrategias para el almacenamiento seguro de las credenciales como: No almacenarlas en el SCV y en su lugar utilizar el sistema manejador de Credenciales de Jenkins, y estrategias enfocadas en el uso adecuado de las credenciales, con el fin de evitar la pérdida de estas o su exposición accidental, tales como: Utilizar el Snippet Generator en lugar de la funcion WithCredentials o limitar el acceso de los usuarios y de los Jobs a las credenciales.

La tercera categoría con más estrategias reportadas fue la de Control de accesos con un total de once, estas se pueden consultar en la Tabla 54.

Tabla 54. Estrategias de seguridad enfocadas en el control de accesos.			
Dominio de Jenkins	Aspecto de seguridad	Estudios que reportan	Total
Control de acceso	Implementar control de acceso (Basado en roles o IAM)	EP24, EPI00, EPI01, ERLG05, ERLG14, ERLG14, ERLG38, ERLG46, ERLG52.	9
	Definir un grupo con reglas de seguridad específicas para acceder a la IP y puerto donde se aloja Jenkins	EP05, EP07, EP97, EPI01.	4
	Ejecutar los procesos con un usuario sin privilegios de root	EP48, ERLG46.	I
	No deshabilitar el sistema de control de accesos de los agentes.	ERLG46.	I
	Utilizar el plugin Matrix Authorization Strategy.	ERLG12.	I
	Utilizar el Plugin Role-Base Authorization Strategy	ERLG14.	I
	Evitar conceder permisos a los usuarios anónimos.	ERLG46.	I
	Utilizar el Plugin Authorize Project Plugin	ERLG46.	I
	Asignar y segregar a los usuarios a distintos equipos de proyectos.	ERLG52.	I
	Crear un grupo de Docker compuesto de usuarios confiables	EP48.	I

	Dejar que los colaboradores decidan cuando realizar el build en lugar de generarlo por cada pull request.	EP48.	1
--	---	-------	---

Es importante mencionar que en esta categoría se encuentra la estrategia de seguridad más reportada de todas con nueve menciones, la estrategia “Implementar controles de acceso basados en roles o IAM” fue la que más se repitió en distintos estudios. Otras estrategias que se encontraron están relacionadas con el uso de ciertos plugins específicos que permiten la implantaciones y configuración de mecánicas de privilegios, así como la segmentación de usuarios y creación de grupos para una gestión más eficiente.

Al igual que las estrategias de Control de accesos, también se reportaron un total de once estrategias relacionadas a la comunicación en Jenkins, ya sea entre los nodos y agentes que este maneja o con herramientas externas. En la Tabla 55 se pueden consultar estas estrategias.

Tabla 55. Estrategias de seguridad enfocadas en la seguridad de la comunicación.			
Dominio de Jenkins	Aspecto de seguridad	Estudios que reportan	Total
Seguridad en la comunicación	Utilizar HTTPS para la comunicación segura.	ERLG02, ERLG05.	2
	Generar un token SSH RSA para establecer una conexión segura en el modelo master-slave	EP23.	1
	Utilizar credenciales para acceder de manera segura a sitios y aplicaciones externas que interactúan con Jenkins	ERLG46.	1
	Utilizar SSH tunneling para iniciar sesión e interactuar con Jenkins	ERLG02.	1
	Configurar transporte WebSocket para agentes entrantes.	ERLG46.	1
	Proteger la comunicación master-slave con usuario y contraseña	EP23.	1
	Para acceder a la API de Jenkins durante la compilación, crear un plugin sencillo en Java que implemente un Safe Wrapper alrededor de la API de Jenkins a la que se quiere acceder	ERLG46.	1
	Utilizar autenticación basada en contraseñas para conectarse a los servidores SSH.	ERLG25.	1
	Si se habilita el puerto TCP, utilizar la configuración fija para facilitar la administración con el firewall.	ERLG46.	1
	Utilizar un Socket Unix en lugar del puerto TCP para el proxy inverso	ERLG46.	1
	Cambiar el puerto por defecto 0.0.0.0 a 127.0.0.1	ERLG46.	1

La principal estrategia de comunicación, con dos menciones, es utilizar el protocolo HTTPS para conectarse y comunicarse con la instancia de Jenkins. Otras estrategias de comunicación que se enfocan en establecer una conexión segura con los agentes de Jenkins se mencionaron únicamente en una ocasión, de la misma forma estrategias relacionadas con la incorporación de aplicaciones externas y de la API de Jenkins fueron reportadas en la literatura, sin embargo todas estas estrategias únicamente fueron mencionadas una vez.

En lo que respecta a protección de datos, se identificaron un total de siete estrategias, de las cuales únicamente dos de estas (Utilizar comillas simples en lugar de comillas dobles para datos sensibles en la interpolaciones de Groovy, y Usar variables de entorno para implementar las credenciales en lugar de hardcodearlas) fueron reportadas en más de un estudio como se muestra en la Tabla 56.

Tabla 56. Estrategias de seguridad enfocadas en la protección de datos.			
Dominio de Jenkins	Aspecto de seguridad	Estudios que reportan	Total
Protección de datos	Utilizar comillas simples en lugar de comillas dobles para datos sensibles en las interpolaciones de Groovy .	ERLG46, ERLG46.	2
	Usar variables de entorno para implementar las credenciales en lugar de hardcodearlas.	ERLG37, ERLG39.	2
	No compartir la URL del servidor de Jenkins con nadie fuera de la organización.	ERLG46.	1
	La interpolación de cadenas de Groovy nunca debe usarse con credenciales, puede exponer variables de entorno sensibles.	ERLG46.	1
	Utilizar Hashicorp Vault para almacenar los secretos	ERLG24.	1
	Jamás incluir la llave de control en el respaldo de Jenkins.	ERLG46.	1
	Implementar técnicas de gestión de secretos y API-keys para evitar la exposición de secretos.	ERLG46.	1

Parte de las estrategias identificadas estan relacionadas con la interpolación que realiza Groovy durante los builds para leer las credenciales y variables de entorno, esto se menciona debido a que el uso incorrecto de comillas puede dejar expuesta información sensible en los logs de Jenkins al momento de realizar los builds, así mismo otras estrategias se enfocan en la gestión adecuada de los secretos y de los recursos que se podrían resultar en alguna brecha de seguridad.

La categoría de Pruebas de Seguridad engloba estrategias y actividades que los desarrolladores y administradores de Jenkins deberían realizar constantemente en los pipelines de CI/CD para identificar vulnerabilidades y fallos tanto en la infraestructura del pipeline, en la propia herramienta de Jenkins y en el software que se está compilando y desplegando. En la Tabla 57 se muestran estas estrategias.

Tabla 57. Estrategias de seguridad enfocadas en pruebas de seguridad.

Dominio de Jenkins	Aspecto de seguridad	Estudios que reportan	Total
Pruebas de Seguridad	Realizar monitoreos de seguridad (IAST)	EP24, EP96, ERLG14, ERLG24.	4
	Realizar pruebas de seguridad (DAST)	EP24, EP96.	2
	Realizar escaneos de seguridad (SAST).	EP24, EP96.	2
	Realizar pruebas A/B	EP98.	1
	Realizar Análisis de Composición de Software (SCA).	EP96.	1
	Implementar un modelo de amenazas de aplicación	EP44.	1

Como se puede ver estas estrategias estan principalmente relacionadas con realizar Monitores, pruebas y escaneos de seguridad, siendo los monitoreos de seguridad los más mencionados con un total de cuatro, seguido de las pruebas de seguridad dinámicas y los escaneos de seguridad estáticos con dos menciones cada uno.

Únicamente se identificaron tres estrategias de seguridad relacionadas al manejo de errores y registros, enfocadas en garantizar la disponibilidad del servicio de Jenkins implementando mecanismos que eviten cuellos de botella o fallos de dependencias que puedan comprometer la integridad y eficiencia de la ejecución de los pipelines. Como se muestra en la Tabla 58.

Tabla 58. Estrategias de seguridad enfocadas en el Manejo de errores y registros.			
Dominio de Jenkins	Aspecto de seguridad	Estudios que reportan	Total
Manejo de errores y registros	Deshabilitar los git hooks en el controlador y agentes de Jenkins para prevenir la ejecución no controlada de scripts	ERLG32.	1
	Implementar TimeOut en los Jenkinsfiles.	ERLG46.	1
	Simular fallos para detectar problemas en los componentes del pipeline	EP44.	1

Por otra parte, solamente se identificaron dos estrategias enfocadas en al gestion de archivos de Jenkins, dichas estrategias se enfocan en aislar y mantener seguras los archivos básicos del sistema de Jenkins como se muestra en la Tabla 59.

Tabla 59. Estrategias de seguridad enfocadas en la gestión de archivos.			
Dominio de Jenkins	Aspecto de seguridad	Estudios que reportan	Total
Gestión de archivos	Los procesos de agentes no deben tener acceso a los archivos de sistema de Jenkins, ni permisos root.	ERLG46.	1
	Jamás incluir el directorio de secretos en los respaldos de Jenkins.	ERLG46.	1

Entre los archivos más importantes que se deben de proteger al utilizar Jenkins, se encuentran el directorio de secretos, y la carpeta raíz que contienen los archivos de sistema y núcleo de Jenkins, es importante mencionar que para realizar un respaldo de los pipelines y de la propia herramienta, solo es necesario copiar y pegar la carpeta raíz de Jenkins, lo que indica que toda la información y configuración importantes e encuentran ahí, y por eso es tan importante mantener segura esta carpeta.

De la misma forma, se encontraron dos estrategias relacionadas a la sanidad y seguridad de los parámetros y variables de entorno que se ingresan en los pipelines y scripts de Jenkins, estas se muestran en la Tabla 60.

Tabla 60. Estrategias de seguridad enfocadas en la validación de entradas.			
Dominio de Jenkins	Aspecto de seguridad	Estudios que reportan	Total
Validación de entradas	Al aprobar script, se debe verificar que los parámetros no son explotables.	ERLG46.	1
	Definir filtros para las variables de entorno que se usan durante el build	ERLG46.	1

Finalmente, solo se identificó una práctica de criptografía la cual consiste en utilizar mecanismos de encriptación (EP24 y EP100). Y una práctica de Gestión de sesión que recomienda dejar habilitada la protección CSRF para evitar ataques XSS (ERGL46).

5 Discusión

El uso de Jenkins en trabajos de investigación se ha mantenido constante durante los últimos ocho años. Entre 2016 y 2018, la cantidad de estudios acerca de la herramienta Jenkins fueron en aumento. Sin embargo, un año después, el número de publicaciones disminuyó drásticamente, reduciéndose a menos de la mitad. Posteriormente, esta cifra volvió a alcanzar el máximo de estudios publicados en años previos.

Esta tendencia se repite en los años 2021 y 2022, donde ocurrió otra disminución seguida de un aumento en la publicación de estudios sobre Jenkins. Es a partir de los últimos dos años que la investigación se ha mantenido constante, y continua en aumento, lo cual podría ser un indicio de que la investigación sobre el tema seguirá siendo constante o incluso podría ir al alza en los próximos años.

Además, es importante mencionar que la investigación en literatura blanca sobre Jenkins parece estar concentrada en un solo tipo de *venue*, las conferencias acaparan más del 70% de las publicaciones, mientras que los *journals* aportan una cantidad significativa pero mucho menor. Por otra parte, los *workshops* y revistas apenas aportan un mínimo de

publicaciones sobre el tema. Esto se podría deber a la naturaleza del tema y a las propias características de los *venues*. Una herramienta como Jenkins suele estar asociada con estudios prácticos. Así mismo, las conferencias pueden ser preferidas para compartir innovaciones técnicas y casos de uso que tengan aplicaciones inmediatas, mientras que los *journals* profundizan en análisis más extensos.

Por otra parte, en la literatura gris, los principales recursos recuperados fueron *success-stories*, blogs y videos, en lo que respecta a las primeras, las *success-stories* son una especie de reportes cortos que describen implementaciones exitosas de Jenkins compartidas por la propia comunidad de Jenkins en la página oficial, por lo que resulta lógico que estos recursos respondan a las preguntas de investigación, algo importante que vale la pena mencionar es que en total hay alrededor de 200 *success-stories* publicadas por diversas empresas, organizaciones y desarrolladores, este dato resalta la importancia de Jenkins en el mercado, así como su popularidad. En lo que respecta a los blogs y videos, en su mayoría estos son tutoriales con el fin de enseñar de manera general el uso de Jenkins, sin embargo, no se abordan aspectos más específicos o implementaciones más complejas. Finalmente, también se identificaron algunos recursos como *White-papers*, informes y artículos electrónicos, sin embargo, estos fueron la minoría.

Contextos y escenarios en los que se suele utilizar Jenkins para la creación de pipelines de Integración y Despliegue Continuo

Se ha encontrado evidencia del uso de Jenkins en distintos contextos, siendo el contexto de laboratorio en el que mayormente se utiliza con el fin de: exponer marcos de trabajo, en los que integran Jenkins con herramientas con fines específicos; proponer modelos de automatización de procesos, en los que utilizan a Jenkins como la herramienta principal para la implementación; o para demostrar la eficiencia de Jenkins y otras herramientas en la automatización de ciertos procesos. En todos estos estudios, la creación y funcionamiento de los pipelines de CI/CD se lleva a cabo en ambientes controlados, y a pesar de que varios estudios describen la evaluación de estas implementaciones, se debe de tomar en cuenta la ausencia de factores de los entornos de desarrollo del mundo real, ya que estos pueden tener un impacto significativo en el rendimiento y comportamiento de los pipelines.

Por otra parte, aunque son menos los estudios que reportan el uso de Jenkins en proyectos y sistemas reales de la Industria, no se pudo pasar por alto que es una cantidad considerable, que demuestra el interés de las empresas y equipos de desarrollo por adoptar

este tipo de herramientas de automatización, para sus procesos. En estos estudios, los autores enfatizan los beneficios que obtuvieron al incorporar Jenkins en sus flujos de trabajo, beneficios como: la reducción de tiempo en las actividades de integración, prueba y despliegue; disminución en la cantidad de *smells* y errores que llegan a producción; o mayor cadencia en la entrega de nuevas versiones y correcciones de los sistemas.

Finalmente, con base en los resultados obtenidos en esta revisión el uso de Jenkins en contextos académicos no ha adquirido tanta relevancia como en los otros dos ámbitos. Esto podría deberse a que Jenkins es una herramienta que requiere un tiempo considerable para su aprendizaje. Además, debido a su alta curva de aprendizaje, su uso se puede ver limitado en la mayoría de los cursos de Ingeniería de Software, como talleres de pruebas, desarrollo o diseño de software, entre otros. En estos casos, el usar Jenkins no resulta eficiente por la duración limitada de los cursos. Por lo que su uso se podría ver restringido a cursos donde se enfocan en aplicar conceptos y prácticas de DevOps y de CI/CD.

En cuanto a los escenarios, se ha encontrado evidencia de que Jenkins se usa para crear pipelines de CI/CD para una gran variedad de sistemas. Los sistemas web son el principal escenario que se identificó en los estudios, lo que podría estar relacionado con los contextos antes descritos. Como se mencionó previamente, el contexto más recurrente es el de laboratorio, donde el enfoque principal son las herramientas y modelos propuesto. Esto hace que el sistema o software que se está integrando, probando o desplegando no sea tan relevante, lo que convierte a los sistemas web en una opción más accesible para la implementación de los pipelines.

Además de los sistemas web, Jenkins también es usado para microservicios y otros sistemas basados en la nube, ya que, con la configuración adecuada, Jenkins puede utilizar el mismo conjunto de pipelines para integrar, probar y desplegar distintos microservicios.

Por otra parte, también hay bastante evidencia del uso de Jenkins con sistemas que usan fundamentalmente hardware, como los sistemas ciber-físicos, embebidos y de IoT, y

que reportan el uso de Jenkins para realizar principalmente pruebas y en algunos casos desplegar el software directamente en los dispositivos de hardware.

Jenkins es tan flexible que se ha utilizado en una amplia variedad de actividades que no están relacionadas directamente con el desarrollo de software. Se ha encontrado que Jenkins se puede usar para automatizar varios procesos de aprendizaje máquina y de procesamiento de lenguaje natural, así como para crear pipelines que evalúen automáticamente proyectos de software de estudiantes, desplegar software para computación de alto rendimiento, entre otros.

Con el fin de tener un mayor entendimiento del uso de Jenkins en distintos contextos se realizó un análisis más profundo de los resultados, trazando los escenarios con los contextos reportados. En la Figura 45 se muestra la distribución de los escenarios por contextos obtenida.

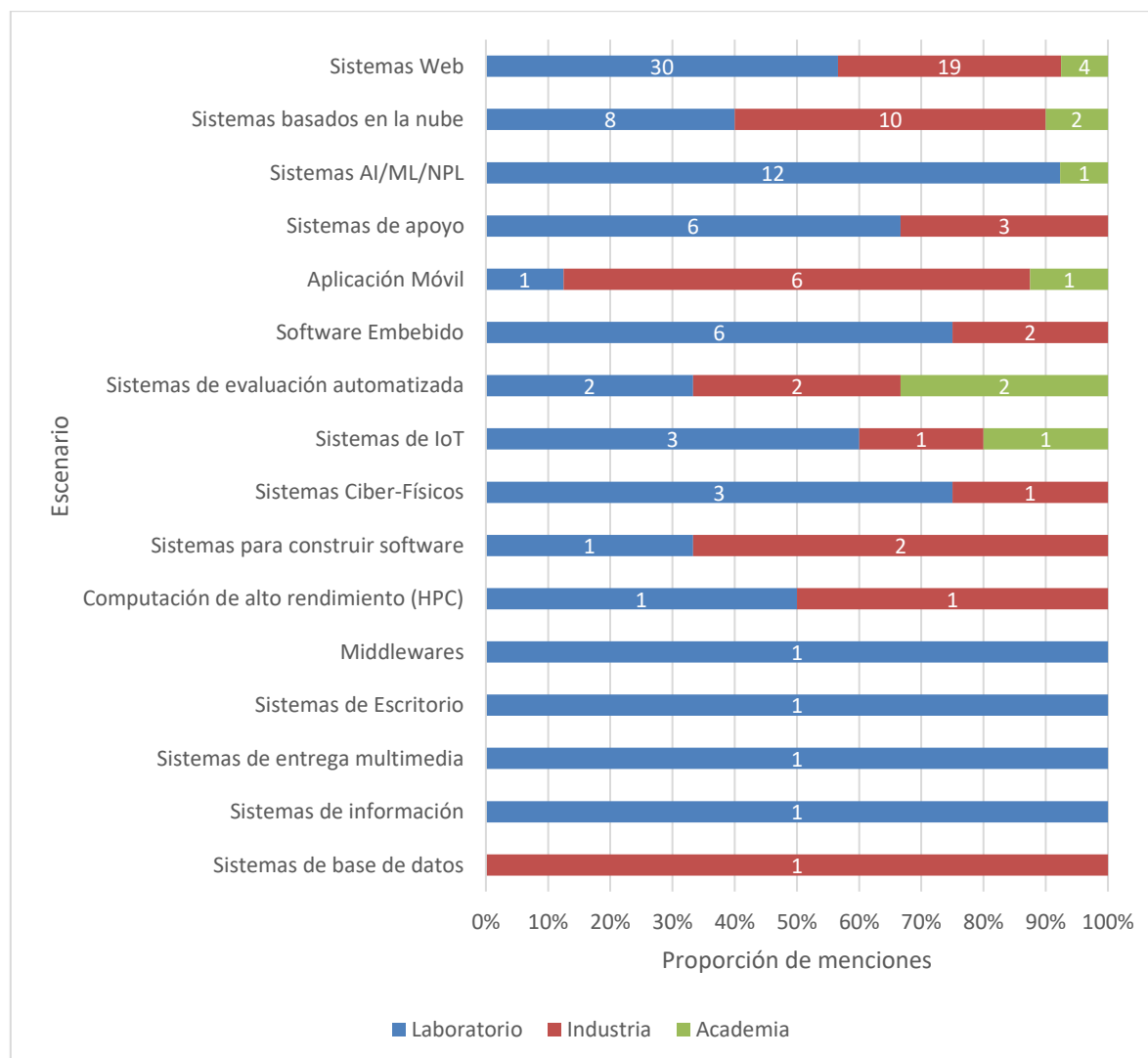


Figura 45. Distribución contextual del Uso de Jenkins por Tipo de sistema

Como era de esperarse, en la mayoría de los casos el contexto de laboratorio predomina, sin embargo en el escenario de Aplicación Móvil y Sistemas Basados en la Nube es el contexto de la Industria el que más estudios aporta, esto puede deberse a que a diferencia de los sistemas web, estos sistemas son implementaciones más complejas, por lo que utilizar este tipo de sistemas para un estudio en un ambiente controlado de laboratorio donde el sistema que se despliega no es la prioridad, no sería eficiente, por lo que se opta por un software más simple como los sistemas web, por otro lado, los sistemas basados en la nube como los microservicios si se ven altamente beneficiados por las prácticas de CI/CD, ya que permiten manejar y gestionar una amplia cantidad de microservicios de manera eficaz, algo similar ocurre con las aplicaciones móviles, la cuales por su naturaleza tienden a recibir una mayor cantidad de actualizaciones en comparación con otro tipo de sistemas, por lo que no es extraño que la mayor parte de los estudios que reportan aplicaciones móviles sean en contextos reales y no de laboratorio.

Tanto en la academia, la industria y el laboratorio, el principal tipo de sistema que se utiliza en Jenkins son los sistemas web, teniendo una cobertura de estudios que lo reportan de un 36%, 40% y 39% respectivamente para cada contexto. En el caso de la academia el segundo tipo de escenario más reportados son los sistemas de evaluación automatizada utilizados para evaluar de los proyectos y tareas de los estudiantes de manera automática, mientras que en la industria el segundo escenario más reportado son los sistemas basados en la nube, lo que se coincide con las tendencias del mercado y de desarrollo de software, las cuales utilizan cada vez más las tecnologías y herramientas para el cómputo en la nube, por otro lado en el contexto de Laboratorio el segundo escenario más reportado son los sistemas de AI/ML/NLP que tiene el propósito general de automatizar ciertas etapas de estos procesos del área de Inteligencia artificial, y presentan modelos y propuestas para integrar los pipelines de CI/CD en estos ámbitos.

Otro de los aspectos que se revisó relacionado al uso de Jenkins fue el sector o ámbito de aplicación de la herramienta, con el fin de obtener un panorama general del uso de la herramienta y ver hasta qué punto la Jenkins resulta útil y adaptivo a los distintos entornos en los que se puede utilizar.

Los resultados mostraron considerable cantidad de sectores de aplicación distintas y variadas entre sí, demostrando que Jenkins se puede utilizar desde un sector tradicional como el Desarrollo de Software Comercial, de Telecomunicaciones o Ciberseguridad hasta un sector totalmente diferente como la Aeronáutica, área de la Salud o Automotriz. Siendo este último el más reportado, es importante mencionar que muchos de los estudios que mencionan un contexto no detallan en el sector de aplicación ya sea porque no es necesario para entender el estudio o por cuestiones de confidencialidad, sin embargo en los estudios revisados que pertenecen a los sectores Automotriz, del área de la Salud, Aeronáutica o de Telecomunicaciones, si hacen descripciones más explícitas del ámbito de aplicación ya que es necesario entender el contexto y entorno en el que se está utilizando Jenkins, por ejemplo en los estudios EP11, EP16 y EP42 se hace mención explícita que se está tratando de automatizar las pruebas del software de vehículos y dispositivos ECU.

De la misma igual manera a los escenarios, se realizó un análisis más profundo de los resultados, comparando las sectores de aplicación con los contextos, en la Figura 46 se muestran los resultados de esta comparación.

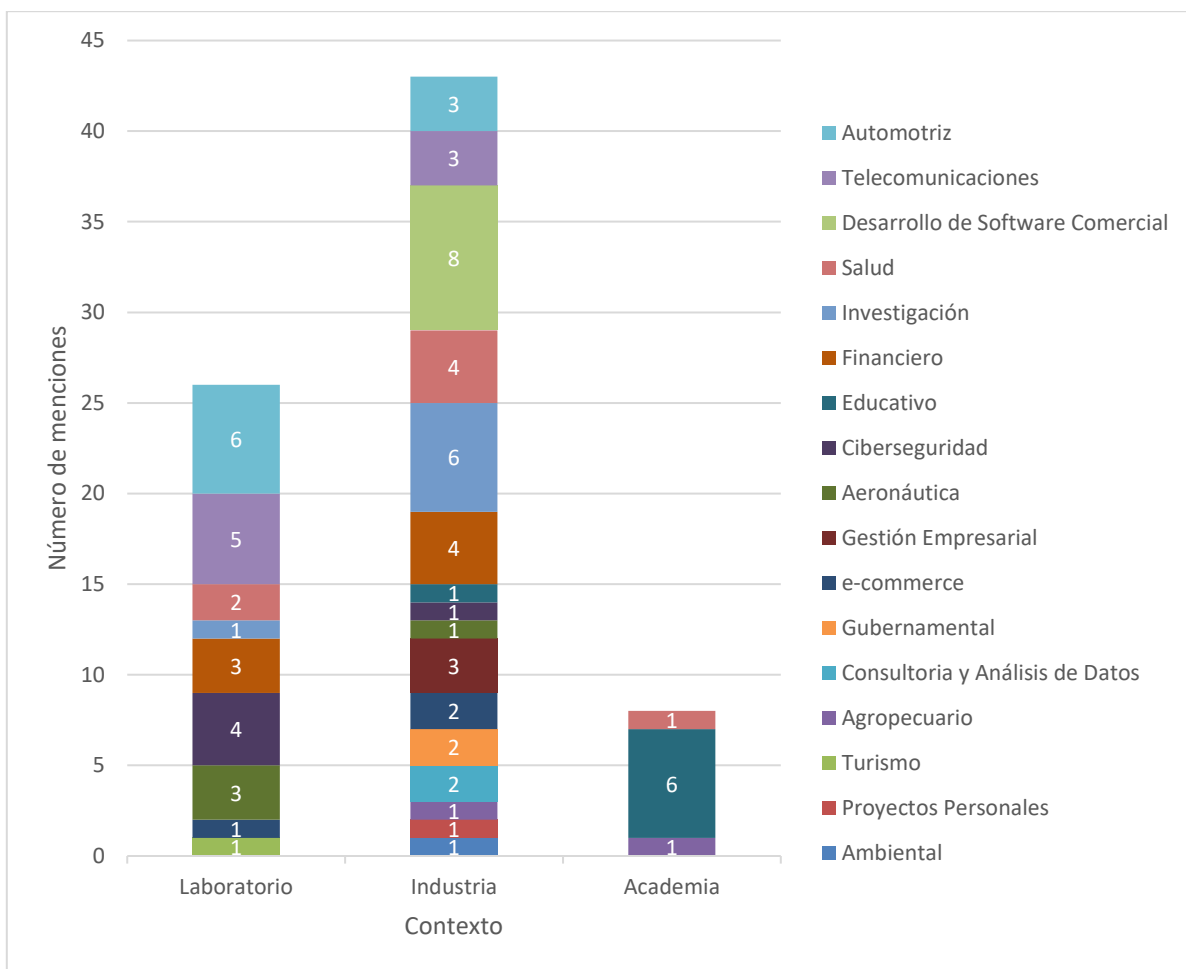


Figura 46. Distribución contextual del uso de Jenkins según el Sector de aplicación.

Un detalle particular que revela este análisis es que a pesar de que hay más estudios de laboratorio, es mayor la cantidad de estudios de la industria que reportan un sector de aplicación siendo casi el doble (43 estudios) que el contexto de laboratorio (26 estudios), esto puede ocurrir debido a que los estudios de la industria son implementaciones reales que requieren del contexto y detalles del entorno en que se llevaron a cabo para poder entenderlas de mejor manera, mientras que los estudios de laboratorio en su mayoría son propuestas de *framework*, o modelos generales de automatización de algún proceso, y que no es necesario detallar respecto al contexto social, cultural o laboral en el que se desarrollan.

En lo que respecta a la industria, el principal sector de aplicación de Jenkins es el desarrollo de software comercial, tales como aplicaciones móviles de la playstore -Catrobat's Pocket Code- (EP34), aplicaciones y servicios de GoDaddy (ERGL10), proyectos comerciales como el "Network UPS Project" (ERGL57) , entre otros. En segundo lugar se encuentra el sector de Investigación, en el que utilizan Jenkins para automatizar proceso o dar soporte a sistemas que se usan en este campo, tales como la "EGI e-infrastructure" en 300 centros de investigación europeos (EP79), herramientas de analisis de patrones (EP80) o dar soporte a los sistemas de un centro de investigación multinacional como el CERN (ERGL40).

Mientras que en el contexto de laboratorio, el principal área de aplicación es la automotriz seguida de las Telecomunicaciones, estos se deben a que los estudios que reportan esto, son estudios en los que se está integrando Jenkins en los procesos automotrices (Especialmente en las pruebas del software) y de telecomunicaciones como algo novedoso y que trata de ser revolucionario en su sector, por lo que no son implementaciones en contextos de producción real.

Por otro lado, en la Academia, como es de esperarse, la principal área de aplicación es en el ámbito educativo, donde se usa Jenkins para evaluar a proyectos y tareas de los alumnos (EP50), así como para dar retroalimentación de manera activa (EP58), esto se realizar en específico en cursos de Ingeniería de software o computación donde los estudiantes tienen que desarrollar algun proyecto al cual realizar pruebas, análisis estático de código, detección de *smells*, entre otros. En segundo lugar se encuentra el área de la salud, así como el sector agropecuario.

Herramientas que se suelen integrar con Jenkins para la creación de pipelines de Integración y Despliegue Continuo.

Las principales herramientas que se utilizan con Jenkins tienen relación directa con los procesos y etapas fundamentales de los pipelines de Integración y Despliegue Continuo. Las herramientas que ayudan a la integración, compilación, ejecución de pruebas y despliegue del software, son las que más se mencionan, debido a que estas actividades son las que normalmente se automatizan en los pipelines. Consecuentemente, herramientas como Git, Apache Maven, Selenium, SonarQube, Ansible, Docker, Kubernetes o AWS son altamente utilizadas e integradas con Jenkins.

Además, se puede apreciar el uso de herramientas suplementarias que también se usan moderadamente en conjunto con Jenkins, tales como repositorios de artefactos, herramientas de monitoreo, notificaciones y de virtualización. Estos tipos de herramientas permiten ampliar las capacidades y funcionalidades de los pipelines de Integración y Despliegue Continuo.

Por otra parte, también se reportan herramientas que tienen un uso muy específico en el contexto de los estudios, y que normalmente no se utilizarían en un pipeline de Integración y Despliegue Continuo. Pero que demuestran la capacidad y flexibilidad de Jenkins para poder integrarse con una amplia gama de herramientas, como ofusadores de código, marcos de trabajo de ingeniería inversa, gestores de bases de datos de grafos, entre otras.

Tratando de tener un mayor entendimiento de las herramientas que se usan en la industria y en la academia, se hizo el esfuerzo por realizar un análisis más profundo de estos aspectos, trazando las herramientas reportadas en cada estudio al contexto del mismo estudio. Los resultados se presentan en la Figura 47.

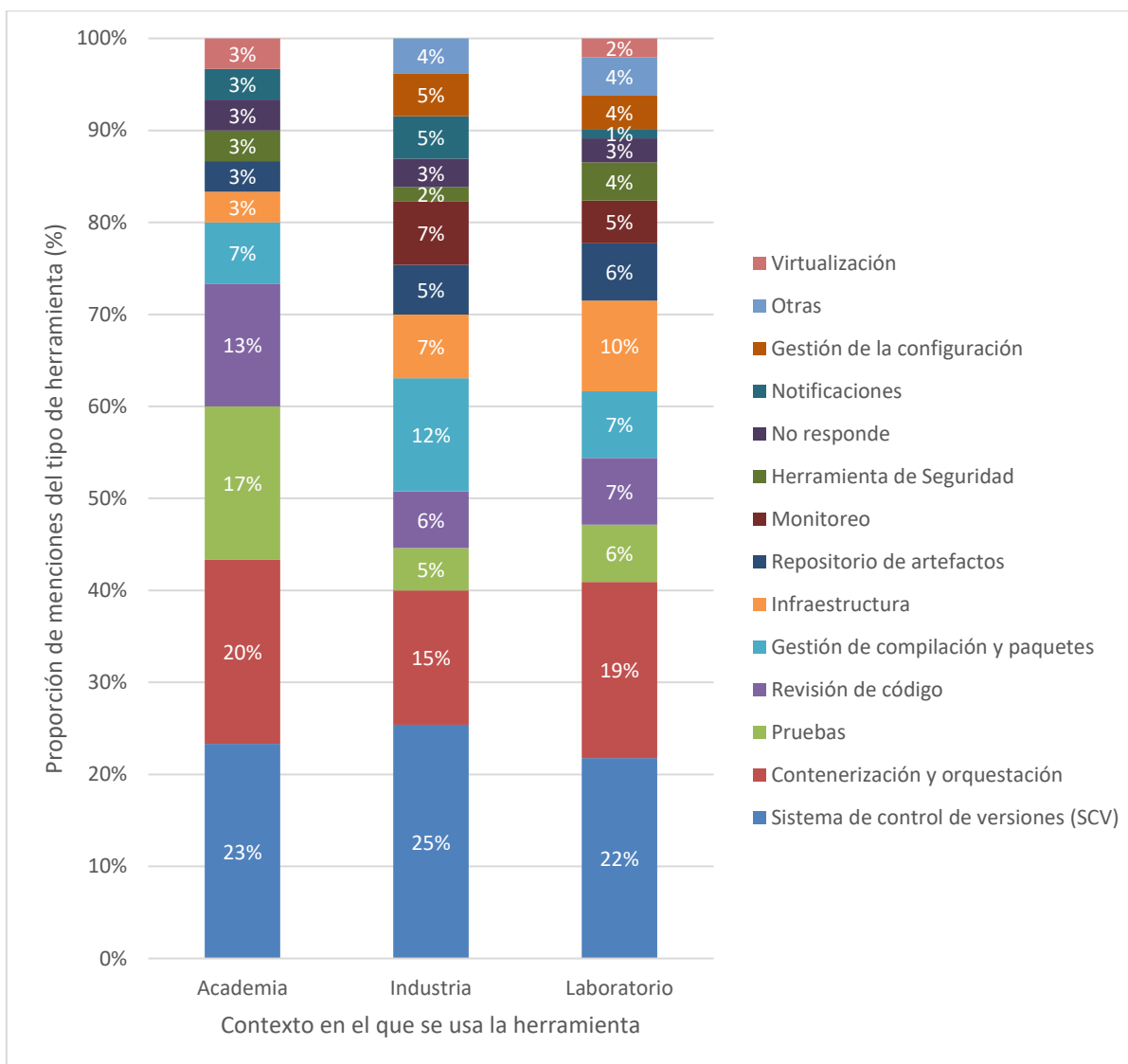


Figura 47. Distribución contextual del uso de herramientas complementarias con Jenkins.

Es evidente que en los tres contextos se mantiene proporciones similares en cuanto al uso de las herramientas complementarias a Jenkins, en términos generales en los tres contextos se están contemplando el uso de herramientas básicas como SCV, herramientas contenerización, de pruebas o de revisión de código o de seguridad, sin embargo hay una tendencia que es importante resaltar, tanto en la industria y en el laboratorio se usan herramientas de Gestión de la configuración, de monitoreo y de infraestructura, mientras que en la academia no se reportó el uso de ninguna herramienta de Gestión de la configuración, ni de monitoreo, así mismo el porcentaje de herramientas de infraestructura, repositorio de artefactos, notificaciones y de seguridad es muy poco, No obstante, si se reportaron en mayor medida el uso de herramientas de pruebas, de revisión de código y contenerización, en comparación con la industria y laboratorio, esto puede deberse a que en los estudios con

contexto académico se priorizan herramientas que den resultados inmediatos y que estén estrechamente relacionadas con etapas primordiales del ciclo de vida de desarrollo de software, como lo son las pruebas, que generan artefactos y resultados de manera inmediata ajustándose a los tiempos y recursos de los cursos de ingeniería de software, mientras que actividades como el monitoreo del pipeline, la gestión de la configuración o el despliegue en ambientes de la nube requieren de más tiempo y recursos tanto monetarios como de cómputo, aspectos con los que normalmente no se cuentan en curso de ingeniería de software, pero si en un ambiente real de trabajo.

Con el objetivo de realizar un análisis más detallado sobre las herramientas específicas que se ocupan en cada contexto, para cada tipo de herramienta se examinó si nivel de adopción en cada contexto. En la Figura 48 Y Figura 49 se muestran los sistemas control de versiones reportados según el contexto.

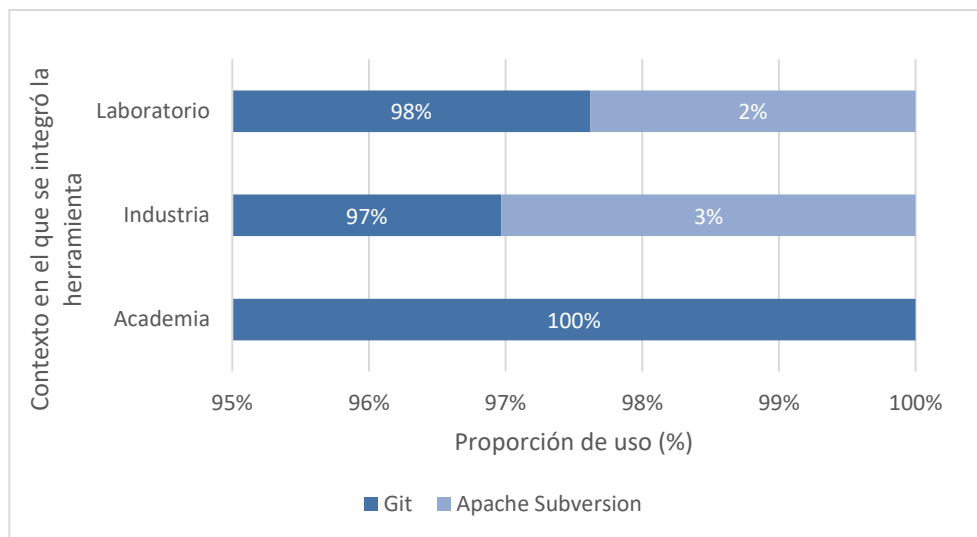


Figura 48. Nivel de integración de herramientas de SCV segun el contexto.

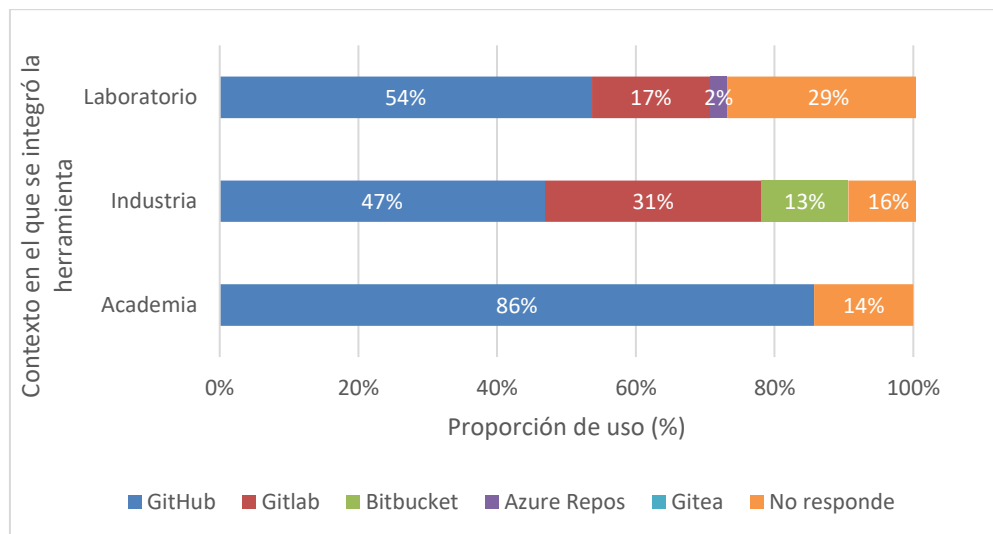


Figura 49. Nivel de integración de gestores de repositorios según el contexto.

Es importante mencionar que algunos estudios reportan el uso de más de un Sistema para la gestión de los repositorios (ERGL35, ERGL45, ERGL46, ERGL51, ERGL58). Con base en estos resultados, se puede observar que en el ámbito académico se utiliza primordialmente Git y GitHub, mientras que en los ámbitos de la industria y laboratorio aunque también predomina Git y GitHub, también existe más variedad de herramientas, esto se debe a que GitHub se centra en la gestión de repositorios y el trabajo colaborativo, un aspecto que en la academia es muy relevante y que se enseña en los cursos de ingeniería de Software, además de que GitHub es una herramienta fácil de usar y que cubre las necesidades básicas que tendría un estudiante para colaborar con sus compañeros en el desarrollo de un proyecto. Mientras que por el otro lado, otras herramientas como GitLab, BitBucket, Gitea o Azure Repos, no solo se enfocan en la gestión de repositorios, también expanden las funcionalidades y permiten realizar implantaciones más complejas (Como integrar Pipelines de CI/CD en GitLab) o especializadas (Como integrarse con un ambiente de Azure de manera más sencilla).

En cuanto a herramientas de Contenerización ocurre algo similar, a pesar de que en los tres contextos se utilizan ampliamente las herramientas de Contenerización, en la academia existe una tendencia a utilizar únicamente Docker y Kubernetes, esto es lógico ya que estas dos herramientas son las más populares en ese aspecto, sin embargo es importante resaltar que en la industria también se utilizan una variedad de herramientas de contenerización más amplia: Singularity, OpenShift, DockerCompose.

Otro aspecto que vale la pena mencionar, es el uso de repositorios de imágenes en la industria, donde es importante mantener un control y organización de los artefactos (Imágenes de Docker en este caso) que se van desarrollando con el tiempo, mientras que en la academia este aspecto no es indispensable ya que los cursos únicamente duran alrededor de seis meses, por lo que utilizar herramientas que permitan un control de artefactos a largo plazo no resulta indispensable. Estas tendencias se pueden ver en la Figura 50.

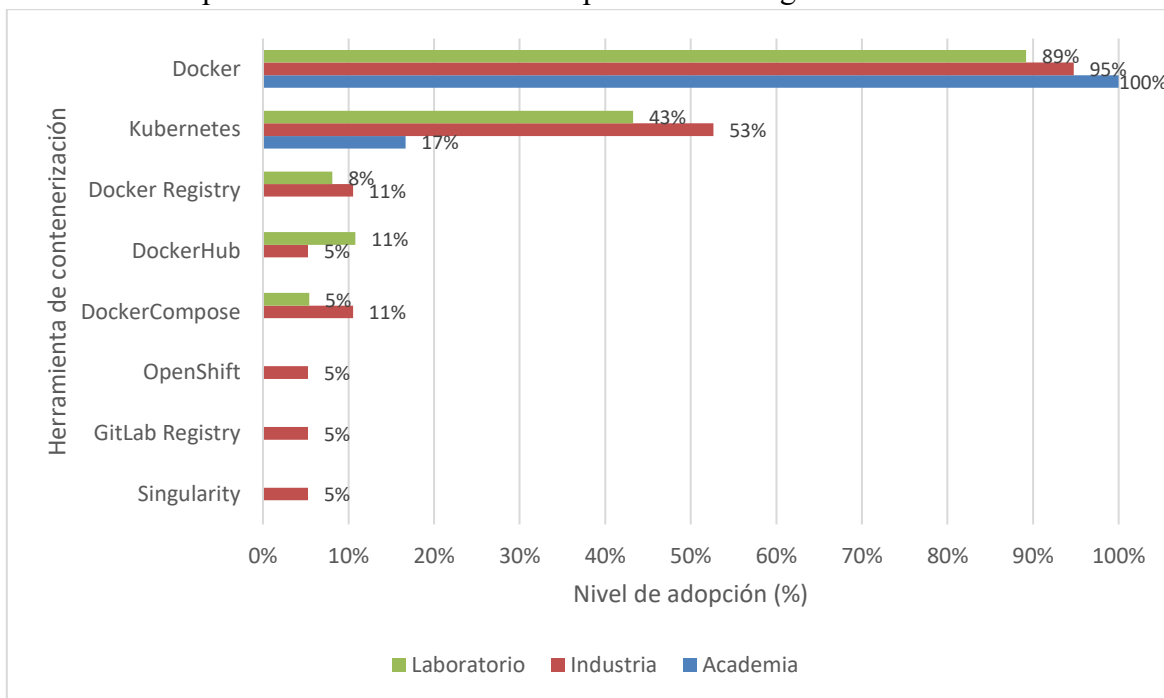


Figura 50. Nivel de integración de Herramientas de Contenerización según su contexto.

En lo que respecta a las herramientas de compilación y de infraestructura, se presenta una tendencia muy parecida a las categorías anteriores, en las cuales el contexto académico propende a utilizar una o dos herramientas únicamente, mientras que en la industria se reportan una variedad de herramientas más amplia, como se muestra en las Figura 51 y Figura 52.

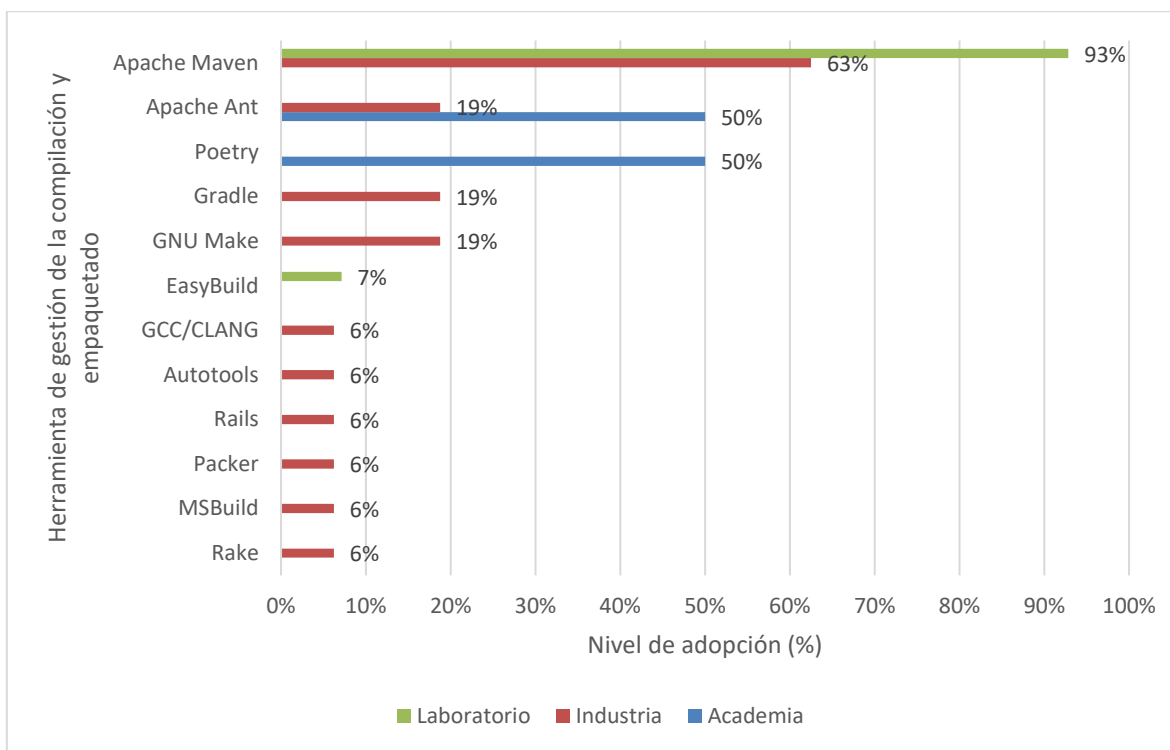


Figura 51. Nivel de integración de herramientas de Compilación y empaquetado según el contexto

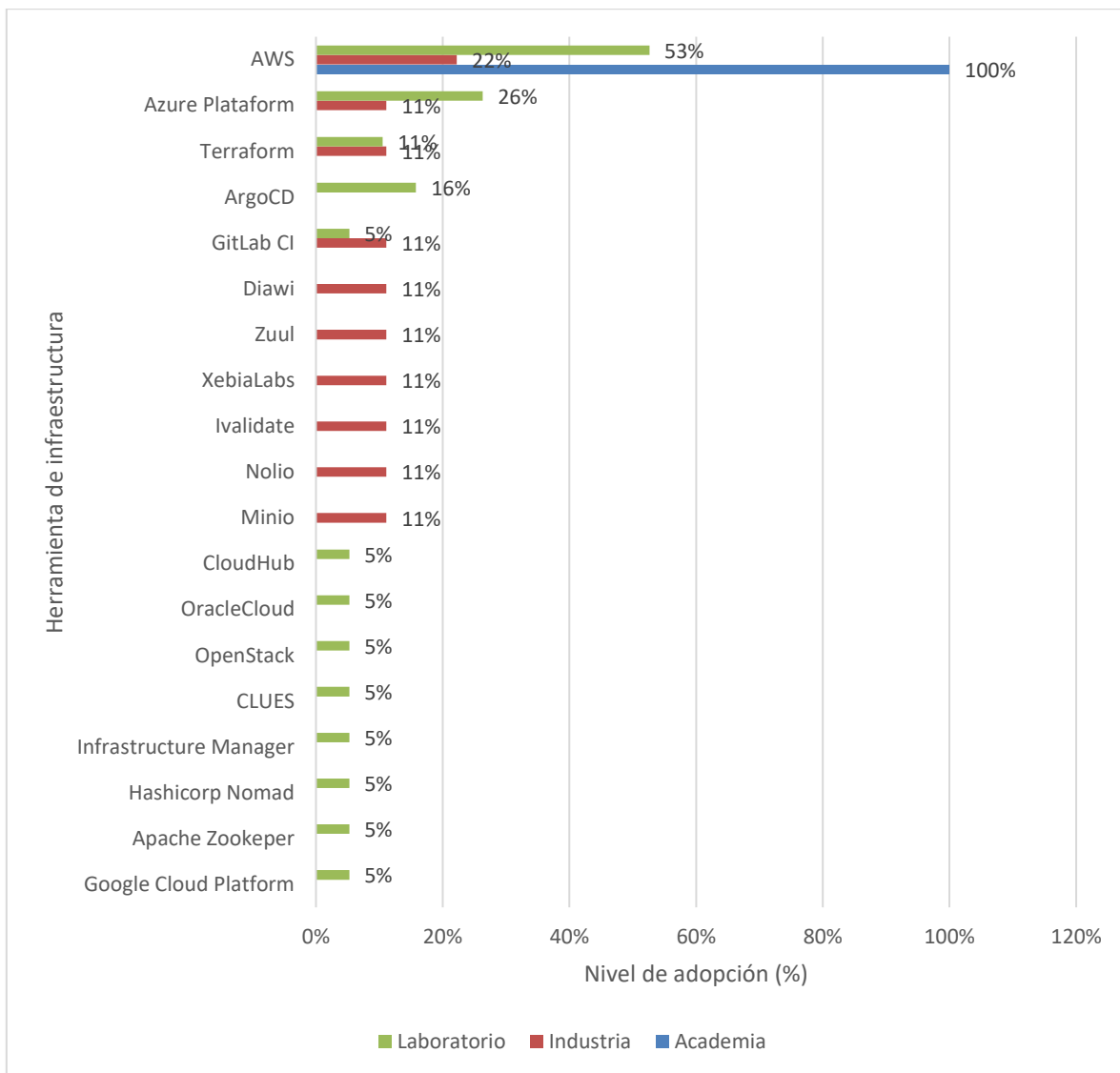


Figura 52. Nivel de Integración de herramientas de infraestructura según el contexto.

En el caso de las herramientas de compilación, la academia reporta el uso de dos herramientas de las cuales una no se utilizó en ningún otro contexto, mientras que la industria reportó el uso de al menos diez herramientas distintas de compilación, de las cuales únicamente Apache Ant fue utilizada en la académica. Por otro lado, a pesar de que la academia reportó únicamente una sola herramienta de infraestructura, y la industria reportó

diez herramientas distintas, esta única herramientas utilizada en la academia es tambien la herramientas más popular y utilizada tanto en la Industria como en el contexto de laboratorio, esto quiere decir, que en esta categoría “Herramientas de Infraestructura” si existe una cierta correspondencia entre lo que se espera en la industria y lo que se enseña en la academia.

Como se pude observar en las gráficas y análisis previos, en la industria se reporta una variedad más amplia de herramientas que en la academia, sin embargo, esta tendencia cambia radicalmente al analizar las herramientas de pruebas y revisión de código, como se muestra en las Figuras 53 y Figura 54.

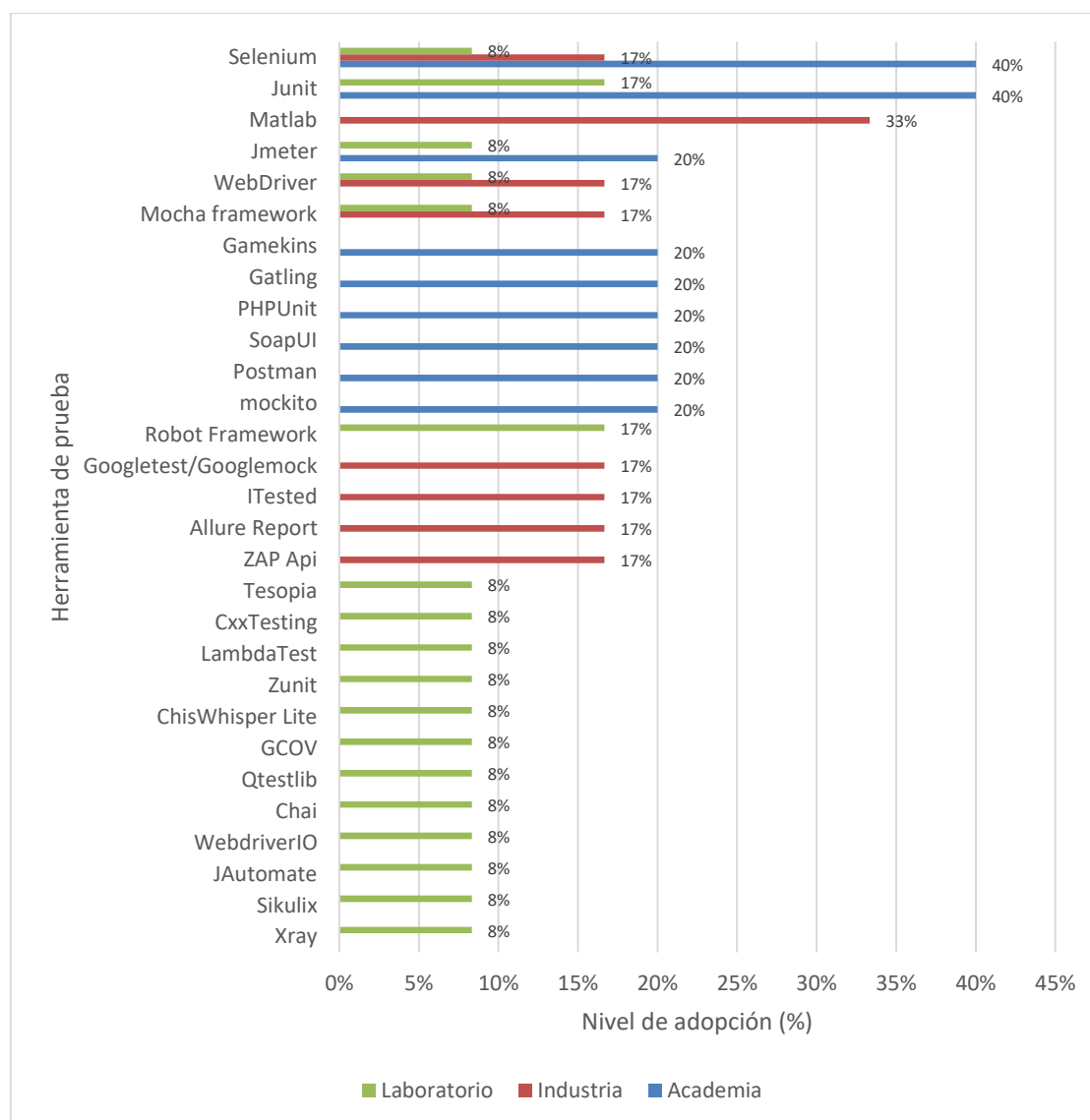


Figura 53. Nivel de integración de herramientas de Pruebas según el contexto.

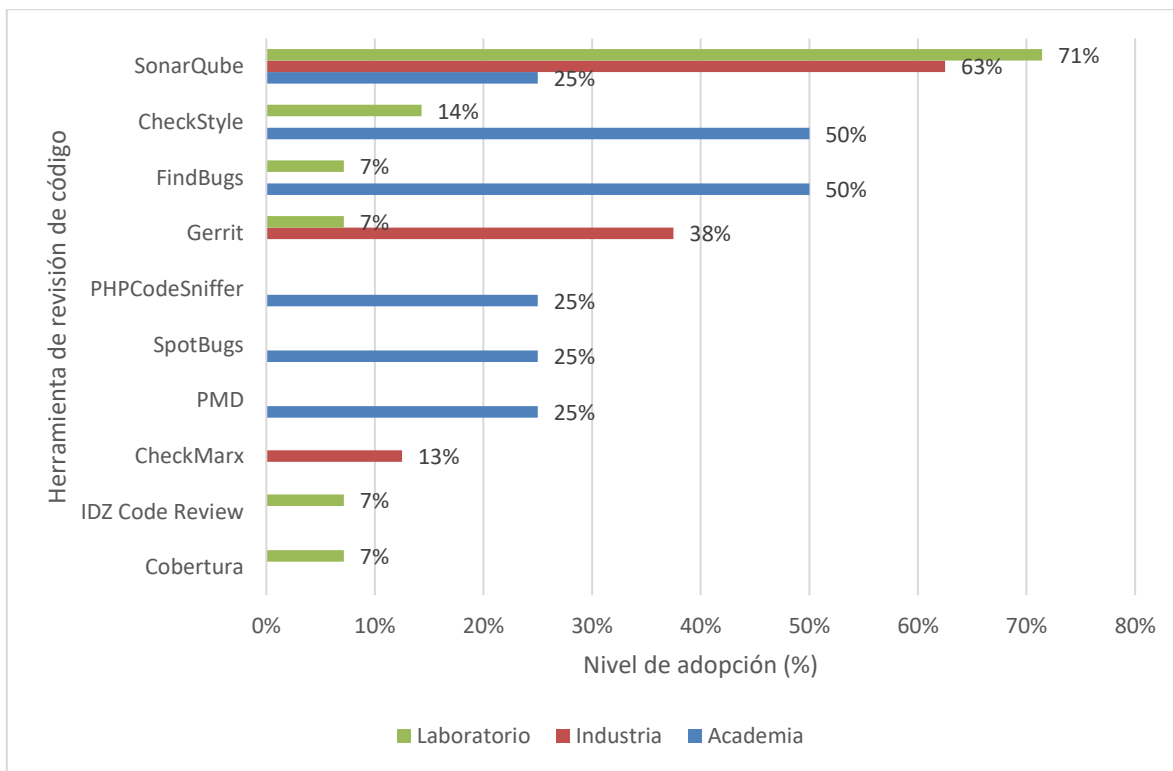


Figura 54. Nivel de Integración de Herramientas de Revisión de Código.

Como se puede observar en las figuras anteriores, es la academia la que ahora reporta una mayor diversidad de herramientas de pruebas y de revisión de código en comparación con la industria y el laboratorio. De la misma forma las principales herramientas que se mencionan en la industria, también se reportan en la academia. Estas particularidades se explicarían ya que en los cursos reportados en los estudios se le dedica bastante tiempo e importancia a las actividades de pruebas y de revisión de código en comparación a otras actividades del ciclo de vida de software.

Como se mencionó previamente, en cuanto a herramientas de monitoreo y para la gestión de la configuración, no se reportó ninguna de estas en los estudios con contexto académico, mientras que en los estudios con contexto industrial, se mencionan una amplia cantidad de herramientas de monitoreo, y unas cuantas herramientas para la gestión de la

configuración (en especial Ansible), esto es otro aspecto para resaltar y a tomar en cuenta en los cursos de Ingeniería de Software.

En cuanto a las herramientas de repositorios de artefactos, y de notificaciones, ocurre lo mismo que en las tendencias previas, los estudios con contexto de industria reportan una amplia variedad de herramientas, mientras que los estudios con contextos académicos reportan una sola herramienta, aunque al igual que en los otros tipos de herramientas (Contenerización o infraestructura), la única herramienta reportada en la academia es la herramienta más utilizada en la industria (Slack para notificaciones y JFrog artefactory como repositorio de artefactos). Como se puede ver en la Figura 55 y la Figura 56.

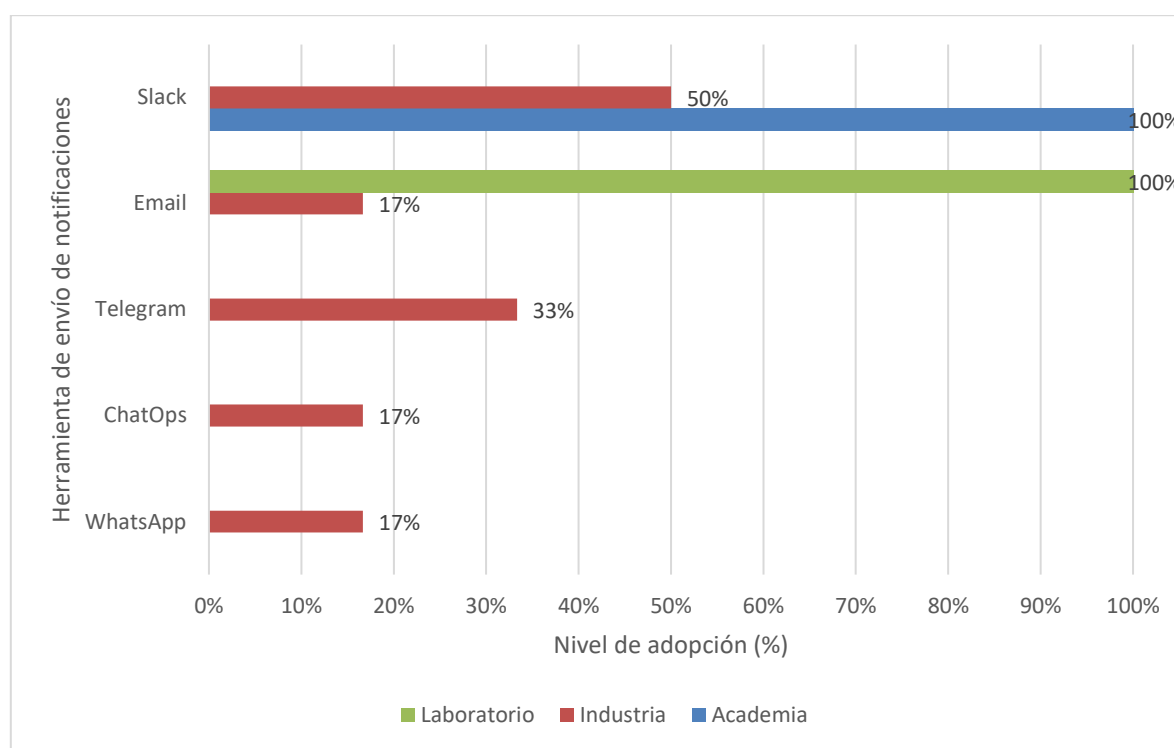


Figura 55. Nivel de Integración de herramientas de Notificaciones según el Contexto.

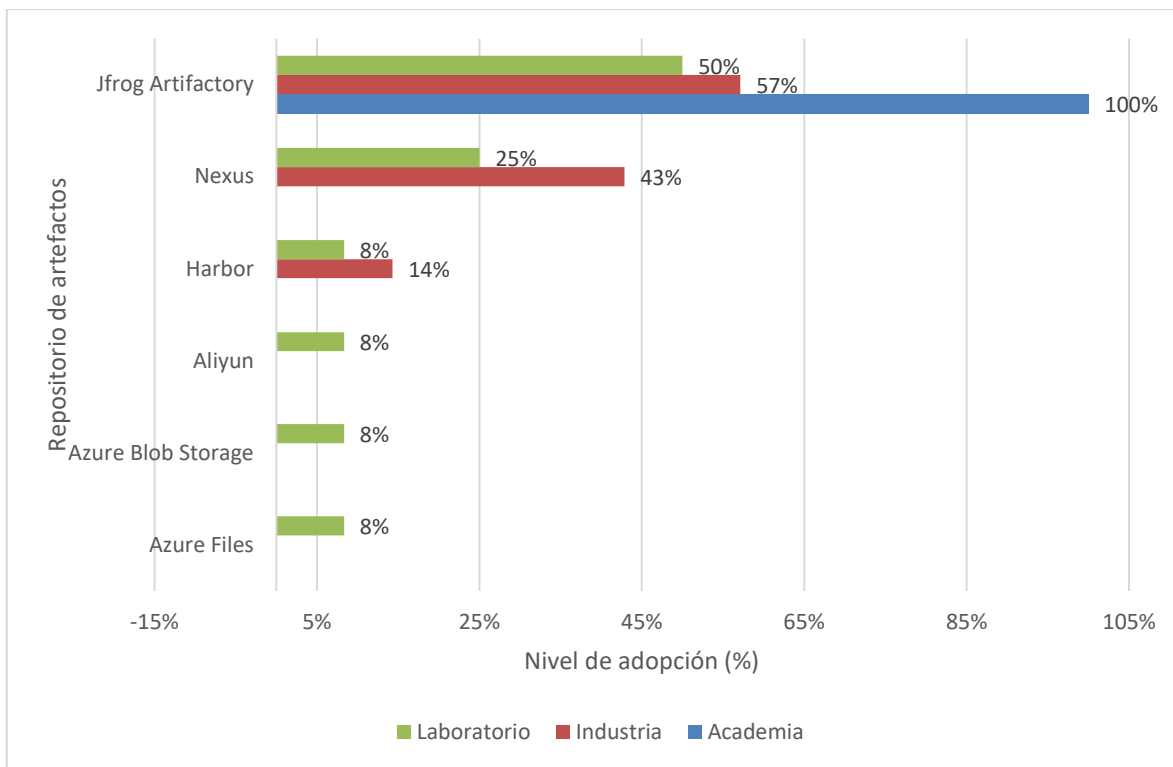


Figura 56. Nivel de integración de herramientas de Repositorio de Artefactos según el contexto.

Por otro lado, en el caso de las herramientas de seguridad, ninguna de las seis herramientas reportadas se utilizó en más de un contexto diferente, lo que podría indicar que no existe una herramienta de seguridad ampliamente utilizada en la industria como si ocurre con otras herramientas con Docker, GitHub, AWS o SonarQube. Finalmente, el único tipo de herramienta que no se reportó en la industria son las herramientas de Virtualización, sin embargo estas sí se usaron en dos ocasiones en los estudios de la academia, esto por sí solo no revela nada importante, pero puede tener origen en que la industria prefiere utilizar herramientas como Docker o Singularity para crear agentes de Jenkins antes que usar herramientas como VirtualBox o Vagrant, cuya configuración puede resultar más compleja, tardada y menos eficiente en términos de recursos.

En general, tanto los estudios con contexto académico, de laboratorio y de industria utilizan en proporciones similares los mismos tipos de herramientas, con excepciones específicas como: herramientas para la gestión de la configuración, virtualización de recursos físicos y de monitoreo. Sin embargo, a pesar de que en los tres contextos se están utilizando los mismos tipos de herramientas, si existe una clara discrepancia entre la cantidad de herramientas que se reportan en la industria y laboratorio en comparación con la academia, en este último contexto predominan una o dos herramientas para cada categoría, mientras que en la industria se utiliza una mayor variedad, esto es un reflejo de las condiciones y factores contextuales de los estudios, por una parte los estudios de la industria reportan una amplia cantidad de herramientas porque responden a las necesidades de su entorno, del proyecto desarrollado y de las organizaciones, por lo que es lógico que no sea posible generalizar el uso de una sola herramienta, mientras que en la academia, no existen estos factores. Siendo el tiempo limitado de los cursos la mayor restricción, por lo que los estudios tienden a utilizar únicamente las herramientas más populares como: Docker, GitHub, AWS, SonarQube, Slack, Junit, Selenium, entre otras. Dejando de lado herramientas menos conocidas, pero que siguen siendo importantes

Prácticas de configuración y administración de Jenkins recomendadas por los expertos

Como ya se mencionó previamente, se identificaron un total de 106 prácticas de configuración y administración de Jenkins distribuidas en seis grupos, cada uno de ellos con agrupaciones internas.

Es indiscutible que las prácticas con mayor relevancia entre los estudios son las relacionadas con los pipelines en Jenkins. Representando un 48% de las 106 prácticas, este es además el que cuenta con más grupos internos, con un total de siete de ellos. También, en este grupo de prácticas es claro el interés en la correcta ejecución de los *pipelines*, así como su mejora continua para soportar procesos más complejos, pues casi 60% de estas prácticas pertenecen a los grupos de escalado de *pipelines* y ejecución de los *pipelines*, lo que representa casi el 30% del total de las prácticas.

Prácticas como modelar *pipelines* mediante archivos de definición (o *Jenkinsfiles*) siguiendo una sintaxis declarativa, realizar limpiezas periódicas de los *builds* y artefactos, el

uso de *shared libraries* en Jenkins o manejar los disparadores de los *pipelines* de forma austera son prácticas que destacan en este grupo.

La práctica encapsular la lógica del pipeline en unidades reutilizables, que forma parte de este grupo de prácticas, es interesante. Los estudios en los que se reportó mencionaban diferentes estrategias para su implementación, como la creación de funciones y *scripts* (EP03), los *steps* personalizados en Groovy (EP29) o la más mencionada, el uso de *shared libraries*.

Las prácticas identificadas son muy generales en su mayoría. Ejemplo claro de esto son las ya previamente mencionadas. Sin embargo, en este grupo de prácticas hubo algunas muy específicas que aplican solamente en escenarios particulares, como usar la directiva `withEnv` en vez de sobrescribir directamente `env.PATH` cuando se utilizan múltiples agentes o ajustar las opciones de recolección de basura cuando Jenkins utilice más de 6 GB de memoria *heap*. Esto revela un conjunto de prácticas que no solo aplica para lo general, también para particularidades y escenarios específicos al trabajar con la herramienta.

Otro aspecto para resaltar de las prácticas es la relevancia que tiene el uso de contenedores (y de Docker) en este contexto de uso de Jenkins. Las prácticas más mencionadas en los grupos de instalación de Jenkins y uso de Jenkins fueron, respectivamente, ejecutar la infraestructura de Jenkins utilizando un enfoque basado en contenedores y utilizar Docker como agente.

Pasando a un análisis más profundo, se realizó una comparación de las prácticas de configuración y administración de Jenkins contra los contextos en los que fueron reportadas, identificados a través de la RQ01 (academia, industria y laboratorio). Para ello, se calculó la proporción de recomendación de las prácticas respecto al total de prácticas reportadas en cada contexto específico, esta proporción por cada grupo de prácticas.

En otras palabras, se analizó la distribución de prácticas por grupo para cada uno de los contextos. La siguiente figura muestra los resultados de esta comparación de forma

gráfica. Como se puede observar, solo aparecen cinco grupos de prácticas de configuración y administración de Jenkins, faltando el de administración del sistema. Esto se debe a que todas las prácticas de este grupo fueron reportadas en estudios donde no se pudo identificar un contexto particular.

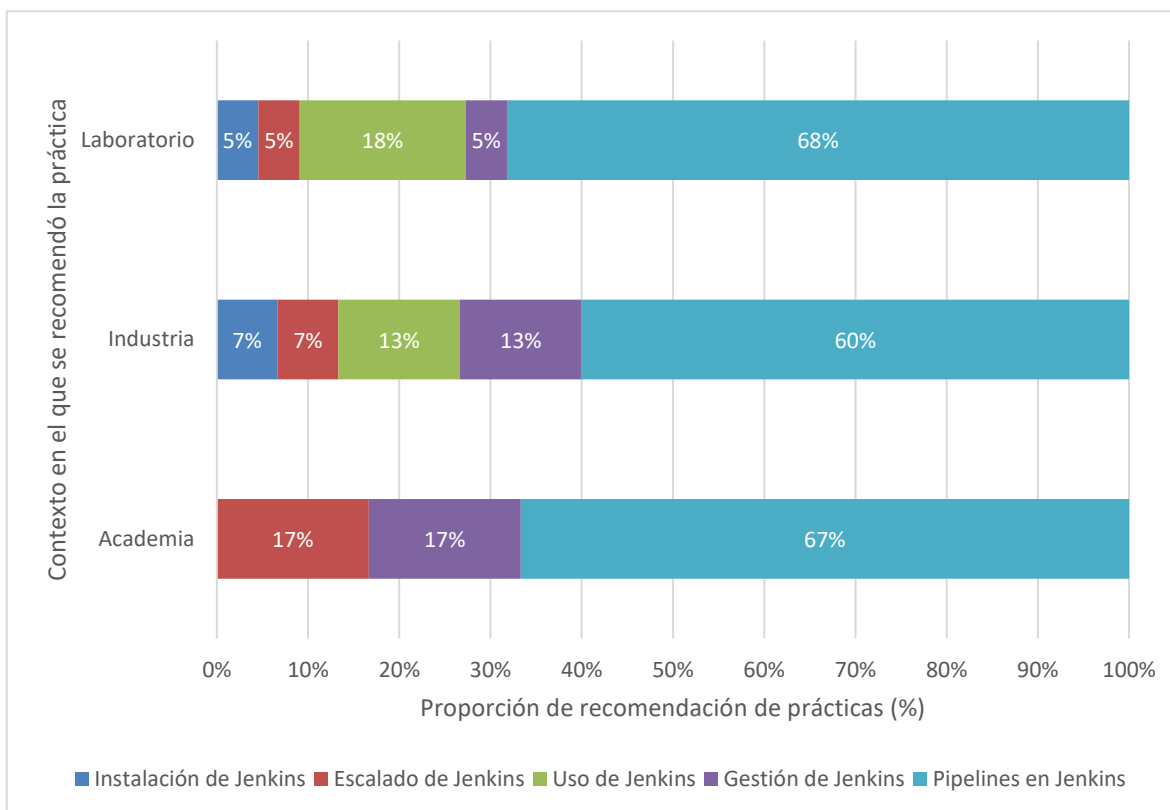


Figura 57. Proporción de recomendaciones de prácticas de configuración y administración de Jenkins según el contexto

Se debe tener en mente que la figura previa solo muestra la distribución de las prácticas que fueron reportadas en estudios en los que se identificó un contexto. Como ya se mencionó, algunas prácticas fueron reportadas en estudios sin un contexto; estas prácticas no se consideraron para los porcentajes de recomendación de prácticas presentados previamente.

Pasando de lleno a la interpretación de la información, en la figura se muestra cómo las prácticas de instalación de Jenkins y uso de Jenkins no se reportaron en ningún estudio con contexto educativo. Además, que la proporción de recomendación de prácticas de escalado de Jenkins es mayor en la academia (17%) que en la industria (7%) o el laboratorio (5%). Esto podría revelar el enfoque de la academia por demostrar prácticas más complejas, pero dejando de lado prácticas esenciales, como las relacionadas con la instalación y el propio uso de Jenkins.

Fuera de estos aspectos particulares, las prácticas identificadas plantean un amplio conjunto de directrices para la correcta configuración y administración de Jenkins, considerando aspectos desde su instalación, uso y configuración de pipelines, hasta incluso el escalado de Jenkins y la administración del sistema, y que abarcan los aspectos generales y algunos particulares en cada uno de estos temas (y más).

Actividades de integración, prueba y despliegue integradas y automatizadas utilizando Jenkins

Como se mencionó previamente, las actividades de integración, prueba y despliegue automatizadas e integradas con Jenkins fueron 19 en total, siete de integración, cinco de prueba y siete de despliegue. Además, el grupo de actividades que más menciones tuvo fue el grupo de actividades de integración, lo que hace sentido considerando que Jenkins es un servidor de integración. Sin embargo, encontrar un gran número de menciones de actividades de prueba y despliegue revela la gran flexibilidad y robustez de la herramienta para manejar pipelines de CI/CD, no solo de CI.

Otro hecho que revela la flexibilidad de Jenkins para automatizar actividades dentro del proceso de CI/CD, fueron las actividades clasificadas como “otras” que no son ni de integración, prueba o despliegue. Se identificaron un total de ocho actividades varias automatizadas e integradas con Jenkins, que se presentan de forma gráfica a continuación.

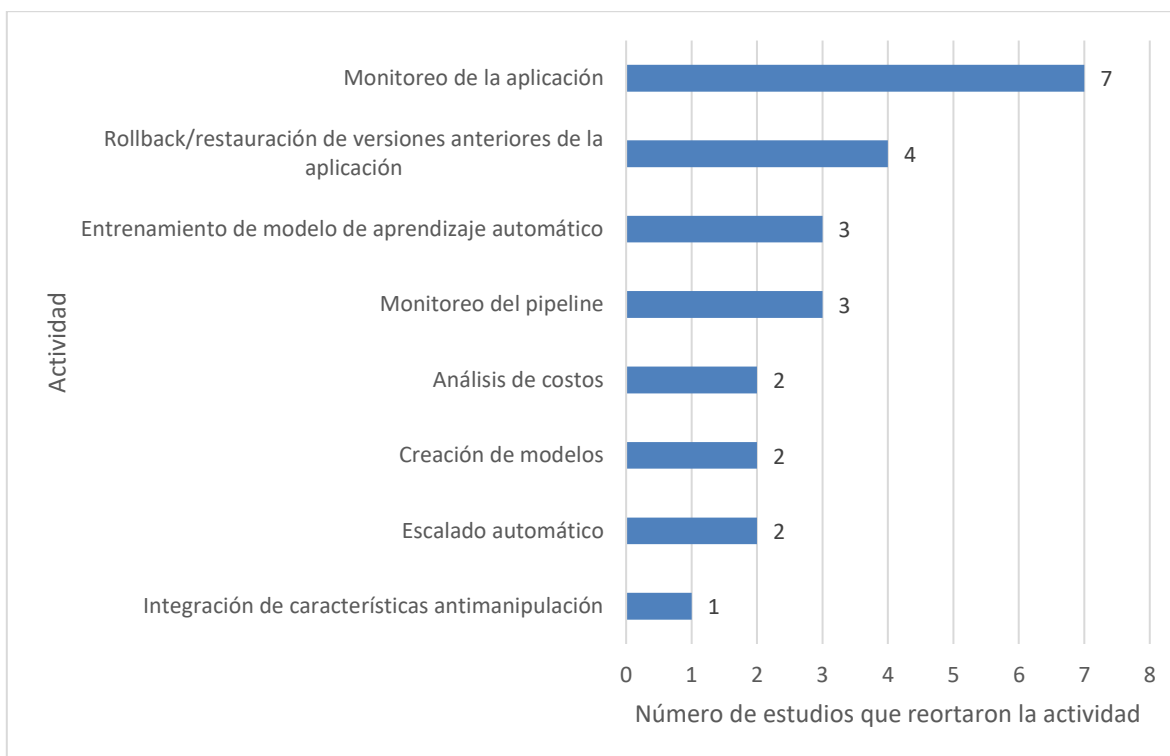


Figura 58. Menciones de otras actividades automatizadas con Jenkins

Entre estas otras actividades identificadas, destacan el monitoreo de la aplicación/software y la restauración de versiones anteriores de la aplicación/software. La primera de estas actividades se mencionó en siete estudios y tenía como objetivo principal la revisión constante del rendimiento del sistema y de su salud. Por otra parte, la restauración de versiones anteriores de la aplicación, mencionada en cuatro estudios, se reportó como un mecanismo para mantener la aplicación en ejecución aun cuando un nuevo despliegue fallara o en caso de que ciertas condiciones de rendimiento no se cumplieran.

Siguiendo con este conjunto de actividades, resalta la creación de modelos; no por el total de sus menciones (dos), sino por su nombre. En el estudio EP77 se automatiza la generación de un documento con las relaciones entre eventos en el ciclo de vida del software. En el EP66 se automatiza la generación de un modelo abstracto a partir del código fuente. Estas generaciones de documentos útiles dentro del proceso seguido en los estudios son las que se engloban a través de esta actividad.

Pasando a las actividades de integración, solo dos de ellas fueron mencionadas en más del 50 % de los estudios que reportaron la automatización de actividades de integración: compilar la aplicación fue reportada en 95 estudios (84% de 113) y descargar el código en el repositorio en 67 estudios (59% de 113). Las cinco actividades restantes tuvieron una

presencia significativamente menor. Esto es relevante, pues indica que solamente el 30% (aproximadamente) de las actividades están teniendo una adopción relevante entre los estudios.

En las actividades de prueba pasó algo similar, pues solamente una de ellas fue mencionada en más del 50% de los estudios que reportaron automatizaciones de actividades de prueba: ejecutar los procedimientos de prueba fue reportada en 81 estudios (86% de 94). Esto significa que el 20% de las actividades de prueba tienen una adopción relevante (menos que integración).

Algo interesante, fue que múltiples estudios mencionaban la automatización de actividades en las etapas de prueba y despliegue, pero no especificaban qué actividades eran las que automatizaban en esas etapas. Para pruebas, 11 fueron los estudios que entraron en este grupo, mientras que para despliegue fueron seis.

Además de las actividades de prueba automatizadas, en los estudios se solían mencionar los niveles de prueba y, en ocasiones, los tipos de prueba. Partiendo de los niveles de prueba y siguiendo nuevamente la (ISO/IEC/IEEE, 2013), pero ahora en la parte 1 del estándar 20119, se identificó la automatización de pruebas en cinco niveles: pruebas unitarias, de integración, de sistema, de integración de sistemas y de aceptación. La siguiente figura muestra el número de menciones entre los estudios para los niveles de prueba.

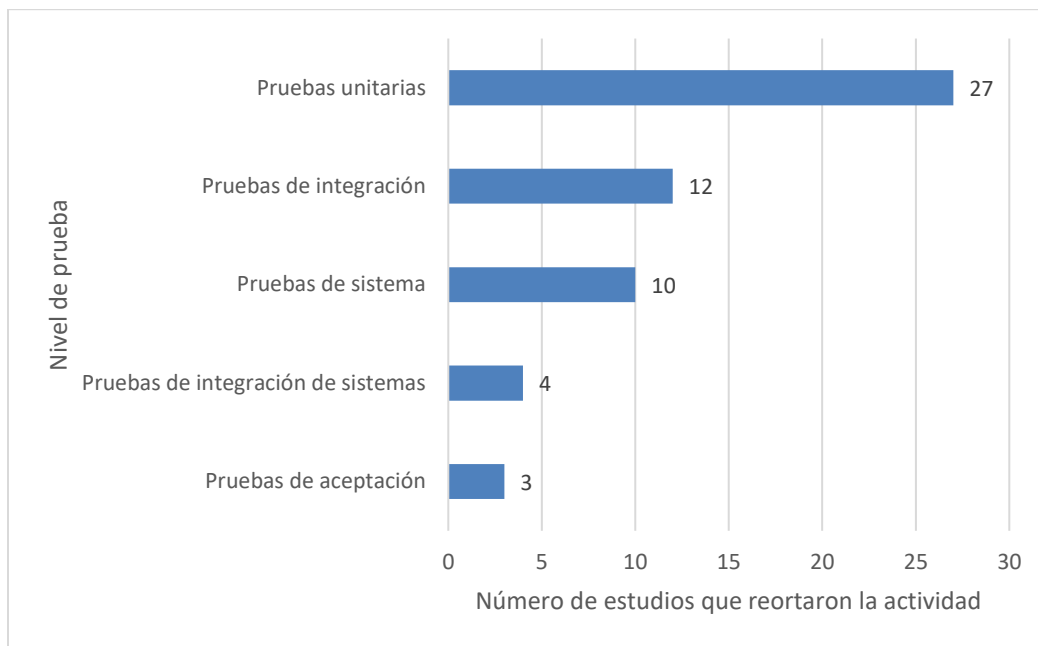


Figura 59. Menciones de niveles de prueba automatizados con Jenkins

Para los tipos de prueba, también se partió del estándar 29119 parte 1, identificando solamente tres tipos de prueba automatizadas: funcionales, de seguridad y relacionadas con el rendimiento (rendimiento y carga). El tipo de prueba más automatizado fueron las pruebas de seguridad, con 13 menciones en los estudios, seguido de las funcionales con 11 menciones y finalmente las relacionadas con el rendimiento, con solo cinco menciones. Esto se puede apreciar de mejor forma en la siguiente figura.

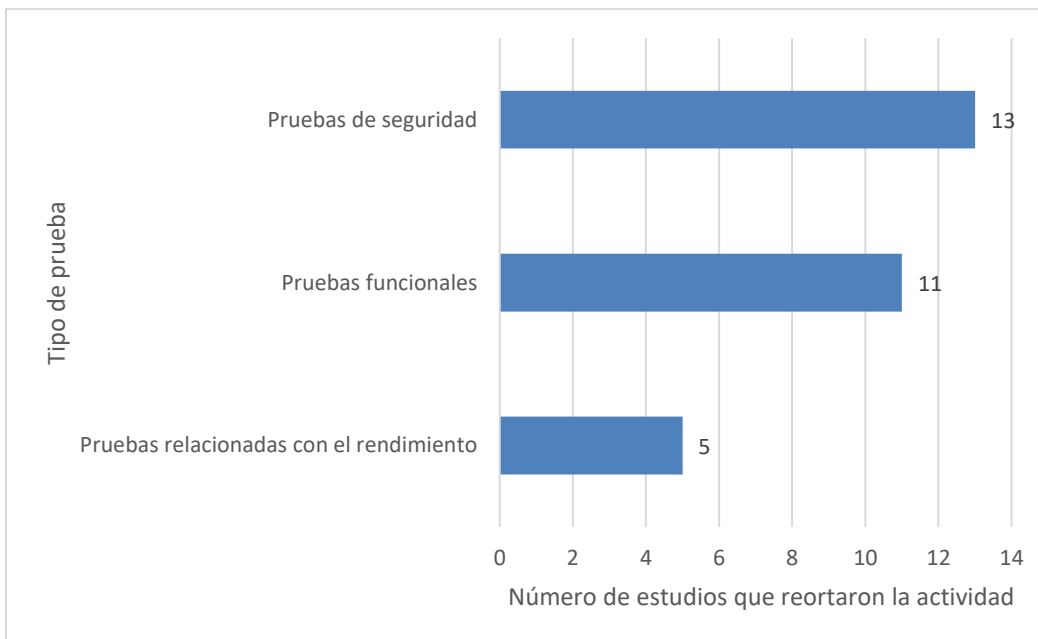


Figura 60. Menciones de tipos de prueba automatizados con Jenkins

Finalmente, en las actividades de despliegue pasó lo mismo que con las actividades de prueba, ya que solo una de ellas fue mencionada en más del 50% de los estudios que reportaron automatizaciones de actividades de despliegue: Instalar el software en diferentes servidores fue reportada en 67 estudios (80% de 84). Es decir, un 14% de las actividades de despliegue tienen una adopción relevante (menos que integración y prueba).

De forma general, las actividades de integración son las que han sido más adoptadas para su automatización con Jenkins, lo cual hace sentido dado el enfoque de la herramienta, siendo esta un servidor de integración continua. Sin embargo, esto no limita a Jenkins a solo permitir la integración continua. Los resultados revelan que Jenkins tiene la capacidad de automatizar actividades de prueba y despliegue, e incluso otras específicas fuera del proceso de CI/CD.

Cambiando de enfoque y con el fin de realizar un análisis más profundo sobre el uso de Jenkins para la automatización de actividades de integración, prueba y despliegue, se llevó a cabo una comparación de este uso en los diversos contextos identificados a través de la pregunta de investigación RQ01: industria, academia y laboratorio. Para esto, partimos del nivel de adopción de las actividades en cada contexto, que simplemente es la proporción de estudios de cierto contexto que reportaron la automatización de actividades específicas.

Comenzando con las actividades de integración, la figura siguiente proporciona una representación gráfica del nivel de adopción de las actividades de integración en cada uno de los contextos. De forma general, parece que las actividades de integración han sido adoptadas de mejor forma en el contexto educativo, pues la mayoría de ellas presenta un porcentaje de adopción mayor en este contexto respecto a los otros.

La afirmación previa es cierta, pero solo de forma parcial. Hay que considerar que solamente hubo nueve estudios que reportaron la automatización de actividades de integración en un contexto académico. En contraste con los números de la industria (37) y laboratorio (57), el total de estudios de la academia es pequeño.

Además, los estudios en un contexto de academia tenían como objetivo principal mostrar los resultados de haber creado algún pipeline de CI/CD en Jenkins con fines educativos en su mayoría. Con esto, tiene sentido que dichos pipelines estén diseñados para abarcar la automatización de muchas actividades y que por ello haya un mayor nivel aparente de adopción en la academia.

Esto anterior resalta más al profundizar más en las actividades de integración. La única de las actividades que tiene un nivel de adopción menor en la academia respecto a la industria y laboratorio es el almacenamiento de artefactos generales. Al ser implementaciones en un contexto académico, es probable que los procesos del pipeline no sean tan pesados, al igual que los artefactos generados por la ejecución del pipeline, por lo que no hay ningún problema en no manejar la persistencia de estos últimos. Sin embargo, estas son solo especulaciones para tratar de explicar el aparente nivel de adopción mayor por la academia para las actividades de integración.

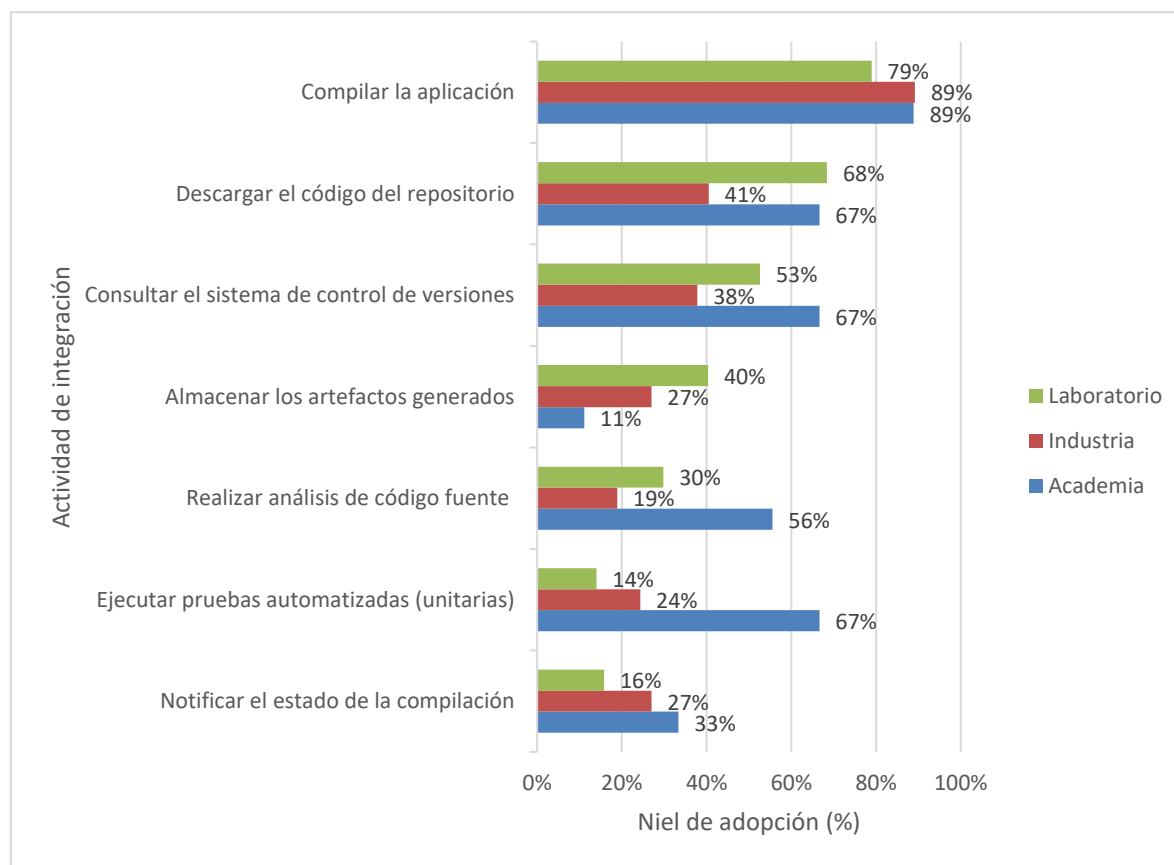


Figura 61. Nivel de adopción de actividades de integración según el contexto

Pasando a las actividades de prueba, en este caso los niveles de adopción se mantuvieron similar a excepción de dos de las actividades: comparar la ejecución de las pruebas y preparar datos de prueba, en donde el nivel de adopción fue 0%. Ningún estudio que reportó la automatización de actividades de integración en un contexto de industria reportó la automatización de estas actividades.

Retomando la idea de la persistencia de los artefactos y el poco énfasis que le da la academia en comparación con la industria o el laboratorio, en las actividades de prueba pasó algo similar. El registro de la ejecución de pruebas fue la actividad con menor nivel de adopción por parte de la academia hablando de actividades de prueba.

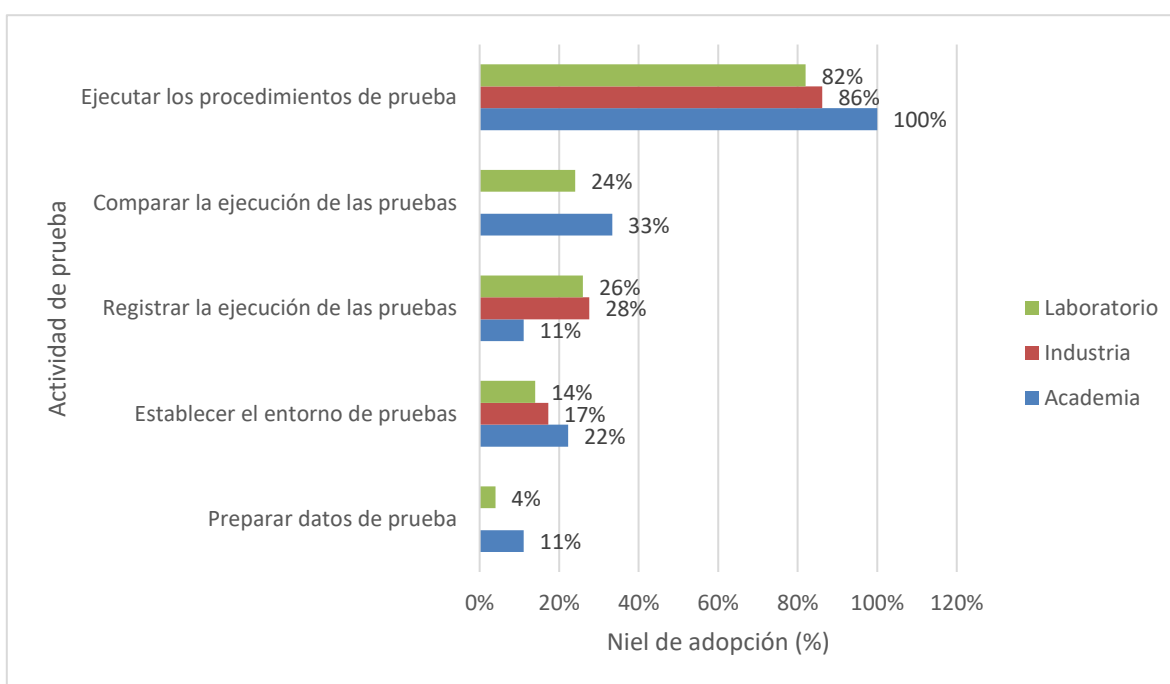


Figura 62. Nivel de adopción de actividades de prueba según el contexto

Finalmente, en las actividades de despliegue fue donde se encontraron los niveles de adopción con mayor disparidad. Cuatro de las actividades presentaron niveles de adopción similares en los diversos contextos, siendo estas: instalar el software en diferentes servidores,

empaqueta el código, iniciar la ejecución de la aplicación y configurar los servidores y las bases de datos; cada una de ellas con niveles de adopción promedio de 81%, 58%, 48% y 17%, respectivamente. Sin embargo, con las actividades reiniciar los servidores, generar archivos de configuración y ejecutar pruebas de humo, el nivel de adopción en el contexto académico fue nulo.

Posiblemente el nivel de adopción nulo de estas actividades en contextos académicos sea resultado de su complejidad, pues al menos dos de ellas refieren a la automatización de la gestión de la infraestructura. Esto hace más sentido al recordar que la mayoría de los estudios que reportaron un contexto académico, implementaron *pipelines* de CI/CD con fines demostrativos y para sentar las bases generales de la automatización del proceso de integración y despliegue continuos. Sin embargo, esta es solamente una especulación, pues el hecho es que existe la disparidad previamente mencionada en la adopción de actividades de despliegue entre los contextos académicos, de la industria y de laboratorio.

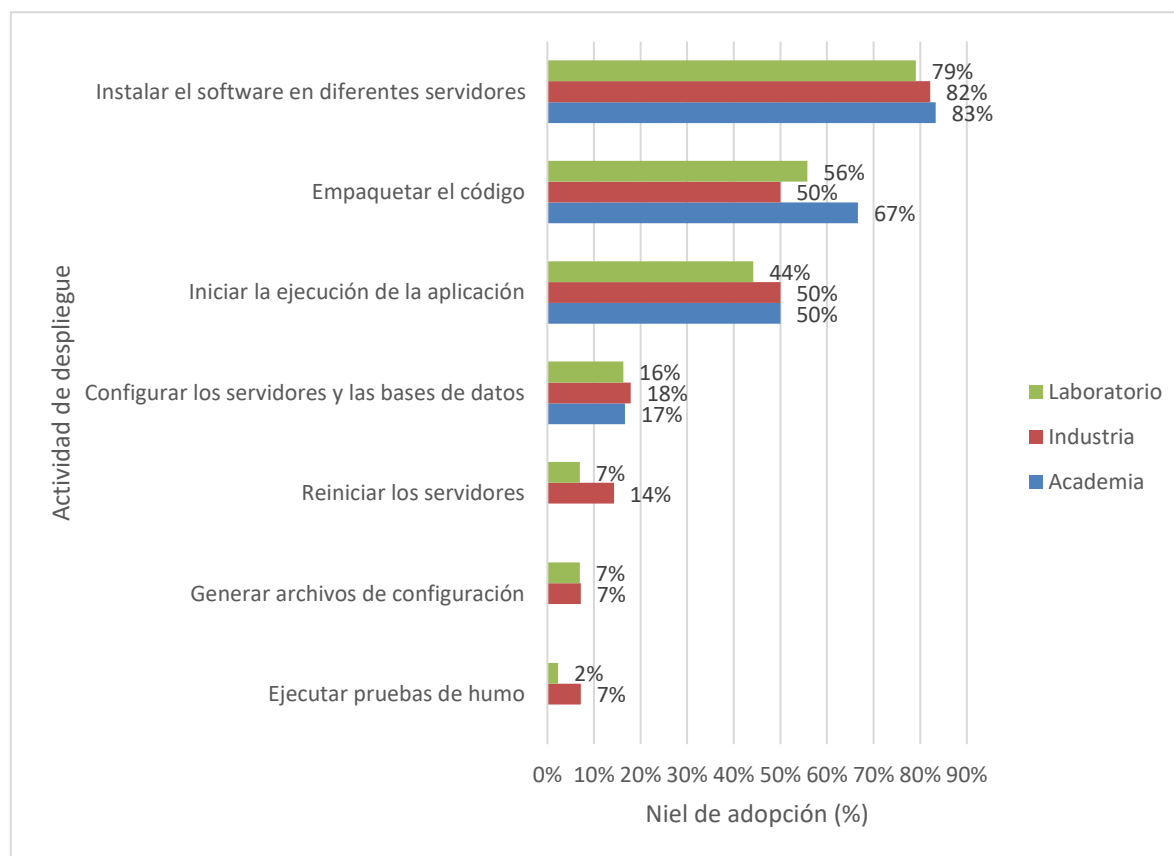


Figura 63. Nivel de adopción de actividades de despliegue según el contexto

En resumen, la respuesta a esta pregunta ofrece una visión general de valor, que permite observar el estado de la automatización de los *pipelines* de CI/CD con Jenkins, hablando particularmente de las actividades que forman parte del *pipeline*. Además, la comparación de estas actividades con el contexto en el que se reportaron permite observar las áreas donde se está enfocando cada contexto, dejando ver aspectos interesantes, como la adopción de actividades de persistencia de información en la industria, aún no adoptadas por la academia; o el nivel de adopción más amplio en actividades de despliegue por parte de la industria difiriendo de la academia.

Aspectos de seguridad y principales estrategias para mitigar vulnerabilidades y asegurar los pipelines de CI/CD

En lo que respecta a los aspectos de seguridad, como se mencionó en el apartado de resultados, los principales aspectos de seguridad están relacionados a el Control de accesos, la seguridad del entorno de Jenkins, la gestión de las credenciales y al aseguramiento de los *builds*.

Es importante mencionar que la mayoría de los aspectos de seguridad se obtuvieron de los recursos revisados en la literatura gris, y tan solo siete estudios de la literatura blanca mencionaron uno o dos aspectos de seguridad al utilizar Jenkins, principalmente relacionados a la exposición de puertos y servicios, y el control de accesos. Mientras que en la Literatura Gris un total de 17 estudios reportaron al menos un aspecto de seguridad, siendo el recurso ERGL46 el estudio en el que más aspectos de seguridad se identificaron (39 aspectos en total), es importante mencionar que este recurso es la documentación oficial de Jenkins, y es debido a eso que se identificaron tantos elementos, sin embargo los estudios restantes solamente mencionan uno o dos aspectos de seguridad, y únicamente tres estudios (ERGL15, ERGL, ERGL32) más de dos aspectos de seguridad.

Estos datos resaltan la limitada atención que se le brinda a las características de seguridad en estos estudios, priorizando aspectos, técnicos, funcionales y de optimización

sobre prácticas y configuraciones de seguridad en la herramienta Jenkins. Estos resultados pueden explicarse por el hecho de que la mayoría de los estudios propone implementaciones de pipelines de CI/CD con el fin de integrar Jenkins en el proceso de desarrollo o mantenimiento de algún software, sin embargo estas implementaciones pretenden resaltar los beneficios y ventajas obtenidas a partir del uso de esta herramienta, así como detallar la forma en que se integró Jenkins en el proceso y con otras herramientas, por lo que no resulta importante detallar aspectos de seguridad con los que se pudieron o no haber enfrentado. Por otra parte solo unos pocos estudios toman un enfoque más apegado a la seguridad, y presentan implementaciones o casos de estudio centrados asegurar los pipelines de CIC/CD y Jenkins (EP44, EP51, EP96).

De la misma forma que con las cuatro preguntas de investigación anteriores, se realizó un análisis más detallado de los aspectos de seguridad dependiendo del contexto reportado, los resultados de este análisis se pueden consultar en la Figura 64.

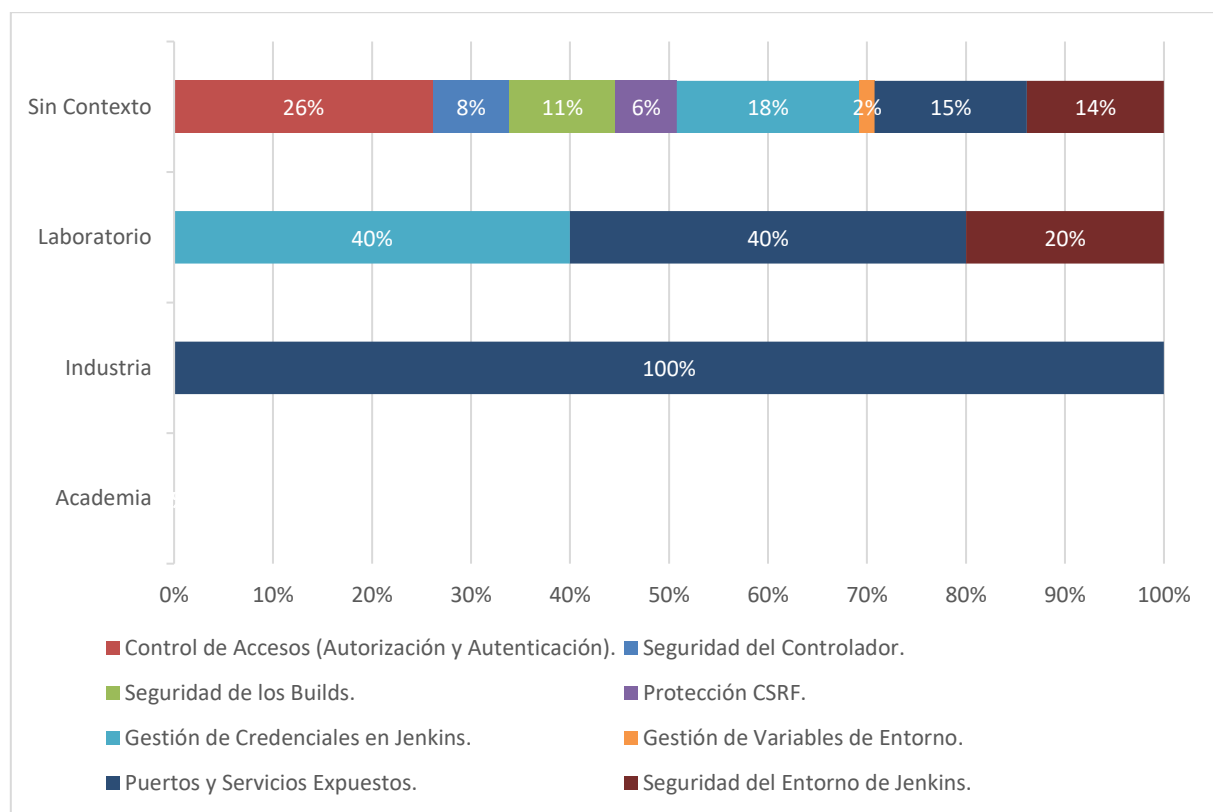


Figura 64. Nivel de consideración de aspectos de seguridad según el contexto.

Como se puede observar en la figura anterior, la mayor parte de los aspectos de seguridad provienen de estudios de los cuales no se logró identificar un contexto específico, es importante mencionar que como la mayoría de los aspectos de seguridad se obtuvieron de

la documentación oficial, la cual no cuenta con un contexto específico por lo que la mayor parte de los aspectos de seguridad se encuentran en la barra de “Sin Contexto”, mientras que solo unos pocos fueron reportados en estudios de laboratorio y de la industria, en cuanto a la industria puede resultar lógico que al ser implementaciones realizadas por empresas, organizaciones o desarrolladoras que trabajan en contexto laborales reales, prefieren omitir las características de seguridad por cuestiones de confidencialidad. Por otra parte en la academia no se identificó ningún aspecto de seguridad en los diez estudios correspondientes, esto podría denotar una carencia en la atención y tiempo que se les brinda a estos temas en los cursos reportados.

En lo que respecta las prácticas para el aseguramiento de los pipelines de Integración y Despliegue Continuo están más relacionadas a la configuración o implementación de mecanismos para asegurar el entorno y ambiente en el que se ejecuta Jenkins, así como el control de accesos y la autenticación para ingresar a la instancia de Jenkins. Gran parte de las estrategias de seguridad se enfocan en acciones de seguridad preventiva y solo unas pocas en seguridad correctiva y de detección. Esto se explica principalmente ya que la mayoría de las estrategias se centran en realizar configuraciones de seguridad en diversos apartados de Jenkins, desde el uso adecuado de herramientas para gestión de las credenciales, hasta la implementación correcta de scripts (Jenkinsfiles) que no dejen hueco de seguridad al momento de ejecutarse.

Nuevamente es importante mencionar que la mayoría de las estrategias de seguridad provienen de la literatura gris, ya que únicamente trece estudios de la literatura blanca reportaron alguna estrategia de seguridad, sin embargo, a diferencia de los aspectos de seguridad, los estudios presentaron más de dos o tres prácticas de seguridad, y en algunos casos se identificaron hasta cuatro o más estrategias (EP24, EP96, EP97).

Por otro lado, en la literatura gris fueron quince los estudios que presentaron una o más estrategias de seguridad, nuevamente, el estudio ERGL46 (La documentación oficial de

Jenkins) fue el recurso que presentó más estrategias de seguridad en comparación con los demás estudios, mientras que la mayoría de los estudios reporta entre tres a seis estrategias, en el handbook se identificaron un total de 44 elementos.

De la misma forma que se hizo con los aspectos de seguridad, también se realizó un análisis más profundo de las estrategias para el aseguramiento de los pipelines de CI/CD en comparación con los contextos reportados, los resultados se pueden ver en la Figura 65.

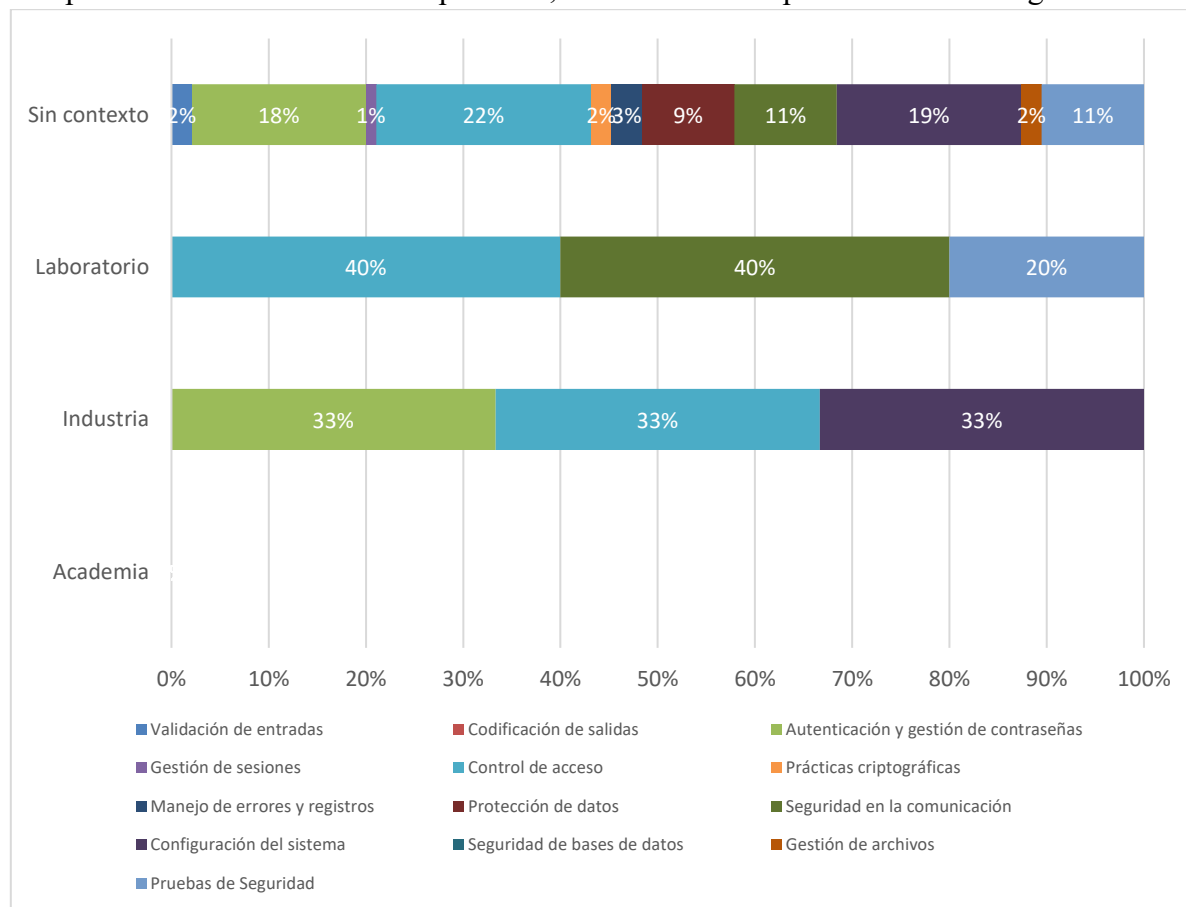


Figura 65. Nivel de adopción de estrategias de seguridad según el contexto.

Como se pudo observar en la figura anterior, los resultados son muy similares a el análisis de los aspectos de seguridad, nuevamente, la mayoría de las estrategias de seguridad se reportaron en estudios sin un contexto específico, mientras que unas cuantas se reportaron en la industria y el laboratorio, aquí vale la pena mencionar que las prácticas de la industria están enfocadas en el control de accesos, la configuración general del sistema, y la autenticación y gestión de las credenciales, justamente estos 3 aspectos son los más populares y los más reportados en los estudios sin contexto, esto se debe a que en su mayoría son estrategias y prácticas muy generales y conocidas, como: el utilizar credenciales o tokens para acceder a las herramientas que se integran con Jenkins, utilizar el sistema gestor de

credenciales que tiene Jenkins por defecto, mantener actualizados los plugins y el propio Jenkins, implementar controles de accesos basados en roles, entre otros. Sin embargo estrategias un poco más complejas de realizar solo se mencionan en estudios sin contexto y unas pocas en estudios de laboratorio.

Otra vez, es importante resaltar que en los estudios con contexto académico que se revisaron, no se encontró ninguna estrategia para el aseguramiento de los pipelines, nuevamente este dato resalta la poca atención e importancia que se le da al tema de seguridad en los cursos de ingeniería de software, y es un área de oportunidad para futuras investigaciones que se enfoquen o den mayor relevancia al aseguramiento de los pipelines de integración y despliegue continuo.

Limitaciones

A continuación, se describen las limitaciones que se identificaron previo y durante el desarrollo de la presente Revisión Sistemática de la Literatura, tanto las limitaciones del trabajo como las identificadas en los estudios primarios.

Limitaciones del trabajo:

- Primeramente, los contextos y escenarios de uso de la herramienta Jenkins se limitan principalmente a la Ingeniería de Software, ciencias computacionales u otras áreas a fines.
- Además, esta Revisión Sistemática no pretende evaluar la calidad de las implementaciones de los pipelines de integración y despliegue continuo.
- Finalmente, todos los estudios recompilados, así como la propia investigación se limita al acceso que se tenga mediante las credenciales institucionales y otros medios legales.

Limitaciones de los estudios:

- En primera instancia, se debe de considerar que en varios de los estudios el uso de la herramienta Jenkins no es la prioridad, por lo que su uso simplemente se reporta en

pequeños fragmentos del estudio y se detalla poco en su funcionamiento, responsabilidades y su integración con las demás herramientas, lo que complica la extracción e interpretación de los datos.

- Además, es importante mencionar que el objetivo de este trabajo es entender y concebir un panorama general del uso real de Jenkins, por lo que se debe tomar en cuenta que se analizaron varios estudios que presentan casos de estudio en los que utilizan Jenkins en ambientes totalmente controlados.

Amenazas a la validez

En primer lugar, es importante mencionar que los dos investigadores principales que condujeron el desarrollo de este trabajo de investigación tienen poca experiencia en enfoques y estrategias sistemáticas de investigación, como los propuestos por B. A. Kitchenham et al. (2015), Popay et al. (2006), Zhang et al. (2011) y Garousi et al. (2019), esto se trató de mitigar mediante la asesoría continua por parte del director y codirector del trabajo de investigación, lo cuales brindaron sus conocimientos y experiencias en el desarrollo de Revisiones Sistemáticas y Multivocales de la Literatura de Ingeniería en Software.

De la misma forma, es relevante mencionar que los investigadores también tienen poca o nula experiencia en el uso de herramientas y prácticas de DevOps, como lo es utilizar Jenkins para crear pipelines de CI/CD, para mitigar esta amenaza, previó a comenzar el proceso de investigación, los investigadores realizaron lecturas exhaustivas sobre el tema, así como realizaron ejercicios prácticos del uso de la herramienta Jenkins, sin embargo se debe de considerar que dichos esfuerzos no compensan en su totalidad la falta de experiencia práctica, especialmente si se compara con el nivel de *expertise* reflejado en los estudios revisados, los cuales abordan enfoques más complejos y avanzados.

Otro aspecto que tomar en cuenta durante el proceso de la búsqueda de estudios primarios es que no se implementaron técnicas de *snowballing*, lo que podría haber omitido más de un estudio primario que respondieran a una de las preguntas de investigación. Este aspecto se mitigó mediante el desarrollo y evaluación de cuatro estándares Quasi-Golds específicos para cada base de datos utilizada. Esta medida tuvo el objetivo de abarcar la mayor cantidad de estudios de interés para el trabajo, aminorando el riesgo de exclusión de estudios significativos.

Por otra parte, es pertinente mencionar la omisión de la etapa de evaluación de los estudios primarios para los recursos de la literatura blanca. Como se mencionó previamente en el documento, para el desarrollo de esta RML se siguieron las pautas propuestas por B. Kitchenham et al. (2015). Sin embargo se tomó la decisión de omitir esta etapa debido a la limitada experiencia de los investigadores principales que llevarían a cabo esta evaluación. Además, debido a que la parte de la literatura blanca de esta RML se centró exclusivamente en recursos publicados en Journals, conferencias, workshops y revistas, se consideró que estos estudios ya garantizan un nivel adecuado de calidad. Por lo que no resulta prudente que dos investigadores con experiencia limitada cuestionaran la calidad de estudios previamente sometidos a procesos de revisión por pares.

Como se mencionó en el apartado designado a la síntesis narrativa, se siguió el proceso propuesto por Popay et al. (2006). Sin embargo, como se especificó en dicho apartado, no se llevó a cabo una evaluación de la robustez, lo cual debe tenerse en cuenta al considerar las conclusiones derivadas de esta RML. Esta etapa se omitió debido al objetivo de las preguntas de investigación planteadas, las cuales no tienen un nivel de interpretación profundo ni subjetivo. Son preguntas exploratorias que buscan respuestas específicas, y poco interpretativas, pero de gran valor. Por lo que las conclusiones obtenidas no están significativamente influenciadas por la perspectiva de los investigadores, lo que mitiga el riesgo de introducir sesgos subjetivos.

Finalmente, otro aspecto a tomar en cuenta es que los métodos y enfoques seguidos para desarrollar la RML tienen un factor de subjetividad intrínseco que depende de las capacidades de los investigadores, por lo que puede haber estudios erróneamente incluidos o excluidos. Para mitigar esta amenaza se llevaron a cabo revisiones con el director principal del trabajo, para validar que se están descartando y aceptando los estudios adecuados.

Conclusiones

Este escrito presentó la revisión multivocal de la literatura (RML) titulada “Análisis del uso de Jenkins en actividades de integración y despliegue continuo: Una Revisión Multivocal de la Literatura”, en la que se analizaron 102 estudios pertenecientes a la literatura blanca y 61 recursos de la literatura gris, siguiendo el proceso de elaboración de RSL que plantea Kitchenham et al. (2015), el proceso de elaboración de RML que plantea Garousi et al. (2019), el proceso de búsqueda y selección de estudios de Zhang et al. (2011) y la aplicación de una síntesis narrativa propuesta por Popay et al. (2006).

El trabajo tuvo como objetivo principal, tal como el título lo menciona, analizar el uso de la herramienta Jenkins. Se buscaban conocer: los contextos y escenarios del uso de la herramienta, las herramientas complementarias con las que se suele integrar, las prácticas de configuración y administración de Jenkins recomendadas por los expertos, las actividades de integración, prueba y despliegue comúnmente integradas y automatizadas con la herramienta y los aspectos de seguridad a considerarse al utilizar Jenkins, así como las principales estrategias para mitigar vulnerabilidades y asegurar los pipelines de CI/CD. Esto a través del planteamiento de cinco preguntas de investigación que servirían de guía.

Tras el análisis de los estudios, el objetivo de esta investigación fue alcanzado y las cinco preguntas se pudieron responder satisfactoriamente. Este trabajo, al ser el primer estudio secundario en analizar el uso de Jenkins, revela vacíos en la investigación que podrían motivar futuros estudios primarios, principalmente en lo relativo a los aspectos de seguridad a considerar al utilizar Jenkins, así como las estrategias de mitigación de vulnerabilidades en los pipelines de CI/CD, pues los resultados revelaron poca información relacionada existente respecto a estos temas en comparación con los resultados obtenidos en las otras preguntas de investigación.

Siguiendo la idea previa, los resultados de este trabajo permitirán proporcionar un panorama general sobre los contextos en los que se usa Jenkins, las herramientas con las que se suele integrar, sus capacidades de automatización y aspectos clave para considerar al configurarlo y administrarlo. Esto establece las bases para el trabajo futuro, pues la RML aquí presentada es el punto de partida de un trabajo de investigación más amplio, que tiene como objetivo desarrollar una guía práctica de Jenkins para el despliegue automatizado de software y, sin duda, los resultados del trabajo proporcionan un buen punto de partida para el trabajo futuro, permitiendo visualizar las capacidades de la herramienta.

Para finalizar, se espera que este trabajo sea de utilidad para otros investigadores y que motive investigaciones futuras, como las ya mencionadas relativas a la seguridad en el uso de Jenkins. También, se espera que el presente trabajo sirva a los profesionales en el área para adentrarse y conocer las capacidades de automatización de la herramienta Jenkins, sus usos reportados, sus integraciones con otras herramientas y las prácticas recomendadas por los expertos.

Referencias

- Amaro, R., Pereira, R., & da Silva, M. M. (2025). Mapping DevOps capabilities to the software life cycle: A systematic literature review. *Information and Software Technology*, 177, 107583. <https://doi.org/https://doi.org/10.1016/j.infsof.2024.107583>
- Bildiri, F., & Akdemir, Ö. (2021). From Agile to DevOps, Holistic Approach for Faster and Efficient Software Product Release Management. *AYBU Business Journal*, 1(1).
- Cruzes, D., & Dybå, T. (2011). Research synthesis in software engineering: A tertiary study. *Information and Software Technology*, 53, 440–455. <https://doi.org/10.1016/j.infsof.2011.01.004>
- Dileepkumar, S. R., & Mathew, J. (2023). Transforming Software Development: Achieving Rapid Delivery, Quality, and Efficiency with Jenkins-Based CI/CD Pipelines. *2023 Annual International Conference on Emerging Research Areas: International Conference on Intelligent Systems, AICERA/ICIS 2023*. <https://doi.org/10.1109/AICERA/ICIS59538.2023.10420251>
- Ebert, C., Gallardo, G., Hernantes, J., & Serrano, N. (2016). DevOps. *IEEE Software*, 33(3). <https://doi.org/10.1109/MS.2016.68>
- Enlyft. (s/f). *Continuous Delivery products*. <https://Enlyft.Com/Tech/Continuous-Delivery>.
- Garousi, V., Felderer, M., & Mäntylä, M. V. (2019). Guidelines for including grey literature and conducting multivocal literature reviews in software engineering. *Information and Software Technology*, 106, 101–121. <https://doi.org/https://doi.org/10.1016/j.infsof.2018.09.006>
- Gowda, P. G., Stanley S A, N., & Eskaline Joyce, J. (2024). DevOps Dynamics: Tools Driving Continuous Integration and Deployment. *2024 IEEE International Conference on Information Technology, Electronics and Intelligent Communication Systems (ICITEICS)*, 1–7. <https://doi.org/10.1109/ICITEICS61368.2024.10624986>
- Humble, J., & Farley, D. (2010). Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation. En *Continuous delivery*.
- International Organization for Standardization., International Electrotechnical Commission., Institute of Electrical and Electronics Engineers., & IEEE-SA Standards Board. (2013). 29119-2-2013 - ISO/IEC/IEEE International Standard - Software and systems engineering —Software testing —Part 2:Test processes. *IEEE Software*.
- ISO/IEC/IEEE. (2013). 29119-1:2013 - ISO/IEC/IEEE International Standard for Software and systems engineering — Software testing — Part 1: Concepts and definitions. *ISO/IEC/IEEE 29119-1:2013(E)*, 2013.

ISO/IEEE. (2021). IEEE Standard for DevOps: Building Reliable and Secure Systems Including Application Build, Package, and Deployment (ISO/IEEE 2675:2021). En *IEEE Std 2675-2021* (Vol. 1).

Jenkins. (s/f-a). *Glossary*. Recuperado el 17 de septiembre de 2024, de <https://www.jenkins.io/doc/book/glossary/>

Jenkins. (s/f-b). *Pipeline*. Recuperado el 17 de septiembre de 2024, de <https://www.jenkins.io/doc/book/pipeline/>

Jetbrains. (2023). *The State of Developer Ecosystem 2023*. <https://www.jetbrains.com/lp/devecosystem-2023/team-tools/>

Jokinen, O. (2020). *Software development using DevOps tools and CD pipelines, A case study* [Master's thesis, University of Helsinki]. <http://urn.fi/URN:NBN:fi:hulib-202003241630>

Kitchenham, B. A., Budgen, D., & Brereton, P. (2015). Evidence-Based Software Engineering and Systematic Reviews. En *Evidence-Based Software Engineering and Systematic Reviews*. <https://doi.org/10.1201/b19467>

Kitchenham, B., Budgen, D., & Brereton, P. (2015). *Evidence-Based Software Engineering and Systematic Reviews*. <https://doi.org/10.1201/b19467>

Kusumadewi, R., & Adrian, R. (2023). Performance Analysis of Devops Practice Implementation Of CI/CD Using Jenkins. *MATICS: Jurnal Ilmu Komputer dan Teknologi Informasi (Journal of Computer Science and Information Technology)*, 15(2). <https://doi.org/10.18860/mat.v15i2.17091>

Leszko, R. (2022). *Continuous delivery with Docker and Jenkins : delivering software at scale*. Packt Publishing.

OWASP Foundation. (2010). *OWASP Secure Coding Practices Quick Reference Guide*.

Popay, J., Roberts, H., Sowden, A., Petticrew, M., Arai, L., Rodgers, M., Britten, N., Roen, K., & Duffy, S. (2006). *Guidance on the conduct of narrative synthesis in systematic reviews: A product from the ESRC Methods Programme*. <https://doi.org/10.13140/2.1.1018.4643>

Révész, Á., & Pataki, N. (2021). Visualisation of Jenkins Pipelines. *Acta Cybernetica*, 25(4). <https://doi.org/10.14232/actacyb.284211>

- Shahin, M., Ali Babar, M., & Zhu, L. (2017). Continuous Integration, Delivery and Deployment: A Systematic Review on Approaches, Tools, Challenges and Practices. *IEEE Access*, 5, 3909–3943. <https://doi.org/10.1109/ACCESS.2017.2685629>
- Theunissen, T., van Heesch, U., & Avgeriou, P. (2022). A mapping study on documentation in Continuous Software Development. *Information and Software Technology*, 142, 106733. <https://doi.org/https://doi.org/10.1016/j.infsof.2021.106733>
- Virmani, M. (2015). Understanding DevOps & bridging the gap from continuous integration to continuous delivery. *Fifth International Conference on the Innovative Computing Technology (INTECH 2015)*, 78–82. <https://doi.org/10.1109/INTECH.2015.7173368>
- Washizaki, H. (2024). *Guide to the Software Engineering Body of Knowledge (SWEBOK Guide), Version 4.0*.
- Zhang, H., Babar, M. A., & Tell, P. (2011). Identifying relevant studies in software engineering. *Information and Software Technology*, 53(6). <https://doi.org/10.1016/j.infsof.2010.12.010>

6 Referencias de los estudios primarios (literatura blanca)

ID	Título del estudio
EP01	Vassallo, C. (2019). Enabling continuous improvement of a continuous integration process. Proceedings - 2019 34th IEEE/ACM International Conference on Automated Software Engineering, ASE 2019, 1246–1249. https://doi.org/10.1109/ASE.2019.00151
EP02	Balalaie, A., Heydarnoori, A., & Jamshidi, P. (2016). Microservices Architecture Enables DevOps: Migration to a Cloud-Native Architecture. <i>IEEE Software</i> , 33(3), 42–52. https://doi.org/10.1109/MS.2016.64
EP03	Dileepkumar, S. R., & Mathew, J. (2023). Transforming Software Development: Achieving Rapid Delivery, Quality, and Efficiency with Jenkins-Based CI/CD Pipelines. 2023 Annual International Conference on Emerging Research Areas: International Conference on Intelligent Systems, AICERA/ICIS 2023. https://doi.org/10.1109/AICERA/ICIS59538.2023.10420251
EP04	Naveen, B., Grandhi, J. K., Lasya, K., Reddy, E. M., Srinivasu, N., & Bulla, S. (2023). Efficient Automation of Web Application Development and Deployment Using Jenkins: A Comprehensive CI/CD Pipeline for Enhanced Productivity and Quality. International Conference on Self Sustainable Artificial Intelligence Systems, ICSSAS 2023 - Proceedings. https://doi.org/10.1109/ICSSAS57918.2023.10331631

EP05	Singh, N., Singh, A., & Rawat, V. (2022). Deploying Jenkins, Ansible and Kubernetes to Automate Continuous Integration and Continuous Deployment Pipeline. 2022 IEEE International Conference on Service Operations and Logistics, and Informatics (SOLI), 1–5. https://doi.org/10.1109/SOLI57430.2022.10294378
EP06	Mysari, S., & Bejgam, V. (2020). Continuous Integration and Continuous Deployment Pipeline Automation Using Jenkins Ansible. 2020 International Conference on Emerging Trends in Information Technology and Engineering (Ic-ETITE), 1–4. https://doi.org/10.1109/ic-ETITE47903.2020.239
EP07	Cepuc, A., Botez, R., Craciun, O., Ivanciu, I. A., & Dobrota, V. (2020). Implementation of a continuous integration and deployment pipeline for containerized applications in amazon web services using jenkins, ansible and kubernetes. Proceedings - RoEduNet IEEE International Conference, 2020-December. https://doi.org/10.1109/RoEduNet51892.2020.9324857
EP08	Vernekar, S. R., Kumar, L., & Kalnoor, G. (2024). Hotel Reservation App Integrated with Docker and Jenkins. 2024 International Conference on Emerging Technologies in Computer Science for Interdisciplinary Applications (ICETCS), 1–6. https://doi.org/10.1109/ICETCS61022.2024.10544088
EP09	Permana, P. A. G., Triandini, E., & Suwirmayanti, N. L. G. P. (2021). Implementation Jenkins Automation Deployment with Scheduler and Notification. 2021 3rd International Conference on Cybernetics and Intelligent System (ICORIS), 1–5. https://doi.org/10.1109/ICORIS52787.2021.9649555
EPI0	Rakshitha, A. H., Jayanthi, P. N., & Manyam, S. (2023). Software Configuration Using Jenkins and Yocto Project. 2023 7th International Conference on Computation System and Information Technology for Sustainable Solutions (CSITSS), 1–4. https://doi.org/10.1109/CSITSS60515.2023.10334213
EPI1	Ankit, A., Nimala, K., & Jadhav, M. (2024). Creation of Continuous Integration Continuous Deployment Pipeline Using Cloud. 2024 5th International Conference on Intelligent Communication Technologies and Virtual Mobile Networks (ICICV), 337–341. https://doi.org/10.1109/ICICV62344.2024.00058
EPI2	Singh, C., Gaba, N. S., Kaur, M., & Kaur, B. (2019). Comparison of Different CI/CD Tools Integrated with Cloud Platform. 2019 9th International Conference on Cloud Computing, Data Science & Engineering (Confluence), 7–12. https://doi.org/10.1109/CONFLUENCE.2019.8776985
EPI3	Deshpande, A., Veenadevi, S. v., & Aleti, S. (2021). Test Automation and Continuous Integration using Jenkins for Smart Card OS. 2021 12th International Conference on Computing Communication and Networking Technologies (ICCCNT), 1–5. https://doi.org/10.1109/ICCCNT51525.2021.9580021
EPI4	Fadil, I., Saeppani, A., Guntara, A., & Mahardika, F. (2020). Distributing Parallel Virtual Image Application using Continuous Integrity/Continuous Delivery Based on Cloud Infrastructure. 2020 8th International Conference on Cyber and IT Service Management (CITSM), 1–4. https://doi.org/10.1109/CITSM50537.2020.9268860
EPI5	Singh, A., Singh, V., Aggarwal, A., & Aggarwal, S. (2022). Improving Business deliveries using Continuous Integration and Continuous Delivery using Jenkins and an Advanced Version control system for Microservices-based system. 2022 5th International Conference on Multimedia, Signal Processing and Communication Technologies (IMPACT), 1–4. https://doi.org/10.1109/IMPACT55510.2022.10029149
EPI6	Kanchana, A., & Murthy B.N., C. (2020). Automated Development and Testing of ECUs in Automotive Industry with Jenkins. 2020 IEEE International Conference on Electronics, Computing and Communication Technologies (CONECCT), 1–5. https://doi.org/10.1109/CONECCT50063.2020.9198423
EPI7	Yiran, W., Tongyang, Z., & Yidong, G. (2018). Design and implementation of continuous integration scheme based on Jenkins and Ansible. 2018 International Conference on Artificial Intelligence and Big Data (ICAIBD), 245–249. https://doi.org/10.1109/ICAIBD.2018.8396203
EPI8	Zhang, S., Fan, D., He, J., & Zhang, P. (2021). A New Approach for End to End Automation Testing Platform with Cloud Computing for 5G Product. 2021 International Conference on Computer Engineering and Application (ICCEA), 322–326. https://doi.org/10.1109/ICCEA53728.2021.00070

EP19	Arachchi, S. A. I. B. S., & Perera, I. (2018). Continuous Integration and Continuous Delivery Pipeline Automation for Agile Software Project Management. 2018 Moratuwa Engineering Research Conference (MERCon), 156–161. https://doi.org/10.1109/MERCon.2018.8421965
EP20	Fathima F., N., & Vani, H. Y. (2020). Building, Deploying and Validating a Home Location Register (HLR) using Jenkins under the Docker and Container Environment. 2020 International Conference on Smart Electronics and Communication (ICOSEC), 1278–1282. https://doi.org/10.1109/ICOSEC49089.2020.9215458
EP21	Chen, T., & Suo, H. (2022). Design and Practice of DevOps Platform via Cloud Native Technology. 2022 IEEE 13th International Conference on Software Engineering and Service Science (ICSESS), 297–300. https://doi.org/10.1109/ICSESS54813.2022.9930226
EP22	Shah, J., Dubaria, D., & Widhalm, J. (2018). A Survey of DevOps tools for Networking. 2018 9th IEEE Annual Ubiquitous Computing, Electronics & Mobile Communication Conference (UEMCON), 185–188. https://doi.org/10.1109/UEMCON.2018.8796814
EP23	Đokić, S., Vujičić, D., Pešović, U., Randić, S., Stojić, D., Jovanović, Ž., Vesković, M., & Marković, D. (2020). Angular and Jenkins Based System for Remote Execution of Tasks on Raspberry Pi Cluster. 2020 19th International Symposium INFOTEH-JAHORINA (INFOTEH), 1–4. https://doi.org/10.1109/INFOTEH48170.2020.9066299
EP24	Rajendra, A., Reddy, P. S., Vignesh, B. S. P., & Rao, T. S. M. (2024). Setting Up A CICD Pipeline in The Cloud for A Web Application. 2024 International Conference on Expert Clouds and Applications (ICOECA), 213–217. https://doi.org/10.1109/ICOECA62351.2024.00048
EP25	Pandi, S. S., Kumar, P., & Suchindhar, R. M. (2023). Integrating Jenkins for Efficient Deployment and Orchestration across Multi-Cloud Environments. 2023 International Conference on Innovative Computing, Intelligent Communication and Smart Electrical Systems (ICSES), 1–6. https://doi.org/10.1109/ICSES60034.2023.10465502
EP26	Yulianto, Y., Utami, E., & Kusnawi. (2022). Automatic Deployment Pipeline for Containerized Application of IoT Device. 2022 6th International Conference on Information Technology, Information Systems and Electrical Engineering (ICITISEE), 81–86. https://doi.org/10.1109/ICITISEE57756.2022.10057699
EP27	Niranjan, D. R., & Mohana. (2022). Jenkins Pipelines: A Novel Approach to Machine Learning Operations (MLOps). International Conference on Edge Computing and Applications, ICECAA 2022 - Proceedings, 1292–1297. https://doi.org/10.1109/ICECAA55415.2022.9936252
EP28	Srithar, S., Vetrimani, E., Vignesh, V., Ulaganathan, M. S., Kumar, B. R., & Alagumuthukrishnan, S. (2022). Cost-Effective Integration and Deployment of Enterprise Application Using Azure Cloud Devops. 2022 International Conference on Computer Communication and Informatics (ICCCI), 1–5. https://doi.org/10.1109/ICCCI54379.2022.9740874
EP29	Campell, C., Graber, J., Tashakkori, R., & O'Brien, W. (2022). IoT Apiary Fleet Management with Jenkins. 2022 IEEE 13th Annual Ubiquitous Computing, Electronics & Mobile Communication Conference (UEMCON), 328–335. https://doi.org/10.1109/UEMCON54665.2022.9965635
EP30	Bani Muhamad, F. P., Sarno, R., Ahmadiyah, A. S., & Rochimah, S. (2016). Visual GUI testing in continuous integration environment. 2016 International Conference on Information & Communication Technology and Systems (ICTS), 214–219. https://doi.org/10.1109/ICTS.2016.7910301
EP31	Meghana, K., Deep, V. D., Chowdary, V. S., Jagadish, G., K. A., & Rao, T. K. R. K. (2023). Deploying a Build Job Orchestration in Docker. 2023 2nd International Conference on Edge Computing and Applications (ICECAA), 476–480. https://doi.org/10.1109/ICECAA58104.2023.10212235
EP32	Paliawadana, S., Wijeweera, C. H., Sanjitha, M. G. T. N., Liyanage, V. K., Perera, I., & Meedeniya, D. A. (2017). Tool support for traceability management of software artefacts with DevOps practices. 2017 Moratuwa Engineering Research Conference (MERCon), 129–134. https://doi.org/10.1109/MERCon.2017.7980469
EP33	Ekanayaka, E. M. I. M., Thathsarani, J. K. K. H., Karunanayaka, D. S., Kuruwitaarachchi, N., & Skandhakumar, N. (2023). Enhancing Devops Infrastructure For Efficient Management Of Microservice Applications. 2023 IEEE International Conference on E-Business Engineering (ICEBE), 63–68. https://doi.org/10.1109/ICEBE59045.2023.00035
EP34	Luhana, K. K., Schindler, C., & Slany, W. (2018). Streamlining mobile app deployment with Jenkins and Fastlane in the case of Catrobat's pocket code. 2018 IEEE International Conference on Innovative Research and Development (ICIRD), 1–6. https://doi.org/10.1109/ICIRD.2018.8376296
EP35	Gota, L., Gota, D., & Miclea, L. (2020). Continuous Integration in Automation Testing. 2020 IEEE International Conference on Automation, Quality and Testing, Robotics (AQTR), 1–6. https://doi.org/10.1109/AQTR49680.2020.9129990
EP36	Sun, B., Shao, Y., & Chen, C. (2019). Study on the Automated Unit Testing Solution on the Linux Platform. 2019 IEEE 19th International Conference on Software Quality, Reliability and Security Companion (QRS-C), 358–361. https://doi.org/10.1109/QRS-C.2019.00073

EP37	Aktas, G., Ipek, B., Ali Konukoglu, E., & Aydin, Y. (2023). Development of Artificial Intelligence Supported Tool for Anomaly Detection in Cloud Computing Systems. 2023 International Conference on Electrical, Communication and Computer Engineering (ICECCE), 1–6. https://doi.org/10.1109/ICECCE61019.2023.10442490
EP38	Arcuri, A., Campos, J., & Fraser, G. (2016). Unit Test Generation During Software Development: EvoSuite Plugins for Maven, IntelliJ and Jenkins. 2016 IEEE International Conference on Software Testing, Verification and Validation (ICST), 401–408. https://doi.org/10.1109/ICST.2016.44
EP39	Lv, K., Zhao, Z., Rao, R., Hong, P., & Zhang, X. (2016). PCCTE: A portable component conformance test environment based on container cloud for avionics software development. 2016 International Conference on Progress in Informatics and Computing (PIC), 664–668. https://doi.org/10.1109/PIC.2016.7949582
EP40	N, V. S., Mohan Saini, L., & Mohan, H. (2022). Cloud Computing in Automation Testing. 2022 International Conference on Edge Computing and Applications (ICECAA), 31–36. https://doi.org/10.1109/ICECAA55415.2022.9936462
EP41	Setiawan, H., Erlangga, L. E., & Baskoro, I. (2020). Vulnerability Analysis Using The Interactive Application Security Testing (IAST) Approach For Government X Website Applications. 2020 3rd International Conference on Information and Communications Technology (ICOIACT), 471–475. https://doi.org/10.1109/ICOIACT50329.2020.9332116
EP42	Birchler, C., Rohrbach, C., Kim, H., Gambi, A., Liu, T., Horneber, J., Kehrer, T., & Panichella, S. (2023). TEASER: Simulation-Based CAN Bus Regression Testing for Self-Driving Cars Software. 2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE), 2058–2061. https://doi.org/10.1109/ASE56229.2023.00154
EP43	Steffens, A., Lichter, H., & Döring, J. S. (2018). Designing a next-generation continuous software delivery system: Concepts and architecture. Proceedings - International Conference on Software Engineering, 1–7. https://doi.org/10.1145/3194760.3194768
EP44	Düllmann, T. F., Paule, C., & Hoorn, A. van. (2018). Exploiting DevOps Practices for Dependable and Secure Continuous Delivery Pipelines. 2018 IEEE/ACM 4th International Workshop on Rapid Continuous Software Engineering (RCoSE), 27–30.
EP45	Straubinger, P., & Fraser, G. (2024). Gamifying a Software Testing Course with Continuous Integration. 2024 IEEE/ACM 46th International Conference on Software Engineering: Software Engineering Education and Training (ICSE-SEET), 34–45. https://doi.org/10.1145/3639474.3640054
EP46	Straubinger, P., & Fraser, G. (2024). An IDE Plugin for Gamified Continuous Integration.
EP47	Straubinger, P., & Fraser, G. (2022). Gamekins: Gamifying Software Testing in Jenkins. 2022 IEEE/ACM 44th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion), 85–89. https://doi.org/10.1145/3510454.3516862
EP48	Zampetti, F., Nardone, V., & di Penta, M. (2022). Problems and Solutions in Applying Continuous Integration and Delivery to 20 Open-Source Cyber-Physical Systems. 2022 IEEE/ACM 19th International Conference on Mining Software Repositories (MSR), 646–657. https://doi.org/10.1145/3524842.3527948
EP49	Light, J., Pfeiffer, P., & Bennett, B. (2021). An evaluation of continuous integration and delivery frameworks for classroom use. Proceedings of the 2021 ACM Southeast Conference, 204–208. https://doi.org/10.1145/3409334.3452085
EP50	Heckman, S., & King, J. (2018). Developing Software Engineering Skills using Real Tools for Automated Grading. Proceedings of the 49th ACM Technical Symposium on Computer Science Education, 794–799. https://doi.org/10.1145/3159450.3159595
EP51	Pecka, N., ben Othmane, L., & Valani, A. (2022). Privilege Escalation Attack Scenarios on the DevOps Pipeline Within a Kubernetes Environment. Proceedings of the International Conference on Software and System Processes and International Conference on Global Software Engineering, 45–49. https://doi.org/10.1145/3529320.3529325

EP52	Antunes, R. V., Navarro, G. M., & Hanazumi, S. (2018). Test Framework for Jenkins Shared Libraries. Proceedings of the III Brazilian Symposium on Systematic and Automated Software Testing, 13–19. https://doi.org/10.1145/3266003.3266008
EP53	Schloyer, P., Knauer, P., Bauer, B., & Merli, D. (2024). Automating Side-Channel Testing for Embedded Systems: A Continuous Integration Approach. Proceedings of the 19th International Conference on Availability, Reliability and Security. https://doi.org/10.1145/3664476.3670436
EP54	Sarmiento-Calisaya, E., Mamani-Aliaga, A., & do Prado Leite, J. C. S. (2024). Introducing Computer Science Undergraduate Students to DevOps Technologies from Software Engineering Fundamentals. 2024 IEEE/ACM 46th International Conference on Software Engineering: Software Engineering Education and Training (ICSE-SEET), 348–358. https://doi.org/10.1145/3639474.3640071
EP55	Eddy, B. P., Wilde, N., Cooper, N. A., Mishra, B., Gamboa, V. S., Patel, K. N., & Shah, K. M. (2017). CDEP: Continuous Delivery Educational Pipeline. Proceedings of the 2017 ACM Southeast Conference, 55–62. https://doi.org/10.1145/3077286.3077301
EP56	Amalfitano, D., de Simone, V., Fasolino, A. R., & Scala, S. (2017). Improving traceability management through tool integration: an experience in the automotive domain. Proceedings of the 2017 International Conference on Software and System Process, 5–14. https://doi.org/10.1145/3084100.3084101
EP57	Schreiber, M., Kraft, B., & Zündorf, A. (2016). Cost-efficient quality assurance of natural language processing tools through continuous monitoring with continuous integration. Proceedings of the 3rd International Workshop on Software Engineering Research and Industrial Practice, 46–52. https://doi.org/10.1145/2897022.2897029
EP58	Ohtsuki, M., Ohta, K., & Kakeshita, T. (2016). Software engineer education support system ALECS utilizing DevOps tools. Proceedings of the 18th International Conference on Information Integration and Web-Based Applications and Services, 209–213. https://doi.org/10.1145/3011141.3011200
EP59	Lorenzo, P. R., Nalepa, J., Ramos, L. S., & Pastor, J. R. (2017). Hands-Free Research Workflow. Proceedings of the 21st International Conference on Evaluation and Assessment in Software Engineering, 70–73. https://doi.org/10.1145/3084226.3084266
EP60	Janes, A., Lenarduzzi, V., & Stan, A. C. (2017). A Continuous Software Quality Monitoring Approach for Small and Medium Enterprises. Proceedings of the 8th ACM/SPEC on International Conference on Performance Engineering Companion, 97–100. https://doi.org/10.1145/3053600.3053618
EP61	Huijgens, H., Lamping, R., Stevens, D., Rothengatter, H., Gousios, G., & Romano, D. (2017). Strong agile metrics: mining log data to determine predictive power of software metrics for continuous delivery teams. Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering, 866–871. https://doi.org/10.1145/3106237.3117779
EP62	Sampedro, Z., Holt, A., & Hauser, T. (2018). Continuous Integration and Delivery for HPC: Using Singularity and Jenkins. Proceedings of the Practice and Experience on Advanced Research Computing: Seamless Creativity. https://doi.org/10.1145/3219104.3219147
EP63	Greising, L., Bartel, A., & Hagel, G. (2018). Introducing a Deployment Pipeline for Continuous Delivery in a Software Architecture Course. Proceedings of the 3rd European Conference of Software Engineering Education, 102–107. https://doi.org/10.1145/3209087.3209091
EP64	Hur, A., & Ryu, Y. (2020). Developing Anti-tamper Functionalities through Continuous Integration. Proceedings of the 2020 5th International Conference on Intelligent Information Technology, 34–38. https://doi.org/10.1145/3385209.3385219
EP65	The Hajek, J., Rashid, M., Sevil, M., Cinar, A., Alvarez Fernandez, P. A., & Jain, D. (2020). The Necessity of Interdisciplinary Software Development for Building Viable Research Platforms: Case Study in Automated Drug Delivery in Diabetes. Proceedings of the 21st Annual Conference on Information Technology Education, 390–396. https://doi.org/10.1145/3368308.3415377
EP66	Mendoza, C., Garcés, K., Casallas, R., & Bocanegra, J. (2021). Detecting architectural issues during the continuous integration pipeline. Proceedings of the 22nd International Conference on Model Driven Engineering Languages and Systems, 589–597. https://doi.org/10.1109/MODELS-C.2019.00090
EP67	López-Huguet, S., García-Castro, F., Alberich-Bayarri, A., & Blanquer, I. (2019). A Cloud Architecture for the Execution of Medical Imaging Biomarkers. In J. M. F. Rodrigues, P. J. S. Cardoso, J. Monteiro, R. Lam, V. v Krzhizhanovskaya, M. H. Lees, J. J. Dongarra, & P. M. A. Sloot (Eds.), Computational Science – ICCS 2019 (pp. 130–144). Springer International Publishing.
EP68	Berger, C. (2016). An Open Continuous Deployment Infrastructure for a Self-driving Vehicle Ecosystem. In K. Crowston, I. Hammouda, B. Lundell, G. Robles, J. Gamalielsson, & J. Lindman (Eds.), Open Source Systems: Integrating Communities (pp. 177–183). Springer International Publishing.
EP69	Biswas, N., Mondal, A. S., Kusumastuti, A., Saha, S., & Mondal, K. C. (2022). Automated credit assessment framework using ETL process and machine learning. Innovations in Systems and Software Engineering. https://doi.org/10.1007/s11334-022-00522-x

EP70	Wei, H., Rodriguez, J. S., & Garcia, O. N.-T. (2021). Deployment Management and Topology Discovery of Microservice Applications in the Multicloud Environment. <i>Journal of Grid Computing</i> , 19(1), 1. https://doi.org/10.1007/s10723-021-09539-1
EP71	Teixeira, J. A., & Karsten, H. (2019). Managing to release early, often and on time in the OpenStack software ecosystem. <i>Journal of Internet Services and Applications</i> , 10(1), 7. https://doi.org/10.1186/s13174-019-0105-z
EP72	Munir, H., Linäker, J., Wnuk, K., Runeson, P., & Regnell, B. (2018). Open innovation using open source tools: a case study at Sony Mobile. <i>Empirical Software Engineering</i> , 23(1), 186–223. https://doi.org/10.1007/s10664-017-9511-7
EP73	Altunel, H., & Say, B. (2021). Software Product System Model: A Customer-Value Oriented, Adaptable, DevOps-Based Product Model. <i>SN Computer Science</i> , 3(1), 38. https://doi.org/10.1007/s42979-021-00899-9
EP74	Kao, C. H. (2017). Testing and evaluation framework for virtualization technologies. <i>Computing</i> , 99(7), 657–677. https://doi.org/10.1007/s00607-016-0517-6
EP75	Song, H., Dautov, R., Ferry, N., Solberg, A., & Fleurey, F. (2022). Model-based fleet deployment in the IoT–edge–cloud continuum. <i>Software and Systems Modeling</i> , 21(5), 1931–1956. https://doi.org/10.1007/s10270-022-01006-z
EP76	Sallin, M., Kropp, M., Anslow, C., Quilty, J. W., & Meier, A. (2021). Measuring Software Delivery Performance Using the Four Key Metrics of DevOps. In P. Gregory, C. Lassenius, X. Wang, & P. Kruchten (Eds.), <i>Agile Processes in Software Engineering and Extreme Programming</i> (pp. 103–119). Springer International Publishing.
EP77	Ståhl, D., Hallén, K., & Bosch, J. (2017). Achieving traceability in large scale continuous integration and delivery deployment, usage and validation of the eiffel framework. <i>Empirical Software Engineering</i> , 22(3), 967–995. https://doi.org/10.1007/s10664-016-9457-1
EP78	Rinker, F., Waltersdorfer, L., Meixner, K., Winkler, D., Lüder, A., & Biffl, S. (2023). Traceable Multi-view Model Integration: A Transformation Pipeline for Agile Production Systems Engineering. <i>SN Computer Science</i> , 4(2), 205. https://doi.org/10.1007/s42979-022-01572-5
EP79	Orviz Fernández, P., Pina, J., López García, Á., Campos Plasencia, I., David, M., & Gomes, J. (2018). umd-verification: Automation of Software Validation for the EGI Federated e-Infrastructure. <i>Journal of Grid Computing</i> , 16(4), 683–696. https://doi.org/10.1007/s10723-018-9454-2
EP80	Bernardo, S., Orviz, P., David, M., Gomes, J., Arce, D., Naranjo, D., Blanquer, I., Campos, I., Moltó, G., & Pina, J. (2024). Software Quality Assurance as a Service: Encompassing the quality assessment of software and services. <i>Future Generation Computer Systems</i> , 156, 254–268. https://doi.org/10.1016/j.FUTURE.2024.03.024
EP81	Lange, M., Tran, K. C., Grunewald, A., Koschel, A., Pakosch, A., & Astrova, I. (2023). Modern Build Automation for an Insurance Company Tool Selection. <i>Procedia Computer Science</i> , 219, 736–743. https://doi.org/10.1016/j.PROCS.2023.01.346
EP82	Yang, D., Wang, D., Yang, D., Dong, Q., Wang, Y., Zhou, H., & Daocheng, H. (2020). DevOps in Practice for Education Management Information System at ECNU. <i>Procedia Computer Science</i> , 176, 1382–1391. https://doi.org/10.1016/j.PROCS.2020.09.148
EP83	Hermosilla, A., Gallego-Madrid, J., Martinez-Julia, P., Ortiz, J., Kafle, V. P., & Skarmeta, A. (2024). Advancing 5G Network Applications Lifecycle Security: An ML-Driven Approach. <i>CMES - Computer Modeling in Engineering and Sciences</i> , 141(2), 1447–1471. https://doi.org/10.32604/CMES.2024.053379
EP84	Chadwick, D. W., Fan, W., Costantino, G., de Lemos, R., di Cerbo, F., Herwono, I., Manea, M., Mori, P., Sajjad, A., & Wang, X. S. (2020). A cloud-edge based data security architecture for sharing and analysing cyber threat information. <i>Future Generation Computer Systems</i> , 102, 710–722. https://doi.org/10.1016/j.FUTURE.2019.06.026
EP85	Sadek, J., Craig, D., & Trenell, M. (2022). Design and Implementation of Medical Searching System Based on Microservices and Serverless Architectures. <i>Procedia Computer Science</i> , 196, 615–622. https://doi.org/10.1016/j.PROCS.2021.12.056

EP86	Krzemien, W., Gajos, A., Kacprzak, K., Rakoczy, K., & Korcyl, G. (2020). J-PET Framework: Software platform for PET tomography data reconstruction and analysis. <i>SoftwareX</i> , 11, 100487. https://doi.org/10.1016/j.SOFTX.2020.100487
EP87	Kayser, H., Herzke, T., Maanen, P., Zimmermann, M., Grimm, G., & Hohmann, V. (2022). Open community platform for hearing aid algorithm research: open Master Hearing Aid (openMHA). <i>SoftwareX</i> , 17, 100953. https://doi.org/10.1016/j.SOFTX.2021.100953
EP88	Gallego-Madrid, J., Sanchez-Iborra, R., & Skarmeta, A. (2021). From Network Functions to NetApps: The 5GASP Methodology. <i>Computers, Materials and Continua</i> , 71(2), 4115–4134. https://doi.org/10.32604/CMC.2022.021754
EP89	Hähner, U. R., Alvarez, G., Maier, T. A., Solcà, R., Staar, P., Summers, M. S., & Schulthess, T. C. (2020). DCA++: A software framework to solve correlated electron problems with modern quantum cluster methods. <i>Computer Physics Communications</i> , 246, 106709. https://doi.org/10.1016/j.CPC.2019.01.006
EP90	Manzano, M., Ayala, C., Gómez, C., Abherve, A., Franch, X., & Mendes, E. (2021). A Method to Estimate Software Strategic Indicators in Software Development: An Industrial Application. <i>Information and Software Technology</i> , 129, 106433. https://doi.org/10.1016/j.INFSOF.2020.106433
EP91	He, X., Tu, Z., Xu, X., & Wang, Z. (2021). Programming framework and infrastructure for self-adaptation and optimized evolution method for microservice systems in cloud-edge environments. <i>Future Generation Computer Systems</i> , 118, 263–281. https://doi.org/10.1016/j.FUTURE.2021.01.008
EP92	Shrestha, R., & Ray, A. (2024). Streamlining Application Deployment: A CI/CD Pipeline for Kubernetes. 2024 IEEE International Conference on Cloud Engineering (IC2E), 253–255. https://doi.org/10.1109/IC2E61754.2024.00038
EP93	Qu, M., He, J., Tucakovic, Z., Bartocci, E., Nickovic, D., Isakovic, H., & Grosu, R. (2024). DeepRIoT: Continuous Integration and Deployment of Robotic-IoT Applications. <i>Proceedings of the 61st ACM/IEEE Design Automation Conference</i> . https://doi.org/10.1145/3649329.3658250
EP94	Mondal, S. K., Wu, X., & Pan, R. (2024). On Rapid Application Deployment with Kubernetes. <i>Proceedings of the 2024 4th International Conference on Artificial Intelligence, Automation and Algorithms</i> , 76–80. https://doi.org/10.1145/3700523.3700538
EP95	Li, H., & Mondal, S. K. (2024). Automation Framework Foundation for Continuous Integration with Docker. <i>ACM International Conference Proceeding Series</i> , 70–75. https://doi.org/10.1145/3700523.3700537
EP96	Saleh, S. M., Sayem, I. M., Madhavji, N., & Steinbacher, J. (2024). Advancing Software Security and Reliability in Cloud Platforms through AI-based Anomaly Detection. <i>Proceedings of the 2024 on Cloud Computing Security Workshop</i> , 43–52. https://doi.org/10.1145/3689938.3694779
EP97	Savant, R. V., Sunder, S. N., Seshadri, S., Panda, N., & Rajagopal, S. M. (2024). Cloud-Native CDN Monitoring Using CI/CD. 2024 15th International Conference on Computing Communication and Networking Technologies (ICCCNT), 1–9. https://doi.org/10.1109/ICCCNT61001.2024.10724159
EP98	Nikolov, L., & Aleksieva-Petrova, A. (2024). Framework for Integrating Threat Modeling into a DevOps Pipeline for Enhanced Software Development. 2024 International Conference on Software, Telecommunications and Computer Networks (SoftCOM), 1–5. https://doi.org/10.23919/SoftCOM62040.2024.10721871
EP99	Baitha, S., Soorya, V., Kothari, O., Rajagopal, S. M., & Panda, N. (2024). Streamlining Software Development: A Comprehensive Study on CI/CD Automation. 2024 4th International Conference on Sustainable Expert Systems (ICSES), 1299–1305. https://doi.org/10.1109/ICSES63445.2024.10763207
EPI00	Kumar, S., Bala, Er. N., Singh, A. P., & Raj, Y. (2024). Cloud-Native Continuous Integration/Continuous Deployment (CI/CD) Pipeline. 2024 Second International Conference on Advanced Computing & Communication Technologies (ICACCTech), 292–297. https://doi.org/10.1109/ICACCTech65084.2024.00054
EPI01	Georgiev, A., Valkanov, V., & Georgiev, P. (2024). A comparative analysis of Jenkins as a data pipeline tool in relation to dedicated data pipeline frameworks. 2024 International Conference Automatics and Informatics (ICAI), 508–512. https://doi.org/10.1109/ICAI63388.2024.10851591
EPI02	Newman, E., Dor, H., Hans, M., Curtis, A., Platt, F., Hemmings, R., & Okoye, O. (2024). Maintaining Mars Rover Operations Software on a Budget. 2024 IEEE 10th International Conference on Space Mission Challenges for Information Technology (SMC-IT), 67–77. https://doi.org/10.1109/SMC-IT61443.2024.00015

Apéndice A) Protocolo de investigación del trabajo recepcional

En el siguiente enlace se puede acceder al protocolo de investigación:

[Protocolo de investigación](#)

Apéndice B) Protocolos de la Revisión multivocal de la literatura

En el siguiente enlace se puede acceder al protocolo de la RSL y de la posterior actualización a RML:

[Protocolo de la RSL](#)

[Protocolo de la RML.docx](#)

Apéndice C) Rúbrica de Evaluación de la Calidad de la Literatura Gris.

Tabla 61. Rúbrica de Evaluación de la Calidad de la Literatura Gris			
Enfoque del Criterio	ID	Descripción del criterio	Escala de Puntaje
Autoridad del autor	QEC01	¿La organización que publica tiene buena reputación?	(no=0 o sí=1)
Metodología.	QEC02	¿La fuente tiene un objetivo claramente establecido?	(no=0, parcialmente=0.5 o sí=1)
	QEC03	¿La fuente tiene una metodología establecida?	(no=0, parcialmente=0.5 o sí=1)
	QEC04	¿La fuente está sustentada con referencias de autoridad y contemporáneas?	(no=0 o sí=1)
	QEC05	¿Hay límites bien establecidos?	(no=0 o sí=1)
	QEC06	¿El trabajo refiere a un caso particular?	(no=0 o sí=1)
Objetividad	QEC07	¿Existen intereses creados? (conflictos de intereses)	(sí=0, parcialmente=0.5, no=1)
Fecha	QEC08	¿El trabajo tiene una fecha bien establecida?	(no=0 o sí=1)
Posición respecto a fuentes relacionadas	QEC09	¿Se han vinculado o discutido fuentes formales o GL clave relacionadas?	(no=0 o sí=1)
Innovación	QEC10	¿Enriquece o añade algo único a la investigación?	(no=0 o sí=1)
	QEC11	¿Fortalece o refuta una posición actual?	(no=0 o sí=1)
Impacto	QEC12	"¿El trabajo tiene un impacto en la comunidad según el recuento de backlinks o vistas (para videos)?	Valor normalizado con base en el recurso con mayor cantidad de <i>backlinks</i> , vistas o citas según el tipo de recurso.

		Utilizar https://www.seoreviewtools.com/valuable-backlinks-checker/ para este propósito, utilizando el recuento de external backlinks."	Puntaje máximo = 1.
Tipo de recurso	QEC13	"Libros, revistas, tesis, informes gubernamentales, artículos de literatura blanca, artículos electrónicos (medida=1) Informes anuales, artículos de noticias, presentaciones, vídeos, sitios de preguntas y respuestas, Artículos wiki (medida=0.5) Blogs, correos electrónicos, tweets (medida=0)"	(nivel 3 = 0, nivel 2 = 0.5 o nivel 1 = 1)

Apéndice D) Formatos de extracción de datos.

A continuación se presenta el formato de extracción de datos utilizado para la literatura blanca y literatura gris.

Tabla 62. Formato de Extracción de datos de la literatura blanca.

DATOS DEL ESTUDIO	
Elemento	Descripción
ID	Identificador único del estudio primario, definido por los responsables. Debe tener el formato "EPXX", donde "XX" representa un número asignado de forma única para cada estudio, de forma secuencial y comenzando por el 01.
Título	Título del estudio.
Autores	Responsables de la publicación del estudio.
Año	Año en el que fue publicado el estudio y que debe encontrarse entre 2016 y 2024 (incluyendo estos años), siguiendo los criterios de inclusión.
Fuente	URL desde donde se puede acceder al estudio.
DOI	Identificador único del estudio.
Base de datos	Nombre del motor de búsqueda desde donde se recuperó el estudio.
Tipo de publicación	Alguno de los siguientes valores: "Journal", "Conferencia" o "Workshop".
Venue	Nombre de la conferencia o journal donde fue publicado el estudio.
Palabras clave	Palabras clave del estudio.
Abstract o resumen	Abstract del estudio.
ANÁLISIS DE LA INFORMACIÓN	
Elemento	Descripción
Pregunta/s de investigación relacionada/s	Lista de las preguntas de investigación que el estudio responde.
Contexto/s o escenario/s en los que se suele utilizar Jenkins	Contenido del estudio que responde a la pregunta de investigación P 01.
Herramienta/s complementarias con las que se complementa Jenkins	Contenido del estudio que responde a la pregunta de investigación P 02.
Práctica/s recomendada/s para la configuración y/o administración de Jenkins en entornos de producción	Contenido del estudio que responde a la pregunta de investigación P 03.
Actividades integradas y/o automatizadas utilizando Jenkins	Contenido del estudio que responde a la pregunta de investigación P 04.

Aspecto/s de seguridad para tomar en cuenta y/o estrategias para mitigar vulnerabilidades al utilizar Jenkins	Contenido del estudio que responde a la pregunta de investigación P 05.
---	---

Tabla 63. Formato de Extracción de datos de la literatura gris.

DATOS DEL ESTUDIO	
Elemento	Descripción
ID	Identificador único del estudio primario, definido por los responsables. Debe tener el formato “EPXX”, donde “XX” representa un número asignado de forma única para cada estudio, de forma secuencial y comenzando por el 01.
Título	Título del estudio.
Autores	Responsables de la publicación del estudio.
Año	Año en el que fue publicado el estudio y que debe encontrarse entre 2016 y 2024 (incluyendo estos años), siguiendo los criterios de inclusión.
Fuente	URL desde donde se puede acceder al estudio.
DOI	Identificador único del estudio.
Base de datos	Nombre del motor de búsqueda desde donde se recuperó el estudio.
Tipo de publicación	Alguno de los siguientes valores: “Journal”, “Conferencia” o “Workshop”.
Venue	Nombre de la conferencia o journal donde fue publicado el estudio.
Palabras clave	Palabras clave del estudio.
Abstract o resumen	Abstract del estudio.
ANÁLISIS DE LA INFORMACIÓN	
Elemento	Descripción
Pregunta/s de investigación relacionada/s	Lista de las preguntas de investigación que el estudio responde.
Contexto/s o escenario/s en los que se suele utilizar Jenkins	Contenido del estudio que responde a la pregunta de investigación P 01.
Herramienta/s complementarias con las que se complementa Jenkins	Contenido del estudio que responde a la pregunta de investigación P 02.
Práctica/s recomendada/s para la configuración y/o administración de Jenkins en entornos de producción	Contenido del estudio que responde a la pregunta de investigación P 03.
Actividades integradas y/o automatizadas utilizando Jenkins	Contenido del estudio que responde a la pregunta de investigación P 04.

Apéndice E) Artefactos generados durante el desarrollo de la RSL

En el siguiente enlace se presentan los diversos artefactos que se generaron durante el desarrollo de la RSL, tales como: hojas de cálculo con los estudios seleccionados, documentos de Word, cronogramas, presentaciones, etc.

[Artefactos de referencia](#)

“Lis de Veracruz: Arte, Ciencia, Luz”

www.uv.mx

